UML gota a gota

Martin Fowler

con

Kendall Scott

TRADUCCIÓN:

Jaime González V.
Especialista en análisis y diseño de sistemas

David Morales Peake
Traductor profesional

REVISIÓN TÉCNICA:

Dr. Gabriel Guerrero
Facultad de Ciencias de la Universidad Nacional Autónoma de México
Doctorado en Matemáticas, París VI, Francia
Consultor en saXsa

PEARSON Educación®

México • Argentina • Brasil • Colombia • Costa Rica • Chile • Ecuador
España • Guatemala • Panamá • Perú • Puerto Rico • Uruguay • Venezuela

Datos de catalogación bibliográfica

FOWLER, MARTIN
UML gota a gota
Addison Wesley Longman de México, S.A. de C.V.
México, 1999

ISBN: 968-444-364-1
Materia: Computación

Formato: 17 x 23 Páginas: 224

Versión en español de la obra titulada *UML Distilled*, de Martin Fowler, publicada originalmente en inglés por Addison Wesley Longman, Inc., Reading, Massachusetts, E.U.A.

Esta edición en español es la única autorizada.

Original English language title by
Addison Wesley Longman, Inc.
Copyright © 1997
All rights reserved
ISBN 0-201-32563-2

Edición en español:
Editor: Pablo Eduardo Roig Vázquez
Supervisor de traducción: Teresa Sanz
Supervisor de producción: Selene Corona Vallejo
Diseñador de portada: Juan Bernardo Rosado

Edición en inglés:
Executive Editor: J. Carter Shanklin
Assisant Editor: Angela Buenning
Project Manager: Sarah Weaver
Copyeditor: Arlene Richman
Proofreader: Maine Proofreading Services
Index and Composition: Kendall Scott
Cover Desing: Simone Payment

Derechos Reservados © 1999
respecto a la primera edición
en español publicada por:

ADDISON WESLEY LONGMAN DE MÉXICO, S.A. DE C.V.
Atlacomulco Núm. 500-5° Piso
Col. Industrial Atoto
53519, Naucalpan de Juárez, Edo. de México

Cámara de la Industria Editorial Mexicana Reg. Núm. 1031

Reservados todos los derechos. Ni la totalidad ni parte de esta publicación pueden reproducirse, registrarse o transmitirse por un sistema de recuperación de información, en ninguna forma, ni por ningún medio, sea electrónico, mecánico, fotoquímico, magnético o electroóptico, por fotocopia, grabación o cualquier otro, sin permiso previo por escrito del editor.

El préstamo, alquiler o cualquier otra forma de cesión de uso de este ejemplar requerirá también la autorización del editor o de sus representantes.

ISBN 968-444-364-1

Impreso en México. *Printed in Mexico.*

1 2 3 4 5 6 7 8 9 0 03 02 01 00 99

Contenido

Prólogo xi

Prefacio xiii

Reconocimientos xv

Capítulo 1: Introducción 1

¿Qué es el UML? 1

Cómo llegamos hasta aquí 2

Notaciones y metamodelos 5

¿Por qué analizar y diseñar? 7

Aprendizaje de OO 8

Comunicación con los expertos del dominio 10

Comprensión del panorama general 11

Para mayor información 12

Capítulo 2: Un bosquejo del proceso de desarrollo 15

Panorámica del proceso 16

Concepción 18

Elaboración 18

Manejo de los riesgos de requerimientos 20

Manejo de los riesgos tecnológicos 25

Manejo de los riesgos de habilidad 27

Manejo de los riesgos políticos 29

Base arquitectónica 29

¿Cuándo se termina la elaboración? 30

La planificación 30

Construcción 33

Reestructuración de factores 35

Cuándo reestructurar 37

Para mayor información 37
Desarrollo y planificación iterativos 38
Empleo del UML en la construcción 38
Patrones 42
Cuándo utilizar patrones 45
Para mayor información 45
Transición 47
Cuándo se debe usar el desarrollo iterativo 47
Para mayor información 48
Capítulo 3: Los casos de uso 49
Objetivos del usuario e interacciones con el sistema 50
Diagramas de casos de uso 51
Actores 52
Uses y extends 55
Cuándo emplear casos de uso 58
Para mayor información 59
Capítulo 4: Diagramas de clase: fundamentos 61
Perspectivas 63
Asociaciones 65
Atributos 72
Operaciones 73
Tarjetas CRC 74
Cuándo usar las tarjetas CRC 75
Para mayor información 76
Generalización 77
Reglas de restricción 79
Diseño por contrato 80
Cuándo utilizar el Diseño por contrato 82
Para mayor información 83
Cuándo emplear los diagramas de clase 83
Para mayor información 84
Capítulo 5: Diagramas de clase: conceptos avanzados 85
Los estereotipos 86

Clasificación múltiple y dinámica 87
Agregación y composición 90
Asociaciones y atributos derivados 93
Interfaces y clases abstractas 95
Objetos de referencia y objetos de valor 98
Colecciones para relaciones de valor múltiple 100
Congelado 101
Clasificación y generalización 102
Asociaciones calificadas 103
Clase de asociación 104
Clase con parámetro 108
La visibilidad 110
Características del alcance de clase 113

Capítulo 6: Diagramas de interacción 115

Diagramas de secuencia 116
Procesos concurrentes y activaciones 118
Diagramas de colaboración 121
Comparación de los diagramas de secuencia y de colaboración 123
El comportamiento condicional 124
Cuándo utilizar los diagramas de interacción 124
Para mayor información 125

Capítulo 7: Diagramas de paquetes 127

Cuándo utilizar los diagramas de paquetes 135
Para mayor información 135

Capítulo 8: Diagramas de estados 137

Diagramas de estados concurrentes 142
Cuándo utilizar los diagramas de estados 144
Para mayor información 145

Capítulo 9: Diagramas de actividades 147

Diagramas de actividades para casos de uso 150
Carriles 156

Descomposición de una actividad 158
Cuándo utilizar diagramas de actividades 159
Para mayor información 160

Capítulo 10: Diagramas de emplazamiento 161

Cuándo utilizar diagramas de emplazamiento 163

Capítulo 11: El UML y la programación 165

Observación del paciente: modelo de dominio 166
Observación del paciente: modelo de especificación 170
Generación del código 173

Apéndice A: Técnicas y sus usos 185

Apéndice B: Cambios del UML 1.0 al 1.1 187

Bibliografía 193

Índice 197

Figuras

Figura 1-1: Extracto del metamodelo del UML 1.1 6

Figura 2-1: Proceso de desarrollo del bosquejo. 16
Figura 2-2: Patrón de diseño de un suplente 43
Figura 2-3: Patrón de análisis de escenarios 44

Figura 3-1: Diagrama de casos de uso 52

Figura 4-1: Diagrama de clase 62
Figura 4-2: Notaciones de cardinalidad 68
Figura 4-3: Diagrama de clase con navegabilidades 70
Figura 4-4: Tarjeta de Clase-Responsabilidad-Colaboración (CRC) 74

Figura 5-1: Clasificación múltiple 88
Figura 5-2: Clasificación dinámica 90
Figura 5-3: Agregación y composición 92
Figura 5-4: Notación alterna para la composición 92
Figura 5-5: Asociaciones y atributos derivados 93
Figura 5-6: Clase de periodo de tiempo 94
Figura 5-7: Ventana como clase abstracta 96
Figura 5-8: Interfaces y clase abstracta: un ejemplo de Java 97
Figura 5-9: Notación de paletas para representar interfaces 98
Figura 5-10: Asociación calificada 103
Figura 5-11: Clase de asociación 104
Figura 5-12: Promoción de una clase de asociación a una clase completa 105
Figura 5-13: Sutilezas de la clase de asociación 106
Figura 5-14: Estereotipo de historia para las asociaciones 107

Figura 5-15: Clase con parámetro 108
Figura 5-16: Elemento enlazado (versión 1) 109
Figura 5-17: Elemento enlazado (versión 2) 109

Figura 6-1: Diagrama de secuencia 116
Figura 6-2: Procesos y activaciones concurrentes 119
Figura 6-3: Diagrama de secuencia: revisión de fallas 121
Figura 6-4: Diagrama de colaboración con numeración simple 122
Figura 6-5: Diagrama de colaboración con numeración decimal 123

Figura 7-1: Diagrama de paquetes 130
Figura 7-2: Diagrama de paquetes avanzado 132

Figura 8-1: Diagrama de estados 138
Figura 8-2: Diagrama de estados sin superestados 140
Figura 8-3: Diagrama de estados con superestados 141
Figura 8-4: Autorización de pagos 142
Figura 8-5: Diagrama de estados concurrentes 143

Figura 9-1: Diagrama de actividades 148
Figura 9-2: Recepción de un pedido 152
Figura 9-3: Recepción de abastecimiento 154
Figura 9-4: Recibe orden y recibe existencias 155
Figura 9-5: Carriles 157
Figura 9-6: Diagrama descompuesto de actividades 158

Figura 10-1: Diagrama de emplazamiento 162

Figura 11-1: Modelo de dominio de observación de pacientes 167
Figura 11-2: Diagrama de objeto de observación del paciente 168
Figura 11-3: Otro diagrama del objeto de observación del paciente 169
Figura 11-4: Modelo de especificación de observación del paciente 171

Figura 11-5: Operaciones de observación de pacientes 172
Figura 11-6: Diagrama de secuencia de observación del paciente 174
Figura 11-7: Otro modelo de especificación de observación del paciente 182

Prólogo

Cuando comenzamos a elaborar el Lenguaje unificado de modelado (UML), esperábamos poder producir un medio estándar para expresar el diseño, que no sólo reflejara las mejores prácticas de la industria, sino que también le restara oscuridad al proceso de modelado de software. Creemos que la disponibilidad de un lenguaje de modelado estándar alentará a más desarrolladores para que modelen sus sistemas de software antes de construirlos. Los beneficios de hacerlo son perfectamente conocidos por la comunidad de desarrolladores.

La creación del UML fue en sí mismo un proceso iterativo y gradual muy similar al modelado de un gran sistema de software. El resultado final es una norma construida sobre las muchas ideas y contribuciones realizadas por numerosos individuos y compañías de la comunidad de la orientación a objetos, y un reflejo de todo esto. Nosotros iniciamos el esfuerzo del UML, pero muchos otros ayudaron a llevarlo a una exitosa conclusión; estamos agradecidos a todos por su apoyo.

El llegar a crear y a acordar un lenguaje estándar de modelado es un reto significativo por sí mismo. Educar a la comunidad de desarrollo y presentar el UML de forma que sea accesible y que esté dentro del contexto del proceso de desarrollo de software también es un reto importante. En este libro, engañosamente breve, Martin Fowler ha superado este reto.

En un estilo claro y ameno, Martin no sólo presenta los aspectos clave del UML, sino que demuestra claramente el papel que desempeña el UML en el proceso de desarrollo. De paso, nos obsequia abundantes perlas, ricas en introspección y sabiduría sobre el modelado, obtenidas de sus más de 10 años de experiencia en el diseño y modelado.

El resultado es un libro que recomendamos a los modeladores y desarrolladores interesados en darle un primer vistazo al UML y en obtener una perspectiva sobre el papel clave que desempeña en el proceso de desarrollo.

Grady Booch
Ivar Jacobson
James Rumbaugh

Prefacio

Nunca esperé escribir un libro sobre métodos.

Me propusieron escribir uno a fines de 1992; sin embargo, en ese momento todos los libros realmente influyentes sobre métodos ya habían sido publicados y no pensé tener algo significativo que añadir a esa información. En lo que a mí se refería, el tema había sido cubierto y había cosas más importantes que hacer. Había decidido no crear una nueva metodología "Fowler" y ya había demasiadas metodologías.

Me llené de alegría cuando Grady Booch, Jim Rumbaugh e Ivar Jacobson ("los tres amigos") unieron sus fuerzas para formar un solo **lenguaje unificado de modelado** (*UML, Unified Modeling Language*). Las discusiones sobre el método a escoger han sido algunas de las más cansadas que he tenido, particularmente debido a que tienen poco impacto sobre el resultado final. Me agradó ver que estas controversias habían quedado en el pasado.

Cuando se me planteó escribir este libro, los "amigos" comenzaban a escribir los suyos. Estos libros van a ser las obras de mayor autoridad sobre el UML. Sin embargo, existe la necesidad de contar con un libro breve que suministre algo acerca de este tema, en tanto los tres trabajan sobre sus obras mayores, y que sea también una guía concisa del UML. Tengo la intención de hacer de este volumen el libro de métodos más corto que jamás se haya escrito.

A pesar de que ésta es una meta noble para mí, ¿será acaso el libro adecuado para el lector?

Comenzaré con una explicación sobre lo que *no* es este libro.

- No es un tutorial sobre análisis y diseño orientados a objetos con UML. La guía del usuario, cuya elaboración estará a cargo de Grady Booch, será ese libro.

- No es una guía de referencia definitiva de la notación y su semántica. La guía de referencia, dirigida por Jim Rumbaugh, será dicho libro.
- No es, tampoco, una guía detallada del proceso de utilización de UML en proyectos orientados a objetos. La guía del proceso, conducida por Ivar Jacobson, será tal obra.

Este libro es una guía abreviada de las *partes clave* de la notación, la semántica, y el proceso. Lo estoy dirigiendo a aquellos que ya han usado la tecnología de objetos, probablemente con alguno de los métodos de análisis y diseño OO que actualmente hay disponibles. Este libro le va a decir rápidamente cuáles son los elementos clave de la notación y cuál es su significado, y sugiere un proceso general para usar estos elementos. He aprovechado también la oportunidad de añadir recomendaciones y sugerencias derivados del uso que he hecho de los métodos orientados a objetos durante la última década.

Debido a que es un libro breve, va a ser más fácil digerir la información y acostumbrarse a lo que tiene qué decir el UML. Va a proporcionar también un buen punto de inicio para la búsqueda de información de referencia.

El capítulo 1 examina qué es el UML, la historia de su desarrollo, y las razones por las cuales pudiera usted querer utilizarlo.

El capítulo 2 explica el proceso de desarrollo orientado a objetos. A pesar de que el UML existe independientemente del proceso, encuentro que es difícil explicar las técnicas de modelado sin hablar acerca de dónde encajan en el proceso de desarrollo orientado a objetos.

Los capítulos 3 al 10 analizan, una por una, las diversas técnicas de modelado del UML. He organizado estos capítulos alrededor de los tipos de diagramas que encuentro útiles. Describo la notación, incluyendo su semántica, y ofrezco recomendaciones acerca del uso de las técnicas. Mi filosofía es dejar en claro lo que dice el UML y, al mismo tiempo, ofrecerle mi opinión sobre cómo sacarle el mayor provecho.

El capítulo 11 proporciona un pequeño ejemplo que muestra cómo encaja el UML dentro de la programación con (por supuesto) Java.

El interior de los forros contiene un resumen de la notación de UML. Puede encontrar de utilidad consultarlo conforme usted lea los capítulos, de tal forma que pueda ir revisando la notación para los diversos conceptos de modelado.

Desperdigados en los capítulos "oficiales" de UML se encuentran una serie de recuadros sobre otras técnicas que he encontrado valiosas, pero que no se enfatizan en el UML. Ciertamente, pueden y deben usarse en conjunto con el UML.

Para cada técnica de UML y de no UML, he proporcionado resúmenes acerca de cuándo usar la técnica y dónde encontrar más información. Al momento de escribir esto, aún no han aparecido libros sobre UML en el mercado, así que sólo uso como referencia libros anteriores a la aparición de UML. A pesar de que la notación es diferente, muchos de los conceptos son los mismos, y va a pasar mucho tiempo antes de que estos libros sean relegados al olvido.

Por supuesto, éste, como cualquier otro libro escrito dentro de nuestra industria, será obsoleto tan pronto como quede terminado. Para combatir esto, estoy aprovechando el inevitable uso de la Web. Para obtener mis últimas reflexiones sobre los métodos, eche un vistazo al sitio de este libro en la Web: <**www.awl.com/cseng/titles/0-201-32563-2.html**>.

Reconocimientos

Publicar el presente volumen a la velocidad que se hizo requirió de mucha ayuda de parte de personas que fueron más allá del esfuerzo normal que se invierte en estos casos y para poder hacer todo de manera mucho más rápida.

Kendall Scott tuvo un papel importante conjuntando el material y trabajando sobre el texto y las gráficas.

Los tres amigos, Grady Booch, Ivar Jacobson y Jim Rumbaugh, han proporcionado abundante apoyo y consejos. Hemos gastado muchas horas en llamadas transcontinentales, y han mejorado en mucho la obra (así como mi comprensión del UML).

Es esencial una buena plantilla de revisores, si se quiere hacer un buen trabajo sobre un libro. No sólo estos revisores me dieron la retroalimentación que necesitaba, sino que regresaron sus comentarios en menos de una semana, a fin de cumplir los apretados plazos de entrega. Mi agradecimiento a: Simmi Kochhar Bhargava, de Netscape Communications Corporation; Eric Evans; Tom Hadfield, de Evolve Software, Inc.; Ronald E. Jeffries; Joshua Kerievsky, de Industrial Logic, Inc.; Helen Klein, de la Universidad de Michigan; James Odell, y Vivek Salgar, de Netscape Communications Corporation. ¡Un doble agradecimiento a Tom Hadfield, porque hizo trabajo doble!

Quiero agradecer a Jim Odell dos cosas: primero, la coordinación del esfuerzo del Object Management Group (OMG) para crear un solo UML estándar, lo cual va a ser un gran paso adelante para nuestra industria; y segundo, su aliento para adentrarme en el campo del análisis y diseño orientado a objetos. ¡Ah, y gracias también por revisar el libro!

Gracias a Cindy, por soportarme cuando estuve ausente, aun estando en casa.

No puedo siquiera imaginar las dificultades por las que atravesaron mi editor, J. Carter Shanklin, y su asistente, Angela Buenning, para poder publicar este libro tan rápidamente como lo hicieron. No importa cuáles hayan sido estas dificultades, estoy seguro de que Carter y Angela merecen mi agradecimiento.

Por último, pero no menos importante, doy mi agradecimiento a mis padres por ayudarme a comenzar con una buena educación, a partir de la cual surge todo lo demás.

Martin Fowler
Melrose, Massachusetts
Mayo de 1997
martin_fowler@compuserve.com

Capítulo 1

Introducción

¿Qué es el UML?

El **lenguaje unificado de modelado** o **UML** (*Unified Modeling Language*) es el sucesor de la oleada de métodos de análisis y diseño orientados a objetos (OOA&D) que surgió a finales de la década de 1980 y principios de la siguiente. El UML unifica, sobre todo, los métodos de Booch, Rumbaugh (OMT) y Jacobson, pero su alcance llegará a ser mucho más amplio. En estos momentos el UML está en pleno proceso de estandarización con el OMG (*Object Management Group* o Grupo de administración de objetos) y estoy seguro de que se convertirá en el lenguaje de modelado estándar del futuro.

Decimos, pues, que el UML es un lenguaje de modelado, y no un método. La mayor parte de los métodos consisten, al menos en principio, en un lenguaje y en un proceso para modelar. El **lenguaje de modelado** es la notación (principalmente gráfica) de que se valen los métodos para expresar los diseños. El **proceso** es la orientación que nos dan sobre los pasos a seguir para hacer el diseño.

Las partes que tratan sobre el proceso en muchos libros de métodos son más bien esquemáticas. Más aún, considero que la mayoría de las personas que dicen estar usando un método están usando en realidad un lenguaje de modelado, pero rara vez siguen el proceso. Así pues, en gran medida el lenguaje de modelado es la parte más importante

del método. Ciertamente, es la clave para la comunicación. Si usted desea analizar su diseño con alguien, lo que ambos necesitan comprender es el lenguaje de modelado, *no* el proceso que usted siguió para lograr tal diseño.

Los tres amigos que acabamos de mencionar también trabajan en la creación de un proceso unificado, llamado **Objectory**. No es necesario utilizar Objectory para usar UML, pues ambos están claramente separados. En este libro, sin embargo, hablaré un poco del proceso, con el fin de situar en contexto las técnicas del lenguaje de modelado. En la exposición hago uso de los pasos y términos básicos de Objectory, pero cabe aclarar que la obra *no* es una descripción del proceso de Objectory. Sucede que utilizo muchos procesos diferentes, dependiendo de mi cliente y del tipo de software que esté construyendo. Aunque pienso que es valioso tener un lenguaje de modelado estándar, no considero que haya una necesidad comparable de contar con un proceso estándar, si bien sería útil cierta armonización en el vocabulario.

Cómo llegamos hasta aquí

En la década de 1980, los objetos comenzaron a alejarse de los laboratorios de investigación y dieron sus primeros pasos hacia el mundo "real". Smalltalk se estabilizó, quedando en una plataforma que la gente podía usar, y nació C++.

Como muchos desarrollos en el software, los objetos estuvieron guiados por los lenguajes de programación. Muchos se preguntaban cómo se adecuarían los métodos de diseño a un mundo orientado a objetos. Los métodos de diseño se habían vuelto muy populares en el desarrollo industrial durante las décadas de 1970 y 1980. Muchos pensaban que las técnicas para ayudar al buen análisis y diseño eran también importantes en el desarrollo orientado a objetos.

Los libros clave sobre el análisis orientado a objetos y los métodos de diseño aparecieron entre 1988 y 1992:

- Sally Shlaer y Steve Mellor escribieron un par de libros (1989 y 1991) sobre análisis y diseño; la evolución del material de estos libros ha dado como resultado su enfoque de diseño recursivo (1997).

- Peter Coad y Ed Yourdon también escribieron libros en los que desarrollaron el enfoque hacia los métodos ligeros orientados a prototipos de Coad. Véase Coad y Yourdon (1991a y 1991b), Coad y Nicola (1993) y Coad *et al.* (1995).
- La comunidad Smalltalk de Portland, Oregon, aportó el diseño guiado por la responsabilidad (Responsibility-Driven Design) (Wirfs-Brock *et al.* 1990) y las tarjetas de clase-responsabilidad-colaboración (Class-Responsibility-Collaboration) (CRC) (Beck y Cunningham 1989).
- Grady Booch había trabajado mucho con Rational Software, desarrollando sistemas en Ada. En sus libros se daban varios ejemplos (y las mejores caricaturas del mundo en cuanto a libros de métodos). Véase Booch (1994 y 1995).
- Jim Rumbaugh dirigió un equipo en los laboratorios de investigación de General Electric, cuyo resultado fue un popular libro sobre un método llamado técnica de modelado de objetos (Object Modeling Technique) (OMT). Véase Rumbaugh *et al.* (1991) y Rumbaugh (1996).
- Los libros de Jim Odell, escritos junto con James Martin, se basan en su amplia experiencia en los sistemas de información de negocios y de ingeniería de información. El resultado fue el libro más conceptual de todos. Véase Martin y Odell (1994 y 1996).
- Ivar Jacobson escribió con base en la experiencia adquirida en conmutadores telefónicos para Ericsson e introdujo en el primero de sus libros el concepto de casos de uso (use cases). Véase Jacobson (1994 y 1995).

Cuando me preparaba para viajar a Portland y asistir a la OOPSLA '94, el panorama relativo a los métodos se veía muy dividido y competido. Cada uno de los autores antes mencionados dirigía informalmente un grupo de profesionales que estaban de acuerdo con sus ideas. Todos estos métodos eran muy similares; sin embargo, tenían entre sí gran cantidad de diferencias menores, con frecuencia incómodas. Los mismos conceptos básicos aparecían con denominaciones muy diferentes, lo cual confundía a mis clientes.

En ese entonces, ya se hablaba de estandarización, pero nadie parecía dispuesto a hacer algo al respecto. Algunos se oponían por completo a la idea misma de estándares para los métodos. A otros les atraía la idea pero no tenían la menor intención de hacer ningún esfuerzo. Un equipo del OMG que dio muestras de querer considerar la estandarización sólo logró una carta abierta de protesta de parte de los metodólogos más importantes. Grady Booch intentó organizar una reunión informal para abordar el problema, pero tampoco tuvo éxito. (Esto me recuerda un conocido chiste: ¿Cuál es la diferencia entre un metodólogo y un terrorista? Respuesta: con el terrorista se puede negociar.)

Para la comunidad de los métodos orientados a objetos, la gran noticia en la OOPSLA '94 fue que Jim Rumbaugh había dejado General Electric para unirse a Grady Booch en Rational Software, con la intención de unificar sus métodos.

El año siguiente estuvo lleno de acontecimientos amenos.

Grady y Jim proclamaron que "la guerra de los métodos ha terminado: la ganamos nosotros", declarando, en esencia, que iban a lograr la estandarización a la manera de Microsoft. Otro grupo de metodólogos sugirió la idea de formar una coalición en contra de Booch.

Para la OOPSLA '95, Grady y Jim habían preparado la primera descripción pública de su método integrado: la versión 0.8 de la documentación del *Método unificado* (*Unified Method*). De mayor importancia todavía, anunciaron que Rational Software había comprado Objectory y que Ivar Jacobson se uniría al equipo unificado. Con una concurrida fiesta, Rational celebró el lanzamiento del documento preliminar de 0.8; esta fiesta, además, fue muy divertida, a pesar de las canciones de Jim Rumbaugh.

Durante 1996, Grady, Jim e Ivar, ahora ampliamente conocidos como los tres amigos, construyeron su método y le pusieron otro nombre: Unified Modeling Language (UML), lenguaje unificado de modelado. Sin embargo, los demás actores importantes de la comunidad de métodos orientados a objetos no estaban dispuestos a permitir que el UML fuera la última palabra.

Se creó una fuerza de trabajo en el OMG para llevar a cabo la estandarización en el área de los métodos. Ello representaba un intento mucho

más serio que los anteriores por resolver los problemas en esa área de los métodos. Como directora del proyecto fue designada Mary Loomis; más tarde, Jim Odell se unió como codirector y asumió el liderazgo del proyecto. Odell dejó muy claro que estaba dispuesto a renunciar a su método por un estándar, pero no quería que Rational impusiera el suyo.

En enero de 1997, varias organizaciones entregaron sus propuestas de estandarización de métodos, con el fin de simplificar el intercambio de modelos. Estas propuestas se enfocan en un metamodelo y en una notación opcional. Como su propuesta al OMG, la Rational liberó la versión 1.0 de la documentación del UML.

Mientras escribo estas líneas, Jim Odell y el grupo OMG han dedicado mucho tiempo a la elaboración de la semántica del UML y a la armonización de las diversas propuestas. Ahora tenemos una propuesta única del UML 1.1, que cuenta con amplio apoyo de la industria.

Notaciones y metamodelos

En su condición actual, el UML define una notación y un metamodelo.

La **notación** es el material gráfico que se ve en los modelos; es la sintaxis del lenguaje de modelado. Por ejemplo, la denominación de un diagrama de clases define cómo se representan conceptos y temas como clase, asociación y multiplicidad.

Por supuesto, esto nos lleva a la pregunta de qué significan exactamente asociación, multiplicidad e, incluso, clase. Por el uso común se infieren algunas definiciones informales, pero es mucha la gente que exige definiciones más rigurosas.

En el campo de los métodos formales prevalece la idea de contar con lenguajes de especificación y diseño rigurosos. En tales técnicas, los diseños y las especificaciones se representan usando alguna derivación del cálculo de predicados. Tales definiciones son matemáticamente rigurosas y no permiten la ambigüedad. Sin embargo, el valor de dichas definiciones no es de ninguna manera universal. Incluso, cuando se puede probar que un programa satisface una especificación

matemática, no hay manera de probar que esa especificación matemática se adecue en la práctica a los requisitos reales del sistema.

Lo fundamental en el diseño es ver los temas clave para el desarrollo. Los métodos formales se pierden con frecuencia en infinidad de detalles menores. Igualmente, los métodos formales son difíciles de comprender y manejar, a veces, incluso, más que los lenguajes de programación por si fuera poco, ni siquiera son ejecutables.

La mayoría de los métodos orientados a objetos (métodos OO) tienen escaso rigor; su notación es más intuitiva que formal. En general, esto no parece haber causado muchos daños. Estos métodos pueden ser informales, pero mucha gente sigue encontrándolos útiles y es su utilidad la que cuenta.

Sin embargo, los que trabajan con los métodos OO buscan cómo hacerlos más rigurosos sin sacrificar su utilidad. Un modo de lograrlo es mediante la definición de un **metamodelo**: un diagrama, usualmente un diagrama de clases, que defina la notación.

La figura 1-1 es una pequeña parte del metamodelo 1.1 del UML que muestra la relación entre asociaciones y generalizaciones. (Incluyo este extracto sólo para dar una idea de cómo son los metamodelos, pues ni siquiera voy a tratar de explicarlo.)

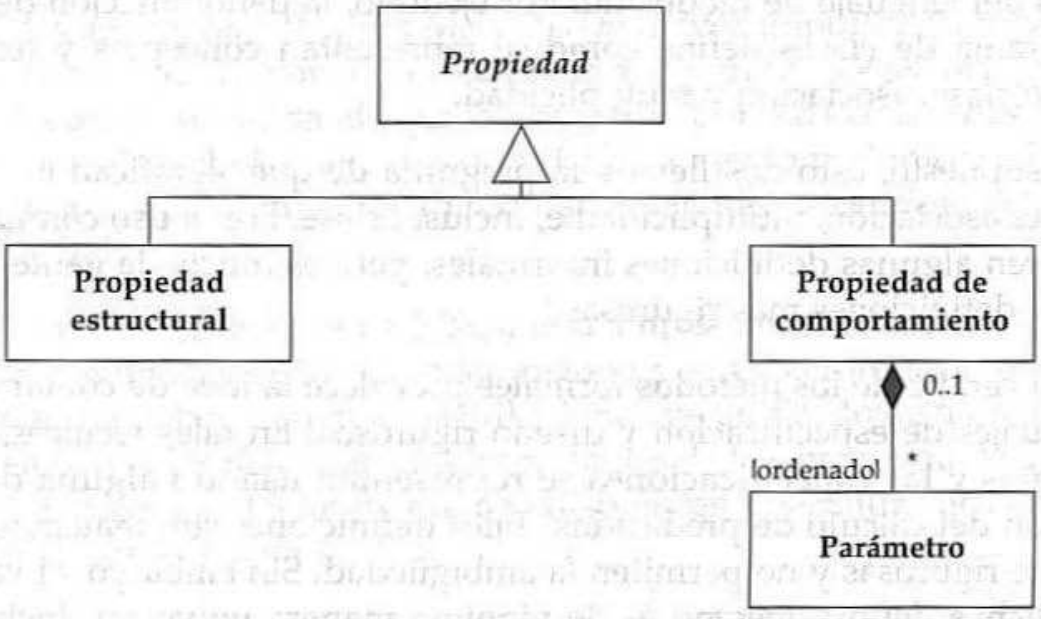


Figura 1-1: *Extracto del metamodelo del UML 1.1*

¿Cuánto afecta el metamodelo al usuario de la notación para modelar? Pues bien, ciertamente, ayuda a definir qué es un modelo bien formado; es decir, uno que sintácticamente está correcto. Como tal, el usuario dedicado a los métodos deberá entender el metamodelo. Sin embargo, la mayoría de los usuarios de los métodos no necesita un conocimiento tan profundo para obtener algunos beneficios de la notación del UML.

He aquí el porqué pude escribir un libro útil antes de que el metamodelo del UML estuviese totalmente definido. Los cambios en el metamodelo entre la versión 1.0 y la 1.1 no provocaron ningún cambio de importancia en el contenido de la obra. Por otra parte, en este libro no seré riguroso, sino que seguiré la ruta tradicional de los métodos y apelaré a la intuición del lector.

¿Qué tan estrictamente debe usted acatar el lenguaje de modelado? Depende del propósito para el que se usa. Si tiene una herramienta CASE que genera código, entonces, para lograr un código aceptable usted deberá apegarse a la interpretación que hace la herramienta CASE del lenguaje de modelado. Por otra parte, si se vale de los diagramas con fines de comunicación, tendrá un poco más de libertad.

Si usted se desvía de la notación oficial, los demás usuarios no comprenderán del todo lo que está diciendo. Sin embargo, hay momentos en que la notación oficial le puede estorbar. Admito que en estos casos no dudo ni un momento en adecuar el lenguaje a mis necesidades. Creo que el lenguaje debe ser flexible para ayudar a comunicarme, y no al contrario. Pero esto no sucede con frecuencia y siempre estoy consciente de que esta desviación es algo malo, si provoca problemas de comunicación. En este libro mencionaré los puntos en los cuales estoy dispuesto a flexibilizar un poco el lenguaje.

¿Por qué analizar y diseñar?

En resumidas cuentas, la cuestión fundamental del desarrollo del software es la escritura del código. Después de todo, los diagramas son sólo imágenes bonitas. Ningún usuario va a agradecer la belleza de los dibujos; lo que el usuario quiere es software que funcione.

Por lo tanto, cuando esté considerando usar el UML, es importante preguntarse por qué lo hará y cómo le ayudará a usted cuando llegue el momento de escribir el código. No existe una evidencia empírica adecuada que demuestre si estas técnicas son buenas o malas, pero en las siguientes subsecciones analizaré las razones que con frecuencia encuentro para justificar su uso.

Aprendizaje de OO

Mucho se habla de la curva de aprendizaje asociada a la OO, el infame cambio de paradigmas. De algún modo, cambiar a OO es fácil. En otros sentidos, hay cierto número de obstáculos cuando se trabaja con objetos, particularmente si se quieren usar en la forma más ventajosa.

No es que sea difícil aprender a programar en un lenguaje OO. El problema es que es necesario cierto tiempo para aprender a aprovechar las ventajas que contiene el lenguaje orientado a objetos. Como lo expresa muy bien Tom Hadfield: los lenguajes de objetos *permiten* ventajas pero no las *proporcionan*. Para aprovechar estas ventajas hay que realizar el infame cambio de paradigma. (¡Sólo asegúrese de estar sentado al momento de hacerlo!)

Las técnicas en el UML fueron diseñadas en cierta medida para ayudar a los usuarios a hacer un buen desarrollo de OO, pero cada técnica tiene distintas ventajas a las de las demás.

- Una de las técnicas más valiosas para aprender OO es la de las tarjetas CRC (véase la página 74), que no son parte del UML oficial (aunque pueden y deben ser usadas con él). Originalmente, las tarjetas CRC fueron diseñadas para enseñar a trabajar con objetos. Como tales, deliberadamente son diferentes de las técnicas de diseño tradicionales. Su énfasis en las responsabilidades y la ausencia de notación compleja las hace particularmente valiosas.

- Los diagramas de interacción (véase el capítulo 6) son muy útiles, pues hacen muy explícita la estructura de los mensajes y, en consecuencia, tienen la ventaja de resaltar los diseños demasiado centralizados o sobrecentralizados, en los que un objeto realiza todo el trabajo.

- Los diagramas de clases (véanse los capítulos 4 y 5), usados para ilustrar modelos de clases, son tanto buenos como malos para el aprendizaje de objetos. Los modelos de clases son muy similares a los modelos de datos, por lo que resultan cómodos; muchos de los principios que hacen que un modelo de datos sea bueno también hacen que un modelo de clases sea bueno. El mayor problema en el uso de los diagramas de clases es que es más fácil desarrollar un modelo de clases que esté orientado a datos que desarrollar uno orientado a responsabilidades.
- El concepto de patrones (véase la página 42) es vital para el aprendizaje de la OO, pues el empleo de patrones le hace centrarse en lograr buenos diseños de OO y aprender con base en ejemplos. Una vez logrado el dominio de algunas técnicas para modelar, tales como los diagramas de clases sencillos y los diagramas de interacción, ha llegado el momento de comenzar a ver los patrones.
- Otra técnica importante es el desarrollo iterativo (véase el capítulo 2). Esta técnica no le ayuda a aprender OO de manera directa, pero es la clave para explotar de manera eficaz la OO. Si desde el principio el desarrollo se hace de manera iterativa, entonces aprenderá, en contexto, cuál es el tipo de proceso adecuado y comenzará a ver por qué los diseñadores sugieren hacer las cosas de la manera en que lo hacen.

Cuando se empieza a usar una técnica, se tiende a hacerlo siguiendo el libro al pie de la letra. Mi recomendación es que vaya usted iniciándose con las sencillas notaciones sobre las que he hablado aquí, en particular con los diagramas de clases. En la medida en que se vaya sintiendo seguro, podrá seleccionar las ideas más avanzadas a medida que sean necesarias. Quizá descubra también que desea ampliar el método.

El UML tiene un mecanismo de extensión que utiliza estereotipos. Hablo de estereotipos sólo en el contexto de los diagramas de clases, pero usted puede emplear estereotipos con cualquier diagrama, ampliando su significado. Los libros de los tres amigos entran en más detalles al respecto. Sólo asegúrese de comprender realmente el significado de la construcción. Con este fin, busco contemplar cualquier construcción desde tres perspectivas: conceptual, especificación y realización (véase el capítulo 4).

Comunicación con los expertos del dominio

Uno de nuestros mayores desafíos en el desarrollo es el de construir el sistema *adecuado*, el que resuelva las necesidades de los usuarios a un costo razonable. Esto se hace aún más difícil porque nosotros, con nuestra jerga, tenemos que comunicarnos con usuarios que también tienen su propia jerga, incluso más críptica. (He trabajado mucho en el sector salud y la jerga que allí se usa sólo la entienden ellos.) Lograr una buena comunicación, junto con una comprensión adecuada del mundo del usuario, es la clave para el desarrollo de buen software.

La técnica obvia que debe emplearse en esta situación es la de los casos de uso (véase el capítulo 3). Un **caso de uso** es una toma instantánea de algún aspecto de su sistema. La suma de todos los casos de uso constituye la vista externa del sistema, que es un gran avance hacia la explicación de lo que hará el sistema.

Un buen conjunto de casos de uso es clave para comprender lo que quieren sus usuarios. Los casos de uso también ofrecen un buen vehículo para la planificación de proyectos, ya que controlan el desarrollo iterativo, que es en sí mismo una técnica valiosa, puesto que retroalimenta de manera regular a los usuarios sobre el curso que lleva el software.

Además de que los casos de uso sirven para la comunicación de los elementos superficiales, también resulta crucial para observar las cuestiones más profundas. Esto implica saber cómo entienden su mundo los expertos del dominio.

Los diagramas de clases (véanse los capítulos 4 y 5) pueden ser muy valiosos aquí, en la medida en que se usen de modo conceptual. En otras palabras, usted debe tratar cada clase como si fuese un concepto en la mente del usuario, como parte de su lenguaje. Los diagramas de clases que usted diseña no son, por tanto, diagramas de datos o de clases, sino diagramas del lenguaje de sus usuarios. El libro sobre los "fundamentos" de James Martin y Jim Odell (1994) es una buena fuente para este tipo de ideas vinculadas a los diagramas de clases.

Me he percatado de que los diagramas de actividades (véase el capítulo 9) son muy útiles en aquellos casos en los que los procesos de

flujo de trabajo (workflow processes) son una parte importante del mundo de los usuarios. Dado que los diagramas de actividades manejan procesos paralelos, pueden ayudarle a usted a deshacerse de secuencias innecesarias. La forma en que estos diagramas dejan de poner énfasis en los vínculos con las clases, las cuales pueden ser un problema en el diseño posterior, es una ventaja durante esta etapa más conceptual del proceso de desarrollo.

Comprensión del panorama general

Como consultor, con frecuencia debo zambullirme en un proyecto complejo y dar la impresión de ser inteligente en un plazo breve. Para ello, considero que las técnicas de diseño explicadas antes poseen un valor incalculable, pues me ayudan a adquirir una visión completa del sistema. Una ojeada a un diagrama de clases me puede decir rápidamente qué tipos de abstracciones se hallan presentes en el sistema y en dónde se encuentran las partes cuestionables que necesitan más trabajo. A medida que profundizo más, quiero saber cómo colaboran las clases, de modo que solicito ver diagramas de interacción que ilustren comportamientos clave en el sistema.

Si, como observador externo, esto me es útil, lo será igualmente para el equipo encargado del proyecto. Es fácil perder de vista el bosque al caminar entre los árboles, cuando se trata de un proyecto grande. Teniendo a la mano unos cuantos diagramas seleccionados, es más fácil solucionar la cuestión del software.

Para construir un mapa del camino, utilice diagramas de paquetes (véase el capítulo 7) en los niveles más altos; con ellos se tendrá el alcance de los diagramas de clases. Cuando se dibuja un diagrama de clases destinado a un mapa del camino, hay que centrarse en las especificaciones. Es muy importante ocultar las implementaciones en este tipo de trabajo. No documente interacción por interacción; en cambio, céntrese en las que son clave.

Utilice patrones (véase la página 42) para describir las ideas clave en el sistema; le ayudarán a explicarle por qué el diseño es como es. También es útil describir los diseños que usted haya rechazado y por qué los ha rechazado. Yo siempre termino olvidando este tipo de decisiones.

Para mayor información

Este libro no es una referencia completa ni definitiva sobre el UML, por no hablar de análisis y diseño orientado a objetos (OO). Se ha dicho mucho y hay mucho, todavía, por leer. Conforme vaya explicando los temas particulares, me referiré a otros libros que debe usted consultar para lograr mayor información con respecto a las ideas del UML y del OOA&D, en general.

Por supuesto, el primer paso más allá de este libro debe ser con los libros sobre el UML de los tres amigos. Mientras escribo estas líneas, cada uno de ellos prepara su libro.

Grady Booch encabeza el trabajo sobre la guía del usuario. Será un libro tutorial que contendrá una serie de estudios de caso minuciosos sobre la manera de aplicar el UML a problemas prácticos. Será más detallado que éste que tiene usted en sus manos y contendrá más consejos sobre la manera de emplear bien el UML.

Jim Rumbaugh está dirigiendo la redacción del libro de referencia, la guía definitiva de la notación y el metamodelo del UML. Será la fuente autorizada de información sobre el significado del lenguaje UML.

Ivar Jacobson está trabajando en un libro que describirá el proceso de utilización del UML. El UML es, estrictamente hablando, un lenguaje de modelado y no contiene nada sobre el proceso de desarrollo del software. Por ello, los tres amigos utilizan el término "lenguaje de modelado" y no la palabra "método", ya que, de hecho, un método debe incluir un proceso. En el libro he bosquejado un proceso ligero para darle cierto contexto a las técnicas y a las notaciones. El libro de Jacobson entrará en mayores detalles.

Ciertamente, los libros de los tres amigos no son los únicos que debería usted leer para enterarse de lo referente al análisis y diseño orientados a objetos (OOA&D). Mi lista de libros recomendables suele cambiar con frecuencia; le recomiendo consultar la versión más actualizada en la página Survey of Analysis and Design Methods de mi sitio en Web, cuya dirección es **<ourworld.compuserve.com/homepages/Martin_Fowler>** (mi página base de Web).

De manera especial, sugiero leer libros sobre patrones, donde usted encontrará temas que lo llevarán más allá de los conceptos básicos. Ahora que la guerra de los métodos ha terminado, considero que será en los patrones donde se hallará el material más interesante sobre análisis y diseño. Sin embargo, es inevitable que surjan nuevas técnicas de análisis y diseño, y es muy probable que quienes las propongan sugieran la manera de emplearlas con el UML. Ésta es otra de las ventajas del UML: promueve el desarrollo de nuevas técnicas sin duplicar ya el trabajo efectuado.

Capítulo 2

Un bosquejo del proceso de desarrollo

Decíamos que el UML es un lenguaje para modelar, no un método. El UML no asume la noción de lo que es un proceso, el cual constituye una parte importante de un método.

Los tres amigos están trabajando para fusionar sus procesos, y el resultado se llamará *Rational Objectory Process*. No creo que sea posible contar con un solo proceso para el desarrollo de software. Por otra parte, distintos factores relacionados con el desarrollo de software conducen a diferentes tipos de procesos. Entre estos factores se incluye el tipo de software que se está desarrollando (tiempo real, sistema de información, producto para computadora de escritorio), la escala (un solo desarrollador, un pequeño equipo, un equipo de más de cien miembros) y así sucesivamente. Por lo tanto, los amigos intentan lograr una estructura de procesos, algo que atrape los elementos comunes pero que al mismo tiempo permita la flexibilidad de emplear técnicas apropiadas para su proyecto.

El título que he dado a este libro es el de *UML destilado*, en virtud de lo cual yo muy bien habría podido ignorar los procesos. Pero no creo que las técnicas para modelar tengan sentido sin que se sepa cómo se adecuan dentro de un proceso.

Considero que es importante analizar primero el proceso, para poder ver cómo funciona un desarrollo orientado a objetos. No entraré en muchos detalles del proceso; sólo ofreceré lo necesario para que usted tenga una idea del modo típico como se lleva a cabo un proyecto que utiliza estas técnicas.

Conforme vaya exponiendo el bosquejo del proceso, utilizaré la terminología y resaltaré la estructura de Objectory. (Tengo que usar algo, y esto parece tan adecuado como cualquier otra cosa.) No he tratado de describir Objectory, pues ello rebasa el alcance de esta obra. Sin embargo, describiré un proceso de peso ligero, de bajo perfil, que es consistente con Objectory. Para la información completa sobre Objectory, consulte el libro acerca del proceso escrito por los amigos.

Aunque el proceso Objectory contiene detalles sobre los tipos de modelos para desarrollar en las diferentes etapas del proceso, no profundizaré al respecto. Tampoco especificaré tareas, entregas o papeles. Mi terminología es más libre que la de Objectory; es el precio que se paga por una descripción superficial.

Sin importar cuál sea el análisis del proceso, no olvide que puede emplearse *cualquier* proceso con el UML. El UML es independiente del proceso. Usted debe seleccionar algo adecuado para su tipo de proyecto. Sea cual fuere el proceso con el que trabaje, el UML le puede servir para registrar las decisiones de análisis y diseño que resulten.

Panorámica del proceso

La figura 2-1 muestra la secuencia al nivel más alto del proceso de desarrollo.

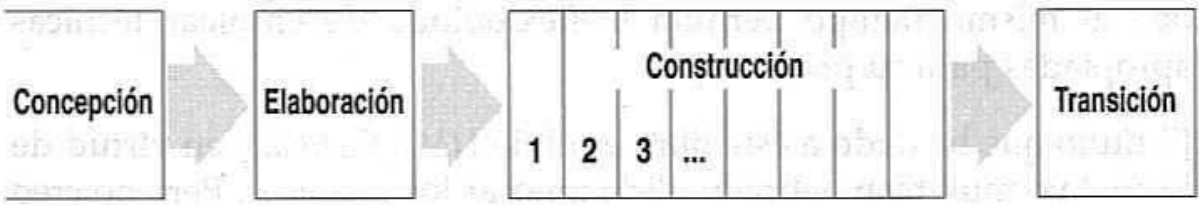


Figura 2-1: *Proceso de desarrollo del bosquejo*

Este proceso es un proceso de desarrollo iterativo y gradual, en el sentido de que el software no se libera de un solo gran golpe al final del proyecto, sino que, al contrario, se desarrolla y se libera por partes. La etapa de **construcción** consta de muchas **iteraciones**, donde cada iteración construye software de calidad para producción, probado e integrado, que cumple un subconjunto de los requerimientos del proyecto. La entrega puede ser externa, destinada a los primeros usuarios, o puramente interna. Cada iteración contiene todas las etapas usuales del ciclo de vida: análisis, diseño, implementación y experimentación.

En principio, se puede comenzar por el inicio: seleccione cierta funcionalidad y constrúyala, escoja otra más, y así sucesivamente. Es importante, sin embargo, dedicar cierto tiempo a la planificación.

Las dos primeras etapas son las de concepción y elaboración. Durante la **concepción**, se establece la razón de ser del proyecto y se determina su alcance. Es aquí cuando se obtiene el compromiso del patrocinador del proyecto para proseguir. En la **elaboración**, se reúnen requerimientos más detallados, se hacen análisis y diseños de alto nivel, a fin de establecer una arquitectura base, y se crea el plan de construcción.

Incluso con este tipo de proceso iterativo, hay trabajos que deben quedar para el final, la etapa de **transición**. Entre ellos están las pruebas beta, la afinación del desempeño y el entrenamiento del usuario.

Los proyectos varían en virtud de la cantidad de **ceremonia** que llevan consigo. Los proyectos de alto ceremonial tienen muchas entregas formales en papel, reuniones formales, autorizaciones formales. Los proyectos de bajo ceremonial pueden tener una etapa de concepción que consista en una plática de una hora con el patrocinador del proyecto y un plan asentado en una hoja de cálculo. Por supuesto, cuanto más grande sea el proyecto, más ceremonia se necesitará. Los pasos fundamentales de las etapas también se llevan a cabo, pero de modo muy diferente.

Yo trato de mantener el ceremonial al mínimo, y mi análisis lo reflejará así. Habrá muchos procesos de alto ceremonial que se puedan escoger en otras obras.

He presentado las iteraciones en la fase de construcción, pero no en las demás fases. De hecho, se pueden tener iteraciones en todas las fases y, con frecuencia, es buena idea tenerlas en las grandes fases. No obstante, la construcción es la fase clave donde se debe iterar.

Ésta es la perspectiva de alto nivel. Ahora nos sumergiremos en los detalles, de modo que tengamos la suficiente información para ver dónde encajan, dentro del panorama global, las técnicas que estudiaremos más adelante. Al hacerlo, hablaré un poco sobre dichas técnicas y cuándo usarlas. Podrá encontrar esto algo confuso si no está familiarizado con las técnicas. De ser éste el caso, pase por alto estas partes y vuelva a ellas después.

Concepción

La concepción puede adoptar muchas formas. Para algunos proyectos será, quizá, una plática frente a la cafetera automática: "Piensa cómo podemos poner nuestro catálogo de servicios en la Web." Para proyectos mayores, podría ser un amplio estudio de factibilidad que tardara meses.

Durante la etapa de concepción, se definirá la situación económica del proyecto: cuánto costará aproximadamente y cuánto redituará. También se necesitará tener una idea del alcance del proyecto. Tal vez haga falta cierto trabajo de análisis inicial para tener una idea de la magnitud del proyecto.

Trato de no darle demasiada importancia a la concepción. La concepción debe consistir en trabajar durante algunos días para determinar si vale la pena dedicar algunos meses de mayor investigación durante la elaboración (véase más adelante). En este momento, el patrocinador del proyecto no se compromete más que a una seria mirada al mismo.

Elaboración

De esta manera, usted ya tiene luz verde para iniciar un proyecto. En esta etapa, lo normal es que sólo posea una vaga idea de los requerimientos. Por ejemplo, podrá decir lo siguiente:

Vamos a construir la próxima generación del sistema de apoyo al cliente de la Watts Galore Utility Company. Tenemos la intención de construir un sistema más flexible, que esté más orientado al cliente, mediante tecnología orientada a objetos; en específico, uno de apoyo a las cuentas consolidadas de los clientes.

Ciertamente, su documento de requerimientos será más extenso que éste, pero en la práctica no dirá mucho más.

A estas alturas, usted querrá comprender mejor el problema.

- ¿Qué es lo que va a construir en realidad?
- ¿Cómo lo va a construir?
- ¿Qué tecnología empleará?

Al tomar decisiones durante esta etapa acerca de dichas cuestiones, lo primero y más importante que debe considerar son los riesgos de su proyecto. ¿Cuáles son los factores que pueden descarrilarlo? Cuanto mayor sea el riesgo, habrá que prestarle más atención.

Según mi experiencia, los riesgos se pueden clasificar, desde un punto de vista práctico, en cuatro categorías:

1. *Riesgos de requerimientos.* ¿Cuáles son los requerimientos del sistema? El gran peligro es que se construya el sistema erróneo, un sistema que no haga lo que quiere el cliente. Durante la etapa de elaboración, usted deberá entender bien los requerimientos y sus prioridades relativas.
2. *Riesgos tecnológicos.* ¿Cuáles son los riesgos tecnológicos que habrá de enfrentar? Plantéese las siguientes preguntas:
   a. Va a usar objetos. ¿Tiene ya la suficiente experiencia en el trabajo de diseño orientado a objetos (diseño OO)?
   b. Se le ha aconsejado que use Java y la Web. ¿Qué tan bien funciona esta tecnología? ¿Puede en realidad proporcionar las funciones que los usuarios necesitan a través de un browser explorador de Web conectado a una base de datos?

3. *Riesgos de habilidades.* ¿Puede conseguir la asesoría y los expertos que necesita?
4. *Riesgos políticos.* ¿Existen fuerzas políticas que se puedan interponer en su camino y afectar seriamente el proyecto?

En su caso, podrá haber más riesgos, pero los que se incluyen en estas categorías casi siempre están presentes.

Manejo de los riesgos de requerimientos

Los requerimientos son importantes y es donde las técnicas del UML son especialmente provechosas. El punto de partida son los casos de uso. Éstos, por lo tanto, son los motores de todo el proceso de desarrollo.

Los casos de uso se estudian en detalle en el capítulo 3, y a continuación sólo los describiré brevemente.

Un caso de uso es una interacción típica entre el usuario y el sistema con el fin de lograr cierto objetivo. Imagínese el procesador de texto con el que estoy trabajando. Un caso de uso sería "poner en negritas el texto seleccionado", otro, "crear el índice de un documento".

Como podrá apreciar en estos ejemplos, el tamaño de los casos de uso puede variar considerablemente. La clave es que cada uno indica una función que el usuario puede entender y, por tanto, tiene un valor para él. Un desarrollador puede responder de manera más concreta.

> *Me tomará dos meses hacer el índice de funciones que usted necesita. También tengo un caso de uso para manejar la revisión ortográfica. Sólo tengo tiempo de hacer uno. ¿Cuál necesita primero? Si quiere texto en negritas, lo puedo hacer en una semana, y puedo encargarme, al mismo tiempo, de las cursivas.*

Los casos de uso son la base para la comunicación entre los patrocinadores y los desarrolladores durante la planificación del proyecto.

Una de las cosas más importantes en la etapa de elaboración es el descubrimiento de los casos de uso potenciales del sistema en construcción. Por supuesto, en la práctica no va a descubrirlos todos. Sin

embargo, querrá encontrar la mayor cantidad posible, en especial los más importantes. Es por esta razón que, durante la etapa de elaboración, deberá programar entrevistas con los usuarios, con el fin de recopilar los casos de uso.

No hay necesidad de detallar los casos de uso. Normalmente considero que bastan uno o dos párrafos de texto descriptivo. El texto debe ser lo bastante específico para que los usuarios comprendan la idea básica y para que los desarrolladores tengan una idea general de lo que abarcan.

Los casos de uso no son, sin embargo, todo el panorama. Otra tarea importante es elaborar el esqueleto del modelo conceptual del dominio. Dentro de las cabezas de uno o varios usuarios es donde se encuentra el panorama del funcionamiento del negocio. Por ejemplo:

> *Nuestros clientes pueden tener varios sitios y nosotros les suministramos diversos servicios a estos sitios. En la actualidad, a cada cliente se le entrega un recibo por todos los servicios proporcionados en un sitio. Queremos que al cliente se le facturen todos los servicios de todos los sitios. A esto lo llamamos facturación consolidada.*

Este pasaje contiene las palabras "cliente", "sitio" y "servicio". ¿Qué significan estos términos? ¿Cómo se relacionan entre ellos? Un modelo conceptual del dominio comienza a contestar estas preguntas y, al mismo tiempo, establece los fundamentos para el modelo de objetos con el que se representarán los objetos del sistema, posteriormente, durante el proceso. Empleo el término **modelo de dominio** para describir cualquier modelo cuyo sujeto primario sea el mundo al que da apoyo el sistema de cómputo y cualquiera que sea la etapa del proceso de desarrollo en que se encuentre.

En Objectory usted se vale de distintos modelos para captar diversos aspectos del desarrollo. Los modelos de dominio y los casos de uso capturan los requerimientos funcionales; los modelos de análisis abarcan las implicaciones de estos requerimientos para una aplicación particular; los modelos de diseño agregan la infraestructura interna que hace que funcione la aplicación. El modelo de dominio de Objectory casi se termina de construir antes de que usted encuentre casos

de uso; su propósito es explorar el vocabulario del dominio en términos comprensibles para los expertos del dominio.

Una vez que usted cuenta con un modelo de dominio y un modelo de casos de uso, desarrolla un **modelo de diseño**, el cual reconoce tanto la información en los objetos del dominio como el comportamiento de los casos de uso. El modelo de diseño agrega clases que se encargan de llevar a cabo el trabajo y que proporcionan además una arquitectura reutilizable, que servirá de ayuda para extensiones futuras. En los proyectos mayores, usted puede desarrollar un modelo de análisis intermedio con el que se pueden explorar las conscuencias de los requerimientos externos antes de tomar decisiones sobre el diseño.

Objectory no requiere que se construya todo el sistema a manera de "cascada". Es importante dejar correctas las clases de dominio clave y los casos de uso clave para después construir una arquitectura de sistema reutilizable que pueda manejar ampliaciones posteriores. Luego, se pueden agregar de manera progresiva casos de uso, los cuales se pueden implementar en el modelo de diseño como parte de un proceso iterativo de desarrollo. No se debe construir el sistema completo de un solo golpe.

Encuentro especialmente valiosas dos técnicas de UML para la construcción de modelos de dominio.

- Los diagramas de clases, cuando se dibujan desde una perspectiva conceptual (véase el capítulo 4), son excelentes para capturar el lenguaje del negocio. Estos diagramas le pueden servir a usted para establecer los conceptos de que se valen los expertos del negocio al pensar en él, y para plantear cómo estos expertos vinculan los conceptos entre sí.

- Los diagramas de actividades (véase el capítulo 9) complementan los diagramas de clase describiendo el flujo del trabajo del negocio; es decir, los pasos que siguen los empleados para llevar a cabo sus labores. El aspecto crucial de los diagramas de actividad es que fomentan la búsqueda de procesos paralelos, lo cual es importante en la eliminación de secuencias innecesarias en los procesos del negocio.

A algunas personas les gusta apoyarse en los diagramas de interacción (véase el capítulo 6) para investigar cómo interactúan diversas actividades en la empresa. Al considerar como unidad tanto a los trabajadores como a las actividades, logran comprender con mayor facilidad el proceso. En mi caso, prefiero utilizar los diagramas de actividad para primero darme una idea de lo que es necesario hacer y después determinar quién se encarga de qué.

Los diagramas de interacción son más útiles durante este último paso. Además, los diagramas de interacción no promueven los procesos paralelos de la manera en que lo hacen los diagramas de actividad. Usted puede emplear los diagramas de actividad con carriles para encargarse tanto de las personas como del paralelismo, pero este procedimiento hace más complicados los diagramas (también puede usar diagramas de estado [véase el capítulo 8] en conjunto con el flujo de trabajo, pero encuentro muy engorroso aplicarlos en este contexto).

El modelado de dominios puede ser un magnífico complemento de los casos de uso. Cuando recopilo casos de uso, me gusta llamar a un experto del dominio e indagar la opinión que tiene acerca del negocio, apoyado con diagramas conceptuales de clases y de actividades.

En este caso, hago el mínimo de anotaciones, no me preocupo por ser riguroso y anoto muchas observaciones en el diagrama. No intento atrapar todos los detalles. En cambio, me concentro en los aspectos y las áreas importantes que implican un riesgo. Dibujo muchos diagramas no relacionados, sin preocuparme por la consistencia y la relación entre ellos.

He encontrado que este proceso me puede dar una gran comprensión con rapidez. Con este conocimiento, puedo identificar más fácilmente los casos de uso de los diferentes usuarios.

Una vez cubiertas la mayoría de las áreas importantes, me gusta consolidar los diversos diagramas en un solo modelo de dominio consistente. Para ello, consulto uno o dos expertos en el dominio a los que les interesa profundizar en el modelado. Conservo una perspectiva conceptual pero, al mismo tiempo, me vuelvo más riguroso.

Trato de desarrollar un solo modelo de área que maneje todos los requerimientos expresados en los modelos discretos anteriores. Este

modelo puede entonces servir como punto de partida para la formación de clases y para un diseño de clases más profundo en la etapa de construcción. Si el modelo resulta muy grande, mediante paquetes divido el modelo en partes. Llevo a cabo la consolidación de los diagramas de clase y de actividad y, tal vez, dibujo un par de diagramas de estado para las clases que tengan ciclos de vida interesantes.

Debe pensarse que el primer modelo de dominio es un esqueleto, no un modelo de alto nivel. El término "modelo de alto nivel" significa que faltan muchos detalles. He visto que se comete este error en varias situaciones, por ejemplo, "no mostrar los atributos en estos modelos". El resultado son modelos sin sustancia. Es fácil entender por qué los desarrolladores se mofan de tales esfuerzos.

Sin embargo, no se puede hacer lo opuesto y construir un modelo detallado. De hacerlo, le tomará demasiado tiempo y morirá de parálisis analítica. La clave está en encontrar y concentrarse en los detalles importantes. La mayor parte de los detalles se cubrirán durante el desarrollo iterativo. Por ello, prefiero considerar como esqueleto este modelo. El esqueleto es el fundamento del resto del modelo. Es detallado, pero representa sólo una parte pequeña de la historia.

Naturalmente, esto no le dice cómo determinar la diferencia entre carne y hueso; en esto consiste el arte del analista talentoso y yo no he descubierto aún la manera de embotellarlo.

El modelado de dominios también es dirigido por los casos de uso, a medida que se descubren. Conforme aparecen los casos de uso, el equipo de modelado debe estudiarlos para determinar si contienen alguna cosa que pudiera influir en el modelo de dominio. De ser así, es preciso investigar más a fondo; de lo contrario, entonces los casos de uso se deben hacer a un lado, por el momento.

El equipo que construye el modelo de dominio debe ser un grupo pequeño (de dos a cuatro personas) que incluya desarrolladores y expertos en el dominio. El equipo viable más pequeño será de un desarrollador y un experto en el dominio. El experto en el dominio (y de preferencia también el desarrollador) debe estar entrenado en el manejo de los diagramas del UML apropiados para el modelado conceptual.

El equipo deberá trabajar intensamente durante el periodo de elaboración hasta concluir el modelo. Durante este periodo, el líder deberá asegurar que el equipo no se empantane en los detalles ni que opere a un nivel tan alto que pierda contacto con la realidad. Una vez que entienden lo que están haciendo, el empantanamiento es el mayor peligro. Una fecha límite inflexible funciona de maravilla para lograr que las mentes se concentren.

Como parte de la comprensión de los requerimientos, se debe construir un prototipo de las partes intrincadas de los casos de uso. La de los prototipos es una técnica valiosa para entender mejor cómo funcionan las situaciones más dinámicas. A veces siento que entiendo bien la situación a partir de los diagramas, pero en otras ocasiones descubro que necesito un prototipo para apreciar adecuadamente lo que pasa. En general no hago un prototipo de todo el panorama, sino que, por el contrario, uso el modelo general del dominio para resaltar las áreas que necesitan prototipos.

Cuando emplee prototipos, no se restrinja al ambiente en el que hará sus entregas. Con frecuencia me he beneficiado enormemente con el análisis de prototipos en Smalltalk, incluso cuando estoy construyendo un sistema C++.

Manejo de los riesgos tecnológicos

Lo más importante que hay que hacer al abordar los riegos tecnológicos es construir prototipos que prueben las partes tecnológicas con las que uno piensa trabajar.

Por ejemplo, digamos que usted está trabajando en C++ y con una base de datos relacional. He aquí los pasos que deberá seguir:

1. Conseguir los compiladores de C++ y las demás herramientas.
2. Construir una parte sencilla de una de las primeras versiones del modelo de dominio. Vea qué tal funcionan para usted las herramientas.
3. Construir la base de datos y conectarla al código C++.
4. Probar diversas herramientas. Vea cuáles son las más fáciles de manejar y las más apropiadas para el trabajo. Adquiera familiaridad con las herramientas que escoja.

No olvide que los riesgos tecnológicos mayores son inherentes a la manera en que se integran los componentes de un diseño, en lugar de hallarse en los componentes mismos. Usted puede conocer bien C++ y las bases de datos correspondientes, pero integrarlos no es tan fácil. Por eso es tan importante obtener todos los componentes con los que se pretende trabajar e integrarlos en esta etapa temprana del proceso.

También en esta etapa deberá ocuparse de cualquier decisión de diseño arquitectónico. Estas decisiones por lo general toman la forma de ideas acerca de lo que son los componentes y sobre la manera como se construirán. Esto es importante, en particular si se contempla un sistema distribuido.

Como parte de este ejercicio, concéntrese en las áreas que parezca que más adelante van a ser más difíciles de cambiar. Trate de llevar a cabo su diseño de tal forma que le permita cambiar los elementos del diseño en forma relativamente fácil. Pregúntese lo siguiente:

- ¿Qué sucederá si no trabaja una pieza de la tecnología?
- ¿Qué ocurrirá si no podemos conectar dos piezas del rompecabezas?
- ¿Cuál es la probabilidad de que algo vaya mal? ¿Qué haremos, si sucede esto?

Del mismo modo que en el modelo de dominio, usted deberá analizar los casos de uso a medida que aparezcan, a fin de determinar si contienen algo que pueda echar por tierra su diseño. Si abriga el temor de que contengan un "gusano púrpura", ahonde en su investigación.

Durante este proceso, usted utilizará, típicamente, cierta cantidad de técnicas UML para esbozar sus ideas y documentar lo que está probando. En este momento, no intente tener una visión detallada; todo lo que necesita son bosquejos breves y, por tanto, eso es lo que debe usar.

- Los diagramas de clase (véanse los capítulos 4 y 5) y los diagramas de interacción (véase el capítulo 6) son útiles para mostrar la manera en que se comunican los componentes.

- Los diagramas de paquetes (véase el capítulo 7) pueden mostrar, en esta etapa, un cuadro de alto nivel de los componentes.
- Los diagramas de emplazamiento o deployment diagrams (véase el capítulo 10) pueden proveer una visión panorámica de la distribución de las piezas.

Manejo de los riesgos de habilidad

Suelo asistir con regularidad a conferencias y escucho ponencias sobre casos prácticos pronunciadas por gente que acaba de llevar a cabo un proyecto orientado a objetos. A la pregunta de "¿cuál fue su mayor problema?" responden invariablemente con frases del estilo de "deberíamos haber recibido mayor entrenamiento".

Nunca deja de asombrarme que las compañías se embarquen en importantes proyectos OO con poca experiencia y sin idea sobre la manera de ganar más. Se preocupan por los costos del entrenamiento, pero pagan hasta el último centavo cuando el proyecto se alarga.

El entrenamiento es una forma de evitar errores, dado que los instructores ya los han cometido. Cometer errores consume tiempo y el tiempo cuesta. Así, se acaba pagando lo mismo en una u otra forma, pero la falta de entrenamiento provoca que el proyecto tarde más.

No soy un fanático de los cursos de entrenamiento formales. He impartido muchos de ellos y diseñado algunos otros también. Sigo sin convencerme de su eficacia en la enseñanza de las habilidades de la orientación a objetos. Dan a las personas un panorama de lo que necesitan saber, pero en realidad no logran transmitirles las habilidades indispensables para construir un proyecto real. Un breve curso de entrenamiento puede ser útil, pero sólo constituye el principio.

Si decide tomar un curso de entrenamiento, preste mucha atención a la persona que se lo impartirá, el instructor. Vale la pena pagar una buena suma extra por un instructor capaz y dedicado, porque así aprenderá mucho más durante el proceso. También es recomendable recibir el entrenamiento por partes, al momento que se necesiten. Por otra parte, si no pone en práctica de inmediato lo aprendido en el curso, pronto lo olvidará.

La mejor manera de adquirir las habilidades de la OO es a través del método de **tutoría**, mediante el cual un desarrollador experimentado colabora con usted en su proyecto durante un extenso periodo. El tutor le muestra cómo hacer las cosas, observa lo que usted hace y le da consejos, al tiempo que le ayudará con pequeños entrenamientos.

El tutor trabajará en los aspectos concretos de su proyecto y sabrá qué partes de su experiencia deben ponerse en práctica en el momento adecuado. En las etapas iniciales, el tutor es parte del equipo, y le ayudará a usted a encontrar soluciones. Con el paso del tiempo, usted se volverá cada vez más capaz y el tutor se dedicará cada vez más a revisar, en lugar de hacer. Por mi parte, mi meta como tutor es volverme innecesario.

Se pueden encontrar tutores para áreas específicas o para todo el proyecto, y desempeñarán su papel de tiempo completo o bien de medio tiempo. Muchos de ellos prefieren trabajar una semana al mes en cada proyecto; en cambio, otros consideran que una semana es poco tiempo. Busque a un tutor con experiencia y con la habilidad de transmitirla. Él puede ser el factor más importante para el éxito de su proyecto; no olvide que vale la pena pagar por la calidad.

Si no puede conseguir un tutor, considere una revisión del proyecto cada dos meses, más o menos. Dentro de este plan, un tutor experimentado dedicará varios días a revisar los diversos aspectos del diseño. Durante este periodo, el revisor podrá detectar las áreas críticas, sugerir otras ideas y proponer técnicas útiles que el equipo no haya tenido en cuenta. Aunque esto no le da todos los beneficios de un buen tutor, puede ser valioso para señalar aspectos importantes susceptibles de mejorarse.

También podrá complementar sus habilidades mediante la lectura. Trate de leer un buen libro técnico, cuando menos cada dos meses. Mejor aún, intente leerlo como parte de un grupo dedicado a la lectura. Encuentre un par de personas que deseen leer el mismo libro. Acuerden leer un par de capítulos a la semana e inviertan un par de horas para analizarlos en conjunto. Con esta forma de lectura usted entenderá mejor la obra que leyéndola solo. Si usted es gerente, promueva esta forma de lectura. Consiga una habitación para el grupo; proporcione a su equipo dinero para comprar libros técnicos, y asigne el tiempo necesario para el grupo de lectura.

La comunidad de usuarios de patrones (*patterns*) ha descubierto que los grupos de lectura son particularmente valiosos. Han surgido varios grupos de lectura de patrones. Para más información sobre ellos, consulte la página base de patrones (<**http://st-www.cs.uiuc.edu/users/patterns/patterns.html**>).

A medida que avance en la elaboración de su proyecto, manténgase alerta en aquellas áreas en las que no tiene todavía habilidades ni experiencia suficientes. Planee adquirir la experiencia en el momento en que la necesite.

Manejo de los riesgos políticos

No puedo ofrecerle aquí ningún consejo importante, pues no soy un hábil político corporativo. Sin embargo, le recomiendo encarecidamente que busque alguien que lo sea.

Base arquitectónica

Un importante resultado de la elaboración es que se cuenta con una **base arquitectónica** para el sistema. Esta arquitectura se compone de:

- La lista de casos de uso, que le dice cuáles son los requerimientos.
- El modelo del dominio, que contiene lo que usted ha entendido sobre el negocio y sirve de punto de partida para las clases clave del dominio.
- La plataforma tecnológica, que describe las partes clave de la tecnología de implementación y la manera como se acoplan.

Esta arquitectura es el cimiento del desarrollo y funciona como anteproyecto de las etapas posteriores. Los detalles de la arquitectura cambiarán inevitablemente, sin que tenga que sufrir, sin embargo, muchos cambios importantes.

Sin embargo, la importancia de una arquitectura estable varía con la tecnología. En Smalltalk es posible efectuar cambios arquitectónicos significativos con mucha más facilidad, gracias a la rapidez con que suceden los ciclos edición-ejecución y a que no se requiere de una especificación estricta de los tipos de datos. Esto permite que la arquitectura sea

muy evolutiva, tal y como se ilustra en el patrón de procesos *Episodes* (véase Cunningham, 1996). En C++, es más importante tener una arquitectura estable subyacente a la construcción.

¿Cuándo se termina la elaboración?

Mi regla empírica es que la elaboración consume una quinta parte de la duración total del proyecto. Dos circunstancias son indicadores clave que señalan que se ha completado la elaboración:

- Los desarrolladores pueden tener la confianza necesaria para dar estimaciones, con un margen de hasta una semana-persona de esfuerzo, sobre el tiempo que tardará la construcción de cada caso de uso.
- Se han identificado todos los riesgos significativos y se han entendido los principales, al grado de que ya se sabe cómo tratarlos.

La planificación

La esencia de un plan es establecer una serie de iteraciones para la construcción y asignar los casos de uso a las iteraciones.

El plan se termina cuando todos los casos de uso han sido asignados a una iteración y cuando se ha identificado la fecha de inicio de todas las iteraciones. El plan no entra en mayores detalles.

El primer paso es clasificar los casos de uso por categoría.

- Los usuarios deberán indicar el nivel de prioridad de cada caso de uso. Por mi parte, suelo emplear tres niveles.

  –"Es indispensable tener esta función en cualquier sistema."

  –"Puedo vivir sin esta función por breve tiempo."

  –"Es una función importante, pero puedo sobrevivir sin ella durante un rato."

- Los desarrolladores deberán considerar el **riesgo arquitectónico** asociado con cada caso de uso, el cual consiste en que, si el caso de uso se deja de lado hasta muy avanzado el proyecto, el trabajo anterior se verá muy comprometido, lo que resultará en una gran

cantidad de retrabajo. Aquí, nuevamente, tiendo a establecer tres categorías de clasificación: alto riesgo, riesgo posible pero no probable y poco riesgo.

- Los desarrolladores deben evaluar la seguridad que tengan en la estimación del esfuerzo requerido para cada caso de uso. A esto lo llamo **riesgo de calendarización**. También aquí considero valioso el uso de tres niveles:

  –"Estoy bastante seguro de saber cuánto tiempo tardará."

  –"Puedo estimar el tiempo sólo hasta el mes-persona más próximo."

  –"No tengo la menor idea de cuánto tiempo tarde."

Realizado lo anterior, habrá que estimar el tiempo de duración que tomará cada caso de uso, hasta la semana-persona más próxima. Al efectuar esta estimación, suponga que hay que efectuar análisis, diseño, codificación, pruebas individuales, integración y documentación. Suponga también que dispone de un desarrollador que trabaja de tiempo completo en el proyecto (más tarde agregaremos un factor de error).

Nótese que considero que quienes deben hacer las estimaciones son los *desarrolladores*, y *no* los gerentes. Como complemento de esta observación, asegúrese de que quien haga la estimación sea el desarrollador que posea el mayor conocimiento del caso de uso dado.

Una vez efectuadas las estimaciones, se puede decidir si se está listo para hacer el plan. Vea los casos de uso con mayor riesgo de calendarización. Si gran parte del proyecto está atada a estos casos de uso o si estos casos tienen muchos riesgos arquitectónicos, entonces será necesaria una mayor elaboración.

El siguiente paso es la determinación de la longitud de iteración. Se busca una longitud de iteración fija para todo el proyecto, de modo que se pueda lograr un ritmo regular de entrega de iteraciones. Cada iteración debe ser lo suficientemente larga para realizar varios casos de uso. En el caso de **Smalltalk**, puede ser de un mínimo de dos a tres semanas, por ejemplo; para C++, puede ser de hasta seis a ocho semanas.

Ahora se puede considerar el esfuerzo para cada iteración.

Un buen punto de inicio es suponer que los desarrolladores operarán con un promedio de eficiencia del 50%; es decir, la mitad de su tiempo se dedicará al desarrollo de los casos de uso. Multiplique la longitud de la iteración por el número de desarrolladores y divida entre dos. El resultado será el esfuerzo de desarrollo por cada iteración. Por ejemplo, dados ocho desarrolladores y una longitud de iteración de tres semanas, tendrá 12 semanas-desarrollador ((8*3)/2) de esfuerzo por iteración.

Sume el tiempo de todos los casos de uso, divida entre el esfuerzo por iteración y sume uno por si hace falta. El resultado es su primera estimación de la cantidad de iteraciones que necesitará para su proyecto.

El siguiente paso es la asignación de los casos de uso a las iteraciones.

Los casos de uso de alta prioridad, riesgo arquitectónico y/o riesgos de calendarización deben tratarse antes que los demás. ¡*No* deje los riesgos para el final! Tal vez necesite dividir los casos de uso más grandes y probablemente tenga que revisar las estimaciones de los casos de uso de acuerdo con el orden en el que está realizando las cosas. Tal vez tenga menos trabajo por hacer que el esfuerzo que requiere la iteración, pero nunca debe calendarizar más de lo que le permita su esfuerzo.

Para la transición, asigne del 10% al 35% del tiempo de construcción a la afinación y el empaquetado para entrega (aplique una cifra mayor si no tiene experiencia en afinación y empaquetado en el ambiente actual).

Después, agregue un factor de contingencia: del 10% al 20% del tiempo de construcción, dependiendo de lo arriesgada que parezca ser la situación. Agregue este factor al final de la etapa de transición. Se debe planificar la entrega sin indicar el tiempo de contingencia (esto es, dentro de su fecha límite interna) pero comprométase a entregar al final del tiempo de contingencia.

Después de seguir todas estas instrucciones generales, usted deberá contar con un plan que muestre los casos de uso que se realizarán durante cada iteración. Este plan simboliza el compromiso entre los desarrolladores y los usuarios; un buen nombre para este plan es el de calendario de compromisos. Este itinerario no es inflexible; de hecho, todo el mundo debe esperar que el calendario de compromisos cam-

bie a medida que avanza el proyecto. Sin embargo, debido a que se trata de un compromiso entre desarrolladores y usuarios, los cambios se deben hacer en conjunto.

Como podrá apreciar por todo lo anterior, los casos de uso son el fundamento de la planificación del proyecto, razón por la cual UML pone tanto énfasis en ellos.

Construcción

La construcción confecciona el sistema a lo largo de una serie de iteraciones. Cada iteración es un miniproyecto. Se hace el análisis, diseño, codificación, pruebas e integración de los casos de uso asignados a cada iteración. Ésta termina con una demostración al usuario y haciendo pruebas del sistema con el fin de confirmar que se han construido correctamente los casos de uso.

El propósito de este proceso es reducir el riesgo. Los riesgos surgen con frecuencia debido a que las cuestiones difíciles se posponen para el final del proyecto. He visto proyectos en los que la prueba y la integración se dejan para el final. Las pruebas y la integración son tareas mayores y generalmente tardan más de lo que se cree.

Fred Brooks estimaba, allá en la época del OS/360, que la mitad del tiempo de un proyecto se iba en pruebas (con la inevitable depuración). Las pruebas y la integración son más difíciles cuando se dejan para el final, y son más desmoralizadoras.

Todo este esfuerzo va acompañado de un gran riesgo. Con el desarrollo iterativo, se lleva a cabo todo el proceso para cada iteración, lo que produce el hábito de lidiar con todos los aspectos cada vez.

Cuanto más envejezco, más agresivo me vuelvo en lo que respecta a las pruebas. Me gusta la regla empírica de Kent Beck que afirma que un desarrollador debería escribir cuando menos la misma cantidad de código de pruebas que de producción. El proceso de pruebas debe ser continuo. No se debe escribir ningún código hasta saber cómo probarlo. Una vez escrito, haga sus pruebas. Hasta que éstas funcionen, no se podrá considerar que se haya terminado de escribir el código.

Una vez escrito, el código de pruebas debe conservarse para siempre. Prepare su código de pruebas de forma que pueda ejecutarlas todas con un comando simple o mediante la pulsación de un botón GUI. El código debe responder con "OK" o con una lista de fallas. Además, las pruebas deben revisar sus propios resultados. No hay mayor pérdida de tiempo que tener que investigar el significado de un número enviado a la salida por una prueba.

Divida las pruebas en pruebas individuales y de sistema. Las pruebas individuales deberán ser escritas por los desarrolladores. Habrán de organizarse con base en paquetes y codificarse de modo que prueben todas las clases de las interfaces. Las pruebas de sistemas deberán ser desarrolladas por un equipo pequeño independiente cuya única tarea sea probar. Este equipo deberá tener una visión completa del sistema y sentir un placer especial en descubrir fallas. (Los bigotes retorcidos y las risas siniestras, aunque opcionales, son deseables.)

Las iteraciones dentro de la construcción son tanto incrementales como iterativas.

- Funcionalmente, las iteraciones son *incrementales*. Cada iteración se construye sobre los casos de uso desarrollados en las iteraciones anteriores.

- Son *iterativas* en términos del código de base. Cada iteración implicará la reescritura de algún código ya existente con el fin de hacerlo más flexible. La **reestructuración de factores** (véase el recuadro) es una técnica muy útil para la iteración del código. Es buena idea saber la cantidad de código desechado con cada iteración. Desconfíe si se descarta menos del 10% del código en cada ocasión.

Reestructuración de factores

¿Se ha topado alguna vez con el principio de la entropía de software? Este principio sugiere que los programas comienzan en un estado de buen diseño pero, a medida que se añade código para nuevas funciones, gradualmente van perdiendo su estructura, deformándose de tal modo que acaban como una masa de espagueti.

Parte de este hecho se debe a la escala. Se escribe un programa pequeño que hace bien un trabajo específico. Luego se pide mejorar el programa, por lo que se vuelve más complejo. Esto puede suceder incluso cuando se lleva el registro del diseño.

Una de las razones por las que surge la entropía en el software es que, cuando se añade una función al programa, se construye encima del programa existente, con frecuencia de una manera para la que no estaba diseñado. En tal situación, se puede rediseñar el programa existente para que maneje mejor los cambios, o bien adecuar esos cambios al código que se añada.

Aunque teóricamente es mejor rediseñar el programa, este procedimiento por lo general cuesta más trabajo, ya que cualquier reescritura de su programa generará nuevas fallas y problemas. Recuerde el viejo adagio de la ingeniería: "¡si no está descompuesto, no lo arregle!". Sin embargo, si no rediseña su programa, las adiciones serán más complejas de lo que deberían.

A la postre, esta complejidad extra tendrá un alto costo. Por tanto, existe un intercambio de una cosa por otra: el rediseño provoca molestias de corto plazo a cambio de una ganancia a largo plazo. Comprendiendo lo que es la presión sobre los calendarios de trabajo, la mayoría de las personas prefieren posponer las molestias para el futuro.

La **reestructuración de factores** es un término que describe técnicas con las que se reducen las molestias a corto plazo del rediseño. Cuando se reestructuran los factores, no cambia la funcionalidad del programa; lo que se hace es cambiar su estructura interna, a fin de simplificar su lectura y su modificación.

Los cambios por la reestructuración de factores usualmente se hacen a pasos pequeños: renombrar un método; mover un campo de una clase a otra; consolidar dos métodos similares en una superclase. Cada paso es pequeño; sin embargo, un par de horas invertidas en estos pequeños pasos pueden hacer maravillas por el programa.

La reestructuración de factores se vuelve más sencilla con los siguientes principios:

- No reestructure un programa y agregue funcionalidad al mismo tiempo; separe claramente ambas acciones mientras trabaja. Puede alternarlas en pasos cortos; por ejemplo, media hora de reestructuración, una hora agregando una nueva función, y otra media hora reestructurando el código recién añadido.
- Asegúrese de tener a la mano buenas pruebas antes de comenzar la reestructuración. Ejecute las pruebas con la mayor frecuencia posible. De este modo, sabrá pronto si sus cambios han echado a perder algo.
- Avance con pasos cortos y deliberados. Mueva un campo de una clase a otra. Fusione dos métodos similares, creando una superclase. La reestructuración implica con frecuencia hacer muchos cambios localizados cuyo resultado sea un cambio de mayor escala. Si sus pasos son cortos y los prueba de uno en uno, evitará depuraciones prolongadas.

Se deberá reestructurar en los casos siguientes:

- Cuando se añada alguna funcionalidad al programa y se descubra que el código viejo estorba. En cuanto esto se vuelva un problema, suspenda la adición de la nueva función y reestructure el código viejo.
- Cuando se tengan dificultades para comprender el código. La reestructuración es un buen modo de ayudarle a comprender el código y de retener este entendimiento para el futuro.

Frecuentemente le sucederá que quiera reestructurar algún código escrito por otro. Cuando lo haga, llévelo a cabo con el autor del código. Es muy difícil escribir un código de modo que otros lo puedan comprender con facilidad. La mejor forma de reestructurar es trabajando junto con alguien que entienda el código. Así podrá combinar el conocimiento de él con la inexperiencia de usted.

Cuándo reestructurar

La reestructuración de factores es una técnica que no se aprovecha suficientemente. Apenas ha comenzado a ser reconocida, en especial en la comunidad de **Smalltalk**. Creo, sin embargo, que se trata de una técnica clave para mejorar el desarrollo del software en cualquier medio ambiente. Asegúrese de que entiende la manera de llevar a cabo la reestructuración de manera disciplinada. Una forma de lograrlo es que su tutor le enseñe las técnicas.

Para mayor información

Debido a su novedad, es poco lo que se ha escrito sobre la reestructuración. La tesis de doctorado de William Opdyke (1992) es probablemente el tratado más completo sobre el tema, aunque está orientado a las herramientas automáticas de reestructuración, más que a las técnicas que le puedan servir a la gente. Kent Beck es uno de los más destacados exponentes de la reestructuración; su libro sobre patrones (1996) incluye muchos de éstos que son medulares para la reestructuración. Véase también el artículo de Beck de 1997, que da una buena idea del proceso de reestructuración.

Si usa VisualWorks o Smalltalk de IBM, deberá bajar Refactory, una herramienta que maneja reestructuración (véase <**http://st-www.cs.uiuc.edu/users/droberts/Refactory.html**>). Esta herramienta fue desarrollada por Don Roberts y John Brandt, quienes trabajan con Ralph Johnson en la Universidad de Illinois. Considero que esta herramienta es el desarrollo más importante en herramientas de codificación desde el Ambiente integrado de desarrollo (*Integrated Development Environment*).

La integración debe ser un proceso continuo. Para los principiantes, la integración total es parte del fin de cada iteración. Sin embargo, puede y debe ocurrir con mayor frecuencia. El desarrollador deberá integrar después de cada pieza de trabajo importante. La secuencia completa de pruebas individuales deberá ejecutarse con cada integración, a fin de garantizar pruebas regresivas completas.

Desarrollo y planificación iterativos

También se puede hacer, con cada iteración, una planificación más detallada. Una de las partes fundamentales de todo programa es enfrentar cosas que no van de acuerdo con lo planeado. Reconozcámoslo, siempre sucede así.

La característica clave del desarrollo iterativo es que los tiempos están limitados; no se permite retrasar ninguna fecha. En cambio, los casos de uso se pueden trasladar a una iteración posterior mediante negociaciones y acuerdos con el patrocinador. De lo que se trata es de habituarse a cumplir con las fechas, evitando la mala costumbre de retrasarlas.

Sin embargo, nótese que si se están posponiendo demasiados casos de uso, es momento de rehacer el plan, incluyendo la reestimación de los niveles de esfuerzo de los casos de uso. En esta etapa, el desarrollador ya deberá tener una mejor idea de cuánto tardarán las cosas.

Empleo del UML en la construcción

Todas las técnicas del UML son útiles durante esta etapa. Debido a que me referiré a técnicas sobre las que no he hablado aún, siéntase con libertad para saltarse esta sección y volver a ella después.

Cuando se contemple la adición de un caso de uso específico, utilice, en primer lugar, el caso de uso para determinar su alcance. Un diagrama conceptual de clases (véase el capítulo 4) puede ser útil para esbozar algunos conceptos del caso de uso y ver cómo encajan en el software que ya se ha construido. Si el caso de uso contiene elementos importantes del flujo de trabajo, se pueden ver mediante un diagrama de actividades (véase el capítulo 9).

La ventaja de estas técnicas durante esta etapa es que se pueden usar en conjunto con el experto del dominio. Como dice Brad Kain: el análisis sólo se hace cuando el experto del dominio se encuentra en la sala (de otra forma, se trata de pseudoanálisis).

He descubierto que, para pasar al diseño, un diagrama de clases correspondiente a la perspectiva de especificaciones (véase el capítulo 4) puede ser útil para proyectar con mayor detalle las clases. Los diagramas de interacción (véase el capítulo 6) son valiosos para mostrar la manera en que interactuarán las clases para implementar el caso de uso.

Se pueden dibujar directamente los diagramas de clases y de interacción o emplear las tarjetas CRC (véase la página 74) para indagar su comportamiento y después documentarlos mediante diagramas, si se desea. Cualquiera que sea el enfoque escogido, me parece importante prestar mucha atención a las responsabilidades en esta etapa del trabajo.

Considero que los diagramas UML son valiosos para entender de manera general el sistema. Sin embargo, debo subrayar que no creo en el hecho de diagramar de manera detallada todo el sistema. Citando a Ward Cunningham (1996):

> *Los memorandos cuidadosamente escritos y seleccionados pueden sustituir con toda facilidad a la tradicional documentación detallada del diseño. Esta última es sobresaliente en pocas ocasiones, excepto en algunos puntos aislados. Destaque estos puntos... y olvídese del resto.*

Restrinja la documentación a aquellas áreas en las que es útil. Si descubre que la documentación no le está ayudando, es señal de que algo anda mal.

Yo me apoyo en un diagrama de paquetes (véase el capítulo 7) como mapa lógico del sistema. Este diagrama me ayuda a comprender las piezas lógicas del sistema y a ver las dependencias (y mantenerlas bajo control).

Me gusta utilizar herramientas que me ayuden a identificar las dependencias y asegurarme de no pasarlas por alto. Incluso herramientas sencillas, como los scripts scripts de Perl, pueden ser de utilidad. Java,

con su manejo explícito de paquetes, es una gran ayuda. También puede ser de utilidad en esta etapa un diagrama de emplazamiento (véase el capítulo 10), el cual muestra el cuadro físico de alto nivel.

En cada paquete me gusta ver un diagrama de clases de perspectiva de especificaciones. No muestro todas las operaciones sobre todas las clases, sino sólo las asociaciones y los atributos y operaciones clave que me ayudan a comprender lo que hay allí.

Este diagrama de clases actúa como índice gráfico de materias. Muchas veces ayuda a mantener un glosario de clases que contiene definiciones breves de cada clase, con frecuencia por medio de declaraciones de responsabilidades. También es una buena idea mantener las declaraciones de responsabilidades en el código, en forma de comentarios, los cuales se pueden extraer con alguna herramienta adecuada.

Si una clase tiene un comportamiento de ciclo de vida complicado, lo describo dibujando un diagrama de estado (véase el capítulo 8). Sólo lo hago si el comportamiento es lo bastante complejo, lo cual no sucede con mucha frecuencia. Lo que es más común son las interacciones complicadas entre clases, para las que genero un diagrama de interacciones.

Mi forma favorita de documentación tiene tres elementos básicos:

1. Una o dos páginas que describen unas cuantas clases en un diagrama de clases.
2. Unos cuantos diagramas de interacción que muestren cómo colaboran las clases.
3. Cierto texto para integrar los diagramas.

Con frecuencia incluyo algún código importante, escrito en estilo de programa literario. Si está implicado un algoritmo particularmente complejo, consideraré la posibilidad de utilizar un diagrama de actividades (véase el capítulo 9), pero sólo si me ayuda a entender mejor que el propio código. En esos casos, utilizo un diagrama de clases correspondiente a la perspectiva de especificación o a la perspectiva de implementación, o tal vez ambos; todo depende de lo que esté tratando de comunicar.

En Objectory, se deberán dibujar diagramas de interacción (véase el capítulo 7) para cada caso de uso que se identifique. Estos diagramas de interacción deberán cubrir todos los escenarios. No se necesita un diagrama individual para cada escenario, pero asegúrese de que la lógica de todos ellos se capte en el diagrama de interacciones que abarque los casos de uso asociados. Para mayor información sobre los detalles de Objectory, véase el libro de los amigos acerca del proceso.

Si descubro conceptos que aparecen continuamente, capto las ideas básicas mediante patrones (véase el recuadro).

Utilizo los patrones de diversos modos; también tiendo a emplear varias formas de patrones. Un patrón de diseño penetrante sugeriría el uso de un patrón estilo "pandilla de los cuatro" (véase Gamma *et al.* 1994). No obstante, empleo también otras formas, dependiendo de lo que se adecue mejor a una situación particular (como usted podrá suponer, presto especial atención a los patrones de análisis; véase Fowler 1997).

Los patrones son útiles dentro del alcance de un proyecto y también para la comunicación de buenas ideas fuera del proyecto. De hecho, considero que los patrones son particularmente valiosos también para comunicación entre proyectos.

Patrones

El UML le dice cómo expresar un diseño orientado a objetos. Los **patrones**, por el contrario, se refieren a los resultados del proceso: modelos ejemplo.

Muchas personas han comentado que los proyectos tienen problemas debido a que la gente que intervino no conocía diseños que a personas con más experiencia les son familiares. Los patrones describen maneras comunes de hacer las cosas. Son conjuntados por personas que detectan temas que se repiten en los diseños. Estas personas toman cada tema y lo describen de tal forma que otros lo puedan leer y sepan cómo aplicarlo.

Veamos un ejemplo. Digamos que se están ejecutando varios objetos en un proceso en su computadora personal y necesita comunicarse con otros objetos en ejecución en otro proceso. Tal vez este proceso también esté en su computadora; o tal vez se encuentre en otra parte. Usted no quiere que los objetos de su sistema tengan que preocuparse por encontrar otros objetos en la red ni que tengan que ejecutar llamadas a procedimientos remotos.

Lo que puede hacer es crear un objeto *suplente* dentro del proceso local para el objeto remoto. El suplente tiene la misma interfaz que el objeto remoto. Los objetos locales le hablan al suplente mediante el envío de mensajes normales del proceso. El suplente es responsable de pasar los mensajes al objeto real, dondequiera que resida.

La figura 2-2 es un diagrama de clases (véase el capítulo 4) que ilustra la estructura del patrón *Suplente*.

Los suplentes son una técnica común que se emplea en redes y otras partes.

Hay una gran experiencia en el empleo de suplentes, en términos del conocimiento de la manera de usarlos, las ventajas que pueden ofrecer, sus limitaciones y la manera de implementarlas. Los libros de métodos como el presente no explican este conocimiento; todo lo que explican es la manera de diagramar suplentes. Aunque

esto es útil, no lo es tanto como la explicación de la experiencia relacionada con los suplentes.

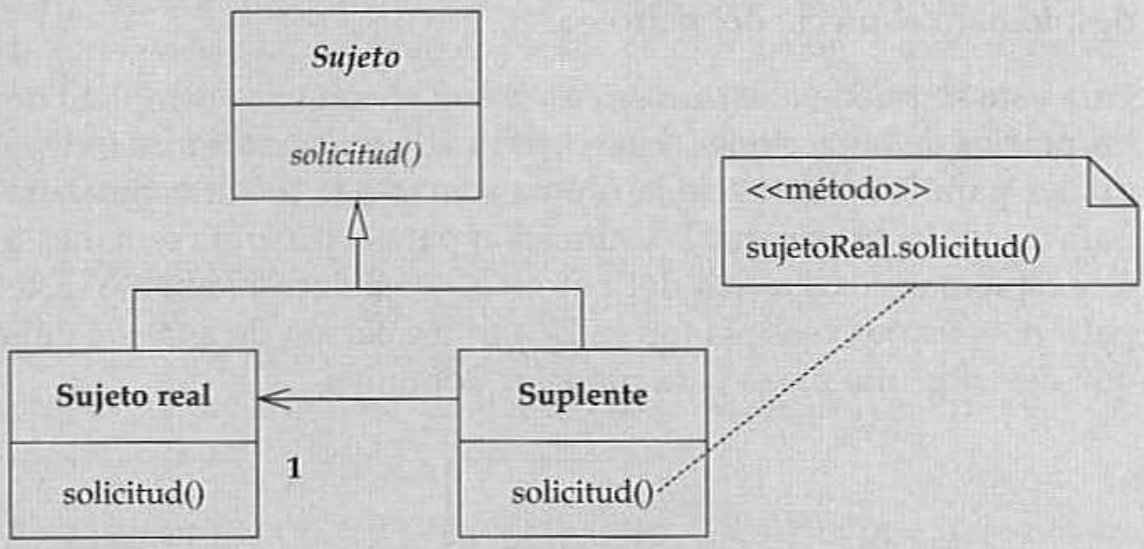


Figura 2-2: *Patrón de diseño de un suplente*

A principios de la década de los noventa, algunos comenzaron a documentar esta experiencia. Formaron una comunidad interesada en la escritura de patrones. Estas personas patrocinan conferencias y han escrito varios libros.

El libro más famoso sobre patrones que ha surgido de este grupo es el libro de "la pandilla de los cuatro" (Gamma *et al.* 1994), que explica con detalle 23 patrones de diseño.

Si quiere saber más sobre los suplentes, ésta es la fuente. El libro de "la pandilla de los cuatro" dedica 10 páginas al tema, dando detalles sobre la operación en conjunto de los objetos, los beneficios y las limitaciones del patrón, los usos y variaciones comunes y sugerencias de implementación para **Smalltalk** y C++.

El suplente es un patrón de diseño porque describe una técnica de diseño. También pueden existir patrones en otras áreas. Digamos que se diseña un sistema de administración de riesgos en los mercados financieros. Se necesita comprender cómo cambia con el transcurso del tiempo el valor de una cartera de acciones. Se pue-

de hacer lo anterior manteniendo un precio por cada acción y fechando el precio. Sin embargo, también querrá considerar lo que sucederá en casos hipotéticos (por ejemplo, "¿qué pasaría si se desplomara el precio del petróleo?").

Para esto se puede crear un *escenario* con el conjunto completo de los precios de las acciones. Luego, se puede tener *escenarios* individuales para los precios de la última semana, la mejor estimación para la siguiente semana, la estimación para la próxima semana, si se desploman los precios del petróleo, y así sucesivamente. Este patrón *escenario* (véase la figura 2-3) es un patrón de análisis porque describe una pieza para modelar dominios.

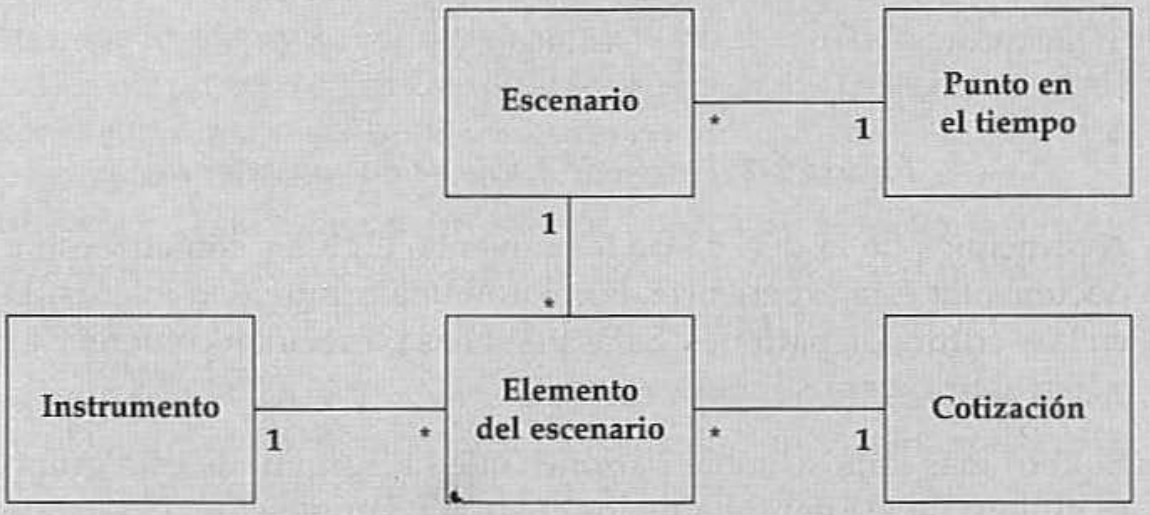


Figura 2-3: *Patrón de análisis de escenarios*

Los patrones de análisis son valiosos porque permiten un mejor comienzo cuando se trabaja con un dominio nuevo. Yo comencé a reunir patrones de análisis porque estaba frustrado por los nuevos dominios. Sabía que no era la primera persona en modelarlos; sin embargo, cada ocasión tenía que volver a comenzar desde cero.

Lo interesante sobre los patrones de análisis es que se aparecen en los lugares más inesperados. Cuando comencé a trabajar en un proyecto de análisis financiero corporativo, fueron de enorme ayuda una serie de patrones que había descubierto en un trabajo previo en la rama de salud.

Para mayor información sobre *escenarios* y otros patrones de análisis, véase Fowler (1997).

Un patrón es mucho más que un modelo. El patrón debe incluir la razón por la cual es como es. Se dice con frecuencia que un patrón es una solución a un problema. El patrón debe dejar claro el problema, explicar por qué lo resuelve y además explicar las circunstancias bajo las que funciona y bajo las que no.

Los patrones son importantes porque son la siguiente etapa tras el dominio de los elementos básicos de un lenguaje o técnica para modelar. Los patrones ofrecen una serie de soluciones y enseñan lo que constituye un buen modelo y cómo se construye. Enseñan con el ejemplo.

Cuando comencé, me preguntaba por qué tenía que inventar las cosas de la nada. ¿Por qué no había manuales que me enseñaran la manera de hacer las cosas más elementales? La comunidad de los patrones está intentando construir estos manuales.

Cuándo utilizar patrones

Se deben usar patrones todo el tiempo. Siempre que se pretenda desarrollar algo relacionado con análisis, diseño, codificación o administración de proyectos, se deberán buscar patrones que pueden ser de utilidad.

Para mayor información

En la actualidad, el campo de los patrones aún es muy joven, por lo cual no hay mucho material (lo cual es una ventaja en cierto modo, ya que la comunidad de los patrones no ha determinado aún la manera de indizar el material).

La fuente central de información sobre patrones es la Patterns Home Page (Página base de patrones) en la red: <**http://st-www.cs.uiuc.edu/users/patterns/patterns.html**>. Aquí encontrará información clave sobre libros, conferencias y demás. Hay una serie de

patrones e información acerca de éstos en la página *Portland Patterns Repository* (Depósito de patrones de Portland) de Ward Cunningham: <**http://c2.com/ppr/index.html**>.

El libro más influyente sobre patrones es Gamma *et al.* (1994) que es un libro de patrones de diseño. También pueden encontrarse patrones de diseño en Buschmann (1996), así como patrones arquitectónicos de alto nivel. Estos libros analizan la operación de canalizaciones y filtros, la arquitectura de pizarrón y el trabajo de reflexión. A un nivel más bajo, se pueden encontrar libros, como el de Kent Beck sobre patrones de Smalltalk (1996), sobre patrones para lenguajes específicos de programación. Muchos de los patrones de Beck sirven para otros lenguajes. Para el modelado de dominios, debo sugerir mi libro (Fowler 1997) sobre patrones de análisis.

La comunidad de patrones sostiene conferencias regulares sobre Lenguajes de programación de patrones (*Pattern Languages of Programming*, o PLoP), que están especialmente diseñadas para ayudar a los escritores de patrones. Una selección de patrones de estas conferencias se publica en la serie de libros PLoPD (Coplien and Schmidt 1995 y Vlissides *et al.* 1996), los cuales incluyen muchos artículos valiosos.

El UML incluye un poco de notación para describir el empleo de un patrón de diseño. No entraré en detalles aquí sobre esto, ya que apenas está en las primeras etapas de su utilización, pero se podrá encontrar más sobre el particular en los libros de los tres amigos.

Transición

De lo que se trata en el desarrollo iterativo es de hacer todo el proceso de desarrollo consistentemente, de tal modo que el equipo del desarrollo se acostumbre a entregar código terminado. Pero hay varias cosas que no deben hacerse demasiado pronto. Un ejemplo primordial es la optimización.

La optimización reduce la claridad y la capacidad de ampliación del sistema con el objeto de mejorar el desempeño. Ésta es una concesión necesaria; a fin de cuentas, un sistema debe ser lo suficientemente rápido para satisfacer las necesidades de los usuarios, pero la optimización demasiado temprana hace más complicado el desarrollo. Así que esto es algo que ha de dejarse para el final.

Durante la transición, no se hacen desarrollos para añadir funciones nuevas (a menos que sean pequeñas y absolutamente indispensables). Ciertamente, *sí* hay desarrollo para depuración.

Un buen ejemplo de una fase de transición es el tiempo entre la liberación beta y la liberación definitiva del producto.

Cuándo se debe usar el desarrollo iterativo

El desarrollo iterativo únicamente se debe utilizar en aquellos proyectos que se quiere que tengan éxito.

Tal vez esto suene un tanto absurdo, pero a medida que envejezco, me he vuelto más intransigente en cuanto al manejo del desarrollo iterativo. Bien realizado, es una técnica esencial, que puede servir para exponer pronto los riesgos y lograr un mejor control sobre el desarrollo. No es lo mismo que no contar con alguna administración (aunque, para ser sinceros, debo señalar que algunos lo han usado de ese modo). Debe planificarse bien. Pero se trata de un enfoque sólido y todos los libros sobre el desarrollo de la OO lo recomiendan, por buenas razones.

Para mayor información

El punto de partida obvio es el libro de los tres amigos acerca del proceso.

Entre mis fuentes favoritas están Goldberg y Rubin (1995), Booch (1994), McConnell (1996) y Graham (1993).

- Goldberg y Rubin hablan mucho sobre los principios generales y cubren mucho terreno.
- El libro de Booch es más directo. Cuenta lo que hace su autor y da muchos consejos.
- McConnell también presenta muchos consejos, en general ajustados al tipo de proceso descrito en este capítulo.
- Si desea una metodología completamente definida, paso a paso, del desarrollo iterativo, el mejor material que he consultado sobre estos temas se encuentra en la obra de Graham.

Rational Software tiene una descripción no publicada de la versión actual del proceso Objectory como producto. Entre los libros publicados, la descripción más cercana es la de Jacobson (1994 y 1995). También debe dar una ojeada a los artículos sobre patrones contenidos en Coplien y Schmidt (1995), así como el artículo sobre "episodios" de Ward Cunningham (1996).

Kent Beck está trabajando en un libro sobre patrones de administración de proyectos. Cuando se publique, sin duda será un recurso excelente. De hecho, muchas ideas en este capítulo surgieron en conversaciones con él y con Ward Cunningham, así como en conversaciones telefónicas con Ivar Jacobson.

Capítulo 3

Los casos de uso

Los casos de uso son un fenómeno interesante. Durante mucho tiempo, tanto en el desarrollo orientado a objetos como en el tradicional, las personas se auxiliaban de escenarios típicos que les ayudaban a comprender los requerimientos. Estos escenarios, sin embargo, se trataban de modo muy informal; siempre se construían, pero pocas veces se documentaban. Ivar Jacobson es ampliamente conocido por haber cambiado esto con su método Objectory y su primer libro sobre el tema.

Jacobson elevó la visibilidad del caso de uso (su nombre para un escenario) a tal punto que lo convirtió en un elemento primario de la planificación y el desarrollo de proyectos. Desde la publicación de su libro (1994), la comunidad de los objetos ha adoptado los casos de uso en un grado notable. El ejercicio de mi profesión ciertamente ha mejorado gracias a que comencé a emplear los casos de uso de este modo.

Veamos, pues, ¿qué es un caso de uso?

Un **caso de uso** es, en esencia, una interacción típica entre un usuario y un sistema de cómputo. Considérese el procesador de palabras con el que escribo estas líneas que usted lee. Dos casos de uso típicos serían "pon una parte del texto en negritas" y "crea un índice". Por medio de estos ejemplos, se puede uno dar una idea de ciertas propiedades de los casos de uso.

- El caso de uso capta alguna función visible para el usuario.
- El caso de uso puede ser pequeño o grande.
- El caso de uso logra un objetivo discreto para el usuario.

En su forma más simple, el caso de uso se obtiene hablando con los usuarios habituales y analizando con ellos las distintas cosas que deseen hacer con el sistema. Se debe abordar cada cosa discreta que quieran, darle un nombre y escribir un texto descriptivo breve (no más de unos cuantos párrafos).

Durante la elaboración, esto es todo lo que necesitará para empezar. No trate de tener todos los detalles justo desde el principio; los podrá obtener cuando los necesite. Sin embargo, si considera que un caso de uso dado tiene ramificaciones arquitectónicas de importancia, necesitará más detalles a la mano. La mayoría de los casos de uso se pueden detallar durante la iteración dada, a medida que se construyen.

Objetivos del usuario e interacciones con el sistema

Un tema importante que me he encontrado en los casos de uso es la diferencia entre lo que llamo objetivos del usuario e interacciones con el sistema. Por ejemplo, considere la funcionalidad de la style sheet hoja de estilos con que cuentan la mayor parte de los procesadores de texto.

Las **interacciones con el sistema** permiten decir que los casos de uso incluirán cosas como: "define estilo", "cambia estilo", y "mueve un estilo de un documento a otro". Sin embargo, estos casos de uso reflejan más bien cosas que el usuario hace con el sistema, en lugar de los verdaderos objetivos que el usuario trata de conseguir. Los verdaderos **objetivos del usuario** se describirían con términos como "garantizar el formateo consistente de un documento" y "hacer que el formato de un documento sea igual que el de otro".

Esta dicotomía entre objetivo del usuario e interacción con el sistema no se presenta en todas las situaciones. Por ejemplo, el proceso de indización de un documento es muy parecido si se le considera como objetivo del usuario o como interacción con el sistema. No obstante, cuando los objetivos del usuario y las interacciones del sistema difieren, es muy importante tener clara la diferencia.

Ambos estilos de casos de uso tienen sus aplicaciones. Los casos de uso de interacción con el sistema sirven más para fines de planificación; conviene no perder de vista los objetivos del usuario, con el fin de poder considerar formas alternas para el cumplimiento de tales objetivos. Si se llega demasiado pronto a la interacción con el sistema, recurriendo a la primera alternativa obvia, se pasarán por alto otras maneras creativas de cumplir con mayor eficacia los objetivos del usuario. En todos los casos, es una buena idea preguntarse "¿por qué hicimos esto?" Esta pregunta generalmente conduce a una mejor comprensión del objetivo del usuario.

En mi trabajo, me centro primero en los objetivos del usuario y después me ocupo de encontrar casos de uso que los cumplan. Hacia el final del periodo de elaboración, espero tener por lo menos un conjunto de casos de uso de interacción con el sistema por cada objetivo de usuario que he identificado (como mínimo, para las metas del usuario que pretendo manejar en la primera entrega).

Diagramas de casos de uso

Jacobson (1994), además de introducir los casos de uso como elementos primarios del desarrollo del software, también diseñó un diagrama para la representación gráfica de los casos de uso. **El diagrama de casos de uso** es ya también parte del UML.

La figura 3-1 muestra algunos de los casos de uso de un sistema de financiamiento.

Comenzaré el análisis de los elementos de este diagrama hablando sobre los actores.

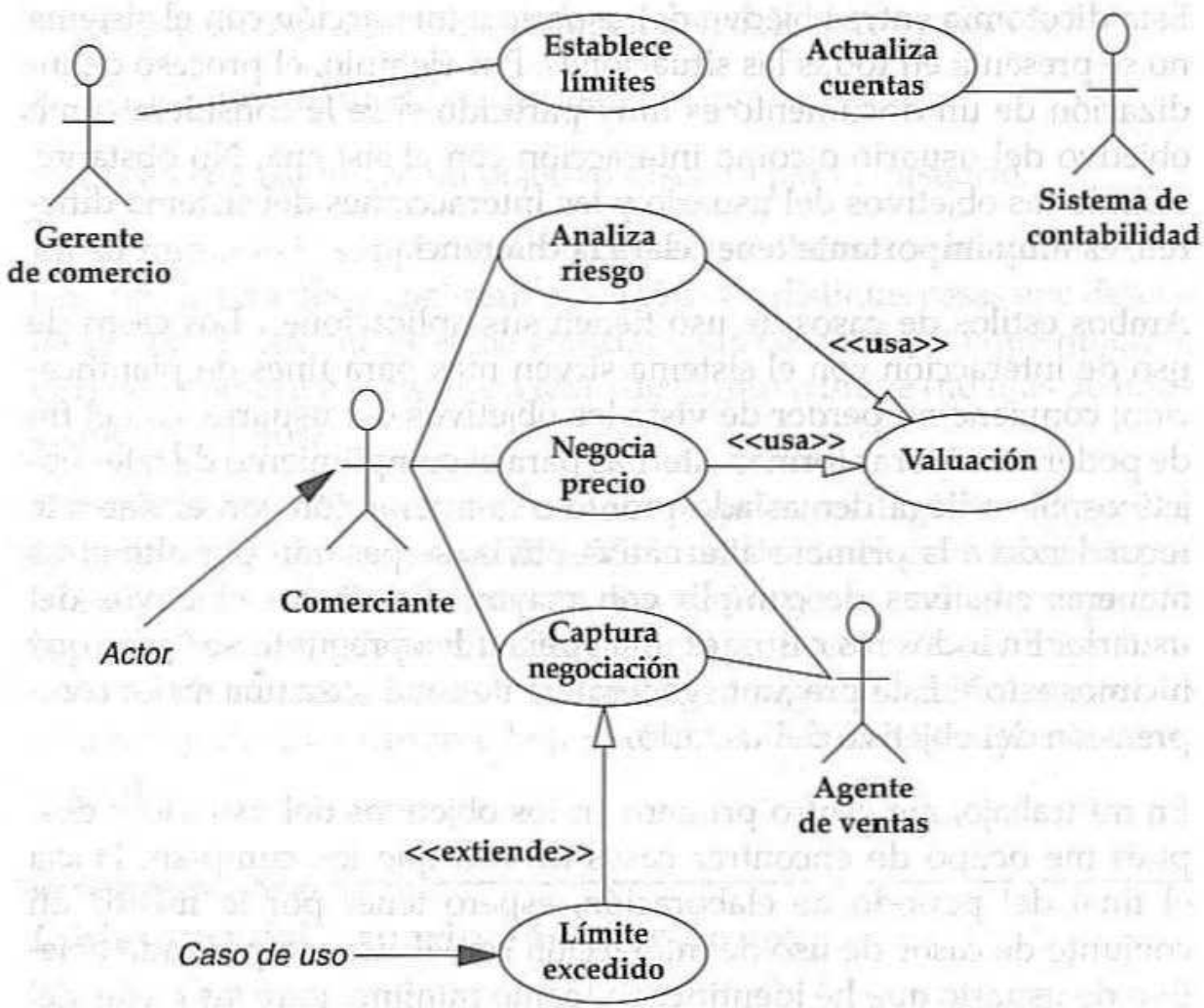


Figura 3-1: *Diagrama de casos de uso*

Actores

Empleamos el término **actor** para llamar así al usuario, cuando desempeña ese papel con respecto al sistema. Hay cuatro actores en la figura 3-1: el gerente de comercio, el comerciante, el agente de ventas y el sistema de contabilidad.

En la mencionada organización, probablemente habrá diversos comerciantes. Además, un usuario puede desempeñar varios papeles. Por ejemplo, un comerciante de edad madura podría desempeñar el papel de gerente de comercio y además ser un comerciante normal. Por otra parte, un comerciante puede ser también agente de ventas. Cuando se trata con actores, conviene pensar en los papeles, no en las personas ni en los títulos de sus puestos.

Los actores llevan a cabo casos de uso. Un mismo actor puede realizar muchos casos de uso; a la inversa, un caso de uso puede ser realizado por varios actores.

En la práctica, considero que los actores son muy útiles cuando se trata de definir los casos de uso. Al enfrentarse con un sistema grande, puede ser difícil obtener una lista de casos de uso. Es más fácil en tales situaciones definir la lista de los actores y después tratar de determinar los casos de uso de cada actor.

Obsérvese que no es necesario que los actores sean seres humanos, a pesar de que los actores estén representados por figuras humanas en el diagrama del caso de uso. El actor puede ser también un sistema externo que necesite cierta información del sistema actual. En la figura 3-1 se puede apreciar la necesidad de actualizar las cifras del sistema de contabilidad.

El tema de las interacciones con sistemas externos produce mucha confusión y variaciones de estilo entre los usuarios de los diagramas de casos de uso.

1. Algunos sienten que todas las interacciones con sistemas remotos deben aparecer en el diagrama. Por ejemplo, si se necesita acceder a Reuters con el fin de cotizar un contrato, se deberá mostrar el vínculo entre el caso Negocia precio y Reuters.
2. Algunas personas consideran que sólo se deben mostrar los casos de uso con interacción externa, cuando quien inicia el contacto es otro sistema. Según esta regla, sólo se mostraría el caso de uso del sistema de contabilidad si dicho sistema invocara algún proceso que le dijera al sistema fuente que lo hiciera.
3. Algunas personas consideran que sólo se deben mostrar los actores del sistema cuando ellos sean los que necesiten el caso de uso. Por tanto, si el sistema genera cada noche un archivo que después es recogido por el sistema de contabilidad, entonces éste es el actor que corresponde, debido a que es quien necesita el archivo generado.
4. Otros más sienten que constituye un enfoque equivocado considerar que un sistema es un actor. Por el contrario, consideran que un actor es un usuario que desea algo del sistema (por ejemplo, un ar-

chivo en particular). En el caso de nuestro ejemplo de sistema, los actores serían los auditores internos de la compañía.

Tomando en cuenta todos los factores, me inclino por la opción 3.

Todos los casos de uso tratan sobre funcionalidad requerida externamente. Si el sistema de contabilidad necesita un archivo, entonces ése es un requerimiento que debe satisfacerse.

El acceso a Reuters es importante, pero no una necesidad del usuario. Si sigue la opción 4, se termina por analizar el sistema de contabilidad, cosa que probablemente usted no quisiera hacer. Dicho lo anterior, siempre se deben cuestionar los casos de uso con los actores del sistema, descubrir los objetivos reales del usuario y considerar formas alternas para lograrlos.

Cuando trabajo con actores y casos de uso, no me preocupo mucho por la relación exacta entre ellos. Lo que me interesa casi siempre son los casos de uso; los actores son sólo un modo de llegar a ellos. Siempre y cuando obtenga todos los casos de uso, no me preocupan los detalles acerca de los actores.

Sin embargo, una situación en la que los actores sí desempeñan un papel es en la configuración del sistema para varios tipos de usuarios. Si el sistema tiene casos de uso que corresponden a funciones de usuario de alto nivel, usted puede emplear los vínculos entre casos de uso y actores para hacer los perfiles de los usuarios. Cada usuario tendría una lista asociada de nombres de actores con la que se determinarían los casos de uso que puede ejecutar cada uno.

Otra buena razón para rastrear a los actores es la necesidad de saber cuáles son los casos de uso que quiere cada uno. Esto puede ser importante cuando usted esté evaluando las necesidades que compiten entre ellos. La comprensión de los actores puede servir para negociar entre demandas de desarrollo que compiten entre sí. También pueden ser útiles para especificar una política de seguridad.

Algunos casos de uso no tienen vínculos claros con actores específicos. Considérese una compañía de servicios públicos. Por supuesto, uno de sus casos de uso es "envío de factura". No es tan fácil, sin embargo, identificar un actor asociado. No existe un papel de usuario en

particular que solicite una factura. La factura se envía al cliente, pero el cliente no protestaría si esto no se llevara a cabo. Lo que más se parece a un actor es el Departamento de facturación, en el sentido de que obtiene un valor del caso de uso. Pero la facturación generalmente no interviene en la ejecución del caso de uso.

Los actores pueden tener varios papeles con respecto a un caso de uso. Pueden ser los que obtienen un valor del caso de uso, o tal vez sólo participen en él. Dependiendo de cómo se utilice la relación entre los actores será la importancia de los papeles que desempeñen los actores. Por mi parte, suelo preocuparme más por controlar el desarrollo del sistema. Así, en general, siento un mayor interés por quién quiere que se construya un caso de uso (por lo general, quienes obtienen un valor del caso de uso).

La clave es tener en cuenta que algunos casos de uso no saltarán a la vista como resultado de ponerse a pensar en los casos de uso de cada actor. Si esto sucede, no se preocupe demasiado. Lo importante es comprender los casos de uso y los objetivos del usuario que satisfacen.

Una buena fuente para identificar los casos de uso son los eventos externos. Piense en todos los eventos del mundo exterior ante los cuales usted quiera reaccionar. Un evento dado puede provocar una reacción en el sistema en la que no intervenga el usuario, o puede causar una reacción que provenga principalmente de los usuarios. La identificación de los eventos ante los que se necesitará reaccionar será de ayuda en la identificación de los casos de uso.

Uses y extends

Además de los vínculos entre los actores y los casos de uso, hay otros dos tipos de vínculos en la figura 3-1. Éstos representan las relaciones de uses (usa) y extends (extiende) entre los casos de uso. Con frecuencia, tales relaciones son fuente de confusión para quienes mezclan los significados de ambos conceptos, de modo que tómese el tiempo necesario para comprenderlos.

Se usa la relación **extends** cuando se tiene un caso de uso que es similar a otro, pero que hace un poco más.

En nuestro ejemplo, el caso de uso es Captura negociación. Éste es un caso en que todo sucede sin contratiempos. Sin embargo, hay situaciones que pueden estropear la captura de una negociación. Una de ellas surge cuando se excede algún límite, por ejemplo, la cantidad máxima que la organización de comercio ha establecido para un cliente en particular. En este ejemplo, dado que no efectuamos el comportamiento habitual asociado con dicho caso de uso, efectuamos una variación.

Podríamos poner esta variación dentro del caso de uso Captura negociación. Sin embargo, esto llenaría dicho caso de uso de una gran cantidad de lógica especial, la cual oscurecería el flujo "normal".

Otro modo de abordar la variación es poner la conducta normal en un caso y la conducta inusual en cualquier otro lado. A continuación describiremos la esencia de la relación extends.

1. Primero obtenga el caso de uso simple y normal.
2. En cada paso de ese caso de uso, pregúntese "¿qué puede fallar aquí?", "¿cómo podría funcionar esto de modo diferente?"
3. Dibuje todas las variaciones como extensiones del caso de uso dado. Con frecuencia habrá un buen número de extensiones. Si se listan por separado serán mucho más fáciles de entender.

Puede suceder que se tenga qué dividir un caso de uso tanto en la etapa de elaboración como en la de construcción. Durante la elaboración, suelo dividir los casos de uso que se estén volviendo muy complicados. No obstante, hay otros casos de uso cuya complejidad no abordo a fondo, sino hasta llegar a la construcción.

En la etapa de construcción del proyecto, realizo la división cuando no puedo construir un caso de uso completo en una sola iteración. Divido los casos de uso complejos en un caso normal y unas cuantas extensiones, y luego construyo el caso normal en una sola iteración y las extensiones como partes de una o varias iteraciones posteriores. (Por supuesto, aquí sucederán cambios en el plan de compromisos, lo que deberá negociarse con los usuarios.)

Las relaciones **uses** ocurren cuando se tiene una porción de comportamiento que es similar en más de un caso de uso y no se quiere copiar

la descripción de tal conducta. Por ejemplo, tanto Analiza riesgo como Negociación precio requieren valuar la negociación. La descripción de la valuación de la negociación requiere de bastante escritura, y yo odio las operaciones de copiar y pegar. Por tanto, elaboro un caso de uso separado, Negociación valor, para esta situación y me refiero a él desde los casos de uso originales.

Adviértanse las diferencias entre extends y uses. Ambos implican la factorización de comportamientos comunes de varios casos de uso, dejando un solo caso de uso común que es empleado, o extendido, por otros varios casos de uso. No obstante, la *intención* es la que cambia.

En sus vínculos con los actores, estos dos tipos de relación implican asuntos diferentes. Tratándose de la relación extends, los actores tienen que ver con los casos de uso que se están extendiendo. Se supone que un actor dado se encargará tanto del caso de uso base como de todas las extensiones. En cuanto a las relaciones uses, es frecuente que no haya un actor asociado con el caso de uso común. Incluso si lo hay, no se considera que esté llevando a cabo los demás casos de uso.

Aplique las siguientes reglas.

- Utilice extends cuando describa una variación de conducta normal.
- Emplee uses para repetir cuando se trate de uno o varios casos de uso y desee evitar repeticiones.

Es probable que oiga usted el término **escenario** en conexión con los casos de uso. Cabe aclarar que este término se emplea de manera inconsistente. Algunas veces, escenario es usado como sinónimo de caso de uso. En el contexto del UML, la palabra escenario se refiere a una sola ruta a través de un caso de uso, una ruta que muestra una particular combinación de condiciones dentro de dicho caso de uso. Por ejemplo, para ordenar algunas mercancías, tendremos un solo caso de uso con varios escenarios asociados: uno en el cual todo va bien; otro donde no hay suficientes mercancías; otro en el que nuestro crédito es rechazado, y así por el estilo.

Conforme vaya realizando sus tareas de modelado, encontrará modelos que expresen la manera de hacer sus casos de uso, ya sea entre el software o entre la gente. Es evidente que hay más de una manera de

llevar a cabo un caso de uso. En la jerga del UML, decimos que un caso de uso puede tener muchas **realizaciones**.

Quizá suceda con frecuencia que usted desee bosquejar varias realizaciones, con el fin de discutirlas y determinar con cuál va a trabajar. Si hace esto, recuerde guardar información acerca de las realizaciones descartadas, incluyendo las notas de por qué se descartaron. No quisiera contarles cuántas horas he desperdiciado en discusiones del estilo de "yo *sé* que existe una razón por la que no hicimos eso, pero..."

Cuándo emplear casos de uso

No puedo imaginar en este momento una situación en la que no emplearía los casos de uso. Son una herramienta esencial para la captura de requerimientos, la planificación, o el control de proyectos iterativos. La captura de los casos de uso es una de las tareas principales durante la fase de elaboración; de hecho, es lo primero que se debe hacer.

La mayoría de los casos de uso se generarán durante esa fase del proyecto, pero irá descubriendo otros a medida que avance. Esté siempre pendiente de ellos. Todo caso de uso es un requerimiento potencial y hasta que no haya usted capturado un requerimiento, no podrá planear cómo manejarlo en el proyecto.

Algunos prefieren listar y analizar los casos de uso primero, y luego llevar a cabo un poco de modelado. Yo a veces lo hago, pero también he encontrado que el modelado conceptual con los usuarios ayuda a descubrir los casos de uso. Mi recomendación es que se prueben ambas maneras y se escoja la que mejor sirva.

Los diseñadores emplean casos de uso con distintos grados de granularidad. Por ejemplo, Ivar Jacobson dice que en un proyecto de 10 personas-año, él esperaría unos 20 casos de uso (sin contar con las relaciones *use* y *extend*). En un proyecto reciente de la misma magnitud, tuve más de 100 casos de uso. Prefiero los casos de uso con menor granularidad, pues con ellos es más fácil trabajar sin perder de vista el calendario de compromisos. Sin embargo, demasiados casos de uso pueden acabar siendo abrumadores. Por el momento no creo

que haya una respuesta correcta, de modo que le recomiendo que sea flexible y trabaje con lo que le resulte más cómodo.

Para mayor información

Me parece que el mundo sigue a la espera de un libro realmente bueno sobre casos de uso. Por supuesto, el primer libro de Jacobson (1994) es una fuente valiosa, ya que constituye en realidad el libro con el que comenzó todo. El siguiente libro de Jacobson (1995) es útil por su énfasis en los casos de uso de procesos de negocios (que, según opiniones encontradas, deben usarse todo el tiempo). Ian Graham (1993) también incluye algunos buenos consejos (él emplea el término "*script*" en lugar del de "caso de uso"). Además, le recomiendo que consulte los textos de casos de uso en el sitio de Web de Alistair Cockburn: **<http://members.aol.com/acockburn>**.

Capítulo 4

Diagramas de clase: fundamentos

La técnica del **diagrama de clase** se ha vuelto medular en los métodos orientados a objetos. Virtualmente, todos los métodos han incluido alguna variación de esta técnica.

El **diagrama de clase**, además de ser de uso extendido, también está sujeto a la más amplia gama de conceptos de modelado. Aunque los elementos básicos son necesarios para todos, los conceptos avanzados se usan con mucha menor frecuencia. Por eso, he dividido mi estudio de los diagramas de clase en dos partes; los fundamentos (en el presente capítulo) y los conceptos avanzados (véase el capítulo 5).

El **diagrama de clase** describe los tipos de objetos que hay en el sistema y las diversas clases de relaciones estáticas que existen entre ellos. Hay dos tipos principales de relaciones estáticas:

- **asociaciones** (por ejemplo, un cliente puede rentar diversas videocintas).
- **subtipos** (una enfermera es un tipo de persona).

Los diagramas de clase también muestran los atributos y operaciones de una clase y las restricciones a que se ven sujetos, según la forma en que se conecten los objetos.

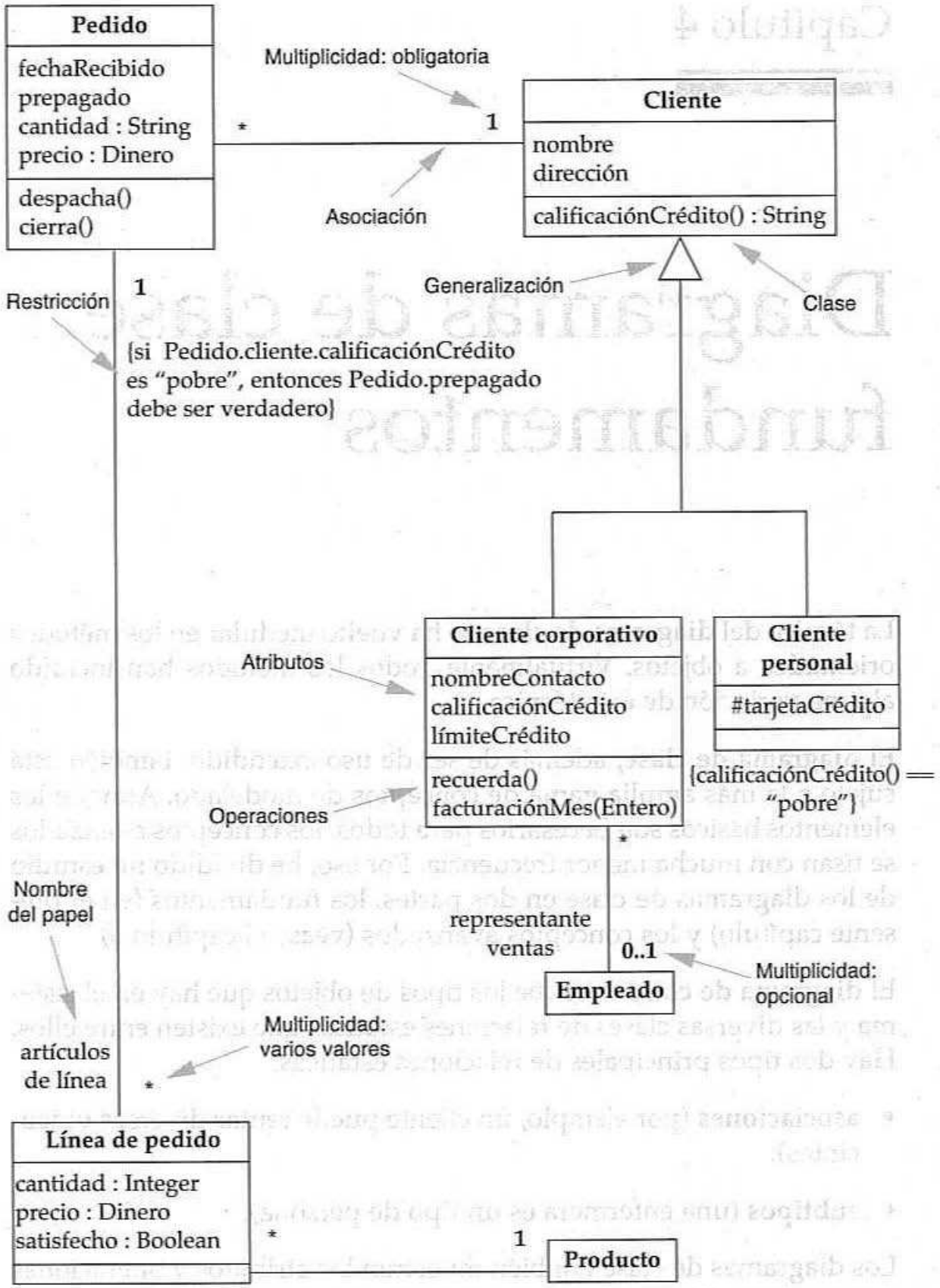


Figura 4-1: *Diagrama de clase*

Los diversos métodos OO utilizan terminologías diferentes (y con frecuencia antagónicas) para estos conceptos. Se trata de algo sumamente frustrante pero inevitable, dado que los lenguajes OO son tan desconsiderados como los métodos. Es en esta área que el UML aportará algunos de sus mayores beneficios, al simplificar estos diferentes diagramas.

La figura 4-1 muestra un diagrama de clase típico.

Perspectivas

Antes de empezar a describir los diagramas de clase, quisiera señalar una importante sutileza sobre el modo en que se usan. Tal sutileza generalmente no está documentada, pero tiene sus repercusiones en el modo en que debe interpretarse un diagrama, ya que se refiere a lo que se va a describir con un modelo.

Siguiendo la recomendación de Steve Cook y John Daniels (1994), considero que hay tres perspectivas que se pueden manejar al dibujar diagramas de clase (o, de hecho, cualquier modelo, aunque esta división se advierte de modo especial en relación con los diagramas de clase).

- **Conceptual**. Si se adopta la perspectiva conceptual, se dibuja un diagrama que represente los conceptos del dominio que se está estudiando. Estos conceptos se relacionan de manera natural con las clases que los implementan, pero con frecuencia no hay una correlación directa. De hecho, los modelos conceptuales se deben dibujar sin importar (o casi) el software con que se implementarán, por lo cual se pueden considerar como independientes del lenguaje. (Cook y Daniels llaman perspectiva esencial a esto; por mi parte, empleo el término "conceptual", pues se ha usado durante mucho tiempo.)
- **Especificación**. Ahora estamos viendo el software, pero lo que observamos son las interfaces del software, no su implementación. Por tanto, en realidad vemos los tipos, no las clases. El desarrollo orientado a objetos pone un gran énfasis en la diferencia entre in-

terfaz e implementación, pero esto con frecuencia se pasa por alto en la práctica, ya que el concepto de clase en un lenguaje OO combina tanto la interfaz como la implementación. Así, a menudo se hace referencia a las interfaces como tipos y a la implementación de esas interfaces como clases. Influidos por este manejo del lenguaje, la mayor parte de los métodos han seguido este camino. Esto está cambiando (Java y CORBA tendrán aquí cierta influencia), pero no con suficiente rapidez. Un tipo representa una interfaz que puede tener muchas implementaciones distintas debido, por ejemplo, al ambiente de implementación, características de desempeño y proveedor. La distinción puede ser muy importante en diversas técnicas de diseño basadas en la delegación; véase el estudio de este tema en Gamma *et al.* (1994).

- **Implementación**. Dentro de esta concepción, realmente tenemos clases y exponemos por completo la implementación. Ésta es, probablemente, la perspectiva más empleada, pero en muchos sentidos es mejor adoptar la perspectiva de especificación.

La comprensión de la perspectiva es crucial tanto para dibujar como para leer los diagramas de clase. Desafortunadamente, las divisiones entre las perspectivas no son tajantes y la mayoría de los modeladores no se preocupa por definirlas con claridad cuando las dibujan. A medida que ahonde en mi exposición de los diagramas de clase, enfatizaré cómo cada elemento de la técnica depende en gran medida de la perspectiva.

Cuando usted dibuje un diagrama, hágalo desde el punto de vista de una sola perspectiva clara. Cuando lea un diagrama, asegúrese de saber desde qué perspectiva se dibujó. Dicho conocimiento es esencial si se quiere interpretar correctamente el diagrama.

La perspectiva no es parte del UML formal, pero considero que es extremadamente valiosa al modelar y revisar los modelos. El UML se puede utilizar con las tres perspectivas. Mediante el etiquetado de las clases con un estereotipo (véase la página 86), se puede dar una indicación clara de la perspectiva. Las clases se marcan con <<clase

Tabla 4-1: *Terminología de diagramas de clase*

UML	Clase	Asociación	Generalización	Agregación
Booch	Clase	Usa	Hereda	Contiene
Coad	Clase y objeto	Conexión de instancias	Espec-gen	Parte-todo
Jacobson	Objeto	Asociación por reconocimiento	Hereda	Consiste en
Odell	Tipo de objeto	Relación	Subtipo	Composición
Rumbaugh	Clase	Asociación	Generalización	Agregación
Shlaer/ Mellor	Objeto	Relación	Subtipo	N/A

de implementación>> para mostrar la perspectiva de implementación y con <<tipo>> para el caso de la perspectiva de especificación y la conceptual. La mayor parte de los usuarios de los métodos OO adopta una perspectiva de implementación, lo cual es de lamentar, porque las otras perspectivas con frecuencia son más útiles.

La tabla 4-1 lista cuatro términos del UML que aparecen en la figura 4-1 y sus términos correspondientes en otras metodologías bien establecidas.

Asociaciones

La figura 4-1 muestra un modelo de clases simple que no sorprenderá a nadie que haya trabajado con un proceso de pedidos. Describiré sus partes y hablaré sobre las maneras de interpretarlas, desde las distintas perspectivas.

Comenzaré con las asociaciones. Las **asociaciones** representan relaciones entre instancias de clases (una persona trabaja para una empresa; una empresa tiene cierta cantidad de oficinas).

Desde la perspectiva **conceptual**, las asociaciones representan relaciones conceptuales entre clases. El diagrama indica que un Pedido debe venir de un solo Cliente y que un Cliente puede hacer varios Pedidos en un periodo de tiempo. Cada uno de estos Pedidos tiene varias instancias de Línea de pedido, cada una de las cuales se refiere a un solo Producto.

Cada asociación tiene dos **papeles**; cada papel es una dirección en la asociación. De este modo, la asociación entre Cliente y Pedido contiene dos papeles: uno del Cliente al Pedido; el segundo del Pedido al Cliente.

Se puede nombrar explícitamente un papel mediante una etiqueta. En este caso, el papel en sentido Pedido a Línea de orden se llama Artículos de línea. Si no hay etiqueta, el papel se puede nombrar de acuerdo con la etiqueta de la clase; de esta forma, el papel de Pedido a Cliente se llamará Cliente (en este libro, me refiero a la clase de donde parte el papel como **origen** y a la clase a donde va el papel como **destino**. Esto significa que hay un papel de Cliente cuyo origen es Pedido y cuyo destino es Cliente).

Un papel tiene también una **multiplicidad**, la cual es una indicación de la cantidad de objetos que participarán en la relación dada. En la figura 4-1, la * entre Cliente y Pedido indica que el primero puede tener muchas Órdenes asociadas a él; el 1 indica que un Pedido viene de un solo Cliente.

En general, la multiplicidad indica los límites inferior y superior de los objetos participantes. El * representa de hecho el intervalo *0...infinito*: el Cliente no necesita haber colocado un Pedido, y no hay un tope superior (por lo menos en teoría) para la cantidad de pedidos que puede colocar. El 1 equivale a 1..1: cada Pedido debe haber sido solicitado por un solo Cliente.

En la práctica, las multiplicidades más comunes son 1, *, y 0..1 (se puede no tener ninguno o bien tener uno). Para una multiplicidad más general, se puede tener un solo número (por ejemplo, *11* jugadores en un equipo de fútbol), un intervalo (por ejemplo, *2..4* para los participantes de un juego de canasta) o combinaciones discretas de números e intervalos (por ejemplo, 2, 4 para las puertas de un automóvil).

La figura 4-2 muestra las notaciones de cardinalidad del UML y de los principales métodos pre-UML.

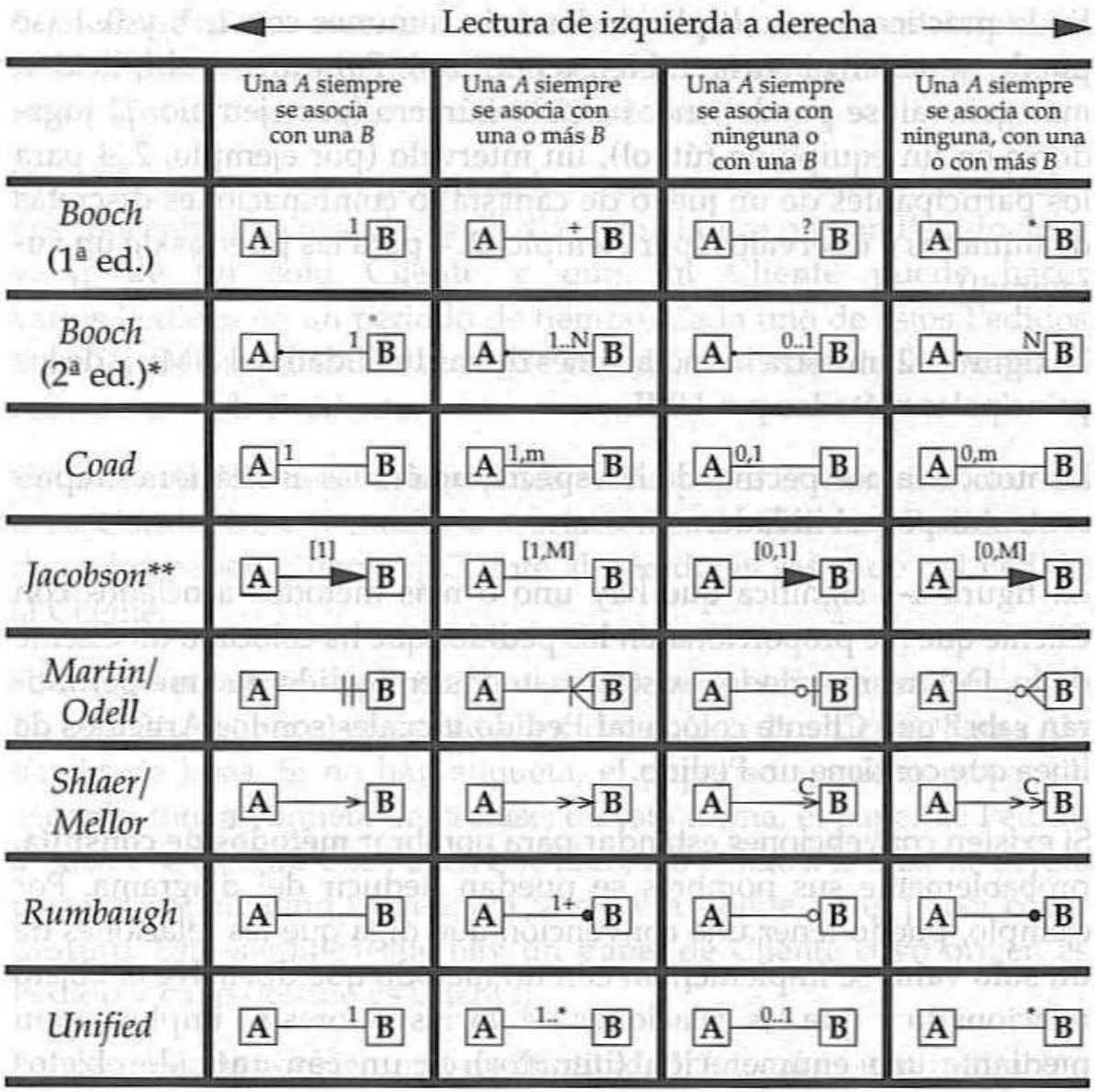
Dentro de la perspectiva de la **especificación**, las asociaciones representan responsabilidades.

La figura 4-1 significa que hay uno o más métodos asociados con Cliente que me proporcionarán los pedidos que ha colocado un Cliente dado. Del mismo modo, existen métodos en Pedido que me permitirán saber qué Cliente colocó tal Pedido y cuales son los Artículos de línea que contiene un Pedido.

Si existen convenciones estándar para nombrar **métodos de consulta**, probablemente sus nombres se puedan deducir del diagrama. Por ejemplo, puedo tener una convención que diga que las relaciones de **un solo valor** se implementan con un método que devuelve el objeto relacionado y que las relaciones de varios valores se implementan mediante una enumeración (iterador) en un **conjunto** de objetos relacionados.

Al trabajar con una convención de nombres como ésta en Java, por ejemplo, puedo deducir la siguiente interfaz para una clase de Pedido:

```
class Pedido {
        public Cliente cliente() ;
        // Enumeración de líneas de Pedido
        public Enumeration lineasPedido() ;
```



* puede ser unidireccional

** unidireccional

Figura 4-2: *Notaciones de cardinalidad*

Las convenciones de programación, por supuesto, variarán de acuerdo con el lugar y no indicarán todos los métodos, pero le serán a usted de gran utilidad para encontrar el camino.

La figura 4-1 implica también cierta responsabilidad al poner al día la relación. Debe haber algún modo de relacionar el Pedido con el Cliente. De nuevo, no se muestran los detalles; podría ser que se especifique el Cliente en el constructor del Pedido. O, tal vez, exista un método

agregarPedido asociado al Cliente. Esto se puede hacer más explícito añadiendo operaciones al cuadro de clase (tal y como veremos más adelante).

Sin embargo, estas responsabilidades *no* implican una estructura de datos. A partir de un diagrama a nivel de especificación, no puedo hacer suposición alguna sobre la estructura de datos de las clases. No puedo, ni debo poder decir si la clase Pedido contiene un apuntador a Cliente, o si la clase Pedido cumple su responsabilidad ejecutando algún código de selección que pregunte a cada Cliente si se refiere a un Pedido dado. El diagrama sólo indica la interfaz, y nada más.

Si éste fuera un modelo de **implementación**, ahora daríamos a entender con ello que hay apuntadores en ambos sentidos entre las clases relacionadas. El diagrama diría entonces que Pedido tiene un campo que es un conjunto de apuntadores hacia Línea de pedido y también un apuntador a Cliente. En Java, podríamos deducir algo como lo que exponemos a continuación:

```
class Pedido {
        private Cliente _cliente;
        private Vector _lineasPedido;
class Cliente {
        private Vector _pedidos;
```

En este caso, no podemos inferir nada sobre la interfaz a partir de las asociaciones. Esta información nos la darían las operaciones sobre la clase.

Ahora véase la figura 4-3. Es básicamente la misma que la figura 4-1, a excepción de que he añadido un par de flechas a las líneas de asociación. Estas líneas indican **navegabilidad**.

En un modelo de especificación, esto indicaría que un Pedido tiene la responsabilidad de decir a qué Cliente corresponde, pero un Cliente no tiene la capacidad correspondiente para decir cuáles son los pedidos que tiene. En lugar de responsabilidades simétricas, aquí tenemos responsabilidades de un solo lado de la línea. En un diagrama de implementación se indicaría qué Pedido contiene un apuntador a Cliente pero Cliente no apuntaría a Pedido.

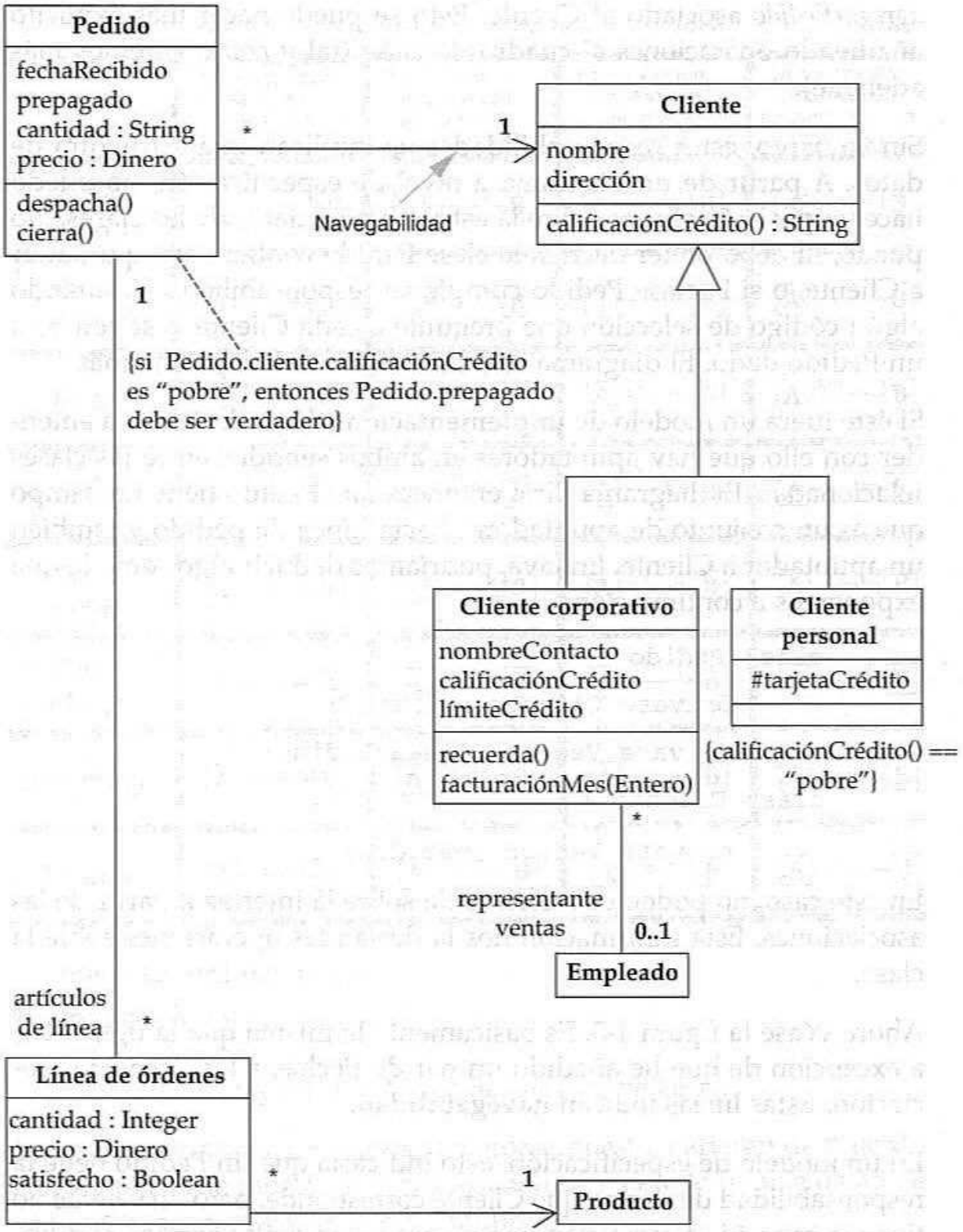


Figura 4-3: *Diagrama de clase con navegabilidades*

Como se podrá apreciar, la navegabilidad es una parte importante de los diagramas de implementación y especificación. Sin embargo, no pienso que la navegabilidad tenga utilidad en los diagramas conceptuales.

Frecuentemente verá diagramas conceptuales que primero no tendrán navegabilidades. Posteriormente, se agregarán las navegabilidades como parte del cambio a perspectivas de especificación y de implementación. Nótese también, que las navegabilidades posiblemente serán diferentes entre la especificación y la implementación.

Si existe una navegabilidad en una sola dirección, a la asociación se le llama **asociación unidireccional**. Una **asociación bidireccional** contiene navegabilidades en ambas direcciones. El UML dice que las asociaciones sin flechas significan que la navegabilidad es desconocida o que la asociación es bidireccional. El proyecto debe definirse en el sentido de uno u otro de los significados. Por mi parte, prefiero que signifique "sin decidir", en el caso de los modelos de especificación e implementación.

Las asociaciones bidireccionales incluyen una restricción adicional, que consiste en que ambos papeles son inversos entre ellos. Esto es igual al concepto de funciones inversas en las matemáticas. En el contexto de la figura 4-3, esto significa que cada Artículo de línea asociado con un Pedido se debe asociar con el Pedido original. De modo similar, si se toma una Línea de pedido y se busca en los artículos de línea el Pedido asociado, se deberá ver la Línea de pedido original en el conjunto. Esta cualidad se mantiene verdadera en las tres perspectivas.

Hay varias maneras de nombrar las asociaciones. A los modeladores de datos tradicionales les gusta denominar a una asociación con un verbo, de modo que se pueda emplear la relación en una oración. La mayoría de los modeladores de objetos prefieren valerse de sustantivos para denominar uno u otro papel, pues esta forma corresponde mejor a las responsabilidades y operaciones.

Algunos les dan nombres a todas las asociaciones. Yo prefiero nombrar las asociaciones sólo cuando mejora la comprensión. He visto demasiadas asociaciones con nombres como "tiene" o "se relaciona con". Si no existe un nombre para el papel, considero que su nombre es el de la clase señalada, como indiqué antes.

Atributos

Los **atributos** son muy parecidos a las asociaciones.

Desde un nivel conceptual, el atributo de nombre de un Cliente indica que los Clientes tienen nombres. Desde el nivel de especificación, este atributo indica que un objeto Cliente puede decir su nombre y tiene algún modo de establecer un nombre. En el nivel de implementación, un Cliente tiene un **campo** (también llamado variable de instancia o miembro de datos) para su nombre.

Dependiendo del detalle del diagrama, la notación de un atributo puede mostrar el nombre, el tipo y el valor predeterminado de un atributo (la sintaxis del UML es *visibilidad nombre: tipo = valor por omisión*, donde la *visibilidad* es igual que la de las operaciones, las cuales se describirán en la sección siguiente).

Por lo tanto, ¿cuál es la diferencia entre un atributo y una asociación?

Desde la perspectiva conceptual, no hay diferencia; un atributo sólo lleva consigo otro tipo de notación, la cual puede servir si se estima conveniente. Los atributos siempre son de valor único. Por lo general, los diagramas no indican si los atributos son opcionales u obligatorios (aunque, hablando estrictamente, deberían indicarlo).

La diferencia se da en los niveles de especificación y de implementación. Los atributos implican navegabilidad sólo del tipo al atributo. Es más, queda implícito que el tipo contiene únicamente su propia copia del objeto de atributo, lo que implica que cualquier tipo que se emplee como atributo tendrá un valor más que una semántica de referencia.

Hablaré sobre el valor y los tipos referenciales más adelante. Por el momento, lo mejor es considerar que los atributos son clases simples y pequeñas, tales como cadenas, fechas, objetos de moneda y valores no objeto, como *int* y *real*.

Operaciones

Las **operaciones** son los procesos que una clase sabe llevar a cabo. Evidentemente, corresponden a los métodos sobre una clase. En el nivel de especificación, las operaciones corresponden a los métodos públicos sobre un tipo. En general, no se muestran aquellas operaciones que simplemente manipulan atributos, ya que por lo común, se pueden inferir. Sin embargo, tal vez sea necesario indicar si un atributo dado es de sólo lectura o está **inmutable** congelado (esto es, que su valor nunca cambia). En el modelo de implementación, se podrían mostrar también las operaciones privadas y protegidas.

La sintaxis del UML completa para las operaciones es

visibilidad nombre (lista-de-parámetros) : expresión-tipo-de-dato-a-regresar {cadena-de-propiedades}

donde

- *visibilidad* es + (public), # (protected), o – (private)
- *nombre* es una cadena de caracteres (string)
- *lista-de-parámetros* contiene argumentos (opcionales) cuya sintaxis es la misma que la de los atributos
- *expresión-tipo-de-dato-a-regresar* es una especificación opcional dependiente del lenguaje
- *cadena-de-propiedades* indica valores de propiedad que se aplican a la operación dada

Un ejemplo de operación sería: *+ últimaCantidadDe (valorTipoFenómeno) : Cantidad*

Dentro de los modelos conceptuales, las operaciones no deben tratar de especificar la interfaz de una clase. En lugar de ello, deberán indicar las principales responsabilidades de dicha clase, usando tal vez un par de palabras que sinteticen una responsabilidad de CRC (véase el recuadro).

Tarjetas de CRC

A fines de la década de 1980, uno de los centros más grandes de tecnología de objetos era el laboratorio de investigación de Tektronix, en Portland, Oregon, Estados Unidos. Este laboratorio tenía algunos de los principales usuarios de Smalltalk y muchas de las ideas clave de la tecnología de objetos se desarrollaron allí. Dos de sus programadores renombrados de Smalltalk eran Ward Cunningham y Kent Beck.

Tanto Cunningham como Beck estaban y siguen preocupados por cómo enseñar los profundos conocimientos de Smalltalk que habían logrado. De esta pregunta sobre cómo enseñar objetos surgió la sencilla técnica de las tarjetas de Clase-Responsabilidad-Colaboración (CRC).

En lugar de utilizar diagramas para desarrollar modelos, como lo hacían la mayoría de los metodólogos, Cunningham y Beck representaron las clases en tarjetas 4 x 6 [pulgadas]. Y en lugar de indicar atributos y métodos en las tarjetas, escribieron responsabilidades.

Ahora bien, ¿qué es una responsabilidad? En realidad es una descripción de alto nivel del propósito de una clase. La idea es tratar de eliminar la descripción de pedazos de datos y procesos y, en cambio, captar el propósito de la clase en unas cuantas frases. El que se haya seleccionado una tarjeta es deliberado. No se permite escribir más de lo que cabe en una tarjeta (véase la figura 4-4).

Nombre de la clase

Responsabilidad — *Pedido*	Colaboración
Revisa si hay elementos en existencia	*Línea de pedido*
Determina precio	*Línea de pedido*
Revisa si el pago es válido	*Cliente*
Despacha a la dirección de entrega	

Figura 4-4: *Tarjeta de Clase-Responsabilidad-Colaboración (CRC)*

La segunda C se refiere a los colaboradores. Con cada responsabilidad se indica cuáles son las otras clases con las que se tiene que trabajar para cumplirla. Esto da cierta idea sobre los vínculos entre las clases, siempre a alto nivel.

Uno de los principales beneficios de las tarjetas de CRC es que alientan la disertación animada entre los desarrolladores. Son especialmente eficaces cuando se está en medio de un caso de uso para ver cómo lo van a implementar las clases. Los desarrolladores escogen tarjetas a medida que cada clase colabora en el caso de uso. Conforme se van formando ideas sobre las responsabilidades, se pueden escribir en las tarjetas. Es importante pensar en las responsabilidades, ya que evita pensar en las clases como simples depositarias de datos, y ayuda a que el equipo se centre en comprender el comportamiento de alto nivel de cada clase.

Un error común que veo que comete la gente es generar largas listas de responsabilidades de bajo nivel. Este procedimiento es completamente fallido. Las responsabilidades deben caber sin dificultad en una tarjeta. Yo cuestionaría cualquier tarjeta que tenga más de tres responsabilidades. Plantéese la pregunta de si se deberá dividir la clase y si las responsabilidades se podrían indicar mejor integrándolas en enunciados de un mayor nivel.

Cuándo usar las tarjetas de CRC

Algunos consideran maravillosas las tarjetas de CRC; en cambio, a otros, esta técnica los deja indiferentes.

Yo considero definitivamente que se deberían probar, a fin de saber si al equipo de trabajo le gusta trabajar con ellas. Se deben usar, en particular, si el equipo se ha empantanado en demasiados detalles o si parecen identificar clases apelmazadas y carentes de definiciones claras.

Se pueden emplear diagramas de clase y diagramas de interacciones (véase el capítulo 6) para captar y formalizar los resultados del modelado CRC en un diseño con notación de UML. Asegúrese de

que cada clase en su diagrama de clase tiene un enunciado de sus responsabilidades.

Para mayor información

Desafortunadamente, Cunningham y Beck no han escrito un libro sobre las CRC, pero usted puede encontrar su artículo original (Beck y Cunningham 1989) en la Web (<**http://c2.com/doc/oopsla89/paper.html**>). En general, el libro que mejor describe esta técnica y, de hecho, todo el concepto del uso de responsabilidades es el de Rebeca Wirfs-Brock (1990). Es un libro relativamente viejo según las normas de la OO, pero se ha añejado bien.

Con frecuencia encuentro útil hacer la distinción entre aquellas operaciones que cambian el estado de una clase y aquellas que no lo hacen. Una **consulta** es una operación que obtiene un valor de una clase sin que cambie el estado observable de tal clase. El **estado observable** de un objeto es el estado que se puede determinar a partir de sus consultas asociadas.

Considérese un objeto de Cuenta que calcula su balance a partir de una lista de entradas. Para mejorar el desempeño, Cuenta puede poner en un campo caché el resultado del cálculo del balance, para consultas futuras. Por tanto, si el caché está vacío, la primera vez que se llama a la operación "balance", pondrá el resultado en el campo caché. La operación "balance" cambia así el estado real del objeto Cuenta, pero no su estado observable, pues todas las consultas devuelven el mismo valor, esté o no lleno el campo caché. Las operaciones que sí cambian el estado observable de un objeto se llaman **modificadores**.

Considero útil tener perfectamente clara la diferencia entre consultas y modificadores. Las consultas se pueden ejecutar en cualquier orden, pero la secuencia de los modificadores es más importante. Yo tengo como política evitar que los modificadores devuelvan valores, con el fin de mantenerlos separados.

Otros términos que algunas veces se presentan son: métodos de obtención y métodos de establecimiento. El **método de obtención** (*get-*

ting) es un método que devuelve el valor de un campo (sin hacer nada más). El **método de establecimiento** (*setting*) pone un valor en un campo (y nada más). Desde afuera, el cliente no debe poder saber si una consulta es un método de obtención ni si un modificador es un método de establecimiento. El conocimiento sobre los métodos de obtención y establecimiento está contenido completamente dentro de la clase.

Otra distinción es la que se da entre operación y método. Una **operación** es algo que se invoca sobre un objeto (la llamada de procedimiento), mientras que un **método** es el cuerpo del procedimiento. Los dos son diferentes cuando se tiene polimorfismo. Si se tiene un supertipo con tres subtipos, cada uno de los cuales suplanta la operación "foo" del supertipo, entonces lo que hay es una operación y cuatro métodos que la implementan.

Es muy común que operación y método se empleen indistintamente, pero hay veces en que es necesario precisar la diferencia. En algunas ocasiones, la gente distingue entre una y otro mediante los términos llamada a un método, o declaración de método (en lugar de operación), y cuerpo del método.

Los lenguajes tienen sus propias convenciones para los nombres. En C++, las operaciones se llaman funciones miembro; en Smalltalk, operaciones de métodos. C++ también emplea el término miembros de una clase para denominar las operaciones y métodos de una clase.

Generalización

Un ejemplo característico de **generalización** comprende al personal y las corporaciones clientes de una empresa. Unos y otros tienen diferencias, pero también muchas similitudes. Las similitudes se pueden colocar en una clase general Cliente (el supertipo) con los subtipos siguientes: Cliente personal y Cliente corporativo.

Este fenómeno también está sujeto a diversas interpretaciones, según el nivel de modelado. Por ejemplo, conceptualmente, podemos decir que el Cliente corporativo es un subtipo de Cliente, si todas las instancias de Cliente corporativo, también son, por definición, instancias de

Cliente. Entonces, un Cliente corporativo es un tipo especial de Cliente. El concepto clave es que todo lo que digamos de un Cliente (asociaciones, atributos, operaciones) también vale para un Cliente corporativo.

En un modelo de especificación, la generalización significa que la interfaz del subtipo debe incluir todos los elementos de la interfaz del supertipo. Se dice, entonces, que la interfaz del subtipo se **conforma con** la interfaz del supertipo.

Otra manera de considerar esto involucra el principio de la **sustituibilidad**. Yo debo poder sustituir a un Cliente corporativo con cualquier código que requiera un Cliente y todo debe operar sin problemas. En esencia, esto significa que, si escribo un código suponiendo que tengo un Cliente, puedo entonces usar con toda libertad cualquier subtipo de Cliente. El Cliente corporativo puede responder a ciertos comandos de manera diferente a otro Cliente (por el principio del polimorfismo), pero el invocador no debe preocuparse por la diferencia.

La generalización desde la perspectiva de implementación se asocia con la herencia en los lenguajes de programación. La subclase hereda todos los métodos y campos de la superclase y puede suplantarlos.

El punto clave aquí es la diferencia entre generalización desde la perspectiva de la especificación (subtipificación o herencia de interfaces) y desde la perspectiva de la implementación (subclasificación o herencia de implementaciones). La subclasificación (formación de subclases) es una manera de implementar la subtipificación. La subtipificación (formación de subtipos) también se puede implementar mediante delegación; de hecho, muchos de los patrones descritos en Gamma *et al.* (1994) se refieren a maneras de tener dos clases con interfaces similares sin servirse de la subclasificación. También se puede consultar el libro de los "pragmáticos" Martin y Odell (1996) o el de Fowler (1997), para más ideas sobre la implementación de subtipificación.

En el caso de estas dos formas de generalización, se deberá asegurar siempre que también tenga validez la generalización conceptual. He encontrado que, si no se hace lo anterior, se puede uno ver en problemas, debido a que la generalización no es estable cuando hay necesidad de efectuar cambios posteriores.

En algunas ocasiones aparecen casos en los que un subtipo tiene la misma interfaz que el supertipo, pero el subtipo implementa sus operaciones de modo diferente. Si éste es el caso, se podría decidir no mostrar el subtipo en el diagrama de perspectiva de especificación. Así lo hago con frecuencia si para los usuarios de la clase es interesante la existencia de los tipos, pero no lo hago cuando los subtipos varían únicamente debido a razones internas de implementación.

Reglas de restricción

Una buena parte de lo que se hace cuando se dibuja un diagrama de clase es indicar las condiciones limitantes o restricciones.

La figura 4-3 indica que un Pedido únicamente puede ser colocado por un solo Cliente. El diagrama implica también que Artículos de línea se considera por separado: digamos 40 adminículos de color marrón, 40 adminículos azules y 40 adminículos rojos, y no 40 artefactos rojos, azules y marrones. El diagrama dice, además, que los Clientes corporativos tienen límites de crédito, pero que los Clientes personales no los tienen.

Los artificios básicos de la asociación, el atributo y la generalización, colaboran de manera significativa con la especificación de condiciones importantes, pero no pueden indicarlas todas. Estas restricciones aún necesitan ser captadas; el diagrama de clase es un lugar adecuado para hacerlo.

El UML no define una sintaxis estricta para describir las restricciones, aparte de ponerlas entre llaves ({}). Yo prefiero escribir en lenguaje informal, para simplificar su lectura. Puede usarse también un enunciado más formal, como un cálculo de predicados o alguna derivada. Otra alternativa es escribir un fragmento de código de programación.

Idealmente, las reglas se deberían implementar como afirmaciones en el lenguaje de programación. Éstas se corresponden con el concepto de invariantes del Diseño por contrato (véase el recuadro). En lo personal, soy partidario de crear un método *revisaInvariante* e invocarlo desde algún código de depuración que ayude a comprobar los invariantes.

Diseño por contrato

El Diseño por contrato es una técnica diseñada por Bertrand Meyer. Esta técnica es una característica central del lenguaje Eiffel, que él desarrolló. Sin embargo, el Diseño por contrato no es específico de Eiffel; es una técnica valiosa que se puede usar con cualquier lenguaje de programación.

En el corazón del Diseño por contrato se encuentra la afirmación. Una **afirmación** es un enunciado Booleano que nunca debe ser falso y, por tanto, sólo lo será debido a una falla. Por lo común, las afirmaciones se comprueban sólo durante la depuración y no durante la ejecución en producción. De hecho, un programa nunca debe suponer que se están comprobando las afirmaciones.

El Diseño por contrato se vale de tres tipos de afirmaciones: poscondiciones, precondiciones e invariantes.

Las precondiciones y las poscondiciones se aplican a las operaciones. Una **poscondición** es un enunciado sobre cómo debería verse el mundo después de la ejecución de una operación. Por ejemplo, si definimos la operación "cuadrado" sobre un número, la poscondición adoptaría la forma de *resultado = éste * éste,* donde *resultado* es el producto y *éste* es el objeto sobre el cual se invocó la operación. La poscondición es una forma útil de decir qué es lo que hacemos, sin decir cómo lo hacemos; en otras palabras, de separar la interfaz de la implementación.

La **precondición** es un enunciado de cómo esperamos encontrar el mundo, antes de ejecutar una operación. Podemos definir una precondición para la operación "cuadrado" de *éste >= 0*. Tal precondición dice que es un error invocar a "cuadrado" sobre un número negativo y que las consecuencias de hacerlo son indefinidas.

A primera vista, ésta no parece ser una buena idea, pues deberíamos poner una comprobación en alguna parte para garantizar que se invoque correctamente a "cuadrado". La cuestión importante es quién es responsable de hacerlo.

La precondición hace explícito que quien invoca es el responsable de la comprobación. Sin este enunciado explícito de responsabilidades, podemos tener muy poca comprobación (pues ambas partes suponen que la otra es la responsable) o tenerla en demasía (cuando ambas partes lo hacen). Demasiada comprobación es mala, ya que provoca que se duplique muchas veces el código de comprobación, lo cual puede complicar de manera importante un programa. El ser explícito sobre quién es el responsable contribuye a reducir esta complejidad. Se reduce el peligro de que el invocador olvide comprobar por el hecho de que, por lo general, las afirmaciones se comprueban durante las pruebas y la depuración.

A partir de estas definiciones de precondición y poscondición, podemos ver una definición sólida del término **excepción**, que ocurre cuando se invoca una operación estando satisfecha su precondición y, sin embargo, no puede regresar con su poscondición satisfecha.

Una **invariante** es una afirmación acerca de una clase. Por ejemplo, la clase Cuenta puede tener una invariante que diga que *balance == sum(entradas.cantidad())*. La invariante "siempre" es verdadera para todas las instancias de la clase. En este contexto, "siempre" significa "siempre que el objeto esté disponible para que se invoque una operación sobre ella".

En esencia, lo anterior significa que la invariante se agrega a las precondiciones y poscondiciones asociadas a todas las operaciones públicas de la clase dada. La invariante puede volverse falsa durante la ejecución de un método, pero debe haberse restablecido a verdadera para el momento en que cualquier otro objeto pueda hacerle algo al receptor.

Las afirmaciones pueden desempeñar un papel único en la subclasificación.

Uno de los peligros del polimorfismo es que se podrían redefinir las operaciones de una subclase, de tal modo que fueran inconsistentes con las operaciones de la superclase. Las afirmaciones

impiden hacer esto. Las invariantes y poscondiciones de una clase deben aplicarse a todas las subclases. Las subclases pueden elegir el refuerzo de estas afirmaciones, pero no pueden debilitarlas. La precondición, por otra parte, no puede reforzarse, aunque sí debilitarse.

Esto parecerá extraño a primera vista, pero es importante para permitir vínculos dinámicos. Siempre debería ser posible tratar un objeto de subclase como si fuera una instancia de su superclase (de acuerdo con el principio de la sustituibilidad). Si una subclase refuerza su precondición, entonces podría fallar una operación de superclase, cuando se aplicara a la subclase.

En esencia, las afirmaciones sólo pueden aumentar las responsabilidades de la subclase. Las precondiciones son un enunciado para trasladar una responsabilidad al invocador; se incrementan las responsabilidades de una clase cuando se debilita una precondición. En la práctica, todo esto permite un mucho mejor control de la subclasificación y ayuda a asegurar que las subclases se comporten de manera adecuada.

Idealmente, las afirmaciones se deben incluir en el código, como parte de la definición de la interfaz. Los compiladores deben poder encender la comprobación por afirmaciones durante las depuraciones y apagarla durante la operación en producción. Se pueden manejar varias etapas de comprobación de afirmaciones. Las precondiciones a menudo ofrecen las mejores posibilidades de atrapar errores con la menor cantidad de sobrecarga de proceso.

Cuándo utilizar el Diseño por contrato

El Diseño por contrato es una técnica valiosa que siempre se debe emplear cuando se programa. Es particularmente útil para la construcción de interfaces claras.

Sólo Eiffel maneja afirmaciones como parte de su lenguaje, pero desafortunadamente Eiffel no es un lenguaje de amplia difusión. La adición de mecanismos a C++ y Smalltalk que manejen algunas

afirmaciones es directa. Es mucho más engorroso hacer algo similar en Java, pero es posible.

El UML no habla mucho acerca de las afirmaciones, pero se pueden usar sin problemas. Las invariantes son equivalentes a las reglas de condición en los diagramas de clase y se deberán usar tanto como sea posible. Las precondiciones y poscondiciones de operación se deben documentar con las definiciones de las operaciones.

Para mayor información

El libro de Meyer (1997) es una obra clásica (aunque, a estas alturas, muy denso) sobre el diseño OO, donde se habla mucho de las afirmaciones. Kim Walden y Jean-Marc Nerson (1995) y Steve Cook y John Daniels (1994) utilizan ampliamente el Diseño por contrato en sus obras.

También se puede obtener mayor información de ISE (la compañía de Bertrand Meyer).

Cuándo emplear los diagramas de clases

Los diagramas de clase son la columna vertebral de casi todos los métodos OO, por lo cual usted descubrirá que se emplean todo el tiempo. Este capítulo ha abarcado los conceptos básicos; el capítulo 5 analizará muchos de los conceptos más avanzados.

El problema con los diagramas de clase es que son tan ricos que pueden resultar abrumadores. A continuación, damos algunos consejos breves.

- No trate de usar todas las anotaciones a su disposición. Comience con el material sencillo de este capítulo: clases, asociaciones, atributos y generalización. Introduzca otras anotaciones del capítulo 5 sólo cuando las necesite.
- Ajuste la perspectiva desde la cual se dibujan los modelos a la etapa del proyecto.

  - Si se está en la etapa del análisis, dibuje modelos conceptuales.
  - Cuando se trabaje con software, céntrese en los modelos de especificación.
  - Dibuje modelos de implementación sólo cuando se esté ilustrando una técnica de implementación en particular.
- No dibuje modelos para todo; por el contrario, céntrese en las áreas clave. Es mejor usar y mantener al día unos cuantos diseños que tener muchos modelos olvidados y obsoletos.

El principal peligro con los diagramas de clase es que se puede quedar empantanado muy pronto en los detalles de la implementación. Para contrarrestar esto, céntrese en las perspectivas conceptual y de especificación. Si cae en estos problemas, encontrará muy útiles las tarjetas de CRC (véase la página 74).

Para mayor información

Los libros de los tres amigos serán la referencia definitiva para todo lo relacionado con los diagramas de clase. Mi consejo sobre otras fuentes depende de si usted prefiere una perspectiva de implementación o conceptual. Para la perspectiva de implementación, es recomendable Booch (1994); para una perspectiva conceptual son recomendables los libros de "fundamentos" de Martin y Odell (1994). Una vez que haya leído su primera selección, lea la otra, pues ambas perspectivas son importantes. Después, cualquier buen libro sobre OO le dará algunos puntos de vista interesantes. Tengo una particular predilección por el de Cook y Daniels (1994), debido a su manejo de las perspectivas y a la formalidad que introducen los autores.

Capítulo 5

Diagramas de clase: conceptos avanzados

Los conceptos descritos en el capítulo 4 corresponden a las notaciones fundamentales de los diagramas de clase. Éstos son los que usted necesita comprender y dominar primero, pues significarán el 90% del esfuerzo que se invertirá en la construcción de los diagramas de clase.

La técnica de diagramas de clase, sin embargo, ha generado decenas de nuevas notaciones para otros conceptos. En realidad, éstas no las empleo con frecuencia, pero resultan útiles cuando son apropiadas. A continuación las explicaré una por una y señalaré algunas de las consideraciones sobre su uso. No obstante, recuérdese que son opcionales y que hay mucha gente que ha obtenido grandes ventajas de los diagramas de clase sin haber utilizado estas características adicionales.

Tal vez encuentre el presente capítulo un poco denso. Sin embargo, tengo una buena noticia al respecto, y es que este capítulo puede usted saltarlo completo, sin ningún problema, en la primera lectura que haga del libro, y volver a él más tarde.

Los estereotipos

La idea de los estereotipos fue acuñada por Rebecca Wirfs-Brock (Wirfs-Brock *et al.*, 1990). El concepto ha sido adoptado con entusiasmo por los inventores del UML, aunque de un modo que en realidad no significa lo mismo. Sin embargo, ambas ideas son valiosas.

La idea original de un estereotipo se refería a una clasificación de alto nivel de un objeto que diera alguna indicación del tipo de objeto que era. Un ejemplo es la diferencia entre un "controlador" y un "coordinador".

Con frecuencia se encuentran diseños OO en los que una clase parece hacer todo el trabajo, muchas veces por medio de un gran método *hazlo* (*doIt*), mientras que las demás clases no hacen otra cosa más que encapsular datos. Éste es un diseño pobre, pues significa que el controlador es muy complejo y difícil de manejar.

Para mejorar esta situación, se traslada el comportamiento del controlador a los objetos de datos relativamente tontos, de modo que éstos se vuelven más inteligentes y adquieren responsabilidades de más alto nivel. Así, el controlador se convierte en un coordinador. El coordinador es el encargado de disparar tareas en una secuencia particular, pero otros objetos son los que saben cómo desempeñarlas.

La esencia del estereotipo consiste en que sugiere ciertas responsabilidades generales de una clase. El UML ha adoptado este concepto y lo ha convertido en un mecanismo general de extensión del lenguaje mismo.

En su trabajo original (1994), Jacobson clasifica todas las clases de un sistema en tres categorías: objetos de interfaz, objetos de control y objetos de entidad (sus objetos de control, cuando están bien diseñados, son parecidos a los coordinadores de Wirfs-Brock). Jacobson sugirió reglas para la comunicación entre estos tipos de clases y les dio a cada una de ellas un icono diferente. Esta distinción no es una parte medular del UML. Al contrario, este tipo de clases constituye en realidad estereotipos de clases; de hecho, son estereotipos, en el sentido que Wirfs-Brock le da al término.

Los estereotipos generalmente se indican en el texto entre comillas francesas (<<objeto de control>>), pero también se pueden mostrar definiendo un icono para el estereotipo. De lo que se trata es de que, si no se está empleando el enfoque de Jacobson, se pueden olvidar los estereotipos. Pero si quiere trabajar con dicho enfoque, se pueden definir los estereotipos y las reglas para su uso.

Muchas extensiones del núcleo de UML se pueden describir como una colección de estereotipos. En los diagramas de clase, pueden ser estereotipos de clases, asociaciones o generalizaciones. Puede pensarse en los estereotipos como subtipos de los tipos Clase, Asociación y Generalización en el metamodelo.

He observado que la gente que trabaja con el UML tiende a confundir las restricciones, es decir, las condiciones limitantes con los estereotipos. Si se marca una clase como abstracta, ¿se trata de una restricción o de un estereotipo? Los documentos oficiales actuales llaman a esto restricción, pero hay que estar conscientes de que se hace un uso confuso de una y otro. Esto no es de sorprender, ya que los subtipos son con frecuencia más limitados que los supertipos.

Clasificación múltiple y dinámica

La **clasificación** se refiere a la relación entre un objeto y su tipo.

La mayor parte de los métodos hacen ciertas suposiciones sobre este tipo de relación (premisas que están presentes también en los principales lenguajes de programación OO). Estas premisas fueron cuestionadas por Jim Odell, quien las consideró demasiado restrictivas para el modelado conceptual. Las suposiciones son para una clasificación simple y estática de los objetos; Odell sugiere emplear una clasificación múltiple y dinámica de los objetos en los modelos conceptuales.

En la **clasificación simple**, un objeto pertenece a un solo tipo, que puede haber heredado de supertipos. En la **clasificación múltiple**, un objeto puede ser descrito por varios tipos que no están conectados necesariamente por medio de herencia.

Obsérvese que la clasificación múltiple es diferente de la herencia múltiple. La herencia múltiple plantea que un tipo puede tener muchos supertipos, pero que se debe definir un solo tipo por cada objeto. La clasificación múltiple permite varios tipos para un objeto sin definir un tipo específico para tal fin.

Como ejemplo, considere una persona subtipificada como hombre o mujer, doctor o enfermera, paciente o no (véase la figura 5-1). La clasificación múltiple permite asignar cualquiera de estos tipos a un objeto en cualquier combinación posible, sin necesidad de que se definan tipos para todas las combinaciones legales.

Si se emplea la clasificación múltiple, habrá de asegurarse que haya quedado claro cuáles son las combinaciones legales. Esto se hace etiquetando una línea de generalización con un **discriminador**, el cual es una indicación de la base de la subtipificación. Varios subtipos pueden compartir el mismo discriminador. Todos los subtipos con el mismo discriminador están desunidos, es decir, cualquier instancia del supertipo puede ser una instancia de sólo uno de los subtipos, dentro del discriminador. Una buena convención es que todas las subclases que emplean un discriminador desemboquen en un triángulo, como se muestra en la figura 5-1. Se pueden tener, como alternativa, varias flechas con la misma etiqueta.

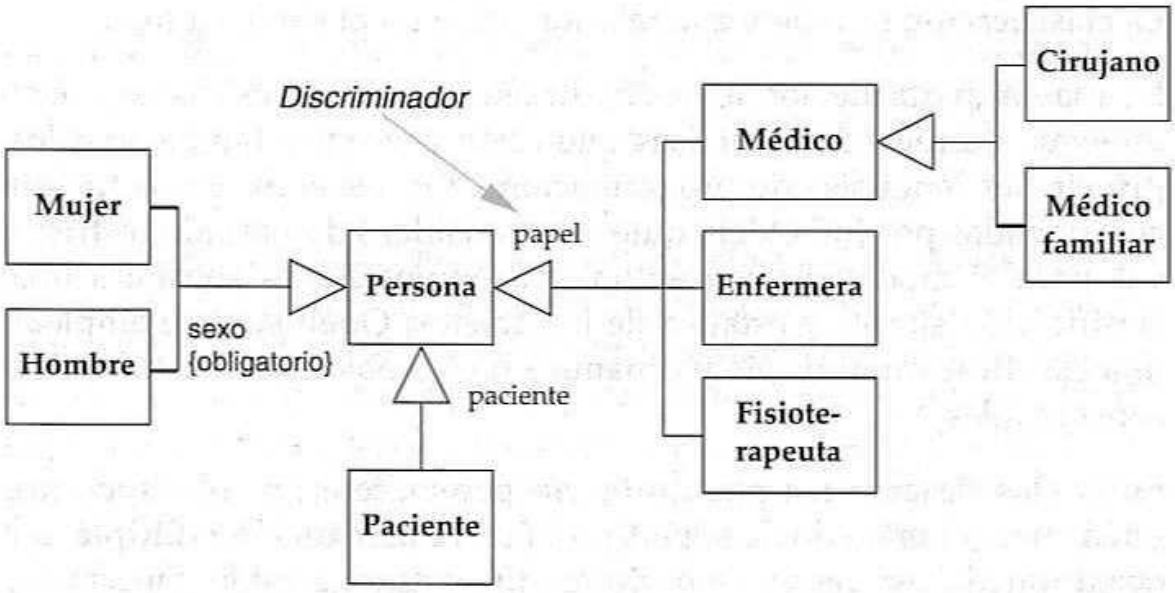


Figura 5-1: *Clasificación múltiple*

Una restricción útil (pero no estándar en el UML) consiste en marcar al discriminador como {obligatorio}. Esto significa que cualquier instancia de la superclase debe ser una instancia de una de las subclases del grupo (por tanto, la superclase es abstracta).

Para ilustrar, obsérvense las siguientes combinaciones legales de subtipos en el diagrama: (Mujer, Paciente, Enfermera); (Hombre, Fisioterapeuta); (Mujer, Paciente); y (Mujer, Médico, Cirujano). Obsérvense también que las combinaciones como (Paciente, Médico) y (Hombre, Médico, Enfermera) son ilegales: la primera, porque no incluye un tipo del discriminador Sexo {obligatorio}; la última, porque contiene dos tipos del discriminador Papel. La clasificación simple, por definición, corresponde a un solo discriminador no etiquetado.

Otra pregunta que cabe plantearse es si un objeto puede cambiar de tipo. Un buen ejemplo para contestarla es una cuenta bancaria. Cuando se sobregira la cuenta, cambia sustancialmente su conducta; específicamente se suplantan varias operaciones (incluyendo la de "retiro" y la de "cierre").

La **clasificación dinámica** permite a los objetos cambiar de tipo dentro de la estructura de subtipificación; la **clasificación estática**, no. En la clasificación estática se hace la distinción entre tipos y estados; en la clasificación dinámica se combinan estos dos conceptos.

¿Debería usarse la clasificación múltiple y dinámica? Considero que es útil para el modelado conceptual. Se puede hacer con modelado por especificaciones, pero conviene sentirse cómodo con las técnicas necesarias para realizarlo. El truco consiste en implementarlo de manera que parezca lo mismo que subclasificar desde la interfaz, de tal suerte que el usuario de una clase no sepa qué implementación se está usando (véase Fowler, 1997, para algunas técnicas). Sin embargo, como en la mayoría de estas cosas, la elección depende de las circunstancias, así que se deberá aplicar lo que se juzgue mejor. La transformación de una interfaz múltiple y dinámica a una implementación estática única bien puede acarrear más problemas de los que resolvería.

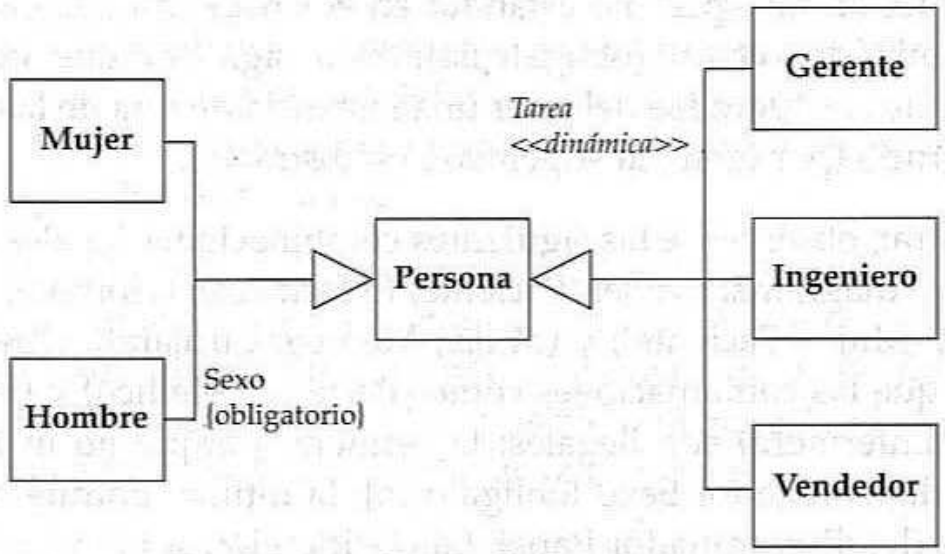


Figura 5-2: *Clasificación dinámica*

La figura 5-2 muestra un ejemplo del empleo de la clasificación dinámica para el trabajo de una persona, que, por supuesto, puede cambiar. Esto puede ser lo apropiado, pero los subtipos necesitarían un comportamiento adicional, en lugar de ser sólo etiquetas. En estos casos, muchas veces vale la pena crear una clase separada para el trabajo y vincular a la persona con ella mediante una asociación. Al respecto, yo escribí un patrón titulado *Modelos de papeles* (*Role Models*); usted puede encontrar información sobre este patrón, así como otra información que sirva de complemento a mi libro *Analysis Patterns*, en <**http://www.aw.com/cp/fowler.html**>.

Agregación y composición

Una de las cosas más *aterradoras* del modelado es la agregación. Ésta es fácil de explicar con pocas palabras: la **agregación** es la relación de componente. Es como decir que un automóvil tiene como componentes motor y ruedas. Esto suena muy bien, pero la parte difícil es determinar cuál es la diferencia entre agregación y asociación.

Peter Coad ejemplificó la agregación como la relación entre una organización y sus empleados; Jim Rumbaugh afirmó que una compañía

no es la agregación de sus empleados. Cuando los gurúes no se ponen de acuerdo, ¿qué debemos hacer? El problema es que los metodólogos no tienen una definición aceptada por todos de la diferencia entre asociación y agregación.

De hecho, muy pocos se sirven de cualquier definición formal. La cuestión práctica que nos interesa aquí es que todos manejan un concepto levemente diferente, de modo que hay que tener precaución con este concepto, y, generalmente, prefiero evitarlo, a menos que el equipo del proyecto se ponga de acuerdo en un significado riguroso y útil.

El UML ofrece, además de la agregación simple, una variedad más poderosa, la cual se denomina composición. Con la **composición**, el objeto parte puede pertenecer a un todo único; es más, se espera, por lo general, que las partes vivan y mueran con el todo. Cualquier borrado del todo se extiende en cascada a las partes.

El borrado en cascada se considera con frecuencia como una parte definitoria de la agregación, pero está implícita en cualquier papel con una multiplicidad de 1..1; si realmente se quiere eliminar a un Cliente, por ejemplo, habrá que aplicar este borrado en cascada a Pedido (y, por tanto, a Línea de pedido).

La figura 5-3 muestra ejemplos de estos artificios. Las composiciones a *Punto* indican que cualquier instancia de *Punto* puede estar en un *Polígono* o en un *Círculo*, pero no en ambos. Sin embargo, una instancia de *Estilo* puede ser compartida por muchos *Polígonos* y *Círculos*. Es más, esto implica que el borrado de un *Polígono* provocaría el borrado de sus *Puntos* asociados, pero **no** del *Estilo* asociado.

Esta restricción (un *Punto* puede solamente aparecer en un solo *Polígono* o *Círculo* a la vez) no se puede expresar mediante las multiplicidades, únicamente. También implica que el punto es un objeto de valor (véase la página 98). Se puede añadir una multiplicidad a un lado compuesto de la asociación, pero yo no me tomo la molestia. El rombo negro dice todo lo que hay que decir.

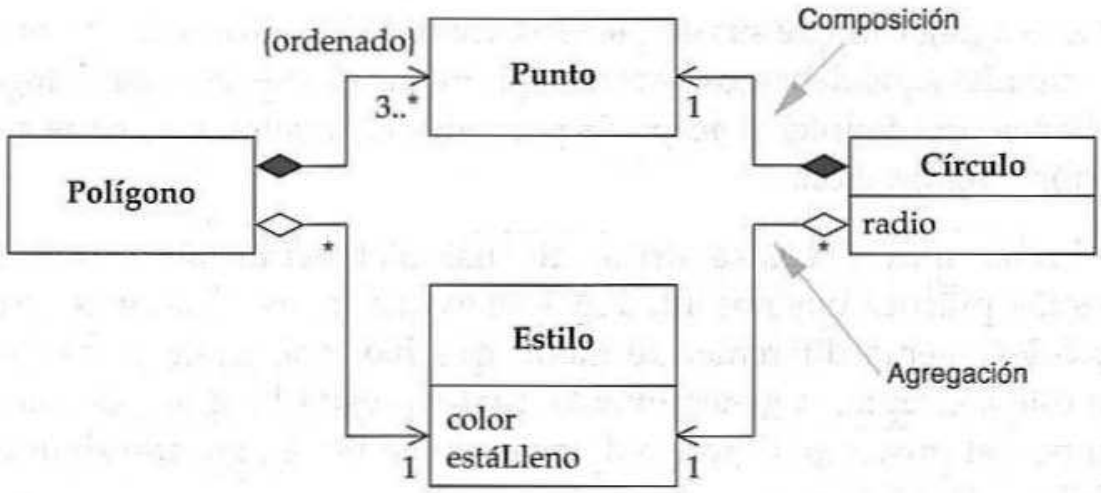


Figura 5-3: *Agregación y composición*

La figura 5-4 muestra otra notación para la composición. En este caso, se pone el componente dentro del todo. El nombre de la clase del componente no está en negritas y se escribe en la forma *nombrepapel: Clase nombre*. Además, se pone la multiplicidad en la esquina superior derecha.

Las diferentes notaciones de composición operan en situaciones diferentes. Hay un par más, aunque la variedad de notaciones de composición ofrecida por el UML es más bien abrumadora. Obsérvese que estas variaciones sólo pueden servir para la composición, no para la agregación.

Figura 5-4: *Notación alterna para la composición*

Asociaciones y atributos derivados

Las **asociaciones derivadas** y los **atributos derivados** son aquellos que se pueden calcular a partir de otras asociaciones y atributos, respectivamente, de un diagrama de clase. Por ejemplo, un atributo de edad de una Persona se puede derivar si se conoce su fecha de nacimiento.

Cada perspectiva contribuye con su propia interpretación de las características derivadas de los diagramas de clase. La más crítica de éstas se relaciona con la perspectiva de especificación. Desde este ángulo, es importante comprender que las características derivadas indican una restricción entre valores y no una declaración de lo que se calcula y lo que se almacena.

La figura 5-5 muestra una estructura jerárquica de cuentas dibujada desde una perspectiva de especificación. El modelo utiliza el patrón *Compuesto* (véase Gamma *et al.* 1994).

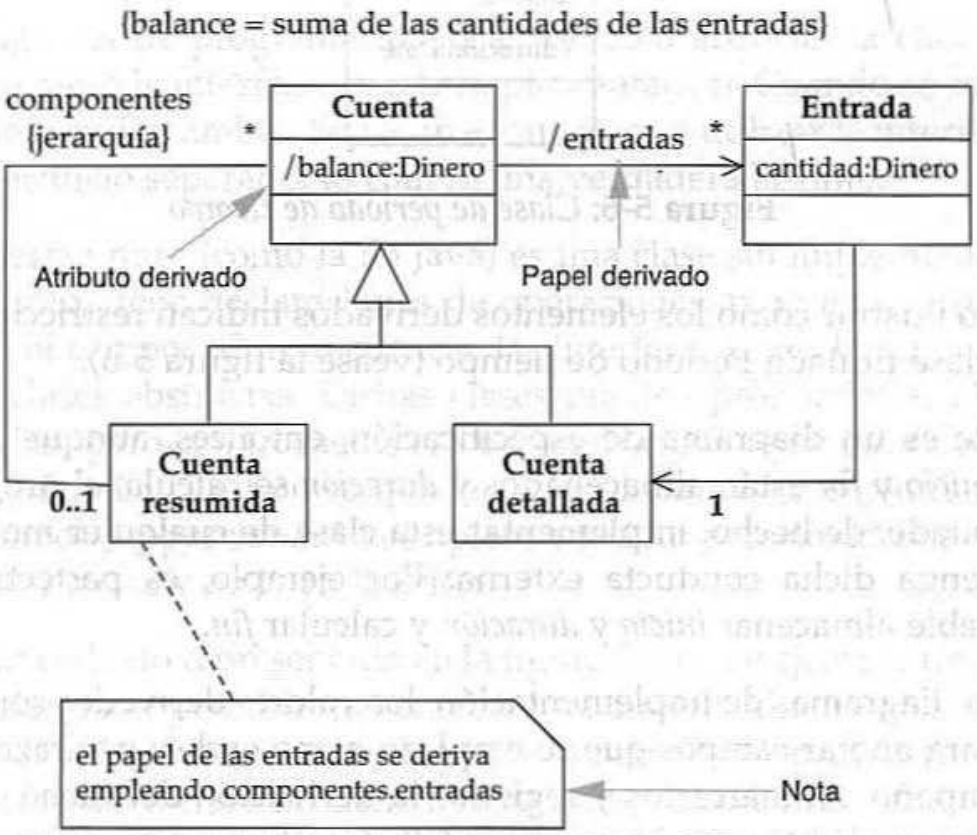


Figura 5-5: *Asociaciones y atributos derivados*

Obsérvese lo siguiente:

- Los objetos de entrada están conectados a Cuentas detalladas.
- El balance de una Cuenta se calcula como la suma de las cantidades de Entrada.
- Las entradas de una Cuenta resumida son las entradas de sus componentes, determinados de manera recurrente.

Debido a que la figura 5-5 ilustra un modelo de especificación, no indica que las clases Cuentas no contienen campos para hacer balances; bien podría estar presente tal caché, pero está oculto para los clientes de la clase Cuenta.

Figura 5-6: *Clase de periodo de tiempo*

Puedo ilustrar cómo los elementos derivados indican restricciones de una clase titulada Periodo de tiempo (véase la figura 5-6).

Si éste es un diagrama de especificación, entonces, aunque sugiera que *inicio* y *fin* están almacenados y *duración* se calcula, el programador puede, de hecho, implementar esta clase de cualquier modo que mantenga dicha conducta externa. Por ejemplo, es perfectamente aceptable almacenar *inicio* y *duración* y calcular *fin*.

En los diagramas de implementación, los valores derivados son valiosos para anotar campos que se emplean como cachés por razones de desempeño. Al marcarlos y registrar la derivación del caché, es más fácil ver explícitamente lo que hace éste. Con frecuencia, refuerzo esto en el código indicando la palabra "caché" en tal campo (por ejemplo, *cachéBalance*).

En los diagramas conceptuales, utilizo marcadores derivados para que me recuerden dónde existen tales derivaciones y confirmar, con los expertos del dominio, qu existen las derivaciones. Entonces, éstas se correlacionan con su empleo en los diagramas de especificación.

En los mundos del OMT y de Odell, las asociaciones derivadas se indican con una diagonal en la línea de asociación. Este uso no es parte del UML, aunque, no obstante, confieso que lo utilizo. Me parece más claro, en particular cuando no le doy nombre a la asociación.

Interfaces y clases abstractas

Una de las grandes cualidades del desarrollo orientado a objetos es que se pueden variar las interfaces de las clases, independientemente de la implementación. Gran parte del poder del desarrollo de objetos surge de esta propiedad. Muy pocos, sin embargo, hacen un buen uso de ella.

Los lenguajes de programación usan un solo artificio, la clase, que contiene tanto la interfaz como la implementación. Cuando se subclasifica, se heredan ambas. Se recurre muy poco a utilizar la interfaz como un artificio separado, lo cual es una verdadera lástima.

Una interfaz pura (como la de Java) es una clase sin implementación y, por tanto, tiene declaraciones de operaciones, pero no cuerpos de método ni campos. Con frecuencia, las interfaces se declaran por medio de clases abstractas. Dichas clases pueden proporcionar alguna realización, pero con frecuencia se usan principalmente para declarar una interfaz. La cuestión es que la subclasificación (o cualquier otro mecanismo) proporcionará la implementación, pero los clientes sólo verán la interfaz y no la realización.

El editor de texto representado en la figura 5-7 es un ejemplo típico de lo anterior. Para permitir que el editor sea independiente de la plataforma, definimos una clase abstracta Ventana, independiente de la plataforma. Esta clase no tiene operaciones; sólo define una interfaz para el editor de texto. Las subclases específicas de la plataforma se pueden emplear como se desee.

Si se tiene una clase o método abstractos, la convención en el UML es poner con cursivas el nombre del elemento abstracto. La restricción {abstracto} también puede utilizarse (y también en sustitución). Por mi parte, manejo {abstracto} en los pizarrones, porque no puedo escribir el texto en cursivas. Sin embargo, teniendo una herramienta de diagramación, prefiero la elegancia de las cursivas.

Pero la subclasificación no es la única manera de hacer lo anterior. Java ofrece una interfaz específica y el compilador revisa que la clase ofrezca una implementación para todas las operaciones definidas para dicha interfaz.

En la figura 5-8, vemos InputStream, DataInput y DataInputStream (definidos en el archivo *java.io* estándar). InputStream es una clase abstracta; DataInput es una interfaz.

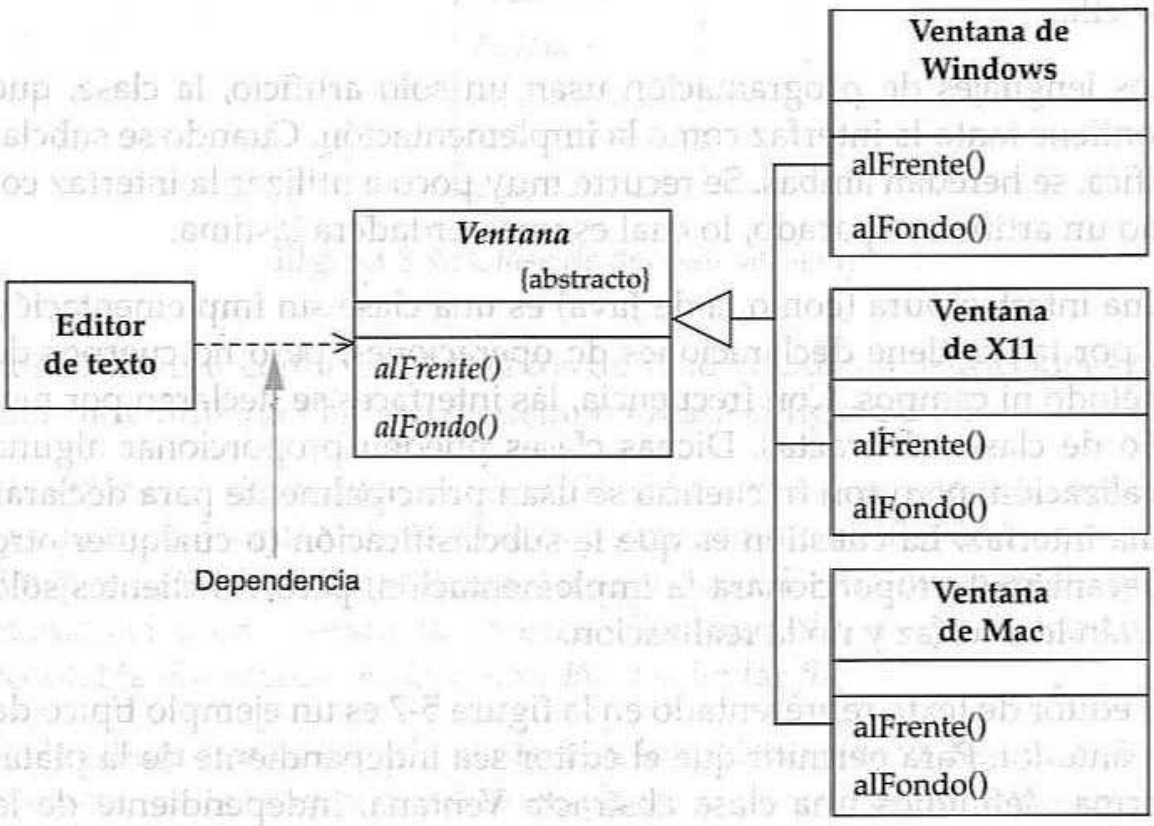


Figura 5-7: *Ventana como clase abstracta*

Alguna clase cliente, digamos, LectorÓrdenes necesita utilizar la funcionalidad de DataInput. La clase de DataInputStream implementa las interfaces DataInput e InputStream y es una subclase de esta última.

El vínculo entre DataInputStream y DataInput es una relación de refinamiento. El **refinamiento** es un término general que se emplea en el UML para indicar un mayor nivel de detalle. Puede usarse para la implementación de interfaces o algún otro fin (véanse los libros de los tres amigos para mayores detalles). El refinamiento deliberadamente es similar a la generalización.

En un modelo de especificación, tanto la subclasificación como el refinamiento se representarían como una subtipificación; la diferencia entre el refinamiento y la generalización es válida sólo en los modelos de implementación. Algunos modeladores prefieren usar <<tipo>> en lugar de <<interfaz>>. En mi opinión, soy partidario de <<interfaz>>, pues así queda explícito el papel de la interfaz.

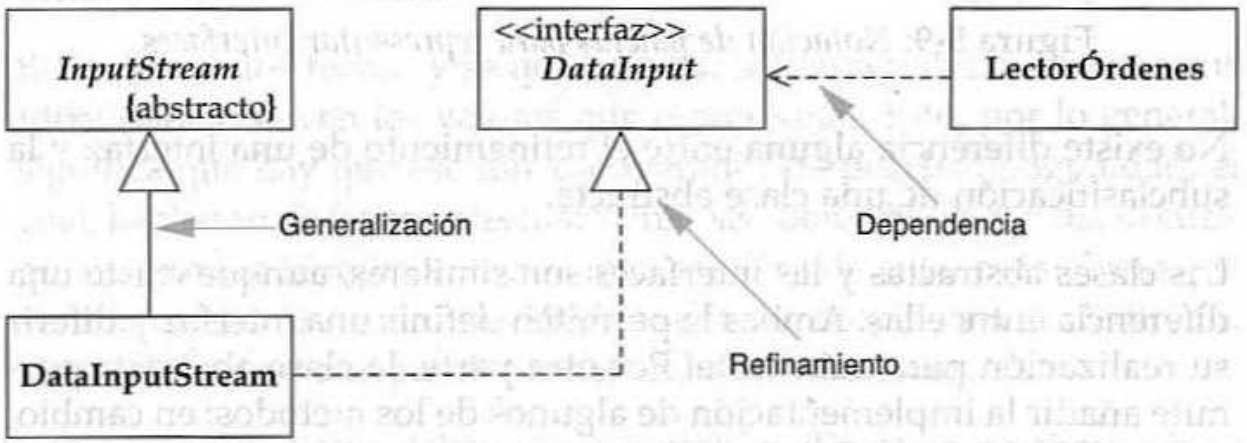


Figura 5-8: *Interfaces y clase abstracta: un ejemplo de Java*

El vínculo entre LectorÓrdenes y DataInput es una dependencia. Muestra que LectorÓrdenes usa la interfaz de DataInput con algún fin. En el capítulo 7 abundaré más sobre las dependencias. En esencia, una dependencia indica que, si la interfaz de DataInput cambia, también tiene que cambiar el LectorÓrdenes. Uno de los objetivos del desarrollo es el de mantener a un mínimo las dependencias, de modo que los efectos de los cambios también sean mínimos.

La figura 5-9 muestra una notación alterna más compacta. En ella, las interfaces están representadas por pequeños círculos llamados con frecuencia paletas (*lollipops*) que surgen de las clases que las implementan.

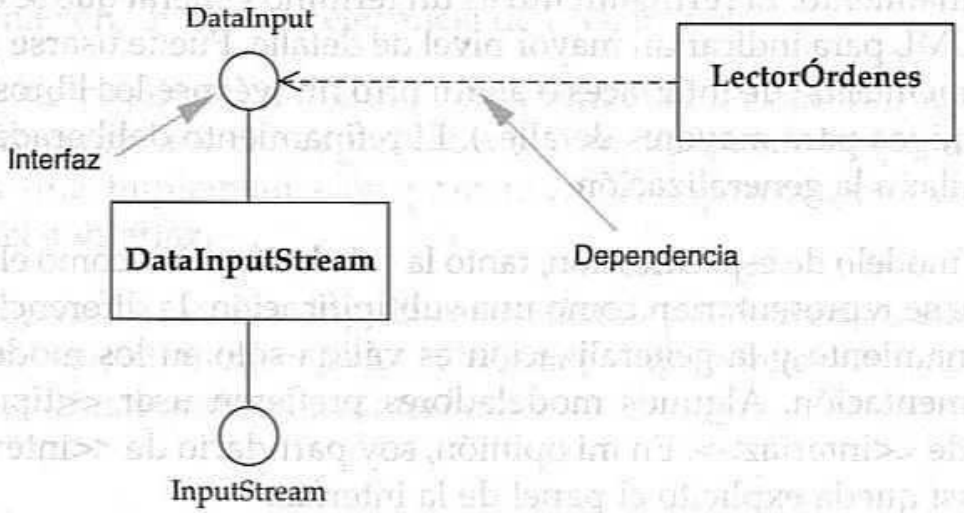


Figura 5-9: *Notación de paletas para representar interfaces*

No existe diferencia alguna entre el refinamiento de una interfaz y la subclasificación de una clase abstracta.

Las clases abstractas y las interfaces son similares, aunque existe una diferencia entre ellas. Ambas le permiten definir una interfaz y diferir su realización para más tarde. Por otra parte, la clase abstracta permite añadir la implementación de algunos de los métodos; en cambio, una interfaz obliga a diferir la definición de todos los métodos.

Objetos de referencia y objetos de valor

Una de las cosas comunes que se dicen de los objetos es que tienen identidad. Esto es cierto, pero el asunto no es tan simple como parece. En la práctica, usted se percata de que la identidad es importante para los objetos de referencia, pero no tanto para los objetos de valor.

Los **objetos de referencia** son cosas como Cliente. La identidad aquí es muy importante, porque usualmente se quiere que un solo objeto

de software designe a un cliente del mundo real. Cualquier objeto que se refiera al objeto Cliente lo hará por medio de una referencia o un apuntador; todos los objetos que se refieran a este Cliente se referirán al mismo objeto de software. De esta manera, los cambios en un Cliente estarán disponibles para todos los usuarios del Cliente.

Si se tienen dos referencias de un Cliente y se desea saber si son iguales, por lo común se comparan sus identidades. Las copias probablemente no estén permitidas y, si lo están, tenderán a hacerse en raras ocasiones, tal vez para propósitos de respaldo o para su réplica en una red. Si se hacen copias, habrá que determinar la manera de sincronizar los cambios.

Los **objetos de valor** son cosas como Fecha. Con frecuencia, se tienen varios objetos de valor que representan al mismo objeto en la realidad. Por ejemplo, es normal tener a cientos de objetos que designen el 1° de enero de 1997. Todas éstas son copias intercambiables. A menudo se crean y destruyen fechas nuevas.

Si se tienen dos fechas y se quiere saber si son iguales, no se ven sus identidades; se ven los valores que representan. Esto, por lo general, significa que hay que escribir un operador de prueba de igualdad, el cual, hablando de fechas, efectuará pruebas sobre año, mes y día (o cualquiera que sea la representación interna). Usualmente, cada objeto que se refiera al 1°de enero de 1997 tendrá su propio objeto dedicado, pero en ocasiones se pueden tener fechas combinadas.

Los objetos de valor deben ser **inmutables**. En otras palabras, no se debe poder tomar un objeto con fecha del 1° de enero de 1997 y cambiarlo al 2 de enero de 1997. En cambio, se deberá crear un nuevo objeto 2 de enero de 1997 y vincularlo al primer objeto. La razón de esto es que si la fecha fuera compartida, se actualizaría la fecha de otro objeto de manera impredecible.

En C++ esto no constituye un problema, pues se hace un gran esfuerzo por no compartir fechas; el compartir objetos de valor conduce a problemas de administración de memoria. En lugar de eso, se puede suplantar la asignación para hacer copias. En los ambientes de memoria administrada, tales como Java, esto es más importante, en especial

porque, en Java, las fechas no son inmutables. Como regla empírica, no cambie un objeto de valor.

En el pasado, la diferencia entre los objetos de referencia y los objetos de valor era más clara. Los objetos de valor eran los valores interconstruidos del sistema de tipos. En la actualidad, se puede extender el sistema de tipos con clases propias del usuario, por lo que esta cuestión requiere de mayor análisis. En el UML, los atributos se usan generalmente para objetos de valor y las asociaciones para objetos de referencia. También se puede manejar composición para objetos de valor.

No considero que la diferencia entre objetos de referencia y objetos de valor sea de utilidad en los modelos conceptuales. No hay diferencia entre las dos construcciones desde la perspectiva conceptual. Si represento un vínculo con un objeto de valor mediante una asociación, marco como * la multiplicidad del papel desde el valor a su usuario, a menos que exista una regla de unicidad (como, digamos, un número de secuencia).

Colecciones para relaciones de valor múltiple

Una **relación de valor múltiple** (*multi-valued role*) es aquella cuyo límite más alto de multiplicidad es mayor que 1 (por ejemplo, *). La convención usual es que las relaciones de valor múltiple se consideran como conjuntos. No hay un ordenamiento para los objetos destino y ningún objeto destino aparece más de una vez en la relación. Sin embargo, se pueden cambiar estas premisas al conectar una restricción.

La restricción *{ordenado}* implica que hay un ordenamiento para los objetos destino (esto es, los objetos destino forman una lista). Los objetos destino pueden aparecer sólo una vez en la lista.

Yo utilizo la restricción *{bolsa}* para indicar que los objetos destino pueden aparecer más de una vez, pero no hay un ordenamiento (si quisiera un ordenamiento y varias apariciones, indicaría *{ordenado bol-*

sa}, aunque aún no he tenido necesidad de hacerlo). También manejo la restricción *{jerarquía}* para indicar que los objetos destino forman una jerarquía y uso la restricción *{gad}* para indicar una gráfica acíclica dirigida.

Congelado

Congelado (frozen) es una restricción que, según la define el UML, se aplica a los papeles, pero también puede aplicarse a los atributos y a las clases.

En un papel o en un atributo, congelado indica que sus valores no pueden cambiar durante la vida del objeto origen. El valor debe establecerse en el momento de la creación del objeto y nunca puede cambiar. El valor inicial puede ser nulo. Ciertamente, si éste es el valor en el momento de la construcción del objeto, así seguirá, mientras el objeto viva. Esto implica que debe haber un argumento para este valor en un constructor y que no debe haber ninguna operación que lo actualice.

Cuando se aplica a una clase, congelado indica que todas las funciones y atributos asociados con esa clase están congelados.

Congelado *no* es igual que la restricción sólo-lectura. La restricción sólo-lectura implica que un valor no puede cambiarse directamente, sino que puede cambiar debido a la modificación de algún otro valor. Por ejemplo, si una persona tiene una fecha de nacimiento y una edad, entonces la edad puede ser de sólo lectura, pero no congelado. Marco "congelamiento" mediante la restricción *{congelado}* y los valores de sólo lectura con *{sólo lectura}*.

Si usted tiene la intención de "congelar" algo, tenga en cuenta que las personas cometen errores. En el software modelamos lo que conocemos sobre el mundo, no cómo es el mundo. Si modeláramos cómo es el mundo, el atributo de "fecha de nacimiento" de un objeto Persona sería inmutable, pero, en la mayoría de los casos, querríamos cambiarlo si nos percatamos de que un registro previo es incorrecto.

Clasificación y generalización

Con frecuencia, oigo a las personas referirse a la subtipificación como la relación de "es un". Le recomiendo muy encarecidamente tener cuidado con esta manera de pensar. El problema es que la frase "es un" puede significar cuestiones diferentes.

Considere las siguientes frases:

1. Shep es un Border Collie.
2. Un Border Collie es un Perro.
3. Los Perros son Animales.
4. Un Border Collie es una Raza.
5. El Perro es una Especie.

A continuación, intente combinar las frases anteriores. Si se combinan las frases 1 y 2, se obtiene "Shep es un Perro"; la combinación de las frases 2 y 3 da "los Border Collie son Animales". Y 1 más 2 más 3 da "Shep es un Animal". Hasta aquí no hay problemas. Ahora, inténtese 1 con 4: "Shep es una Raza". La combinación de 2 con 5 da "un Border Collie es una Especie." Estas combinaciones ya no son tan buenas.

¿Por qué se pueden combinar algunas de estas frases y otras no? La razón es que algunas son **clasificaciones** (el objeto Shep es una instancia del tipo Border Collie) y otras son **generalizaciones** (el tipo Border Collie es un subtipo del tipo Perro). La generalización es transitiva, pero la clasificación no. Puedo combinar una clasificación seguida por una generalización, pero no puedo llevar a cabo la combinación a la inversa.

Puntualizo lo anterior para que desconfíe de la frase "es un". Su uso puede conducir al empleo inapropiado de la subclasificación y a la confusión de responsabilidades. Las frases "los Perros son clases de Animales" y "cada instancia de un Border Collie es una instancia de un Perro" serían, en este caso, mejores pruebas para subtipos.

Asociaciones calificadas

Una **asociación calificada** es el equivalente en UML de un concepto de programación que se conoce de diferentes modos: arreglos asociativos, mapas y diccionarios.

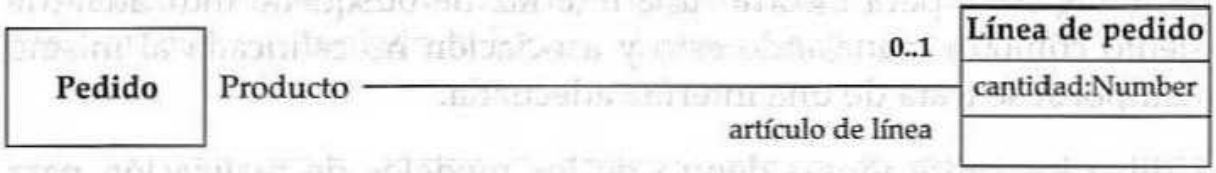


Figura 5-10: *Asociación calificada*

La figura 5-10 muestra un modo de representar la asociación entre las clases Pedido y Línea de pedido que emplea un calificador. El calificador dice que, en conexión con un Pedido, puede haber una Línea de pedido para cada instancia del Producto.

Conceptualmente, este ejemplo indica que no se puede tener dos instancias de Línea de pedido para el mismo Producto dentro de un Pedido. Desde la perspectiva de especificación, esta asociación calificada implicaría una interfaz semejante a

```
class Pedido {

    public LineaPedido articuloLinea (Producto
      unProducto);

    public void agregaArticuloLinea (Numero
      cantidad, Producto paraProducto);
```

Por tanto, todos los accesos a un Artículo de línea dado requieren como argumento la identidad de un Producto. Una multiplicidad de 1 indicaría que debe haber un Artículo de línea para cada Producto; * indicaría que se pueden tener varias Líneas de pedido por Producto, pero que el acceso a los Artículos de línea sigue estando indizado por Producto.

Desde una perspectiva de implementación, lo anterior sugiere el empleo de un arreglo asociativo o de una estructura de datos similar para contener las líneas del pedido.

```
Class Pedido {
        private Dictionary _articulosLinea;
```

En el modelado conceptual, yo utilizo el artificio calificador únicamente para mostrar restricciones del tipo "una sola Línea de pedido por Producto en el Pedido." En el caso de los modelos de especificación, me sirve para mostrar una interfaz de búsqueda indizada. Me siento cómodo manejando esto y asociación no calificada al mismo tiempo, si se trata de una interfaz adecuada.

Utilizo los calificadores dentro de los modelos de realización para mostrar los usos de un arreglo asociativo o de una estructura de datos similar (para mayor información sobre la manera en que empleo los calificadores, véase el análisis del patrón *Correlación indizada (Keyed Mapping)* en Fowler, 1997.

Clase de asociación

Las clases de asociación permiten añadir atributos, operaciones y otras características a las asociaciones, como se muestra en la figura 5-11.

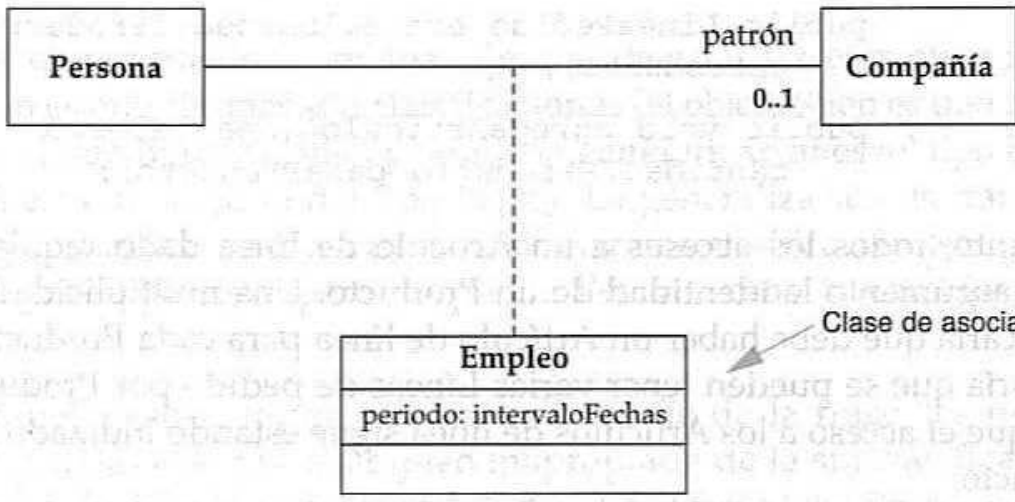


Figura 5-11: *Clase de asociación*

El diagrama nos permite apreciar que una Persona puede trabajar para una sola Compañía. Necesitamos conservar la información sobre el periodo de tiempo que trabaja cada empleado para cada Compañía.

Para lograrlo, añadimos un atributo de *intervaloFechas* a la asociación. Podríamos agregar este atributo a la clase Persona, pero, en realidad, es un hecho sobre la relación entre una Persona y una Compañía, la cual cambiará si también lo hiciera el patrón de la persona.

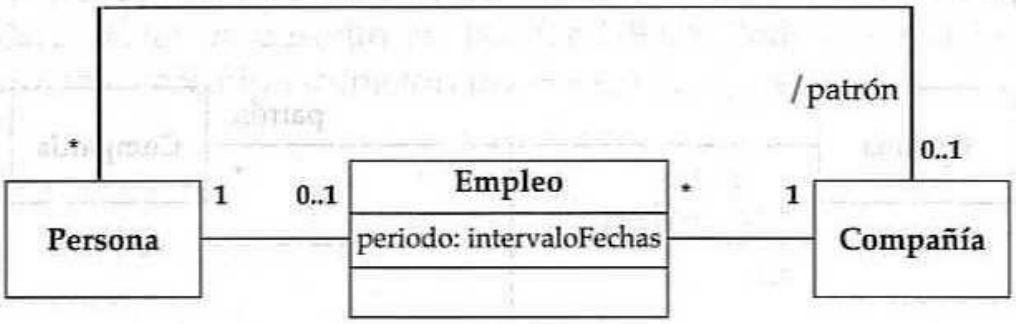


Figura 5-12: *Promoción de una clase de asociación a una clase completa*

La figura 5-12 muestra otra manera de representar esta información: convertir Empleo en una clase completa por derecho propio (obsérvese cómo las multiplicidades se han movido en consecuencia). En este caso, cada una de las clases de la asociación original tiene una función de un solo valor con relación a la clase Empleo. El papel del "patrón" es ahora derivado, aunque no sea necesario mostrar esto.

¿Qué beneficio se logra con la clase de asociación que compense la notación extra que hay que recordar? La clase de asociación añade una restricción nueva, en el sentido de que sólo puede haber una instancia de la clase de asociación entre cualesquiera de dos objetos participantes. Me parece que hace falta un ejemplo.

Observe por un momento los dos diagramas que aparecen en la figura 5-13. Ambos tienen formas muy parecidas. Sin embargo, podríamos imaginarnos a una Persona trabajando para la misma Compañía en diferentes periodos de tiempo (es decir, la persona deja la compañía y después vuelve). Esto significa que una Persona puede tener más de una asociación de Empleo con la misma Compañía a través del tiempo.

En lo que se refiere a las clases Persona y Oficio, sería difícil apreciar por qué una persona tendría más de una Competencia en el mismo oficio; de hecho, probablemente se consideraría como un error.

En el UML sólo es legal el segundo caso. Se puede tener solamente una Competencia para cada combinación de Persona y Oficio. El diagrama superior de la figura 5-13 no permitiría a una Persona tener más de un Empleo en la misma Compañía. Si es necesario permitir esto, habrá que convertir a Empleo en una clase completa, a la manera de la figura 5-12.

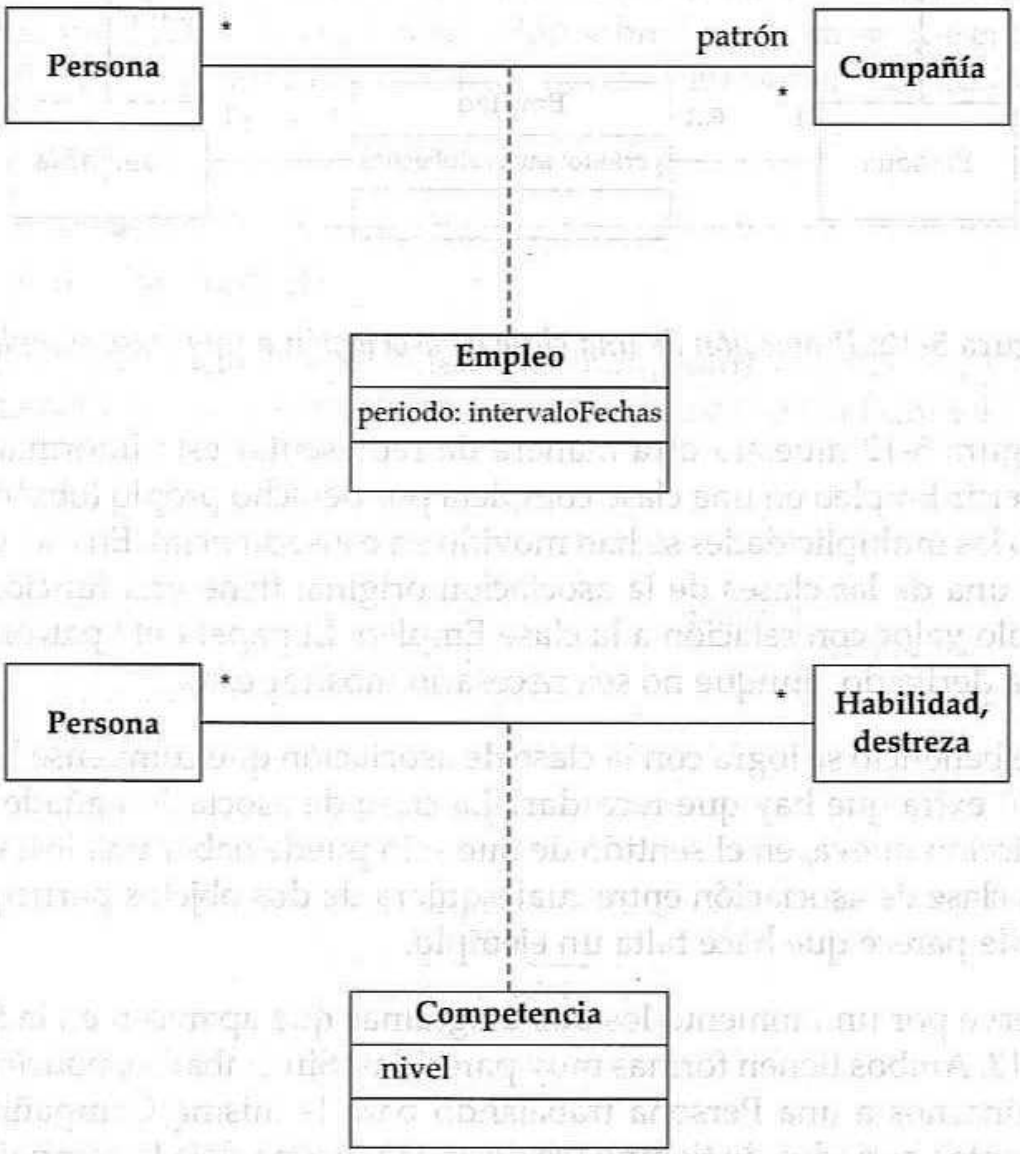


Figura 5-13: *Sutilezas de la clase de asociación*

En el pasado, los modeladores tenían varias premisas sobre el significado de una clase de asociación en estas circunstancias. Algunos suponían que sólo podían tenerse combinaciones únicas (como es el caso de la competencia); mientras que otros más no suponían tal restricción. Muchos no lo tomaban en consideración en lo absoluto y podrían asumir la restricción en algunos lugares y en otros no. Así pues, cuando utilice el UML, recuerde que la restricción siempre está allí.

Con frecuencia, se encuentra este tipo de artificio con información histórica, como en el caso de Empleo anterior. Aquí, un patrón útil es el de *Mapa Histórico*, descrito en Fowler (1997). Podemos usarlo definiendo un estereotipo <<historia>> (véase la figura 5-14).

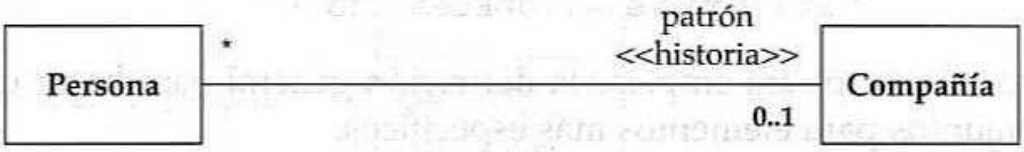


Figura 5-14: *Estereotipo de historia para las asociaciones*

El modelo indica que una Persona únicamente puede trabajar para una sola Compañía en un momento dado. Sin embargo, a través de un tiempo determinado, una Persona puede trabajar para varias Compañías. Esto sugiere una interfaz semejante a las líneas siguientes

```
class Persona {
        // obtiene el patrón actual
        Compañia patron();
        // patron en una fecha dada
        Compañia patron(Date);
        void cambiaPatron(Compañia nuevoPatron,
            Date cambiaFecha);
        void dejaPatron (Date cambiaFecha);
```

El estereotipo <<historia>> no forma parte del UML, pero lo menciono aquí por dos razones. Primera, es un concepto que he encontrado útil en varias ocasiones durante mi ejercicio como modelador. Segundo, muestra cómo se pueden usar los estereotipos para ampliar el UML.

Clase con parámetro

Algunos lenguajes, particularmente C++, tienen el concepto de **clase con parámetro** (también conocida como **plantilla**).

La utilidad más obvia de este concepto es en el trabajo con colecciones en un lenguaje con especificación estricta de tipos de datos. De este modo, se puede definir la conducta de los conjuntos en general por medio de la definición de una clase de plantillas Conjunto.

```
class Conjunto <T> {
        void insert (T nuevoElemento) ;
        void remove (T unElemento);
```

Hecho esto, se podrá emplear la definición general para hacer clases de conjuntos para elementos más específicos.

```
Conjunto <Empleado> conjuntoEmpleados;
```

Una clase con parámetro en UML se declara mediante la notación que aparece en la figura 15-5.

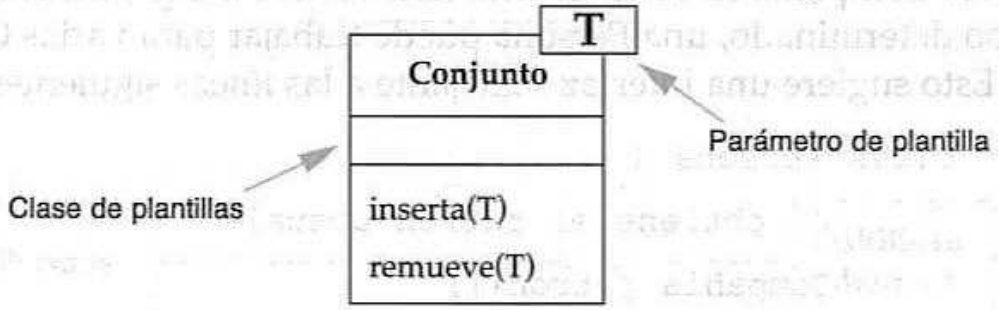


Figura 5-15: *Clase con parámetro*

La T en el diagrama es un marcador para el parámetro de tipo (se podrá tener más de uno). En un lenguaje sin tipos, como Smalltalk, esta cuestión no surge, por lo que el concepto carece de utilidad.

Al uso de una clase con parámetro, tal como *Conjunto<Empleado>* mostrado antes, se le llama **elemento enlazado** (bound).

Se puede mostrar un elemento enlazado de dos maneras. La primera refleja la sintaxis C++ (véase la figura 5-16).

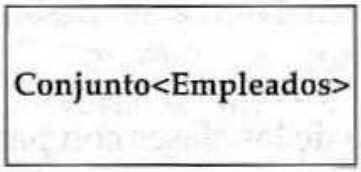


Figura 5-16: *Elemento enlazado (versión 1)*

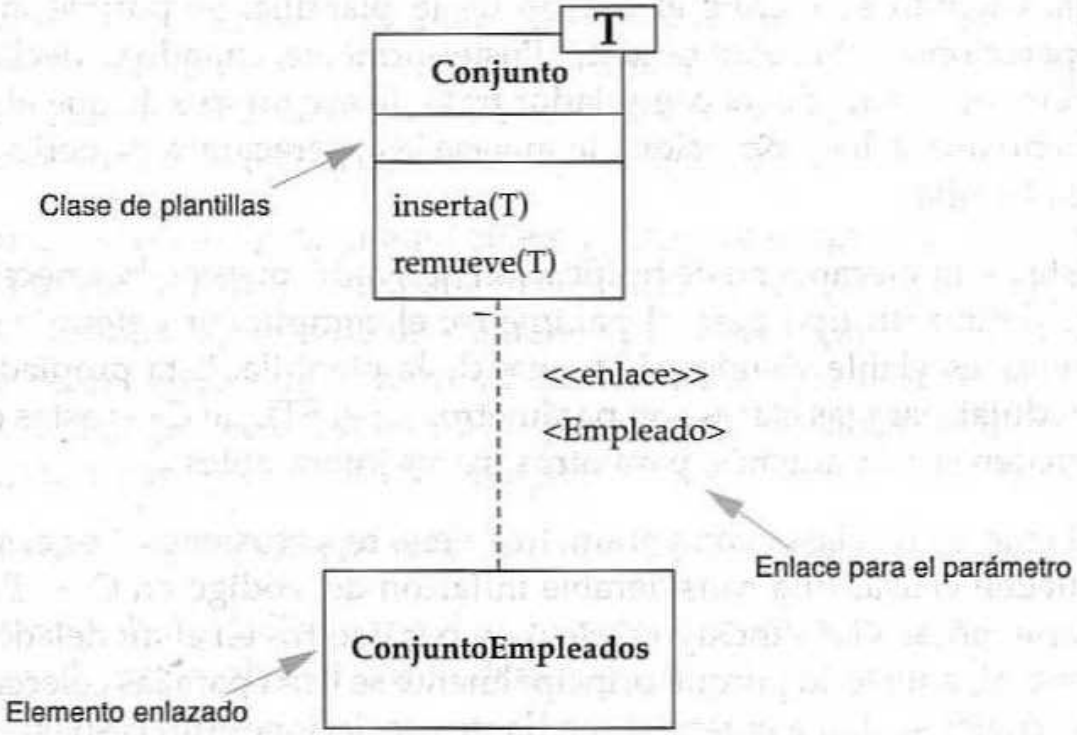


Figura 5-17: *Elemento enlazado (versión 2)*

La anotación alternativa (véase la figura 5-17) refuerza el vínculo con la plantilla y permite renombrar el elemento enlazado.

El estereotipo <<vínculo>> es un estereotipo en la relación de refinamiento. Esta relación indica que el ConjuntoEmpleados se conformará con la interfaz de Conjunto. En términos de especificación, el ConjuntoEmpleados es un subtipo de Conjunto. Esto se ajusta al otro modo de implementar las colecciones específicas de tipos, que consiste en declarar todos los subtipos apropiados.

Sin embargo, usar un elemento enlazado *no* es lo mismo que subtipificar. No se permite añadir características al elemento enlazado: es especificado en su totalidad por su plantilla; sólo se está añadiendo información de tipo restrictivo. Si se desea agregar características, habrá que crear un subtipo.

Aunque el empleo clásico de las clases con parámetros es con colecciones, hay muchas otras maneras de utilizarlas en C++ (véase Koenig, 1996, para otras ideas).

Las clases con parámetros le permiten a usted usar tipificación derivada. Cuando se escribe el cuerpo de la plantilla, es posible invocar operaciones sobre el parámetro. Posteriormente, cuando se declara un elemento enlazado, el compilador trata de asegurarse de que el parámetro que se ha proporcionado maneje las operaciones requeridas por la plantilla.

Éste es un mecanismo de tipificación derivada, pues no hay necesidad de definir un tipo para el parámetro; el compilador determina si el enlace es viable viendo a la fuente de la plantilla. Esta propiedad es medular para las clases con parámetros en el STL de C++; estas clases pueden servir, además, para otros trucos interesantes.

El empleo de clases con parámetros tiene repercusiones. Por ejemplo, pueden causar una considerable inflación del código en C++. Por mi parte, pocas veces incluyo clases con parámetros en el modelado conceptual, ante todo porque principalmente se usan para las colecciones, las cuales se dan a entender mediante asociaciones (un caso en el que sí las utilizo es en el patrón de *Intervalo* (*Range*); véase Fowler, 1997). Sólo uso clases con parámetros en el modelado de especificación y de implementación, si son manejadas por el lenguaje con el que estoy trabajando.

La visibilidad

Debo confesar que esta sección me produce cierta perturbación.

La **visibilidad** es uno de esos temas que, en principio, es simple, pero tiene sutilezas complejas. La idea básica es que cualquier clase tiene

elementos públicos y privados. Los elementos públicos pueden ser usados por cualquier otra clase; los elementos privados sólo pueden ser usados por la clase propietaria. Sin embargo, cada lenguaje tiene sus propias reglas. Aunque todos los lenguajes utilizan términos como "público", "privado" y "protegido" (public, private, protected), su significado varía según el lenguaje. Estas diferencias son pequeñas, pero generan confusión, en especial para quienes trabajamos con más de un lenguaje.

El UML trata de resolver este problema, sin complicarse demasiado. En esencia, en el UML se puede etiquetar cualquier atributo u operación con un indicador de visibilidad. Se puede usar el marcador que se quiera y su significado depende del lenguaje. Sin embargo, el UML proporciona tres abreviaciones de visibilidad (un tanto difíciles de recordar): + (público), - (privado), y # (protegido).

Siento la tentación de dejar aquí el tema pero, por desgracia, cada uno dibuja la visibilidad de una manera específica. Por tanto, para entender en realidad algunas de las diferencias comunes que existen entre los modelos es necesario comprender los enfoques que adoptan los diversos lenguajes, con respecto a la visibilidad. Así pues, aspiremos profundo y sumerjámonos en el fango.

Iniciamos con C++, ya que es la base del empleo regular del UML.

- Un miembro público es visible en todo el programa y puede ser llamado por cualquier objeto dentro del sistema.
- Un miembro privado sólo puede ser usado por la clase que lo define.
- Un miembro protegido sólo puede ser usado por (a) la clase que lo define, o (b) una subclase de esa clase.

Considérese una clase Cliente que tiene una subclase Cliente personal. Considérese también el objeto Martín, que es una instancia de Cliente personal. Martín puede usar cualquier miembro público de cualquier objeto del sistema. También puede usar cualquier miembro privado de la clase Cliente personal. Martín *no* puede usar ningún miembro privado definido en Cliente; sin embargo, sí puede usar los miembros protegidos de Cliente y los usuarios protegidos de Cliente personal.

Ahora veamos Smalltalk. En este lenguaje, todas las variables de instancia son privadas y todas las operaciones son públicas. Sin embargo, lo privado no significa lo mismo en Smalltalk que en C++. En el sistema de Smalltalk, Martín puede acceder a cualquier variable de instancia dentro de su propio objeto, haya sido definida tal variable de instancia dentro de Cliente o de Cliente personal. Así pues, en cierto sentido, lo privado en Smalltalk es similar a lo protegido en C++.

Si aquí acabara la cosa, sería demasiado simple.

Volvamos a C++. Digamos que tenemos otra instancia de Cliente personal llamado Kendall. Kendall también puede acceder a cualquier miembro de Martín que se haya definido como parte de la clase Cliente personal, ya sea público, privado o protegido. Kendall también puede acceder a todos los miembros protegidos y públicos de Martín que se haya definido dentro de Cliente. Sin embargo, en Smalltalk, Kendall no podrá acceder a las variables de instancia privadas de Martín. Sólo podrá hacerlo a las operaciones públicas de Martín.

En C++, se puede acceder a miembros de otros objetos de su propia clase, del mismo modo que se accede a los miembros propios. En Smalltalk, no tiene importancia si otro objeto es de la misma clase o no; se puede acceder únicamente a las partes públicas de otro objeto.

Yo prefiero no acceder a miembros privados o protegidos de otros objetos de la misma clase. Muchos otros se apegan a esta convención (no estipulada).

Java es similar a C++ en el hecho de que ofrece libre acceso a los miembros de otros objetos de su misma clase. Java también añade un nuevo nivel de visibilidad al que llama paquete (package). Un miembro con visibilidad de paquete sólo puede ser accedido por instancias de otras clases dentro del mismo paquete.

Siguiendo con nuestro tema, para asegurar que las cosas no sean tan sencillas, Java redefine levemente la visibilidad protegida. En Java, a un miembro protegido puede accederse por subclases, pero también por cualquier otra clase que esté en el mismo paquete que la clase propietaria. Esto quiere decir que, en Java, protegido es más público que paquete.

Java también permite que las clases sean marcadas como públicas o paquetes. Los miembros públicos de una clase pública pueden ser usados por cualquier clase que importe el paquete al que pertenezca la clase. Una clase de paquete puede ser usada sólo por otras clases del mismo paquete.

C++ añade el toque final. Un método o clase C++ puede ser convertido en amigo de una clase. Un **amigo (friend)** tiene completo acceso a todos los miembros de una clase. De aquí viene la frase: "en C++, los amigos se tocan entre ellos las partes privadas".

Cuando esté manejando visibilidad, emplee las reglas del lenguaje con que trabaja. Cuando vea un modelo UML que provenga de otra parte, desconfíe del significado de los marcadores de visibilidad y tenga presente la manera en que pueden cambiar tales significados de un lenguaje a otro.

En general, encuentro que las visibilidades cambian a medida que se trabaja en el código, así que no las tome demasiado en cuenta, desde un principio.

Características del alcance de clase

Se pueden definir operaciones o atributos en el nivel de alcance de clase. Esto significa que éstos son características de la clase, en lugar de ser de cada objeto. Estas operaciones o atributos corresponden a los miembros estáticos en C++ y Java y a los métodos y variables de clase en Smalltalk. Estas características se muestran de la misma manera que cualquier otra operación o atributo, a excepción de que se subrayan.

Capítulo 6

Diagramas de interacción

Los **diagramas de interacción** son modelos que describen la manera en que colaboran grupos de objetos para cierto comportamiento.

Habitualmente, un diagrama de interacción capta el comportamiento de un solo caso de uso. El diagrama muestra cierto número de ejemplos de objetos y los mensajes que se pasan entre estos objetos dentro del caso de uso.

Ilustraré este enfoque mediante un caso de uso simple que exhibe el comportamiento siguiente:

- La ventana Entrada de pedido envía un mensaje "prepara" a Pedido.
- El Pedido envía entonces un mensaje "prepara" a cada Línea de pedido dentro del Pedido.
- Cada Línea de pedido revisa el Artículo de inventario correspondiente.
  - Si esta revisión devuelve "verdadero", la Línea de pedido descuenta la cantidad apropiada de Artículo de inventario del almacén.

– Si no sucede así, quiere decir que la cantidad del Artículo de inventario ha caído más abajo del nivel de reorden y entonces dicho Artículo de inventario solicita una nueva entrega.

Hay dos tipos de diagramas de interacción: diagramas de secuencia y diagramas de colaboración.

Diagramas de secuencia

En un diagrama de secuencia, un objeto se muestra como caja en la parte superior de una línea vertical punteada (véase la figura 6-1).

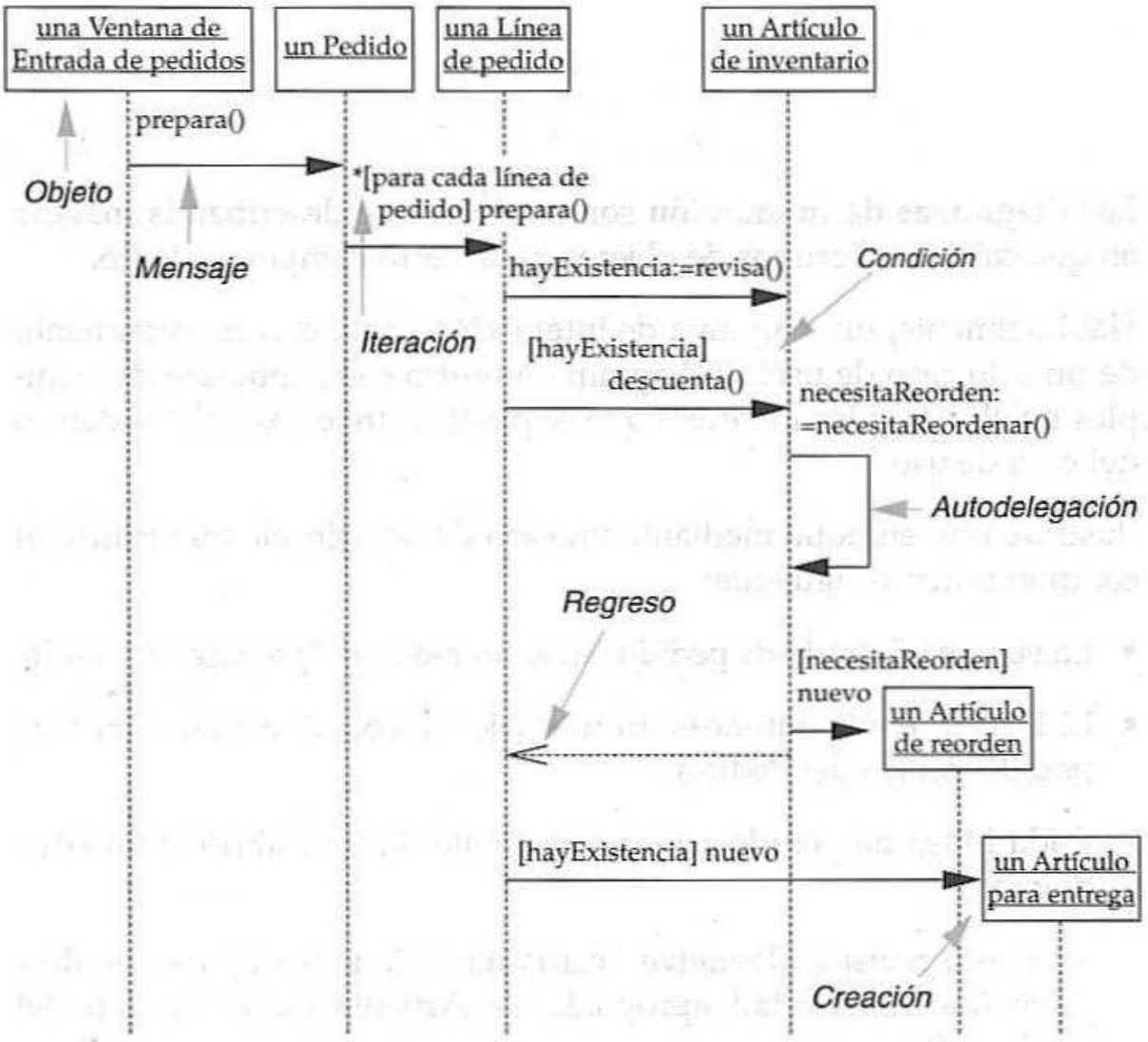


Figura 6-1: *Diagrama de secuencia*

Esta línea vertical se llama **línea de vida** del objeto. La línea de vida representa la vida del objeto durante la interacción. Esta forma fue popularizada inicialmente por Jacobson.

Cada mensaje se representa mediante una flecha entre las líneas de vida de dos objetos. El orden en el que se dan estos mensajes transcurre de arriba hacia abajo. Cada mensaje es etiquetado por lo menos con el nombre del mensaje; pueden incluirse también los argumentos y alguna información de control, y se puede mostrar la **autodelegación**, que es un mensaje que un objeto se envia a sí mismo, regresando la flecha de mensaje de vuelta a la misma línea de vida.

Dos partes de la información de control son valiosas. Primero, hay una **condición**, que indica cuándo se envía un mensaje (por ejemplo, *[necesitaReorden]*). El mensaje se envía sólo si la condición es verdadera. El segundo marcador de control útil es el **marcador de iteración**, que muestra que un mensaje se envía muchas veces a varios objetos receptores, como sucedería cuando se itera sobre una colección. La base de la iteración se puede mostrar entre corchetes (como en *[para cada línea de pedido]).

Como se puede apreciar, la figura 6-1 es muy simple y tiene un atractivo visual inmediato; en ello radica su gran fuerza.

Una de las cuestiones más difíciles de comprender en un programa orientado a objetos es el flujo de control general. Un buen diseño tiene muchísimos pequeños métodos en diferentes clases, y a veces resulta muy complicado determinar la secuencia global de comportamiento. Podrá acabar leyendo el código, tratando de encontrar dónde está el programa. Esto ocurre así, en especial, para todos aquellos que comienzan a trabajar con objetos. Los diagramas de secuencia le ayudan a ver la secuencia.

Este diagrama incluye un **regreso**, el cual indica el regreso de un mensaje, no un nuevo mensaje. Los regresos difieren de los mensajes normales en que la línea es punteada.

Los diagramas POSA (Buschmann *et al.*, 1996), en los que se basa una gran parte de las anotaciones de la notación de gráficas de secuencia de UML, emplean ampliamente los regresos. Yo no lo hago así. He observado que los regresos saturan el diagrama y tienden a oscurecer

el flujo. Todos los regresos se hallan implícitos por el modo como se secuencian los mensajes. Sólo utilizo regresos cuando aumentan la claridad del diagrama.

Mi consejo es mostrar los regresos cuando ayudan a aumentar la claridad. La única razón por la que usé un regreso en la figura 6-1 fue para demostrar la notación; si se elimina el regreso, a mi juicio el diagrama continúa tan claro como antes.

En el UML 1.0 los regresos se indicaban mediante puntas de flecha tipo pluma, en lugar de líneas punteadas. Las he encontrado tan difíciles de implementar que de todas maneras he utilizado las líneas punteadas, de modo que me alegra ver el cambio.

Esto lo menciono, debido a que quisiera ofrecer aquí un pequeño consejo: evite ir en contra de la notación del UML. Esta notación se volverá cabalmente comprendida, y hacer algo fuera de la norma dañará la comunicación del diseñador con otros diseñadores. Sin embargo, si hay algo que esté causando una gran confusión, yo haría algo fuera de la norma. Después de todo, el propósito principal del diagrama es la comunicación. Si es que llega a romper las reglas del UML, hágalo muy de vez en cuando y defina con claridad lo que ha hecho.

Procesos concurrentes y activaciones

Los diagramas de secuencia son valiosos también para los procesos concurrentes.

En la figura 6-2 vemos algunos objetos que revisan una transacción bancaria.

Cuando se crea una Transacción, ésta crea un Coordinador de transacción que coordina el trámite de la Transacción. Este coordinador crea una cantidad (en el presente caso, dos) de objetos Revisor de transacción, cada uno de los cuales es responsable de una revisión en particular. Este proceso permitirá añadir con facilidad otros procesos de revisión, porque cada registro es llamado asincrónicamente y opera en paralelo.

Cuando termina un Revisor de transacción, se lo notifica al Coordinador de transacción. El coordinador comprueba si todos los revisores respondieron; de no ser así, no hace nada. Si todos han respondido in-

dicando terminaciones exitosas, como en el presente caso, entonces el coordinador notifica a la Transacción que todo está bien.

La figura 6-2 introduce varios elementos nuevos en los diagramas de secuencia. En primer lugar, se ven las **activaciones**, que aparecen explícitamente cuando está activo un método, ya sea porque está efectuando operaciones o porque se encuentra esperando la devolución de una subrutina. Muchos diseñadores utilizan las activaciones todo el tiempo. A mi juicio, éstas no aportan mucho a la ejecución de procedimientos. Por tanto, sólo las uso en situaciones concurrentes.

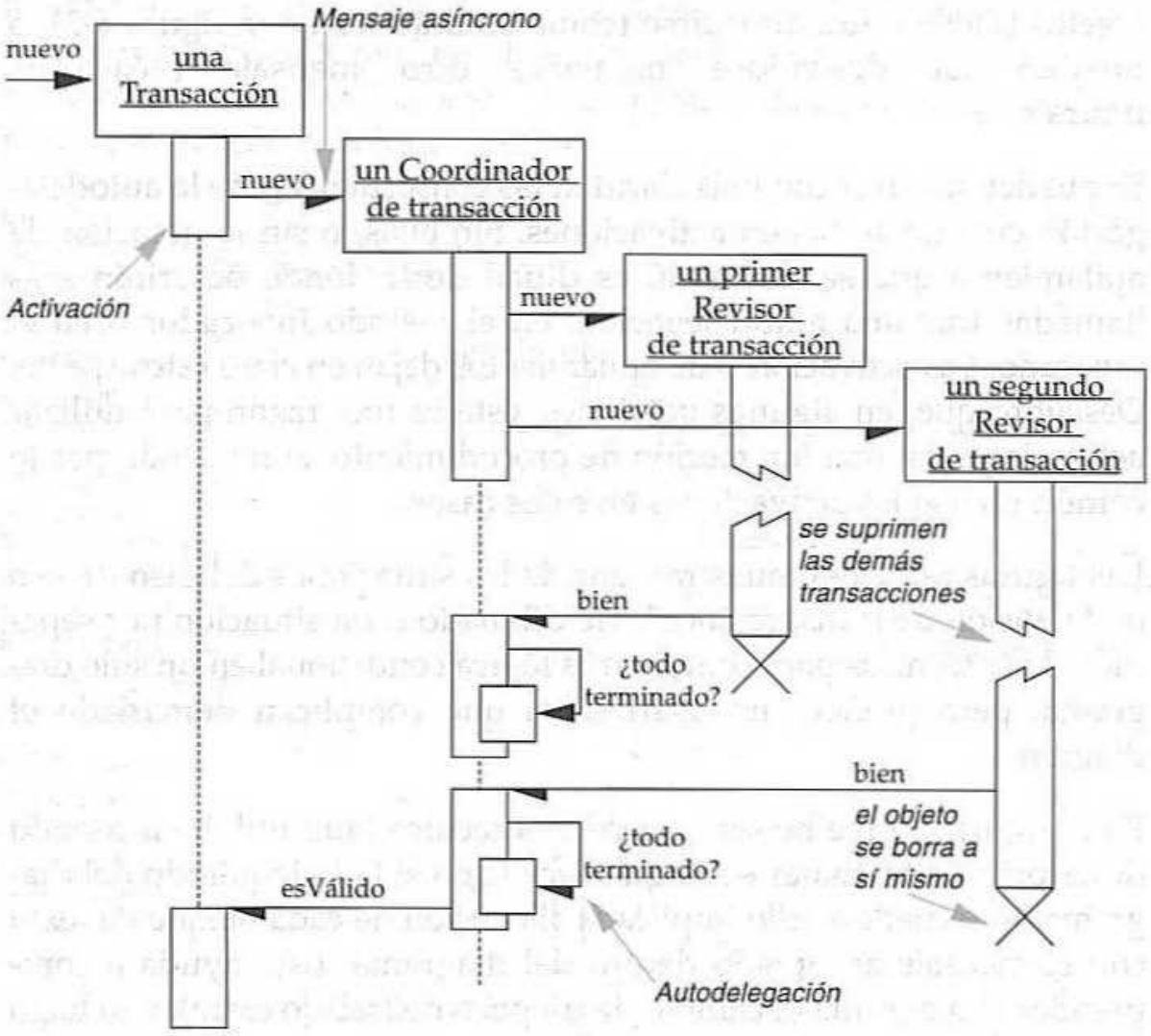


Figura 6-2: *Procesos y activaciones concurrentes*

Las medias cabezas de flecha indican mensajes **asíncronos**. Un mensaje asíncrono no bloquea al invocador, por lo cual puede continuar con su proceso. Un mensaje asíncrono puede hacer alguna de estas tres cosas:

1. Crear un nuevo proceso, en cuyo caso se vincula con el principio de una activación.
2. Crear un nuevo objeto.
3. Comunicarse con un proceso que ya está operando.

El **borrado** (deletion) de un objeto se muestra con una *X* grande. Los objetos pueden autodestruirse (como se muestra en la figura 6-2), o pueden ser destruidos mediante otro mensaje (véase la figura 6-3).

Se pueden mostrar con más claridad las consecuencias de la autodelegación cuando se tienen activaciones. Sin ellas, o sin la notación de apilamiento que se usa aquí, es difícil decir dónde ocurrirán más llamadas tras una autodelegación: en el método invocador o en el invocado. Las activaciones de apilamientos dejan en claro este aspecto. Descubro que, en algunas ocasiones, ésta es una razón para utilizar activaciones en una interacción de procedimiento, aun cuando por lo común no uso las activaciones en estos casos.

Las figuras 6-2 y 6-3 muestran dos de las situaciones del caso de uso de "revisión de transacciones". He dibujado cada situación por separado. Hay técnicas para combinar la lógica condicional en un solo diagrama, pero prefiero no usarlas, ya que complican demasiado el diagrama.

En la figura 6-3 me he servido de una técnica muy útil: he insertado descripciones textuales de lo que sucede en el lado izquierdo del diagrama de secuencia. Ello implica la alineación de cada bloque de texto con el mensaje apropiado dentro del diagrama. Esto ayuda a comprender el diagrama (a cambio de un poco de trabajo extra), y lo hago con aquellos documentos que conservaré, pero no con bosquejos desde cero.

Diagramas de colaboración

La segunda forma del diagrama de interacción es el **diagrama de colaboración**.

En los diagramas de colaboración, los objetos ejemplo se muestran como iconos. Las flechas indican, como en los diagramas de secuencia, los mensajes enviados dentro del caso de uso dado. Sin embargo, en esta ocasión la secuencia se indica numerando los mensajes.

El numerar los mensajes dificulta más ver la secuencia que poner las líneas verticales en la página. Por otra parte, la disposición espacial del diagrama permite mostrar otras cosas mejor. Se puede mostrar cómo se vinculan entre ellos los objetos y emplear la disposición para sobreponer paquetes u otra información.

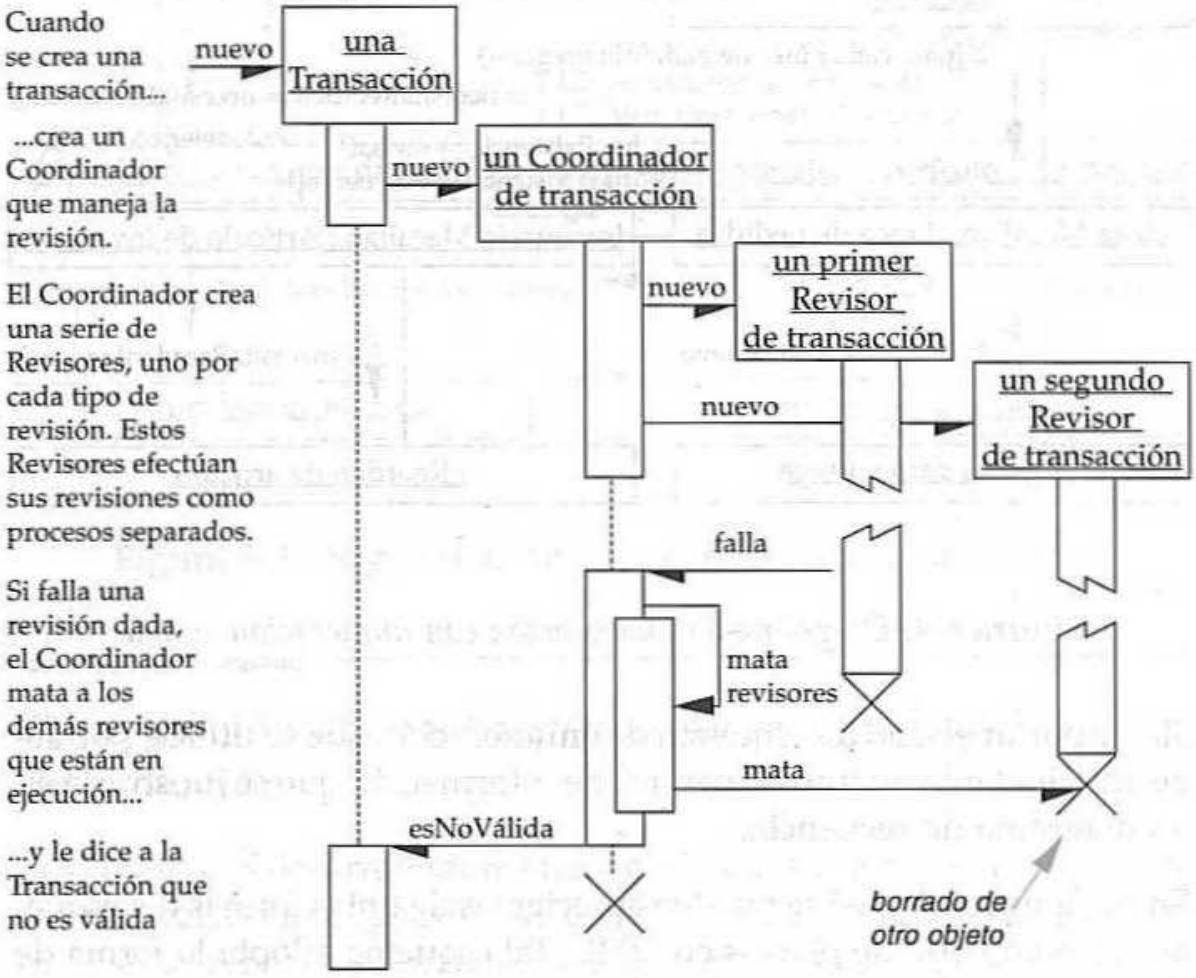


Figura 6-3: *Diagrama de secuencia: revisión de fallas*

Se puede usar alguno de los diversos esquemas de numeración para los diagramas de colaboración. El más simple se ilustra en la figura 6-4. Otro enfoque comprende un esquema de numeración decimal, el cual aparece en la figura 6-5.

En el pasado, el esquema más utilizado era el de la numeración simple. El UML utiliza el esquema decimal, pues hace evidente qué operación llama a otra y cuál es esa otra, aunque pueda ser más difícil apreciar la secuencia general.

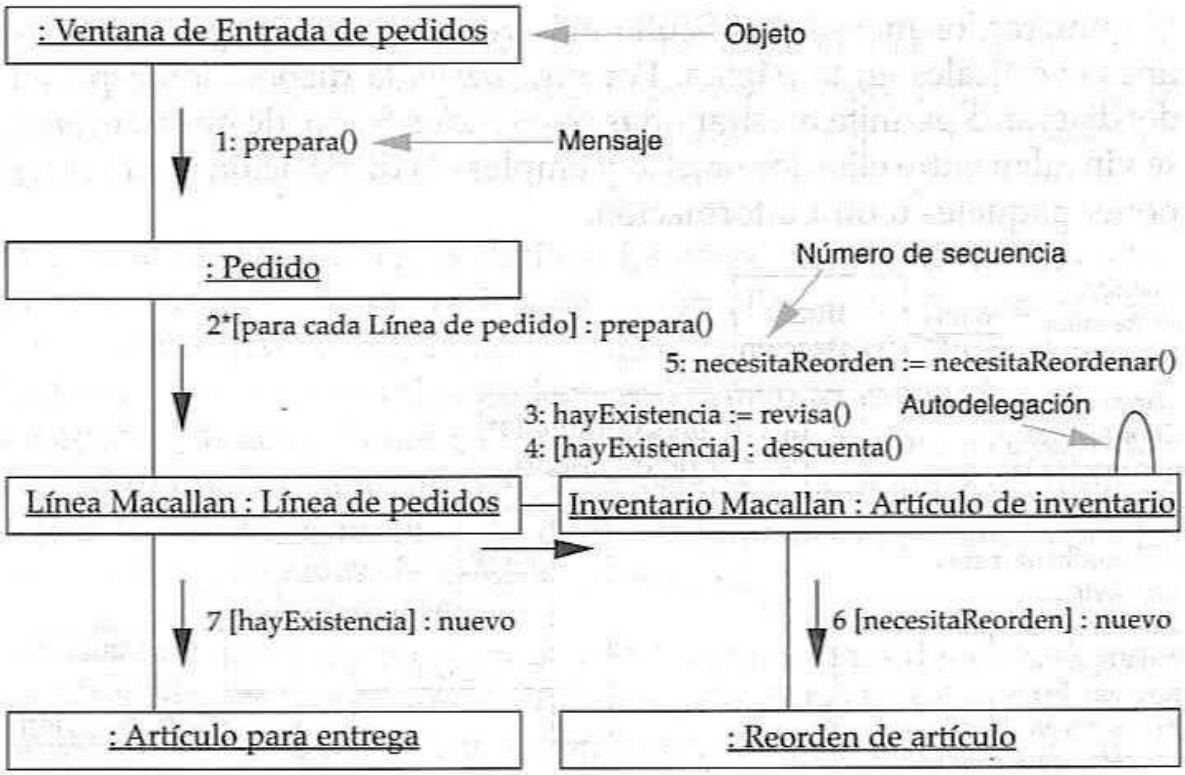


Figura 6-4: *Diagrama de colaboración con numeración simple*

Sin importar el tipo de esquema de enumeración que se utilice, se puede añadir el mismo tipo de control de información que se mostraría en un diagrama de secuencia.

En las figuras 6-4 y 6-5 se pueden apreciar las distintas formas del esquema de nombrado de objetos en UML. Tal esquema adopta la forma de *nombreObjeto : NombreClase*, donde se puede omitir el nombre del objeto o el de la clase. Obsérvese que, si se omite el nombre del objeto, hay que

conservar los dos puntos (:), para que quede claro que es el nombre de la clase y no el del objeto. Así, el nombre "Línea Macallan : Línea de pedidos" indica una instancia de Línea de pedidos llamada Línea Macallan (debo decir que éste es un pedido que yo apreciaría de modo particular). Acostumbro a nombrar objetos con el estilo de Smalltalk que empleé en los diagramas de secuencia (este esquema es válido en UML, pues "unObjeto" es un nombre perfectamente correcto para un objeto).

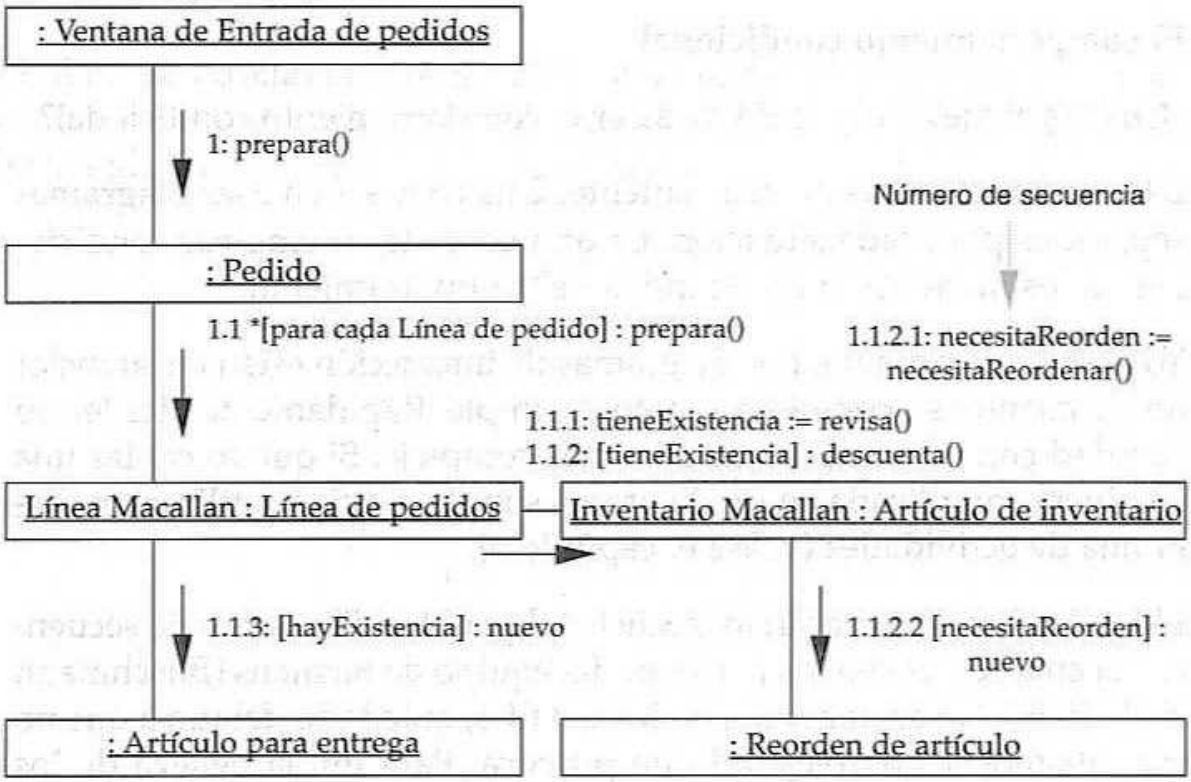


Figura 6-5: *Diagrama de colaboración con numeración decimal*

Comparación de los diagramas de secuencia y de colaboración

Los diferentes desarrolladores tienen distintas preferencias cuando se trata de seleccionar la forma de diagrama de interacción que emplearán. Por lo general, yo prefiero el diagrama de secuencia, debido a que me gusta el énfasis que pone en la secuencia; es fácil apreciar el orden en el que ocurren las cosas. Otros, en cambio, prefieren el diagrama de

colaboración, porque pueden usar la distribución para indicar cómo se conectan estáticamente los objetos.

Una de las características principales de ambos tipos de diagrama de interacción es su simplicidad. Se pueden ver con facilidad los mensajes mirando tan sólo el diagrama. Sin embargo, si trata de representar algo más que un simple proceso secuencial, sin mucho comportamiento condicional o de ciclos, la técnica comienza a fallar.

El comportamiento condicional

¿Cuál es el mejor modo de mostrar el comportamiento condicional?

Existen dos escuelas de pensamiento. Una consiste en usar diagramas separados para cada situación. La otra consiste en emplear condiciones en los mensajes, a fin de indicar el comportamiento.

Yo prefiero la primera. Los diagramas de interacción están en su mejor forma cuando su comportamiento es simple. Rápidamente pierden su claridad con un comportamiento más complejo. Si quiero captar una conducta complicada en un diagrama simple, prefiero utilizar un diagrama de actividades (véase el capítulo 9).

El UML ofrece mucha sintaxis adicional para los diagramas de secuencia, la cual se basa en los patrones del equipo de Siemens (Buschmann *et al.*, 1996). No entraré aquí en los detalles, ante todo debido a que no me satisface la complejidad que provoca. Para mí, la belleza de los diagramas de interacción reside en su simplicidad y muchas de las notaciones adicionales la echan a perder en su intento por ser completos desde el punto de vista computacional. Le aconsejo que no se precipite utilizando las formas más complejas de los diagramas de interacción, pues quizá encuentre que los más simples tienen un valor superior.

Cuándo utilizar los diagramas de interacción

Se deberán usar los diagramas de interacción cuando se desee ver el comportamiento de varios objetos en un caso de uso. Son buenos para mostrar la colaboración entre los objetos; sin embargo, no sirven tan bien para la definición precisa del comportamiento.

Si desea ver el comportamiento de un solo objeto a través de muchos casos de uso, use un diagrama de estado de transición (véase el capítulo 8). Si quiere ver el comportamiento a través de muchos casos de uso o muchos procesos, considere un diagrama de actividad (véase el capítulo 9).

Para mayor información

El libro de Buschmann *et al.* (1996) emplea muchas extensiones que actualmente son parte de la mezcla del UML y le dará a usted una buena idea de lo que se está elaborando.

Capítulo 7

Diagramas de paquetes

Una de las preguntas más antiguas en los métodos de software es: ¿cómo se puede fragmentar un sistema grande en sistemas más pequeños? Preguntamos esto porque, en la medida en que los sistemas se hacen más grandes, se vuelve más difícil comprenderlos, así como entender sus cambios.

Los métodos estructurados se valieron de la **descomposición funcional**, en la cual el sistema en su conjunto se correlacionaba como función y se dividía en subfunciones, que a su vez se dividían en otras subfunciones, y así sucesivamente. Las funciones eran como los casos de uso en un sistema orientado a objetos, en el que las funciones representaban algo que hacía el sistema como un todo.

Eran los días en que el proceso y los datos estaban separados. De tal modo que, además de una descomposición funcional, también había una estructura de datos. Esta última ocupaba el segundo lugar, aunque ciertas técnicas de ingeniería de información agrupaban los registros de datos en áreas temáticas y producían matrices que mostraban la interrelación entre las funciones y los registros de datos.

Es desde este punto de vista que podemos apreciar el gran cambio que han significado los objetos. Ha desaparecido la separación entre el

proceso y los datos, y la descomposición funcional, pero la vieja pregunta sigue en pie. Una idea es agrupar las clases en unidades de nivel más alto. Esta idea aparece, aplicada de manera muy libre, en muchos métodos orientados a objetos. En el UML, a este mecanismo de agrupamiento se le llama **paquete**.

La idea de un paquete se puede aplicar a cualquier elemento de un modelo, no sólo a las clases. Sin cierta heurística que agrupe las clases, el agrupamiento se vuelve arbitrario. El que yo he encontrado más útil, que también es el que recibe mayor énfasis en el UML, es la de pendencia. Empleo el término **diagrama de paquetes** para indicar un diagrama que muestra los paquetes de clases y las dependencias entre ellos.

Hablando estrictamente, los paquetes y las dependencias son elementos de un diagrama de clases, por lo cual un diagrama de paquetes es sólo una forma de un diagrama de clases. En la práctica dibujo estos diagramas por diferentes razones, así que me gusta manejar nombres diferentes.

Existe una dependency **dependencia** entre dos elementos si los cambios a la definición de un elemento pueden causar cambios al otro. En las clases, la dependencia existe por varias razones: una clase envía un mensaje a otra; una clase tiene a otra como parte de sus datos; una clase menciona a otra como parámetro para una operación. Si una clase cambia su interfaz, entonces los mensajes que envía pueden dejar de ser válidos.

En forma ideal, sólo los cambios a una interfaz de clase deberían afectar a otra clase. El arte del diseño en gran escala implica minimizar las dependencias, de modo tal que se reduzcan los efectos del cambio y se requiera de menos esfuerzo para cambiar el sistema.

En la figura 7-1 tenemos las clases de dominio que modelan el negocio, las cuales se agrupan en dos paquetes: Pedidos y Clientes. Ambos paquetes son parte de un paquete que abarca todo el dominio. La aplicación de Captura de pedidos tiene dependencias con los dos paquetes del dominio. La Interfaz de Usuario (IU) para Captura de pedidos

tiene dependencias con la Aplicación Captura de pedidos y con AWT (un juego de herramientas GUI de Java).

Existe una dependencia entre dos paquetes si existe algún tipo de dependencia entre dos clases cualquiera en los paquetes. Por ejemplo, si cualquier clase en el paquete Lista de correo depende de cualquier clase del paquete Clientes, entonces se da una dependencia entre sus paquetes correspondientes.

Existe una similitud obvia entre dependencia de paquetes y dependencias de compilación. Pero también, de hecho, hay una diferencia vital: con los paquetes, las dependencias no son transitivas.

Un ejemplo de relación **transitiva** es aquella en la que Jim tiene una barba más larga que Grady y éste, una más larga que Ivar, por lo que se deduce que Jim tiene una barba más larga que Ivar. Otros ejemplos incluyen relaciones como "está al norte de" y "es más alto que". Por otra parte, "es un amigo de" no constituye una relación transitiva.

Para apreciar por qué es importante esto para las dependencias, obsérvese de nuevo la figura 7-1. El cambio a una clase del paquete Pedidos no indica que el paquete IU Captura de pedidos deba ser cambiado. Indica tan sólo que debe revisarse el paquete de aplicación Captura de pedidos para ver si cambia. Sólo si se altera la interfaz del paquete de aplicación Captura de pedidos hay necesidad de cambiar el paquete IU Captura de pedidos. Si esto es así, la aplicación Captura de pedidos está protegiendo a la IU Captura de pedidos de cambios a los pedidos.

Este comportamiento representa el propósito clásico de una arquitectura en capas. De hecho, ésta es la semántica del comportamiento de las "imports" de Java, pero no la del comportamiento de los "include" de C/C++, ni la de los prerrequisitos de Envy. El include de C/C++ es transitivo, lo que significa que la IU Captura de pedido sería dependiente del paquete de Pedidos. Una dependencia transitiva hace difícil limitar el alcance de los cambios mediante la compilación.

¿Qué significa trazar una dependencia con un paquete que contenga subpaquetes? Los diseñadores se sirven de convenciones diferentes.

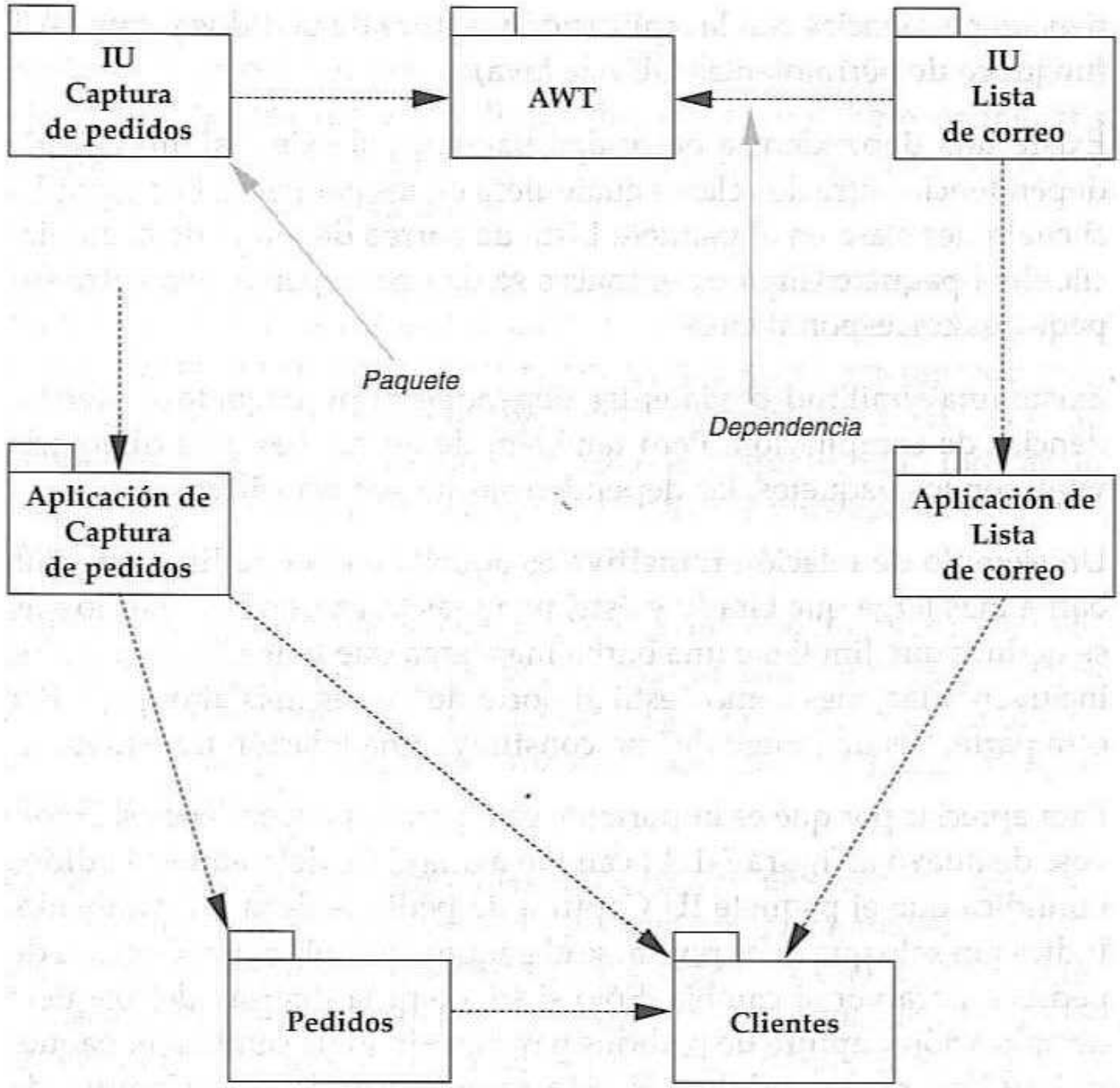


Figura 7-1: *Diagrama de paquetes*

Algunos suponen que dibujar una dependencia hacia un paquete "contenedor" da visibilidad al contenido de todos los paquetes contenidos y a sus respectivos contenidos. Otros consideran que sólo se ven las clases dentro del paquete contenedor, pero no las clases dentro de los paquetes anidados (o sea, la visión es opaca).

Debe declararse explícitamente la convención que se está utilizando en el proyecto o dejarla planteada de manera muy clara colocando estereotipos en los paquetes. Sugiero usar el estereotipo <<transparente>> para indicar que se pueden ver los paquetes anidados, y usar el estereotipo <<opaco>> para indicar que no se pueden ver. Aquí, mi convención es que los paquetes sean transparentes.

¿Qué es lo que se ve, si se tiene una dependencia hacia un paquete? En esencia, se ven todas las clases públicas del paquete y todos sus métodos públicos. Bajo el esquema de visibilidad de C++, esto puede causar un problema, debido a que tal vez se quiera tener una clase que contenga métodos que pueden ser vistos por otros objetos dentro del mismo paquete, pero no por objetos que pertenezcan a otros paquetes.

Ésta es la razón por la cual Java tiene la visibilidad de paquete. Por supuesto, esto le simplifica las cosas a Java. Dentro de C++ se pueden marcar clases y operaciones con visibilidad de paquetes. Aun cuando esta convención no es aplicada por el compilador, sigue siendo útil para el diseño.

Una técnica eficaz aquí es reducir aún más la interfaz del paquete exportando sólo un pequeño subconjunto de las operaciones asociadas con las clases del paquete. Esto se puede hacer dando a todas las clases visibilidad de paquete, de tal modo que sólo puedan ser vistas por otras clases del mismo paquete y añadiendo otras clases públicas para el comportamiento público. Estas clases adicionales, llamadas Fachadas (*Facades*) (Gamma *et al.*, 1994), delegan operaciones públicas a sus compañeras más tímidas dentro del paquete.

Los paquetes no dan respuestas sobre la manera de reducir las dependencias en el sistema, pero sí ayudan a ver cuáles son las dependencias, y sólo se puede efectuar el trabajo para reducirlas, cuando es posible verlas. Los diagramas de paquetes son, desde mi punto de vista, una herramienta clave para mantener el control sobre la estructura global de un sistema.

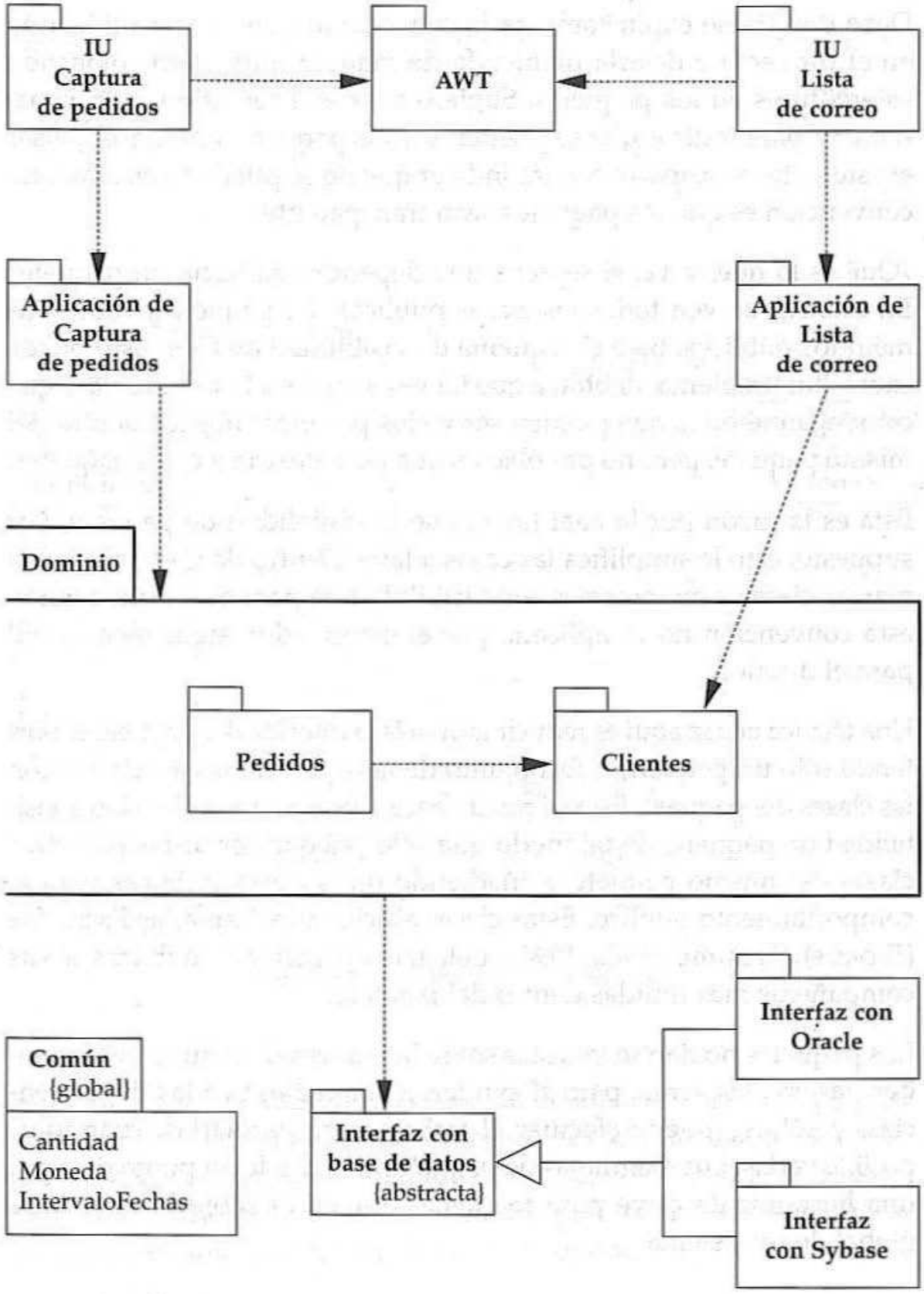


Figura 7-2: *Diagrama de paquetes avanzado*

La figura 7-2 es un diagrama de paquetes más complejo que contiene artificios adicionales.

En primer lugar, vemos que he añadido un paquete de Dominio que contiene paquetes de pedidos y clientes. Esto es de utilidad, pues significa que puedo trazar dependencias desde y hacia el paquete general, en lugar de trazar muchas dependencias separadas.

Cuando se muestra el contenido de un paquete, se pone el nombre del paquete en una "etiqueta" y el contenido dentro del cuadro principal. Estos contenidos pueden ser una lista de clases (tal y como sucede en el paquete Común), otro diagrama de paquetes (como en Dominio) o un diagrama de clases (que no se muestra, pero el concepto ya debe ser obvio).

La mayor parte de las veces, considero suficiente listar las clases clave, pero en algunas ocasiones es útil otro diagrama. En el presente caso he mostrado que, mientras la aplicación Captura de órdenes tiene una dependencia con todo el paquete Dominio, la aplicación Lista de correo sólo depende del paquete Clientes. Estrictamente hablando, el mero listado de clases no es UML puro (se deben mostrar los iconos de clase), pero éste constituye una de las áreas en que me inclinaría a las reglas.

La figura 7-2 muestra el paquete Común marcado como {global}. Esto significa que todos los paquetes del sistema tienen una dependencia hacia él. Por supuesto, se debe emplear este artificio con moderación, pero las clases comunes (como Dinero) se emplean en todas partes.

Con los paquetes se puede aplicar la generalización. Esto significa que el paquete específico debe conformarse a la interfaz del paquete general. Esto es comparable con la perspectiva de especificación de la subtipificación en los diagramas de clases (véase el capítulo 4). Por tanto, de acuerdo con la figura 7-2, el agente de la base de datos puede usar la Interfaz con Oracle o la Interfaz con Sybase. Cuando se usa la generalización de este modo, el paquete general puede marcarse como {abstracto} para mostrar que sólo define una interfaz implementada por un paquete más específico.

La generalización implica una dependencia del subtipo al supertipo (no se necesita mostrar la dependencia extra; basta con la generalización misma). El poner las clases abstractas en un paquete de supertipo es una buena forma de romper ciclos en la estructura de dependencias. En tal situación, los paquetes de interfaz con la base de datos son los responsables de cargar y guardar los objetos de dominio en una base de datos. Por lo tanto, necesitan saber sobre los objetos del dominio. Los objetos del dominio, sin embargo, deben disparar las operaciones de carga y guardado.

La generalización nos permite poner la interfaz disparadora necesaria (varias operaciones de carga y guardado) en el paquete de interfaz con la base de datos. Estas operaciones, posteriormente, son implementadas por clases dentro de los paquetes de subtipo. No necesitamos una dependencia entre el paquete de interfaz con la base de datos y el paquete de interfaz con Oracle, ya que al momento de ejecución será en realidad el paquete de subtipo el que va a ser llamado por el dominio. Pero el dominio sólo piensa que está tratando con el paquete de interfaz (más simple) con la base de datos. El polimorfismo es tan útil para los paquetes como lo es para las clases.

Como regla general, es buena idea quitar los ciclos de la estructura de dependencias. No estoy convencido de que se puedan quitar todos los ciclos, pero ciertamente se deberán minimizar. Si se tienen, habrá que tratar de contenerlos dentro de un paquete contenedor más grande. En la práctica he encontrado casos en que no me ha sido posible evitar ciclos entre paquetes de dominio, pero de todas maneras trato de eliminarlos de las interacciones entre el dominio y las interfaces externas. La generalización de paquetes es un elemento clave para hacerlo.

En un sistema ya existente, las dependencias se pueden deducir observando las clases. Ésta es una tarea muy útil que puede ser llevada a cabo por una herramienta. Encuentro esto muy útil, si trato de mejorar la estructura de un sistema ya existente. Un paso útil inicial es dividir las clases en paquetes y analizar las dependencias entre estos últimos. Después realizo un reordenamiento de factores para reducir las dependencias.

Cuándo utilizar los diagramas de paquetes

Los paquetes son una herramienta vital para los proyectos grandes. Úselos siempre que un diagrama de clases que abarque todo el sistema ya no sea legible en una hoja de papel tamaño carta (o A4).

Usted querrá mantener sus dependencias al mínimo, ya que ello reduce el acoplamiento. Sin embargo, la heurística de esto no está bien comprendida.

Los paquetes son especialmente útiles para pruebas. Aunque yo escribo algunas pruebas para verificar clase por clase, prefiero hacer mis pruebas unitarias en el nivel de paquete por paquete. Cada paquete deberá tener una o más clases de pruebas que verifiquen su comportamiento.

Para mayor información

La fuente original de paquetes era Grady Booch (1994); los llamaba categorías de clase. Su análisis, sin embargo, era muy breve. El mejor estudio que conozco sobre este tema es el de Robert Martin (1995), cuyo libro da varios ejemplos de utilización de Booch y C++, prestando gran atención a la minimización de las dependencias. También puede encontrarse información valiosa en Wirfs-Brock (1990), autora que se refiere a los paquetes como subsistemas.

Capítulo 8

Diagramas de estados

Los **diagramas de estados** son una técnica conocida para describir el comportamiento de un sistema. Describen todos los estados posibles en los que puede entrar un objeto particular y la manera en que cambia el estado del objeto, como resultado de los eventos que llegan a él. En la mayor parte de las técnicas OO, los diagramas de estados se dibujan para una sola clase, mostrando el comportamiento de un solo objeto durante todo su ciclo de vida.

Existen muchas formas de diagramas de estados, cada una con semántica ligeramente diferente. La más popular que se emplea en las técnicas de OO se basa en la tabla de estados de David Harel (Vol. 8). OMT fue quien la usó por primera vez para los métodos de OO y fue adoptada por Grady Booch en su segunda edición (1994).

La figura 8-1 muestra un diagrama de estados de UML para un pedido del sistema de proceso de pedidos que presenté antes. El diagrama indica los diversos estados de un pedido.

Comenzamos en el punto de partida y mostramos una transición inicial al estado de Comprobación. Esta transición está etiquetada como "/obtener el primer artículo". La sintaxis de una etiqueta de transición tiene tres partes, las cuales son optativas: *Evento [Guard Guardia]*

/ *Acción*. En este caso, sólo tenemos la acción "obtiene primer artículo." Una vez realizada tal acción, entramos al estado de Comprobación. Este estado tiene una actividad asociada con él, la cual se indica mediante una etiqueta con la sintaxis *hace/actividad*. En este caso, la actividad se llama "comprueba artículo".

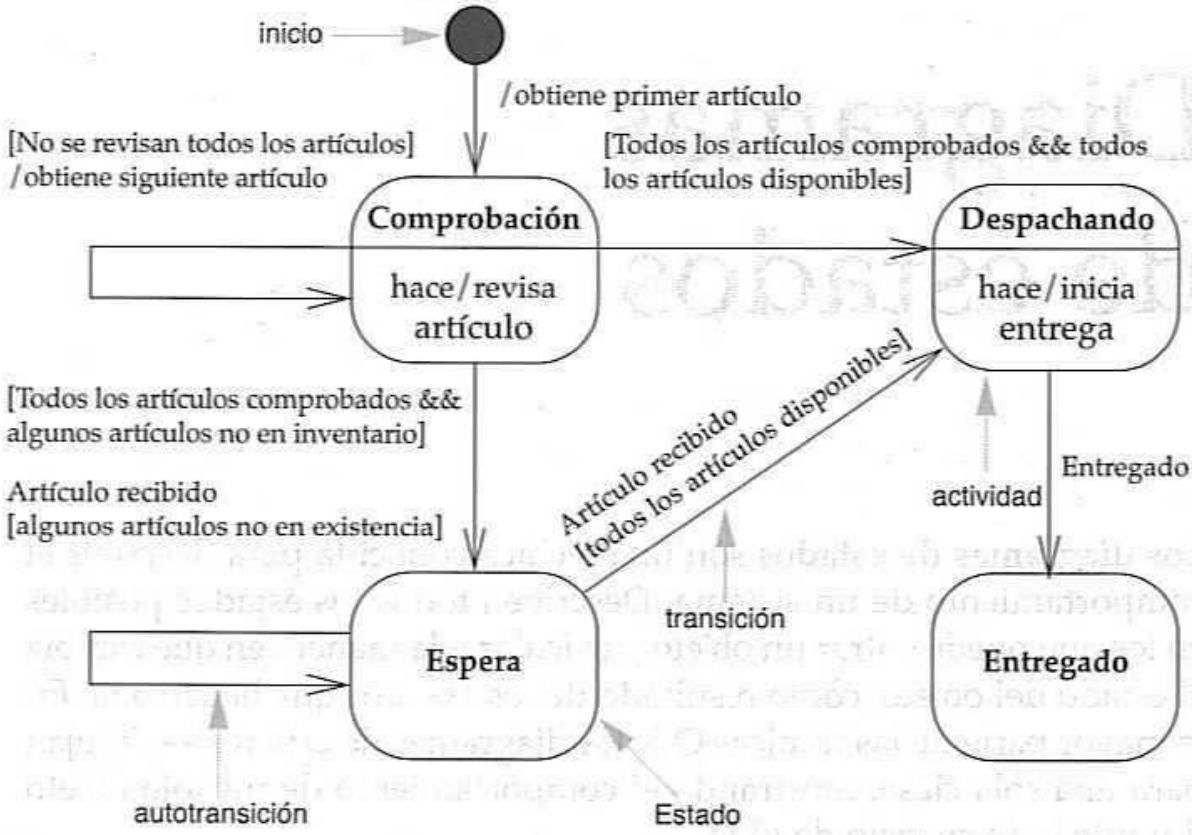


Figura 8-1: *Diagrama de estados*

Nótese que utilizo los términos "acción" para indicar la transición, y "actividad" para indicar el estado. Aunque ambos son procesos, implementados característicamente mediante algún método sobre Pedido, se tratan de manera diferente. Las **acciones** se asocian con las transiciones y se consideran como procesos que suceden con rapidez y no son interrumpibles. Las **actividades** se asocian con los estados y pueden tardar más. Una actividad puede ser interrumpida por algún evento.

Adviértase que la definición de "rápidamente" depende del tipo de sistema que se está produciendo. En un sistema de tiempo real, "Rápida-

mente" puede significar unas pocas instrucciones de código máquina; en los sistemas de información normales, "rápidamente" puede significar menos de unos cuantos segundos.

Cuando una transición no tiene evento alguno en su etiqueta, significa que la transición se da tan pronto como se completa cualquier actividad asociada con el estado dado. En este caso, ello significa tan pronto se termine la Comprobación. Del estado Comprobación se derivan tres transiciones. Las tres sólo tienen guardias en su etiqueta. Un **guardia** es una condición lógica que sólo devuelve "verdadero" o "falso." Una transición de guardia ocurre sólo si el guardia se resuelve como "verdadero".

Sólo se puede tomar una transición de un estado dado, por lo que tratamos de que los guardias sean mutuamente excluyentes para cualquier evento. En la figura 8-1 abarcamos tres condiciones:

1. Si no hemos comprobado todos los artículos, tomamos el siguiente artículo y regresamos al estado de Comprobación para comprobarlo.
2. Si hemos comprobado todos los artículos y todos están en existencia, hacemos la transición al estado de Despachando.
3. Si hemos comprobado todos los artículos pero no todos están en existencia, hacemos la transición al estado Espera.

Veamos, en primer lugar, el estado Espera. Como no hay actividades para este estado, el pedido se detiene en él esperando un evento. Ambas transiciones del estado Espera se etiquetan con el evento Artículo recibido. Esto significa que el pedido espera hasta que detecta este evento. Llegado ese momento, evalúa los guardias de las transiciones y hace la transición apropiada (ya sea a Despachando o de vuelta a Espera).

Dentro del estado Despachando, tenemos actividad que inicia una entrega. También hay una transición simple no guardada, disparada por el evento Entregado. Esto indica que la transición ocurrirá siempre que tenga lugar el evento. Sin embargo, nótese que la transición *no* sucede cuando termina la actividad; por el contrario, una vez terminada la actividad "iniciar entrega", el pedido se mantiene en el estado Despachando, hasta que ocurre el evento Entregado.

La última cuestión a considerar es la transición denominada "cancelado". Queremos tener la posibilidad de cancelar un pedido en cualquier momento, antes de que sea entregado. Podríamos hacerlo dibujando transiciones separadas desde cada uno de los estados, Comprobación, Espera y Despacho. Una alternativa útil es crear un **superestado** con los tres estados, y después dibujar una transición simple, a partir de él. Los subestados simplemente heredan todas las transiciones sobre el superestado.

Las figuras 8-2 y 8-3 muestran cómo estos enfoques reflejan el mismo comportamiento del sistema. La figura 8-2 aparece más bien cargada, a pesar de que sólo tiene tres transiciones duplicadas. La figura 8-3, en cambio, da un cuadro más claro y, si se necesitan posteriormente los cambios, será más difícil olvidar el evento cancelado.

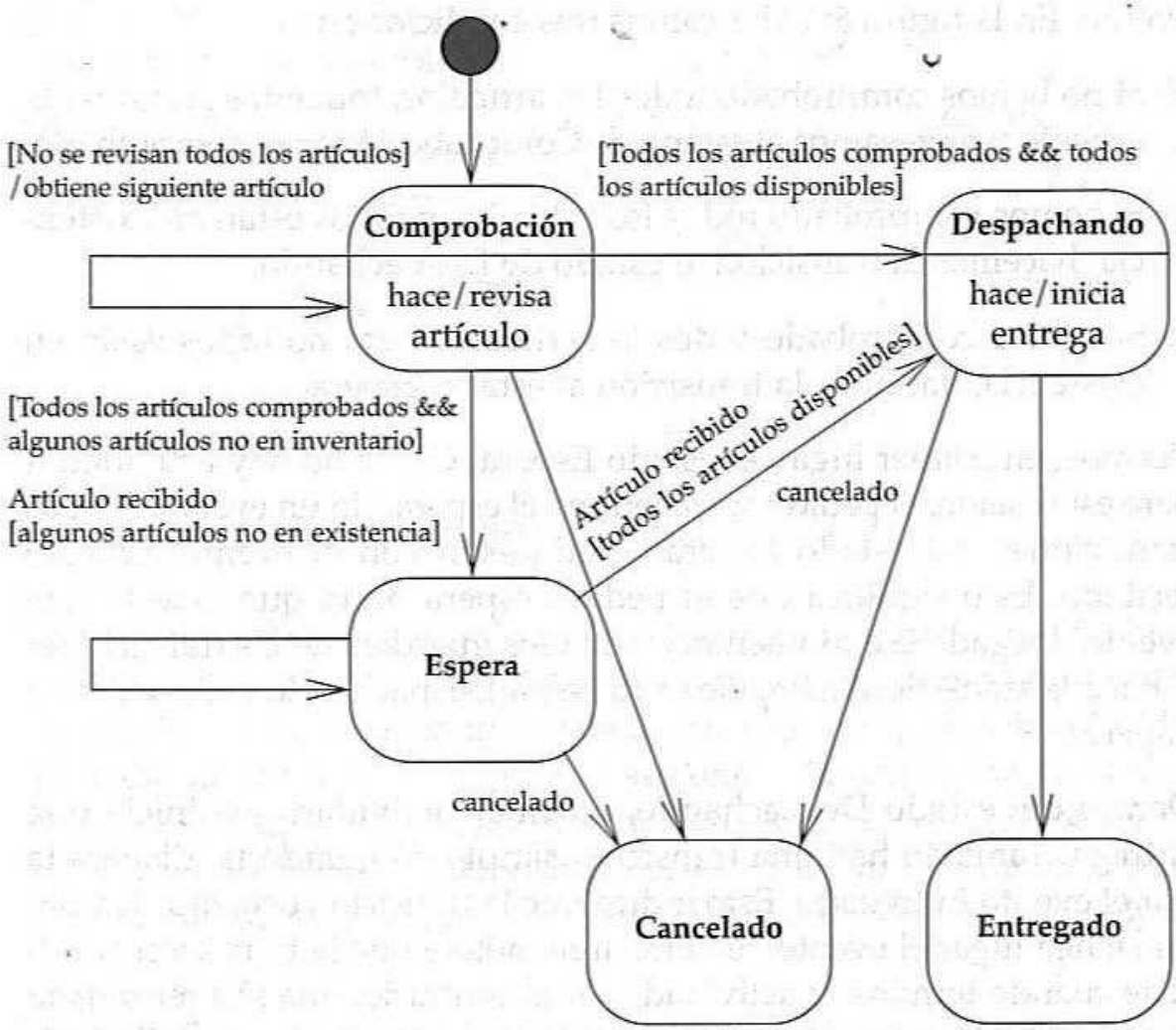


Figura 8-2: *Diagrama de estados sin superestados*

En los ejemplos actuales, he mostrado una actividad dentro de un estado, indicándola con texto en la forma de *hace/actividad*. También se pueden indicar otras cosas en un estado.

Si un estado responde a un evento con una acción que no produzca una transición, dicha condición se puede mostrar poniendo un texto de la forma *nombreEvento/nombreAcción* en el cuadro de estado.

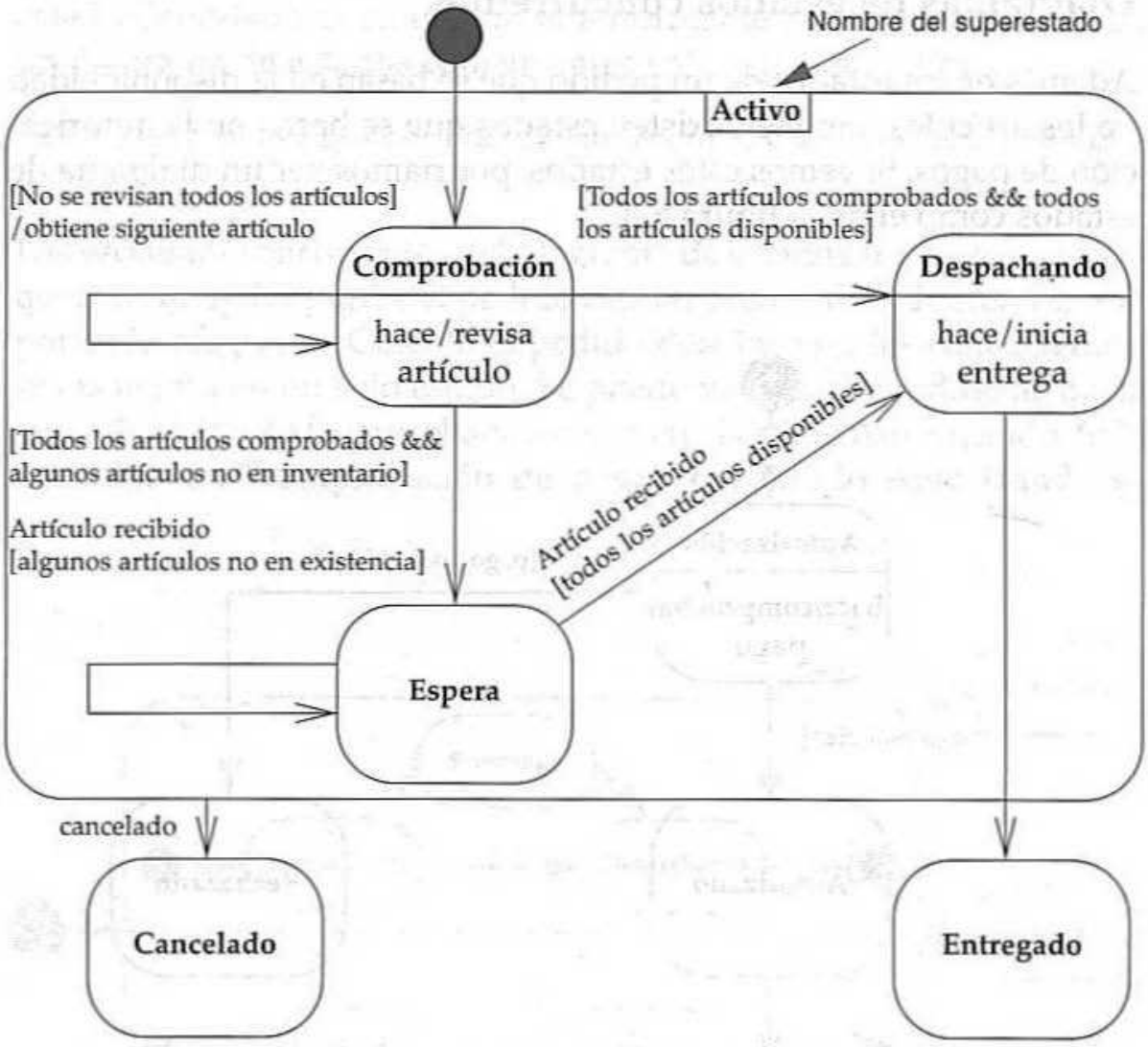


Figura 8-3: *Diagrama de estados con superestados*

Existen también dos eventos especiales, entrada y salida. Cualquier acción que esté marcada como vinculada al evento **entrada** se ejecuta siempre que se entre al estado dado a través de una transición. La acción asociada con el evento **salida** se ejecuta siempre que se sale del

estado por medio de una transición. Si se tiene una transición que vuelve al mismo estado (a esto se le llama **autotransición**) por medio de una acción, se ejecuta primero la acción de salida, luego la acción de transición y, por último, la acción de entrada. Si el estado tiene también una actividad asociada, ésta se ejecuta tras la acción de entrada.

Diagramas de estados concurrentes

Además de los estados de un pedido que se basan en la disponibilidad de los artículos, también existen estados que se basan en la autorización de pagos. Si vemos estos estados, podríamos ver un diagrama de estados como el de la figura 8-4.

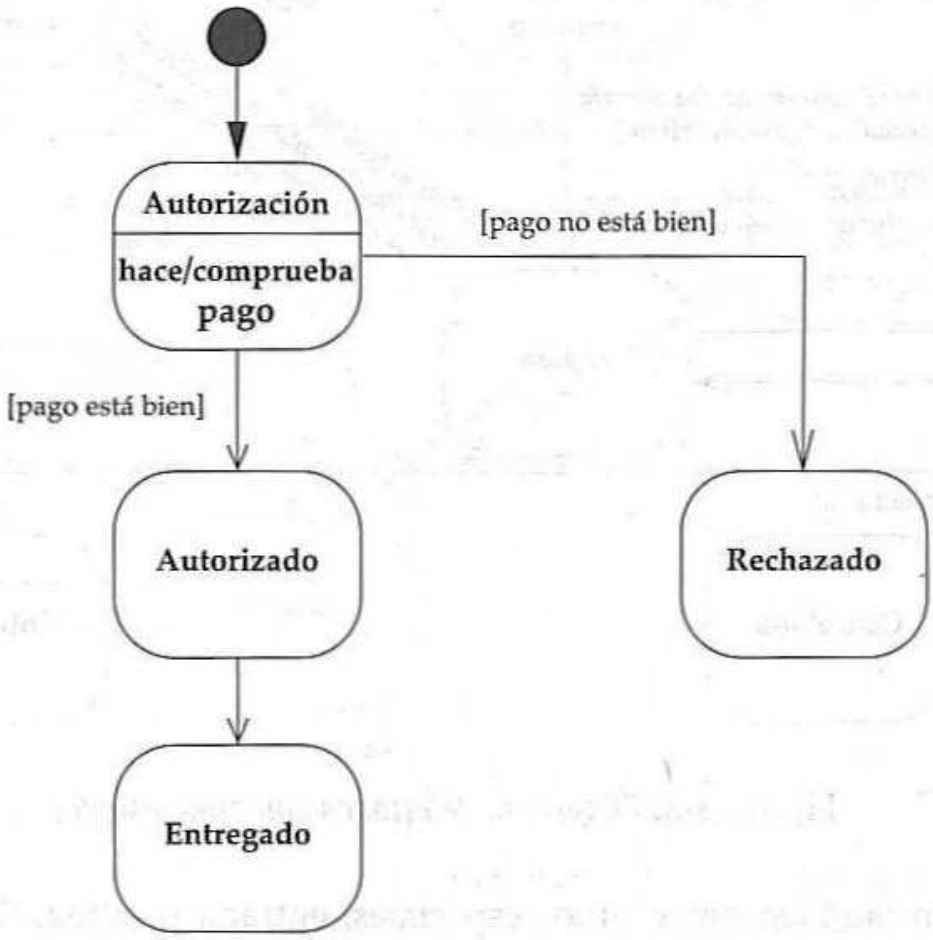


Figura 8-4: *Autorización de pagos*

Aquí comenzamos efectuando una autorización. La actividad de "comprobación de pago" termina señalando que el pago fue aprobado. Si el pago está bien, el pedido espera en el estado Autorizado hasta que sucede el evento de "entrega". De otro modo, el pedido entra al estado de Rechazado.

El objeto Orden presenta una combinación de los comportamientos que se muestran en las figuras 8-1 y 8-2. Los estados asociados y el estado Cancelado mencionados anteriormente pueden combinarse en un **diagrama de estados concurrentes** (véase la figura 8-5).

Nótese que en la figura 8-5 he dejado fuera los detalles de los estados internos.

Las secciones concurrentes del diagrama de estados son lugares en los que, en cualquier punto, el pedido está en dos estados diferentes, uno por cada diagrama. Cuando el pedido deja los estados concurrentes, se encuentra en un solo estado. Se puede ver que un pedido se inicia tanto en el estado Comprobando como en el estado Autorizando. Si la actividad de "comprobación de pago" del estado Autorizando se

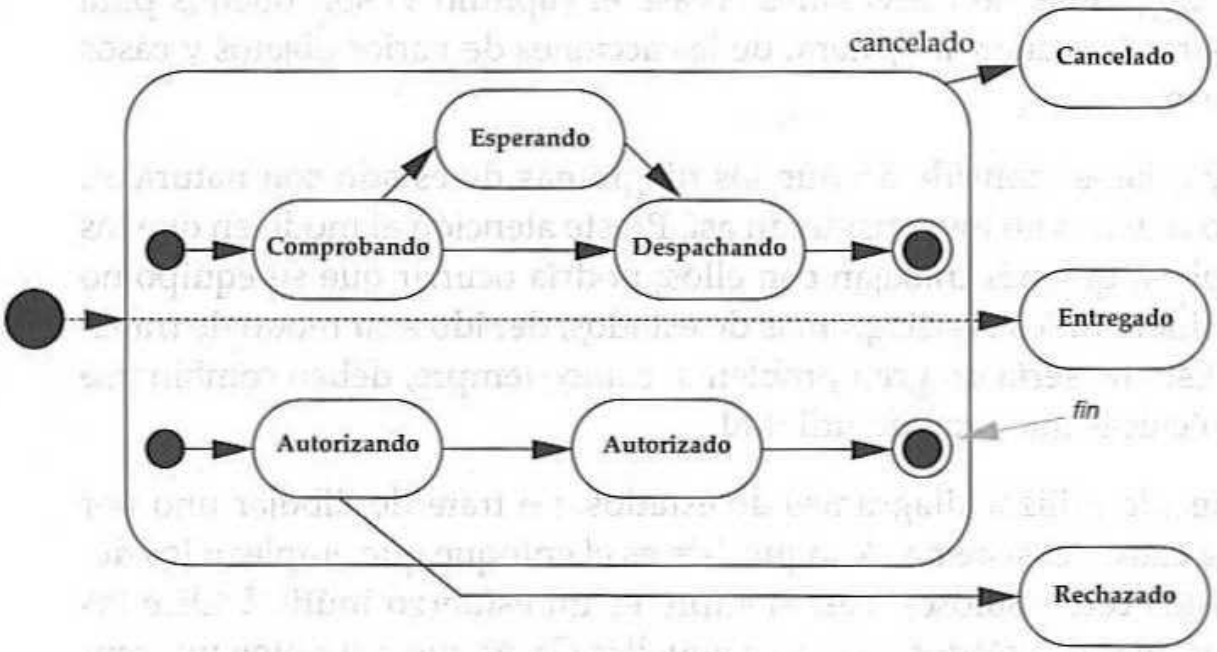


Figura 8-5: *Diagrama de estados concurrentes*

completa inicialmente de modo exitoso, entonces el pedido estará en los estados Comprobando y Autorizado. Si sucede el evento "cancelar", entonces el pedido sólo estará en el estado Cancelado.

Los diagramas de estados concurrentes son útiles cuando un objeto dado tiene conjuntos de comportamientos independientes. Nótese, sin embargo, que no se debe permitir que sucedan demasiados conjuntos de comportamientos concurrentes en un solo objeto. Si se tienen varios diagramas de estados concurrentes complicados para un solo objeto, se deberá considerar la división del objeto en varios.

Cuándo utilizar los diagramas de estados

Los diagramas de estados son buenos para describir el comportamiento de un objeto a través de varios casos de uso. No son tan buenos para describir un comportamiento que involucra cierto número de objetos que colaboran entre ellos. Así pues, es útil combinar los diagramas de estados con otras técnicas. Por ejemplo, los diagramas de interacción (véase el capítulo 6) son buenos para la descripción del comportamiento de varios objetos en un mismo caso de uso. Por su parte, los diagramas de actividades (véase el capítulo 9) son buenos para mostrar la secuencia general de las acciones de varios objetos y casos de uso.

Hay quienes consideran que los diagramas de estado son naturales, pero muchos no los consideran así. Preste atención al modo en que los emplean quienes trabajan con ellos; podría ocurrir que su equipo no considere útiles los diagramas de estados, debido a su modo de trabajar. Esto no sería un gran problema; como siempre, deben combinarse las técnicas que sean de utilidad.

Si decide utilizar diagramas de estados, no trate de dibujar uno por cada clase del sistema. Aunque éste es el enfoque que emplean los detallistas ceremoniosos, casi siempre es un esfuerzo inútil. Utilice los diagramas de estados sólo para aquellas clases que presenten un comportamiento interesante, cuando la construcción de tales diagramas le ayude a comprender lo que sucede. Muchos consideran que los obje-

tos de interfaz de usuario (IU) y de control tienen el tipo de comportamiento que es útil describir mediante diagramas de estados.

Para mayor información

Tanto Grady Booch (1994) como Jim Rumbaugh (1991) tienen material sobre los diagramas de estados, aunque no contiene mucha mayor información de la que hemos mencionado en este capítulo. Cook y Daniels (1994) son quienes han abordado con mayor detalle los diagramas de estados; yo recomiendo ampliamente su libro, si usted emplea con frecuencia los diagramas de estados. La semántica que definen es mucho más detallada que la de cualquier otro libro. Aunque tal semántica no corresponda enteramente con la del UML, los autores tratan con exhaustividad cuestiones que usted debe tener en cuenta, si va usar diagramas de estados.

Capítulo 9

Diagramas de actividades

Los diagramas de actividades constituyen una de las partes más imprevistas del UML.

El diagrama de actividades, a diferencia de la mayor parte de las demás técnicas del UML, no tiene su origen evidente en los trabajos anteriores de los tres amigos. El **diagrama de actividades** combina ideas de varias técnicas: el diagrama de eventos de Jim Odell, las técnicas de modelado de estados de SDL y las redes de Petri. Estos diagramas son particularmente útiles en conexión con el flujo de trabajo y para la descripción del comportamiento que tiene una gran cantidad de proceso paralelo.

En la figura 9-1, que proviene de la documentación del UML 1.0, el símbolo clave es la **actividad**. La interpretación de este término depende de la perspectiva desde la cual se dibuja el diagrama. En un diagrama conceptual, una actividad es cierta tarea que debe ser llevada a cabo, ya sea por un ser humano o por una computadora. En un diagrama de perspectiva de especificación o de perspectiva de implementación, una actividad es un método sobre una clase.

Cada actividad puede ser seguida por otra actividad. Esto simplemente es secuenciación. Por ejemplo, en la figura 9-1, la actividad

Poner el café en el filtro va seguida por la actividad Poner el filtro en la máquina. Hasta ahora, el diagrama de actividades parece un flowchart diagrama de flujo. Podemos investigar las diferencias observando la actividad Encuentra bebida.

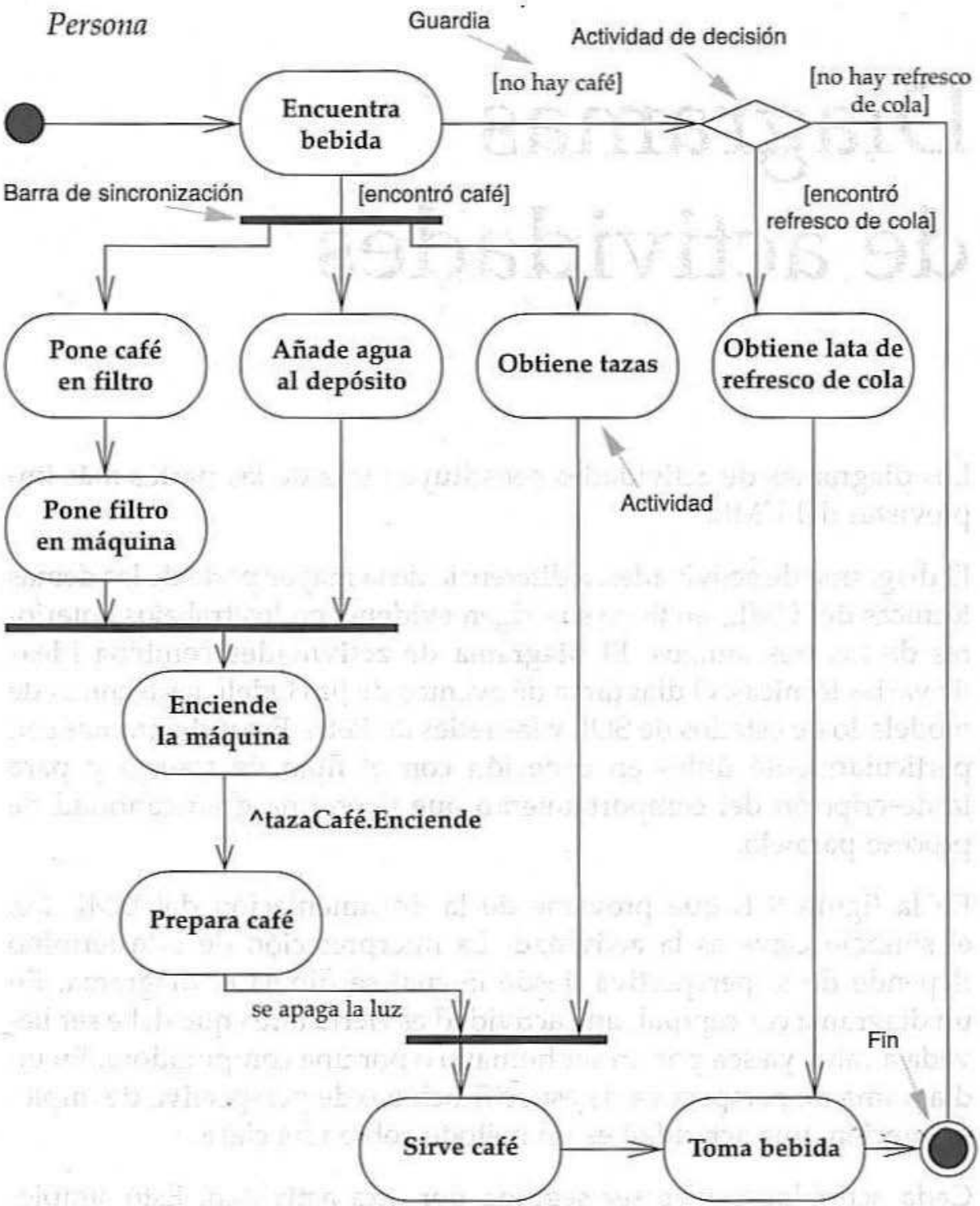


Figura 9-1: *Diagrama de actividades*

De Encuentra bebida salen dos disparadores. Cada disparador tiene un **guardia**, una expresión lógica que se evalúa como "verdadero" o "falso", del mismo modo que en un diagrama de estados (véase el capítulo 8). En el caso de la figura 8-1, la persona seguirá la actividad Encuentra bebida considerando las opciones de café o refresco de cola.

Supongamos que podremos disfrutar de una bolsa de café de Colombia y seguiremos la ruta del café. Este disparador nos conduce a la **barra de sincronización**, a la cual están unidos otros tres disparadores que, a su vez, conducen a las actividades Pone café en filtro, Añade agua al depósito y Obtiene tazas.

El diagrama señala que estas actividades pueden suceder en paralelo, lo cual significa, en esencia, que su orden no es significativo. Se podría poner el café en el filtro primero, después añadir agua al depósito y, por último, obtener las tazas. También podríamos conseguir las tazas, después poner el café en el filtro... ya se entiende la idea.

También puedo llevar a cabo estas actividades en forma intercalada. Podría obtener una taza, añadir después un poco de agua al depósito, obtener otra taza, agregar otro poco más de agua y así sucesivamente. O también podría llevar a cabo estas operaciones simultáneamente: agregar el agua al depósito con una mano y con la otra alcanzar la taza. De acuerdo con el diagrama, cualquiera de estas formas de operar es correcta.

El diagrama de actividades me permite seleccionar el orden en que se harán las cosas. Esto es, simplemente me dice las reglas esenciales de secuenciación que tengo que seguir. Ésta es la diferencia clave entre un diagrama de actividades y un diagrama de flujo. Los diagramas de flujo se limitan normalmente a procesos secuenciales; los diagramas de actividades pueden manejar procesos paralelos.

Esta característica es importante para el modelado de negocios. Los negocios con frecuencia tienen procesos secuenciales innecesarios. Una técnica como ésta, que promueve el comportamiento paralelo, es valiosa en estas situaciones, porque auspicia que las personas se aparten de las secuencias innecesarias en su comportamiento y descubran oportunidades para hacer cosas en paralelo. Esto puede mejorar la eficiencia y capacidad de respuesta de los procesos del negocio.

Los diagramas de actividades también son útiles para los programas concurrentes, ya que se pueden plantear gráficamente cuáles son los hilos y cuándo necesitan sincronizarse.

Cuando se tiene un comportamiento paralelo, se impone la necesidad de sincronizar. No queremos prender la cafetera hasta haberle colocado el filtro y llenado de agua el depósito. Por eso, vemos a los disparadores de estas actividades saliendo juntos de una barra sincronizadora. Una barra de sincronización simple como ésta indica que el disparo de salida ocurre sólo cuando han sucedido ambos disparos de entrada. Como veremos después, estas barras pueden ser más complicadas.

Más adelante se dará otra sincronización: el café debe ser preparado y las tazas deben estar disponibles, antes de poder servir el café.

Vayamos ahora al otro carril.

En este caso, tenemos una decisión compuesta. La primera decisión es sobre el café. Esta decisión es la que determina los dos disparadores que salen de Encuentra bebida. Si no hay café, nos enfrentamos a una segunda decisión, basada ésta en refresco de cola.

Cuando tenemos decisiones como la anterior, señalamos la segunda decisión con un rombo de decisión. Esto nos permite describir decisiones anidadas, de las cuales podemos tener cualquier cantidad.

En la actividad Toma bebida convergen dos disparadores, lo que significa que se llevará a cabo en cualquiera de los dos casos. Por el momento, se puede considerar esto como un caso OR (lo hago, si sucede uno u otro disparador) y a la barra de sincronización como el caso AND (lo hago, si suceden ambos disparadores).

Diagramas de actividades para casos de uso

La figura 9-1 describe un método sobre el tipo Persona. Los diagramas de actividades son útiles para describir métodos complicados. También pueden servir para otras cosas; por ejemplo, para describir un caso de uso.

Considérese un caso de uso para el proceso de pedidos.

> *Cuando recibimos un pedido, comprobamos cada artículo de línea del pedido para ver si lo hay en existencia. Si la respuesta es afirmativa, asignamos la mercancía al pedido. Si esta asignación hace bajar la cantidad de mercancía en existencia por debajo del nivel de reorden, se reordena. Mientras hacemos esto, revisamos el pago para ver si está correcto. Si el pago está bien y hay mercancías en existencia, despachamos el pedido. Si el pago está correcto pero no hay las mercancías en existencia, dejamos en espera el pedido. Si el pago no está bien, cancelamos la orden.*

Véase la figura 9-2 para una representación visual de este caso de uso.

La figura introduce un nuevo artificio al diagrama de actividades. Véase el disparador de entrada asociado con la actividad Comprueba artículo de línea. Está marcado con un *. Éste es un marcador de multiplicidad (se trata del mismo marcador utilizado en los diagramas de clases; véase el capítulo 4) que muestra que, cuando recibimos una orden, tenemos que llevar a cabo la actividad Comprueba artículo de línea para cada artículo de línea del pedido. Esto significa que la actividad Recibe orden va seguida por una invocación de la actividad Autoriza pago y varias invocaciones de la actividad Comprueba artículo de línea. Todas estas invocaciones suceden en paralelo.

Esto resalta la segunda fuente del paralelismo en un diagrama de actividades. Se pueden tener actividades paralelas a través de varias transiciones que salen de una barra sincronizadora; también se pueden tener actividades paralelas cuando se dispara la misma actividad por medio de un **disparador múltiple**. Siempre que se tenga un disparador múltiple, habrá de indicarse en el diagrama cuál es su base, como sucede con [por cada artículo en].

Cuando encuentre un disparador múltiple, generalmente verá una barra de sincronización más abajo del diagrama, que une los hilos paralelos. En este caso, vemos la barra antes de la actividad Despacha pedido. La barra de sincronización tiene aplicada a ella una condición. Cada vez que llega un disparador a la barra de sincronización, se prueba la condición. Si la condición es verdadera, sucede el disparo de

salida (también se puede marcar otro * para indicar la unión de las líneas; yo prefiero no mostrar un segundo *, pues vuelve muy confuso el diagrama, y considero que la condición de sincronización deja las cosas lo bastante claras).

Las barras de sincronización no etiquetadas operan de la misma forma. La ausencia de una condición significa que se está usando la condición predeterminada para las barras sincronizadoras. La condición predeterminada es que todos los disparadores ya han ocurrido. Por esa razón, no hay condiciones en las barras de la figura 9-1.

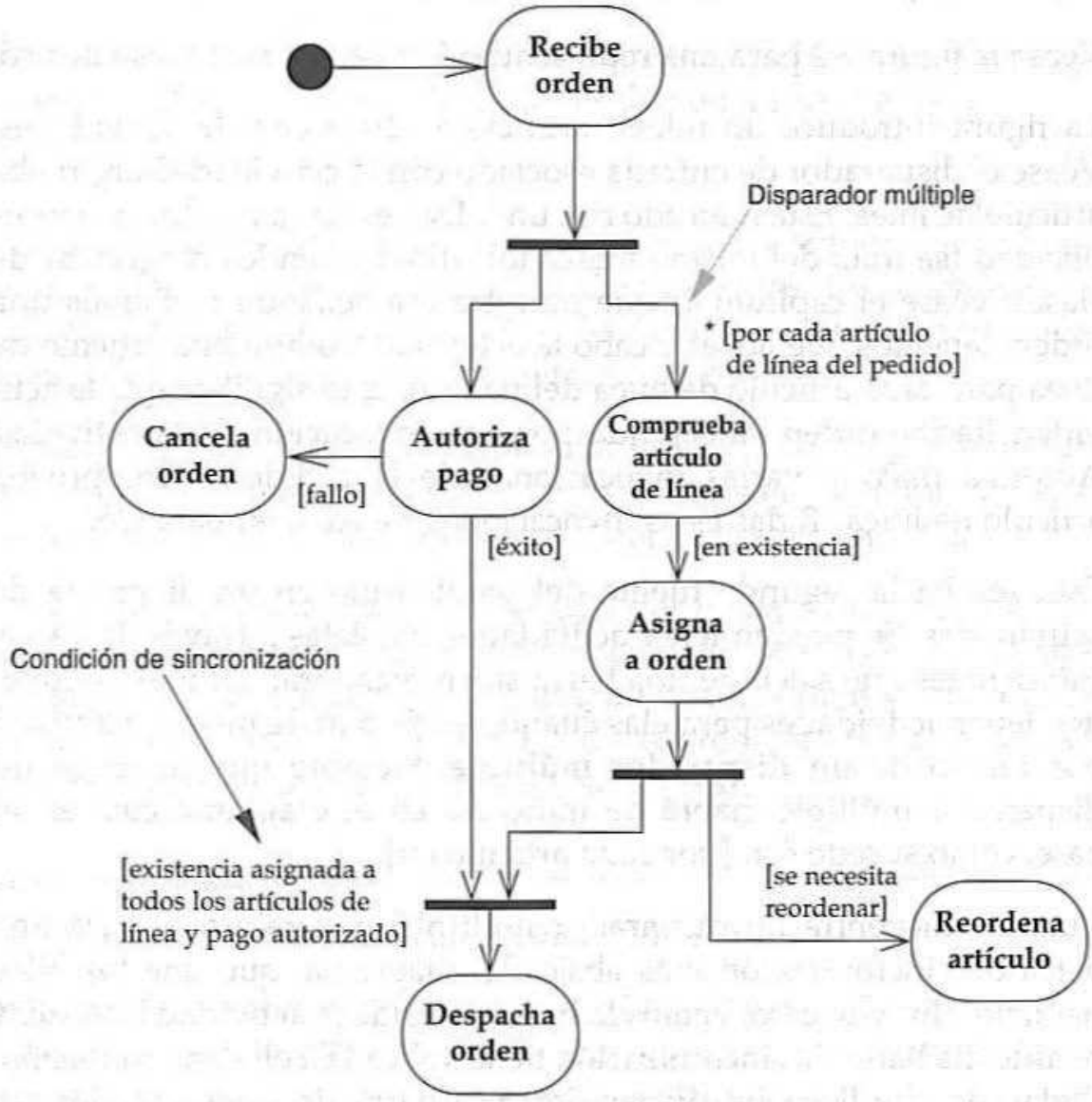


Figura 9-2: *Recepción de un pedido*

Un diagrama de actividades no tiene que tener un punto de terminación definido. El punto de terminación de un diagrama de actividades es el punto en el cual todas las actividades disparadas han sido operadas y no hay nada más por hacer. En la figura 9-2, no tendría utilidad marcar un punto de terminación explícito.

La figura 9-2 tiene también un callejón sin salida: la actividad Reordena artículo. Nada sucede después de realizada esta actividad. Estos callejones sin salida están bien en diagramas de actividades sin terminación, como éste. Algunas veces son obvios, como en este caso. En otras ocasiones no lo son tanto. Obsérvese la actividad Comprobación de artículo de línea; sólo tiene un disparador de salida, el cual tiene una condición. ¿Qué sucede si no hay existencia del artículo de línea? Nada, el hilo del proceso simplemente se detiene allí.

En nuestro ejemplo, no podemos despachar un pedido hasta recibir una orden que reabastezca la existencia. Éste podría ser un caso de uso separado.

> *Cuando llega un reabastecimiento, vemos los pedidos sobresalientes y decidimos cuáles podemos surtir con el material recibido y, entonces, asignamos lo correspondiente a sus respectivos pedidos. Con esto se podrían liberar dichas órdenes para ser despachadas. La mercancía restante la ponemos en el almacén.*

La figura 9-3 es un diagrama de actividades que representa este caso de uso.

Este segundo caso de uso muestra cómo la orden puede quedar en espera de ser despachada hasta que suceda el reabastecimiento.

Cuando cada uno de los dos casos muestra parte del cuadro general, encuentro que es útil dibujar un diagrama combinado, como el de la figura 9-4. Este diagrama muestra los diagramas de actividades para los dos casos de uso superpuestos, de modo que se pueden ver cómo las acciones en un caso de uso afectan las acciones del otro. Un diagrama de actividades como éste tiene múltiples puntos de partida, lo cual está muy bien, pues el diagrama de actividades representa la manera en que el negocio reacciona ante varios eventos externos.

Considero particularmente útil esta capacidad de los diagramas de actividades de mostrar comportamientos que abarcan varios casos de uso. Los casos de uso nos dan rebanadas de información sobre un dominio visto desde afuera; cuando vemos el cuadro interno, necesitamos ver el conjunto. Los diagramas de clases (véase el capítulo 4) nos muestran el cuadro completo de clases interconectadas y los diagramas de actividades hacen lo mismo en lo que respecta al comportamiento.

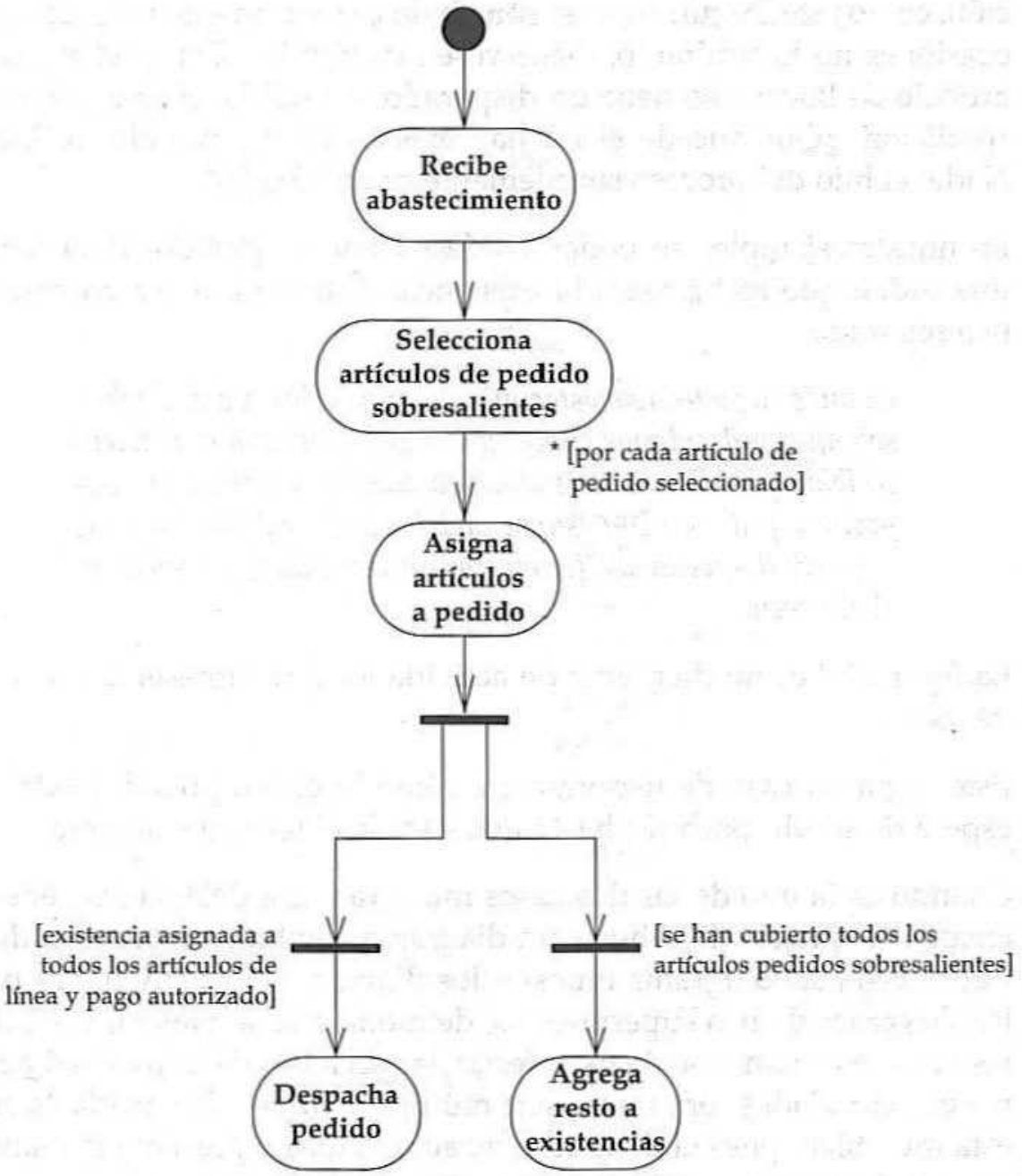


Figura 9-3: *Recepción de abastecimiento*

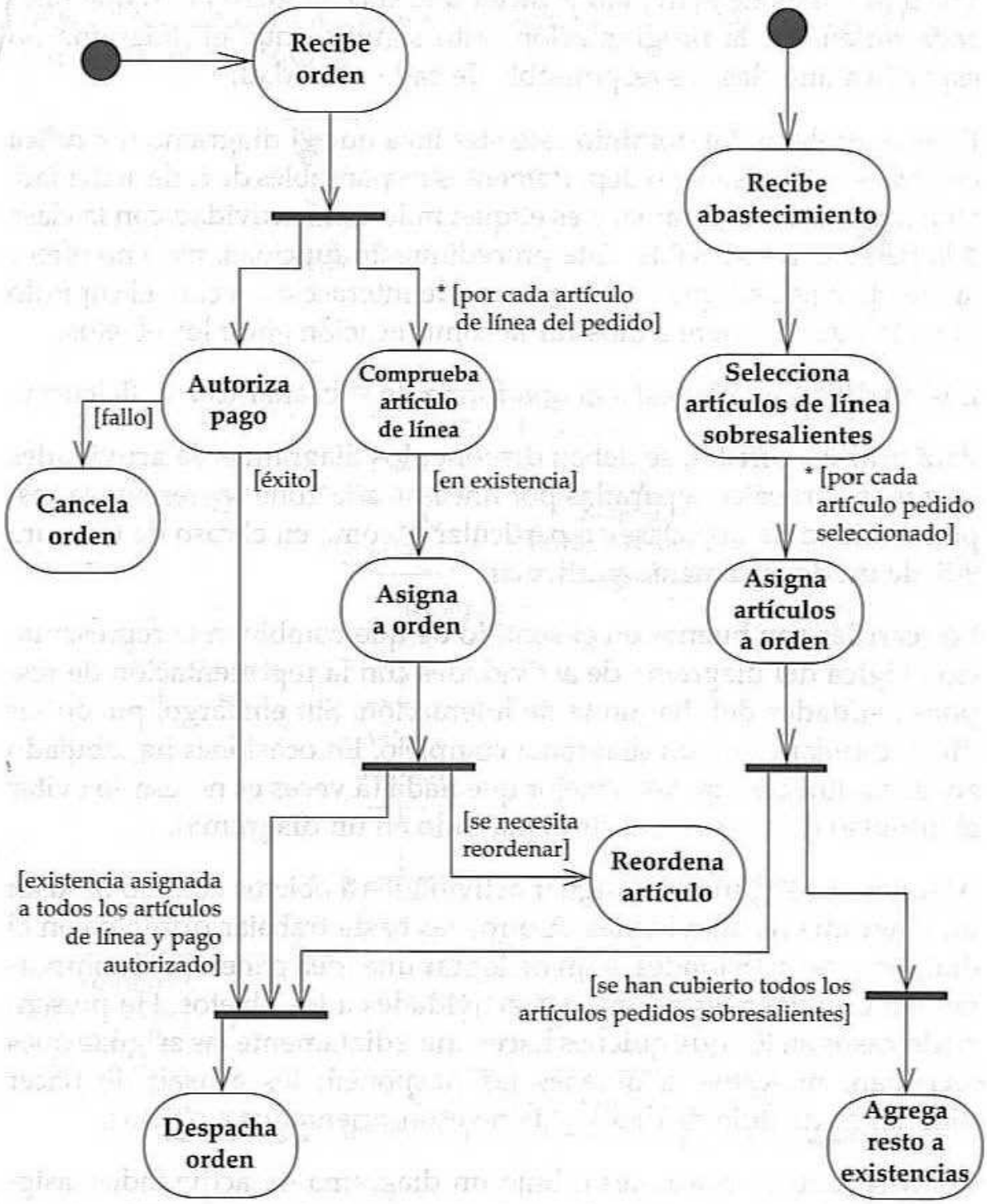


Figura 9-4: *Recibe orden y recibe existencias*

Carriles

Los diagramas de actividades dicen qué sucede, pero no lo que hace cada quién. En la programación, esto significa que el diagrama no especifica qué clase es responsable de cada actividad.

En el modelado del dominio esto significa que el diagrama no indica cuáles son las personas o departamentos responsables de cada actividad. Una manera de superar esto es etiquetando cada actividad con la clase o la persona responsable. Este procedimiento funciona, pero no ofrece la misma claridad que los diagramas de interacción (véase el capítulo 6) en lo que se refiere a mostrar la comunicación entre los objetos.

Los **carriles** (swimlanes) son una forma de subsanar esta deficiencia.

Para usar los carriles, se deben disponer los diagramas de actividades en zonas verticales separadas por líneas. Cada zona representa la responsabilidad de una clase en particular o, como en el caso de la figura 9-5, de un departamento particular.

Los carriles son buenos en el sentido de que combinan la representación lógica del diagrama de actividades con la representación de responsabilidades del diagrama de interacción. Sin embargo, puede ser difícil dibujarlos en un diagrama complejo. En ocasiones he dibujado zonas no lineales, que son mejor que nada (a veces es necesario evitar el impulso de tratar de decir demasiado en un diagrama).

Algunos se aseguran de asignar actividades a objetos cuando dibujan un diagrama de actividades. A otros les basta trabajar primero con el diagrama de actividades, a fin de lograr una idea general del comportamiento, y después asignar las actividades a los objetos. He presenciado casos en los que quienes hacen inmediatamente las asignaciones reclaman, molestos, a quienes las posponen; les acusan de hacer diagramas de flujo de datos y de no estar orientados a objetos.

Confieso que en ocasiones dibujo un diagrama de actividades, asignando sólo hasta después el comportamiento a los objetos. Considero muy útil comprender una cosa a la vez. Esto es particularmente cierto cuando hago modelado de negocios y estoy animando a un experto del dominio a pensar en nuevas formas de hacer las cosas.

Para mí, esto funciona. Otros prefieren asignar inmediatamente el comportamiento a los objetos. Usted debe hacer lo que le parezca más cómodo. Lo importante es asignar las actividades a las clases, antes de terminar. Con frecuencia, recurro a un diagrama de interacción (véase el capítulo 6).

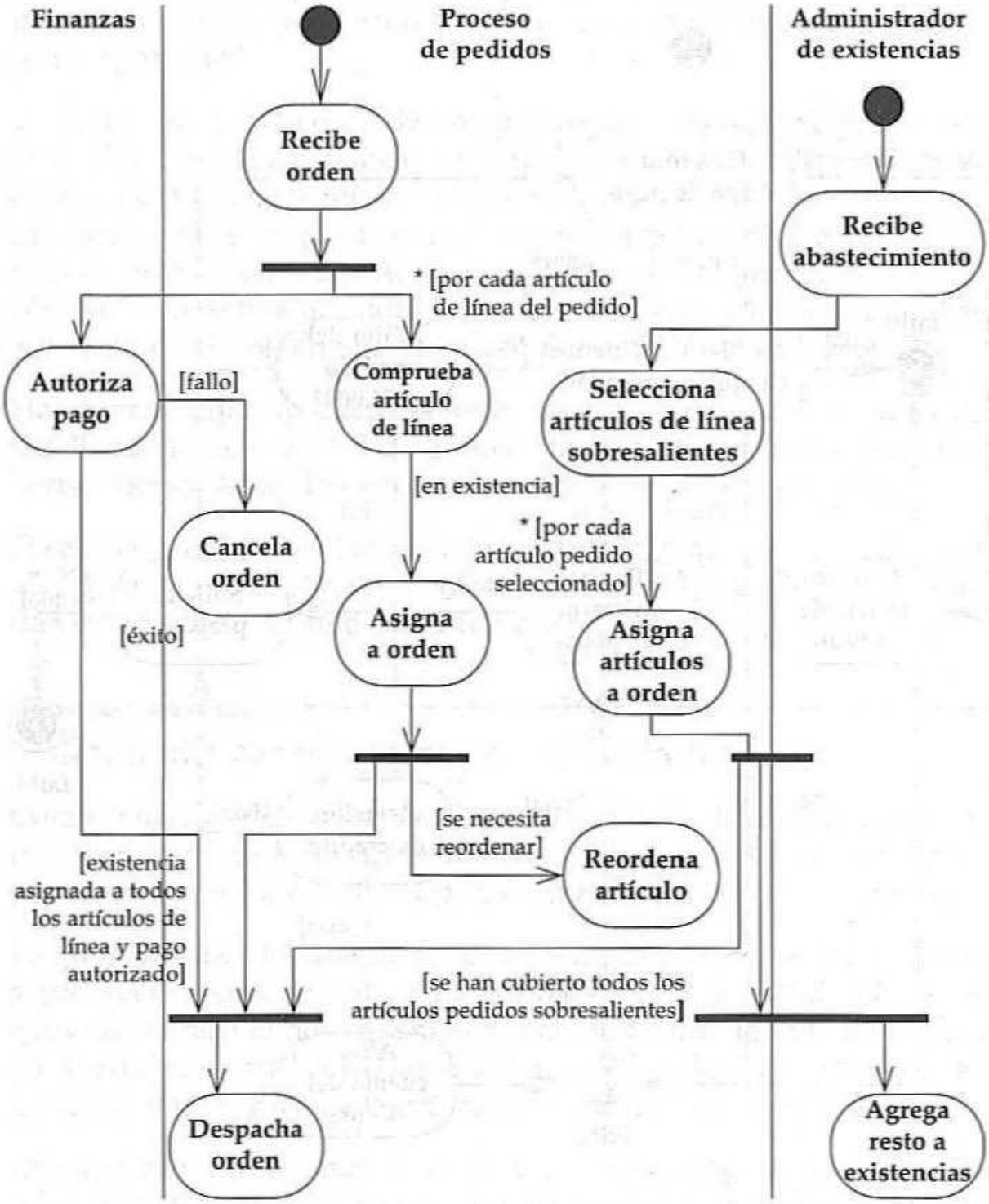


Figura 9-5: *Carriles*

Descomposición de una actividad

Una actividad puede ser descompuesta en una descripción posterior. Esta descripción puede ser un texto, un código u otro diagrama de actividades (véase la figura 9-6.)

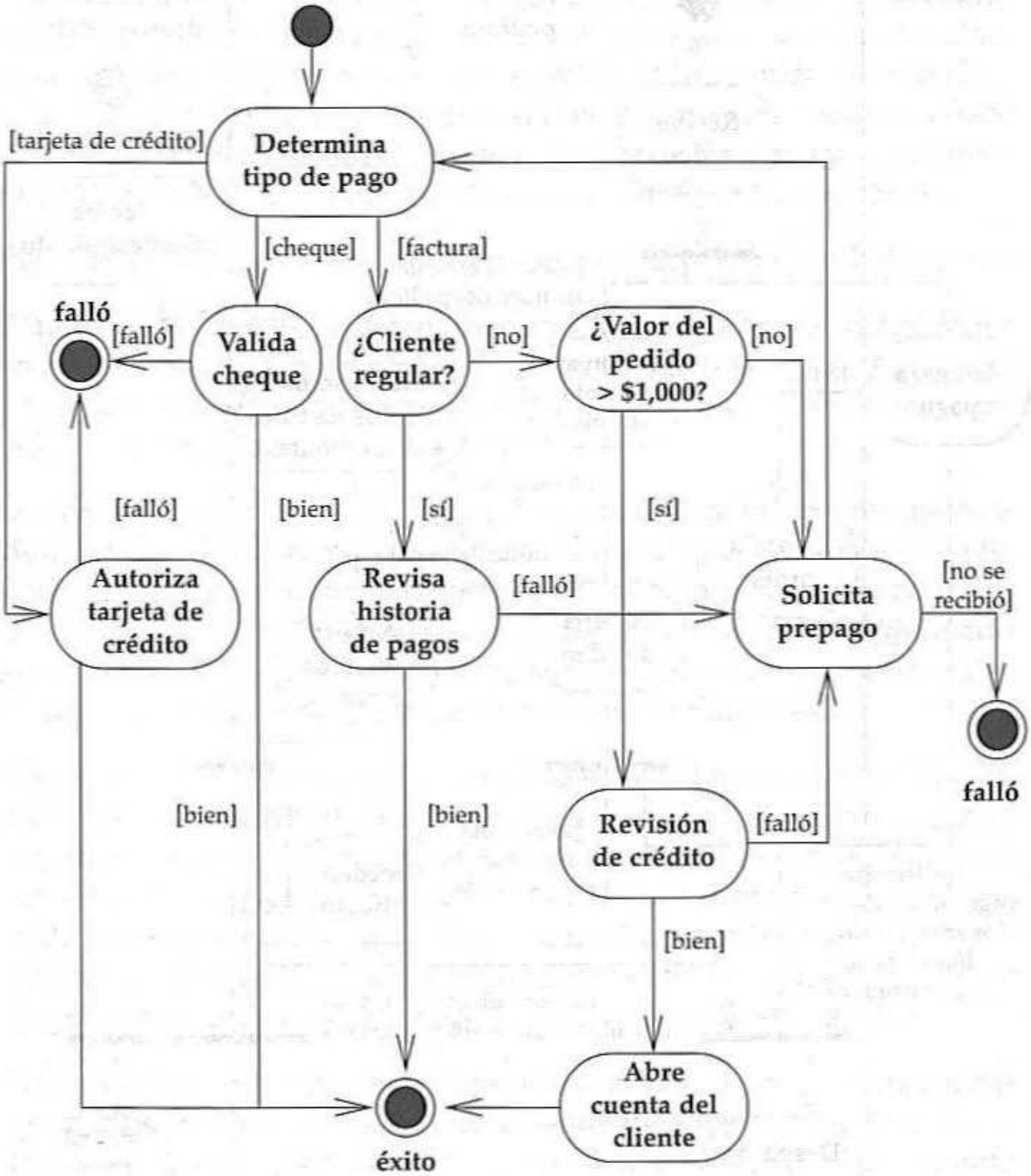


Figura 9-6: *Diagrama descompuesto de actividades*

Cuando se dibuja un diagrama de actividades como una descomposición de una actividad de más alto nivel, se debe proporcionar un solo punto de partida. Sin embargo, se pueden suministrar tantos puntos de terminación como disparadores de salida haya dentro de la actividad de alto nivel. Esto permite al diagrama subsidiario devolver un valor que determina los posteriores disparos. Por ejemplo, la figura 9-6 muestra que la actividad Autoriza tarjeta de crédito devuelve "correcto" o "no aprobado".

La figura 9-6 se dibujó desde una perspectiva conceptual, pero no es muy difícil imaginar lo mismo dibujado como un retrato gráfico del código de programación, tal como un diagrama de flujo. Tengo tendencia a evitar esto, por la misma razón por la que no dibujo diagramas de flujo. Con frecuencia, es más fácil simplemente escribir el código. Si considera que aquí son útiles los diagramas, podría probar este procedimiento, en particular si quiere mostrar varios hilos.

Hay herramientas que pueden ejecutar los diagramas de eventos de Odell, precursores de los diagramas de actividades. Si emplea tales herramientas, las podría encontrar valiosas para crear prototipos.

Si se tiene que representar mucha lógica, es fácil que los diagramas de actividades se compliquen demasiado. En esta situación, una tabla de verdad puede ser una mejor representación.

Cuándo utilizar diagramas de actividades

Como la mayor parte de las técnicas de modelado de comportamiento, los diagramas de actividades tienen virtudes y defectos definidas, por lo que lo mejor es utilizarlos en combinación con otras técnicas.

La gran virtud de los diagramas de actividades reside en que manejan y promueven el comportamiento en paralelo. Esta cualidad hace de ellos una excelente herramienta para el modelado de flujo de trabajo y, en principio, para la programación multihilos. Su gran desventaja es que no dejan muy claros los vínculos entre acciones y objetos.

Usted puede definir a qué se refiere una relación mediante el procedimiento de etiquetar una actividad con un nombre de objeto o bien por medio de carriles (que dividen un diagrama de actividades con base

en responsabilidades), pero este recurso no tiene la sencilla inmediatez de los diagramas de interacción (véase el capítulo 6). Por esa razón, algunos consideran que los diagramas de actividades no están orientados a objetos y que, por tanto, son malos. He encontrado que esta técnica puede ser una herramienta de gran utilidad, y no tengo por norma desechar de entre mis herramientas de trabajo cualquiera que sea útil.

Me gusta emplear los diagramas de actividades en las siguientes situaciones:

- *En el análisis de un caso de uso.* En esta etapa, no me interesa asignar acciones a los objetos: sólo necesito comprender qué acciones deben ocurrir y cuáles son las dependencias de comportamiento. Asigno los métodos a los objetos posteriormente, y muestro tales asignaciones mediante un diagrama de interacción.
- *En la comprensión del flujo de trabajo, a través de numerosos casos de uso.* Cuando los casos de uso interactúan entre ellos, los diagramas de actividades son una gran herramienta para representar y entender este comportamiento. En las situaciones dominadas por el flujo de trabajo, los considero una herramienta formidable.
- *Cuando se trata de aplicaciones multihilos.* No he usado los diagramas de actividades para este propósito, pero son muy adecuados para ello.

Por otra parte, no uso los diagramas de actividades en las siguientes situaciones:

- *Para tratar de ver cómo colaboran los objetos.* Un diagrama de interacción es más simple y da un panorama más claro de las colaboraciones.
- *Para tratar de ver cómo se comporta un objeto durante su periodo de vida.* Utilice un diagrama de estados (véase el capítulo 8) para ese fin.

Para mayor información

Los diagramas de actividades se basan en diversos enfoques orientados al flujo de trabajo. Su antecedente más inmediato es el diagrama de eventos de Jim Odell, sobre el que usted podrá encontrar más información en el libro de "fundamentos" de Martin y Odell (1994).

Capítulo 10

Diagramas de emplazamiento

El **diagrama de emplazamiento** (*deployment diagram*) es aquel que muestra las relaciones físicas entre los componentes de software y de hardware en el sistema entregado. Así, el diagrama de emplazamiento es un buen sitio para mostrar cómo se enrutan y se mueven los componentes y los objetos, dentro de un sistema distribuido.

Cada **nodo** de un diagrama de emplazamiento representa alguna clase de unidad de cómputo; en la mayoría de los casos se trata de una pieza de hardware. El hardware puede ser un dispositivo o un sensor simple, o puede tratarse de un mainframe.

La figura 10-1 muestra una PC conectada a un servidor UNIX por medio de TCP/IP. Las **conexiones** entre nodos muestran las rutas de comunicación a través de las cuales interactuará el sistema.

Los **componentes** en un diagrama de emplazamiento representan módulos físicos de código. En mi experiencia, corresponden exactamente a los paquetes de un diagrama de paquetes (véase el capítulo 7), de tal modo que el diagrama de emplazamiento muestra dónde se ejecuta cada paquete en el sistema.

Las **dependencias** entre los componentes deben ser las mismas que las dependencias de paquetes. Estas dependencias muestran cómo

se comunican los componentes con otros componentes. La dirección de una dependencia dada indica el conocimiento en la comunicación.

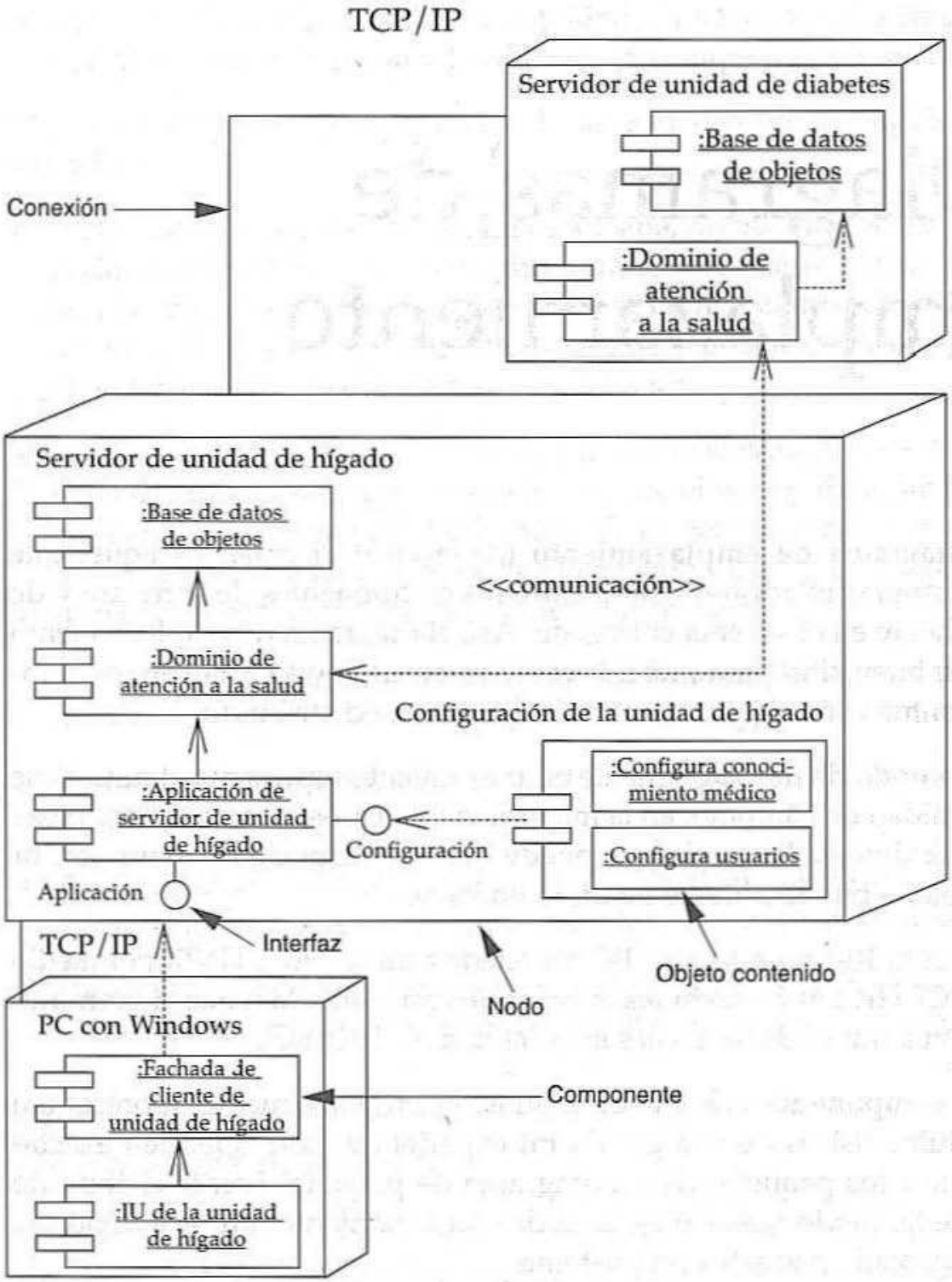


Figura 10-1: *Diagrama de emplazamiento*

Así, en el diagrama, la IU de la unidad de hígado depende de la Fachada de cliente de unidad de hígado, ya que llama a métodos específicos en la fachada. A pesar de que la comunicación es en ambas direcciones, en el sentido de que la Fachada devuelve datos, la Fachada no sabe quién la llama y, por tanto, no depende de la IU. En la comunicación entre ambos componentes del Dominio de atención a la salud, ambos saben que están hablando con otro componente de Dominio de atención a la salud, así que la dependencia de la comunicación es en dos sentidos.

Un componente puede tener más de una interfaz, en cuyo caso usted podrá ver cuáles componentes se comunican con cada interfaz. En la figura 9-1, la PC contiene dos componentes: la IU y la fachada de la aplicación. La fachada de aplicación habla con la interfaz de la aplicación en el servidor. Un componente de configuración separado se ejecuta sólo en el servidor. La aplicación se comunica con su componente local del Dominio de atención a la salud, el cual, a su vez, puede comunicarse con otros componentes de Dominios de atención a la salud de la red.

La utilización de los componentes de diversos Dominios de atención a la salud está oculta para la aplicación. Cada componente del Dominio de atención a la salud tiene una base de datos local.

Cuándo utilizar los diagramas de emplazamiento

En la práctica, no he visto que se use mucho este tipo de diagramas. La mayoría de la gente dibuja diagramas para mostrar este tipo de información, pero se trata de bocetos informales. En general, no tengo problemas con este tipo de diagramas, ya que cada sistema tiene sus propias características físicas que se querrán subrayar. A medida que se tiene que lidiar cada vez más con los sistemas distribuidos, estoy seguro de que se requerirá mayor formalidad, según se vaya entendiendo mejor cuáles son los asuntos que se deben resaltar en los diagramas de emplazamiento.

Capítulo 11

El UML y la programación

Hasta aquí he ahondado en lo concerniente a la notación. Surge, sin embargo, una gran pregunta: en la práctica, ¿cómo *utiliza* un programador el UML, como parte del duro trabajo cotidiano de la programación? Contestaré esta pregunta explicando cómo lo uso yo cuando programo, incluso a pequeña escala. No entraré en detalles, pero espero que lo que sigue a continuación le dé a usted una idea de lo que se puede hacer con el UML.

Imaginemos un sistema de cómputo diseñado para reunir información sobre los pacientes de un hospital.

Varios profesionales del cuidado de la salud hacen observaciones sobre los pacientes. Este sistema simple permite que cualquiera pueda obtener la información incluida en tales observaciones y agregar observaciones nuevas. Como éste es un libro relativamente breve, pasaré por alto los vínculos de la base de datos y la UI, y sólo consideraré las clases de dominio básicas.

Éste es un ejemplo tan simple que no tiene más que un solo caso de uso, llamado "revisar y añadir observaciones sobre el paciente". Podemos ahondar al respecto con unas cuantas situaciones.

- Preguntar por el último ritmo cardiaco de un paciente.
- Preguntar por el grupo sanguíneo de un paciente.
- Actualizar el nivel de conciencia de un paciente.
- Poner al día el ritmo cardiaco de un paciente. El sistema marca el ritmo como lento, normal o acelerado, de acuerdo con los intervalos interconstruidos en el sistema.

Mi primer paso en el proceso es escoger un modelo conceptual que describa los conceptos de este dominio. No me refiero, en esta etapa, a cómo trabajará el software; únicamente me interesa saber cómo organizar los conceptos que hay en las mentes de los médicos y enfermeras. Comenzaré con un modelo que se basa en varios patrones analíticos de Fowler (1997): *Observation Observación, Quantity Cantidad, Range Intervalo, y Phenomenon with Range Fenómeno con intervalo.*

Observación del paciente: modelo de dominio

La figura 11-1 muestra el modelo de dominio inicial para nuestro sistema.

¿Cómo representan estos conceptos la información en el dominio?

Comenzaré con los conceptos simples de Cantidad, Unidad e Intervalo. Cantidad representa un valor que tiene una dimensión, por ejemplo, 2 metros: una cantidad con una cifra de 2 y una unidad de metros. Las unidades son simplemente aquellas categorías de medida con las cuales trabajaremos. El Intervalo nos permite hablar de los intervalos como un solo concepto; por ejemplo, un intervalo de 1 metro a 2 metros se representa como un simple objeto Intervalo con un límite superior de 2 metros y uno inferior de 1 metro. En general, los intervalos se pueden expresar en términos de cualquier cosa que se pueda comparar (mediante los operadores <, >, <=, >=, e =), de tal suerte que los límites superior e inferior de un Intervalo son magnitudes (las cantidades son un tipo de magnitud).

Cada observación hecha por un médico o enfermera es una instancia del concepto de Observación y puede ser tanto una Medición como

una Observación de categoría. De tal forma que una medida de 1.8 metros para Martin Fowler se representaría como una instancia de Medición. Asociada con esta Medición está la cantidad de 1.8 metros, el Tipo de fenómeno "altura" y el Paciente llamado Martin Fowler. Los Tipos de fenómeno representan las cosas que se pueden medir: altura, peso, ritmo cardiaco, etcétera.

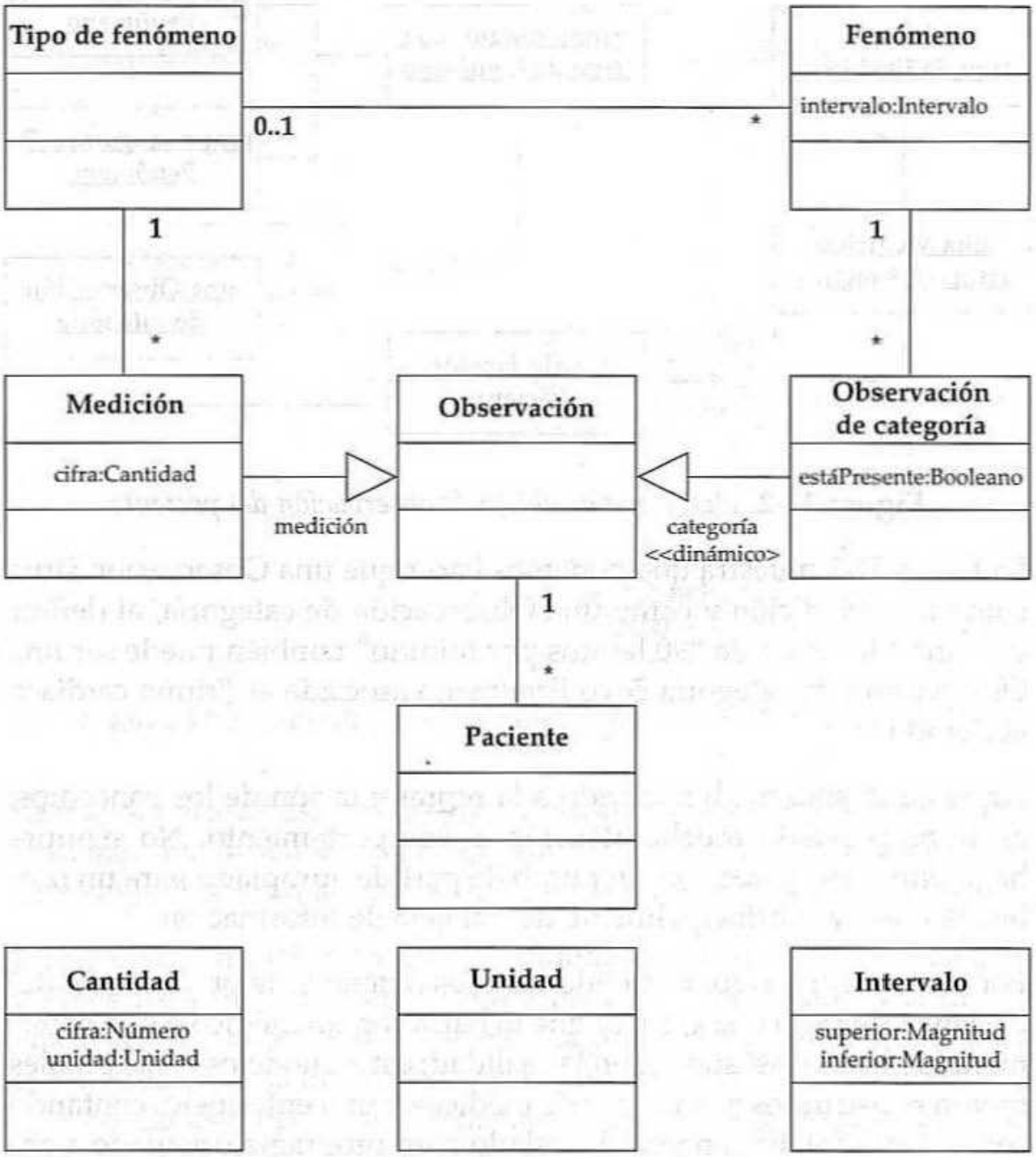


Figura 11-1: *Modelo de dominio de observación de pacientes*

La observación de que el tipo de sangre de Martin Fowler es O se representaría como una Categoría Observación cuyo Fenómeno asociado es el "tipo de sangre O". Este Fenómeno está vinculado al Tipo de fenómeno "grupo sanguíneo".

La figura 11-2 aclarará un poco más las cosas.

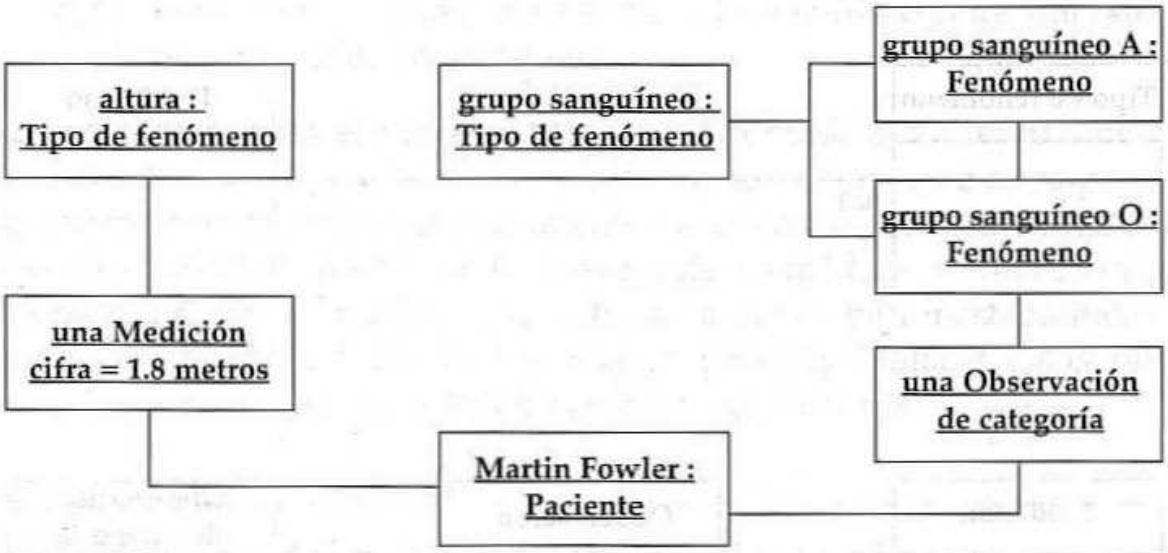


Figura 11-2: *Diagrama de objeto de observación del paciente*

La figura 11-3 muestra que podemos hacer que una Observación sirva como una Medición y como una Observación de categoría, al definir que una Medición de "90 latidos por minuto" también puede ser una Observación de categoría cuyo Fenómeno asociado es "ritmo cardiaco acelerado".

Hasta aquí, sólo me he referido a la representación de los conceptos; no le he prestado mucha atención al comportamiento. No siempre hago esto, pero parece ser un punto de partida apropiado para un problema que trata principalmente del manejo de información.

Por el momento, sigo refiriéndome a los *conceptos* de observación del paciente, de la misma manera que lo haría con un médico o una enfermera (de hecho, así sucede en la realidad; estos modelos conceptuales fueron construidos por un par de médicos y una enfermera, contando con mi ayuda). Para hacer el traslado a un programa orientado a objetos, debo decidir cómo manejar el cuadro conceptual en términos del software. Para este ejercicio he escogido el lenguaje de programación de Java (¡de alguna manera, tenía que incluir a Java en este libro!).

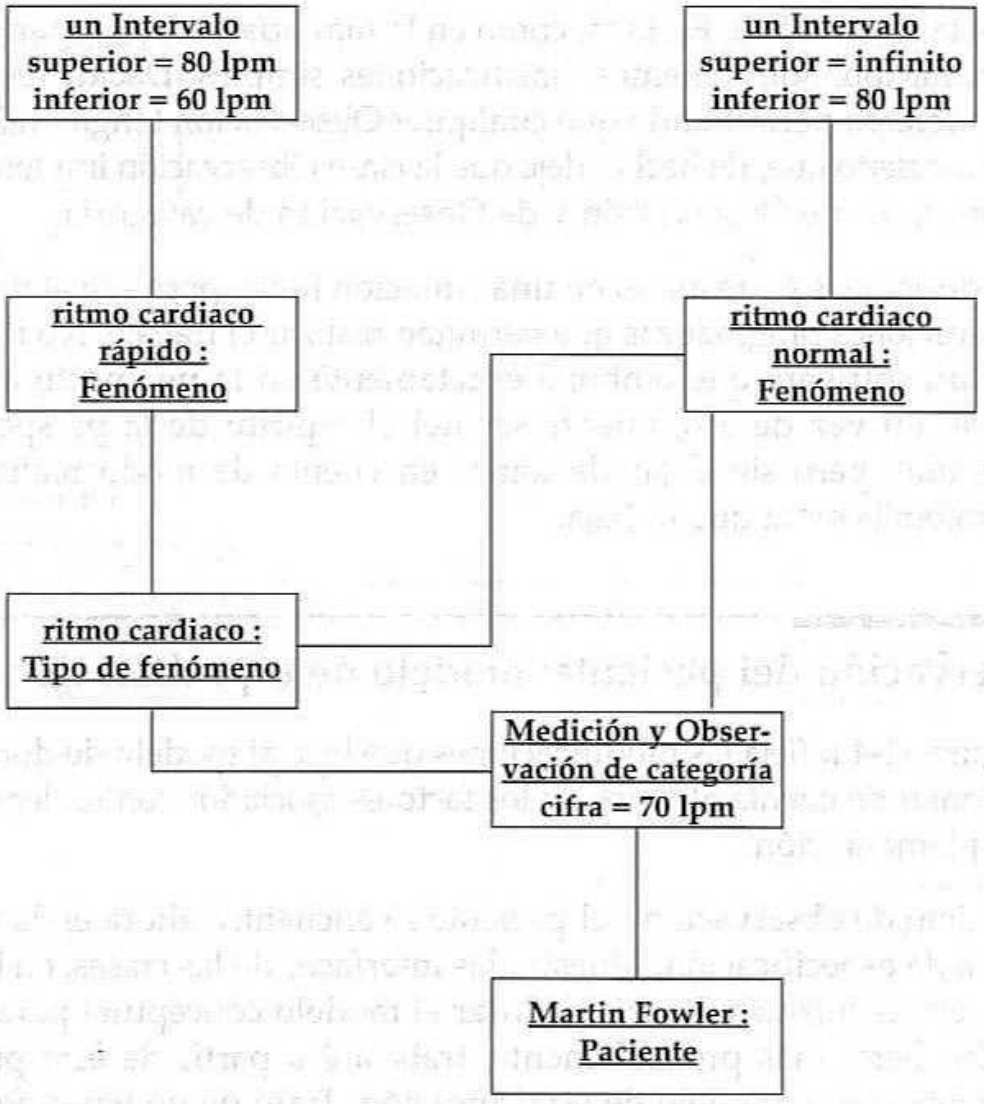


Figura 11-3: *Otro diagrama del objeto de observación del paciente*

La mayor parte de estos conceptos funcionará bien como clases de Java. Paciente, Tipo de fenómeno, Fenómeno, Unidad y Cantidad funcionarán sin problemas. Los elementos difíciles son Intervalo y Observación.

Intervalo es un problema, porque quiero formar un intervalo de cantidades para un Fenómeno. Podría hacerlo creando una interfaz de "magnitud" e indicando que Cantidad implementa esa interfaz, pero eso me complicaría mucho las cosas. Esto no sucede en Smalltalk, y en C++ puedo usar tipos parametrizados. Para este ejercicio, prefiero usar una clase IntervaloCantidad que usa el patrón *Intervalo*.

Mi problema con Observación es que una Observación puede ser al mismo tiempo tanto una Categoría Observación como una Medición

(véase la figura 11-3). En Java, como en la mayoría de los lenguajes de programación, sólo tenemos clasificaciones simples. Decidí resolver este problema permitiendo que cualquier Observación tenga un Fenómeno asociado que, de hecho, deje que la clase Observación implemente los conceptos de Observación y de Observación de categoría.

Estas decisiones no resultan en una situación ideal, pero son el tipo de imperfecciones pragmáticas que permiten realizar el trabajo. No intente hacer un software que embone exactamente en la perspectiva conceptual. En vez de eso, intente ser fiel al espíritu de la perspectiva conceptual, pero sin dejar de tomar en cuenta de modo realista la herramienta con la que trabaja.

Observación del paciente: modelo de especificación

La figura 11-4 refleja las modificaciones que hice al modelo de dominio para tomar en cuenta algunos de los factores asociados con un lenguaje de implementación.

El modelo de observación del paciente se encuentra ahora en la perspectiva de especificación. Muestra las interfaces de las clases, en lugar de las clases mismas. Podría guardar el modelo conceptual para otra ocasión, pero, más probablemente, trabajaré a partir de este punto, solamente con el modelo de especificación. Trato de no tener por ahí muchos modelos mientras trabajo. Mi regla es que, si no puedo mantener actualizado un modelo, se va al cesto (¡también soy un poco perezoso!).

Veamos a continuación el *comportamiento* asociado a nuestro modelo de observación del paciente.

El primer escenario pide la última medición del ritmo cardiaco del paciente. La primera pregunta es: ¿de quién depende la responsabilidad de contestar esta pregunta? La elección natural parece ser el Paciente mismo. El Paciente necesita ver todas sus observaciones, determinar cuáles son las medidas del Tipo de fenómeno "ritmo cardiaco" y encontrar la última cifra. Para hacerlo, tendré que añadir una marca de tiempo a Medición. Dado que esto podría aplicarse igualmente a otras observaciones, también la añadiré a Observación.

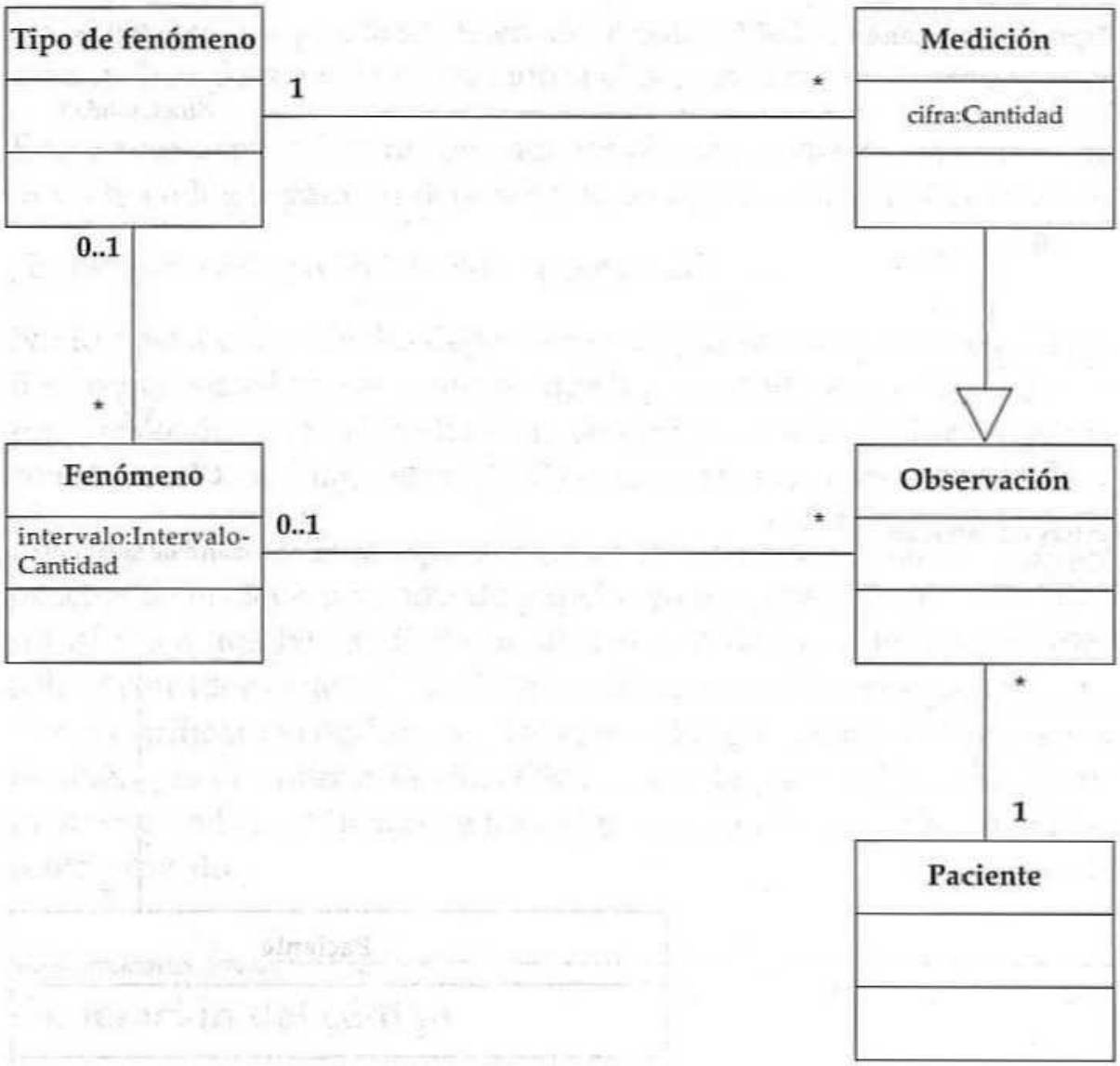


Figura 11-4: *Modelo de especificación de observación del paciente*

Existe una responsabilidad similar para Fenómeno: encontrar la última Observación de categoría que tiene un Fenómeno para el Tipo de fenómeno dado.

La figura 11-5 muestra las operaciones que he añadido a Paciente para representar mis ideas.

No haga demasiados esfuerzos por determinar operaciones, si aún no son obvias. Lo más importante que hay que buscar es una declaración de responsabilidad. Si puede convertirla dándole la forma de una operación, perfecto; pero si no, una frase breve bastará para describir la responsabilidad.

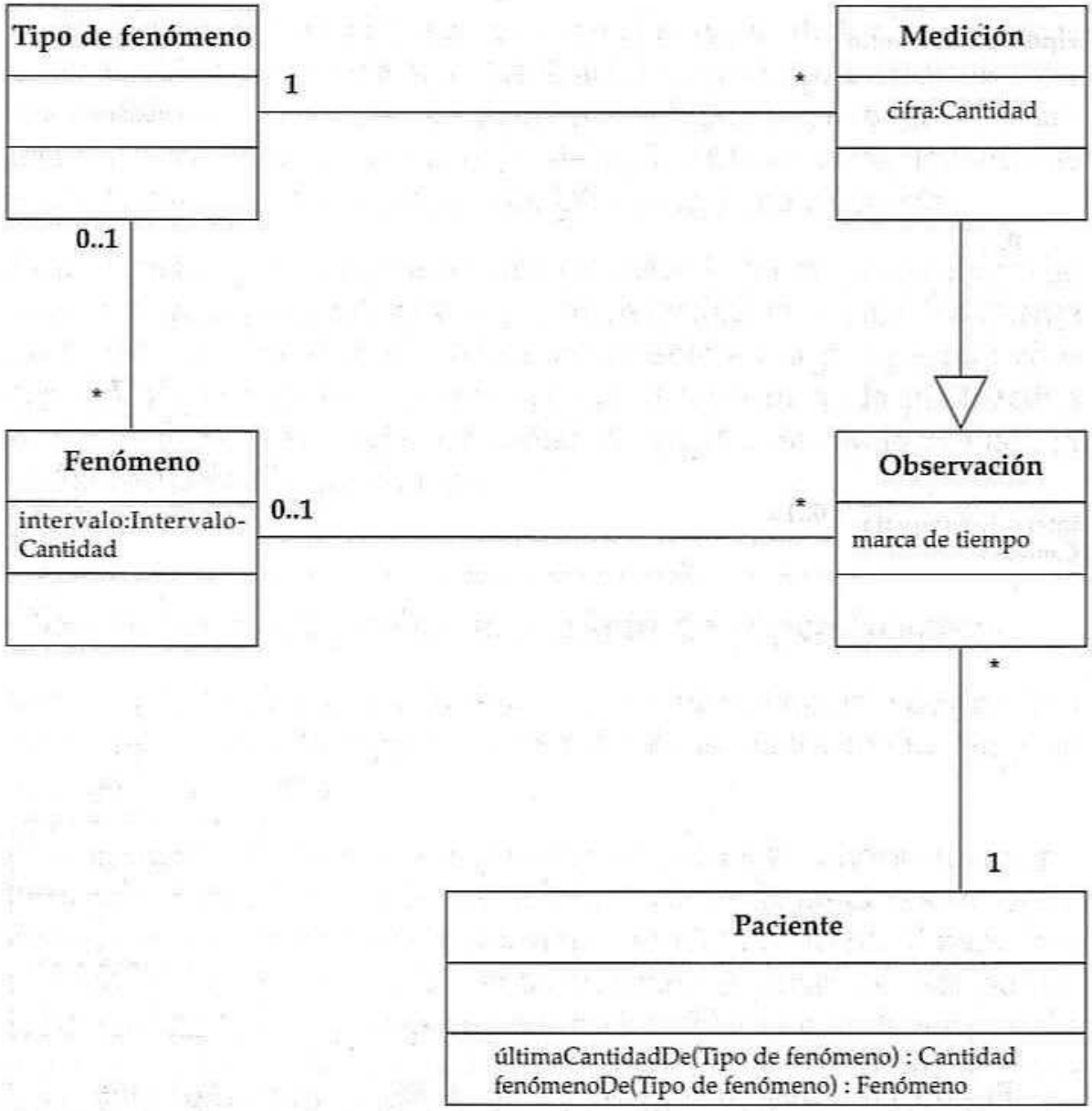


Figura 11-5: *Operaciones de observación de pacientes*

Para actualizar el nivel de conciencia del paciente es necesario crear una nueva Observación del Fenómeno apropiado. Al hacer esto, el usuario preferirá generalmente seleccionar un Fenómeno de una lista desplegable de cierta clase. Esto se puede manejar asociando los objetos Fenómeno con un Tipo de fenómeno en particular, en tanto que esta responsabilidad está implícita en la asociación entre los dos.

Al añadir una medición, necesitamos crear una nueva Medición. Ciertas complicaciones surgen del hecho de que la Medición necesita ver si hay

un Fenómeno que pueda ser asignado. Aquí, la Medición puede preguntar a su Tipo de fenómeno asociado si existe un Fenómeno por asignar.

Existe aquí cierta colaboración entre los objetos, la cual sugiere que éste es un buen lugar para un diagrama de secuencia (véase la figura 11-6).

¿Es *necesario* dibujar todos estos diagramas?

No forzosamente. Mucho dependerá de la forma en que usted visualice lo que sucede y de cómo se sienta trabajando en su lenguaje de programación. En Smalltalk es, en general, tan fácil escribir el código como pensar con diagramas. En C++, son más útiles los diagramas.

Los diagramas no tienen que ser obras maestras. Yo usualmente hago bocetos de ellos en una hoja de papel o en un pizarrón pequeño. Los transfiero a una herramienta de dibujo (o a una herramienta CASE) sólo si considero que vale la pena mantenerlos al día, porque me ayudan a clarificar la conducta de las clases. En este punto del proyecto, también puedo utilizar tarjetas CRC (véase la página 74) como complemento en la sustitución de los diagramas que he descrito en el presente capítulo.

Generación del código

A continuación, podemos echar una ojeada a algunas partes del código que implementa las ideas expuestas en las secciones anteriores. Iniciaré con el Tipo de fenómeno y el Fenómeno, ya que ambos están estrechamente vinculados.

Lo primero que hay que considerar es la asociación entre ambos: ¿debe la interfaz permitir la navegación en ambas direcciones? En el presente caso, pienso que sí, ya que ambas direcciones son valiosas y, en ambos casos, son conceptos estrechamente vinculados. De hecho, me siento cómodo implementando también la asociación con apuntadores en ambas direcciones. Sin embargo, la haré una asociación inmutable, ya que se trata de objetos que se establecen y luego se dejan solos, no se modifican frecuentemente y, cuando sucede, podemos crearlos de nuevo.

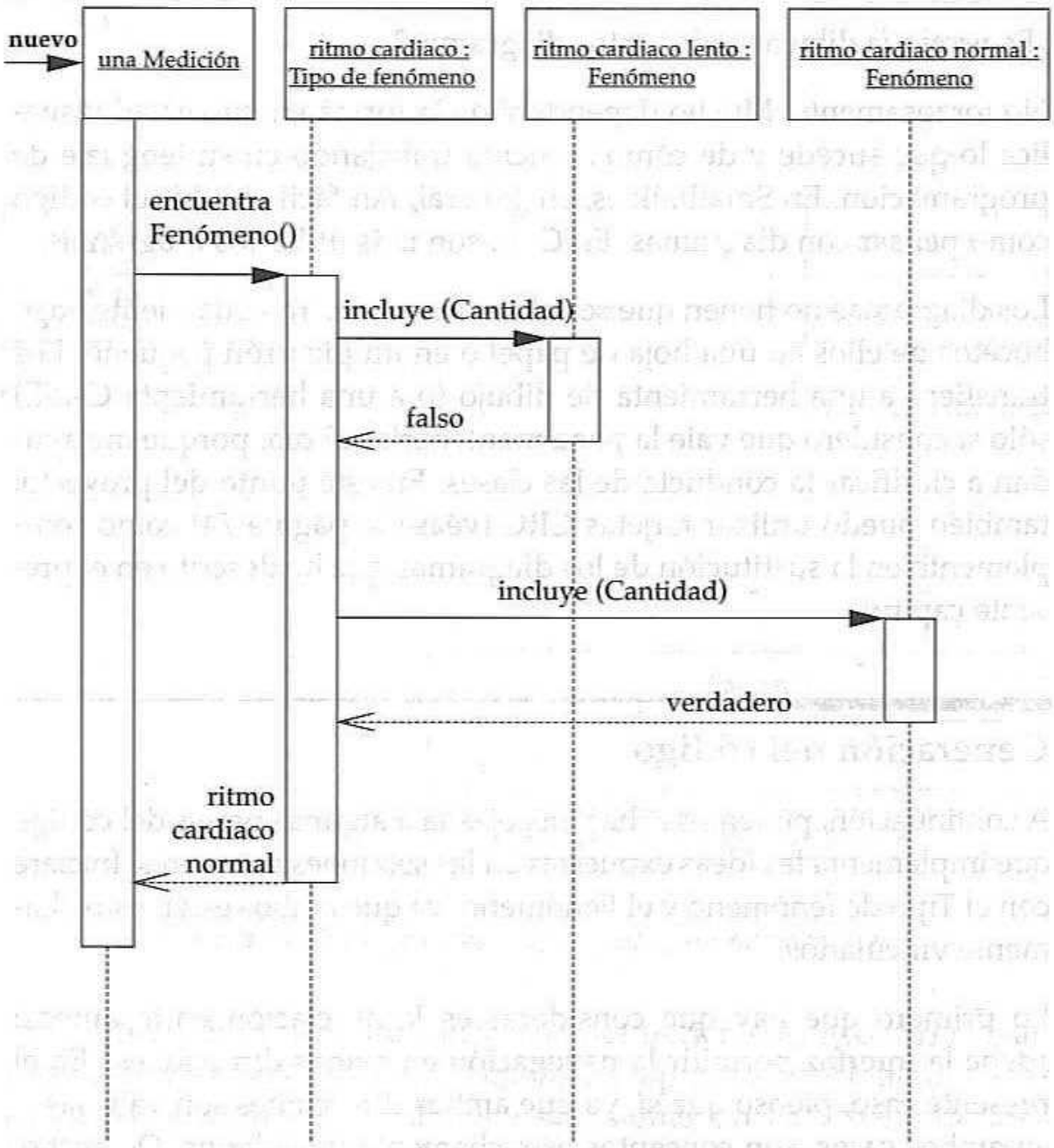


Figura 11-6: *Diagrama de secuencia de observación del paciente*

Algunas personas tienen problemas con los vínculos de doble sentido. Por mi parte, no los encuentro problemáticos, si me aseguran que una clase asume toda la responsabilidad de mantener actualizado el vínculo, ayudado por un "amigo" o un método de asistencia, según sea necesario.

Veamos algunas declaraciones.

```
public class PhenomenonType extends DomainObjet {
 public PhenomenonType (String name) {
  super (name);
 };

 void friendPhenomenonAdd (Phenomenon newPhenomenon) {
  // RESTRICTED: only used by Phenomenon
  _phenomena.addElement (newPhenomenon) ;
 } ;

 public void setPhenomena (String [ ] names) {
  for (int i = 0 ; i < names.length ; i++)
   new Phenomenon (names [ i ], this ;
 } ;

 public Enumeration phenomena ( ) {
  return _phenomena.elements ( ) ;
 } ;

 private Vector _phenomena = new Vector ( ) ;

 private QuantityRange _validRange ;
}
```

Yo aplico la convención de añadir un guión bajo antes de todos los campos. Me ayuda a evitar confundir mis nombres.

```
public class Phenomenon extends DomainObjet {
 public Phenomenon (String name, PhenomenonType type) {
  super (name) ;
  _type = type ;
  _type.friendPhenomenonAdd (this) ;
```

```
 public PhenomenonType phenomenonType ( ) {
  return _type;
 };

 private PhenomenonType _type;

 private QuantityRange _range;
}

package observations;

public class DomainObject {
 public DomainObject (String name) {
  _name = name;
 };

 public DomainObject ( )  { };

 public String name ( ) {
  return _name;
 };

 public String toString ( ) {
  retunr _name;
 };

 protected String _name = "no name" ;
}
```

He añadido una clase DomainObject, que sabe sobre nombres y que hará cualquier otro comportamiento que quiero que hagan todas mis clases de dominio.

Ahora, puedo establecer estos objetos mediante código, de acuerdo con las siguientes líneas.

```
PhenomenonType sex =
 new PhenomenonType ("gender").persist ( ) ;
String [ ] sexes = {"male", "female} ;
sex.setPhenomena (sexes) ;
```

La operación **persist()** almacena el Tipo de fenómeno (PhenomenonType) en un objeto de registro, de modo que se pueda volver a tomarlo después con un método estático **get()**. No entraré en detalles al respecto.

A continuación, ingreso el código que añade observaciones a un paciente. Aquí no deseo que todas las asociaciones sean de doble sentido. Hago que el paciente se cuelgue de un conjunto de observaciones, ya que las observaciones se usan en el contexto de un paciente.

```
public class Observation extends DomainObject {
 public Observation(Phenomenon relevantPhenomenon,
  Patient patient, Date whenObserved) {
   _phenomenon = relevantPhenomenon;
   patient.observationsAdd(this);
   _whenObserved = whenObserved;
 };

 private Phenomenon _phenomenon;

 private Date _whenObserved;
}

public class Patient extends DomainObject {
 public Patient(String name) {
  super(name);
 };

 void observationsAdd(Observation newObs) {
  _observations.addElement(newObs);
 };

 private Vector _observations = new Vector();
}
```

Con lo anterior, puedo crear observaciones.

```
new Patient("Adams").persist();
new Observation(PhenomenonType.get("gender").
 phenomenonNamed("male"), Patient.get("Adams"),
 new Date(96, 3, 1));
```

```
class PhenomenonType {
 public Phenomenon phenomenonNamed(String name) {
  Enumeration e = phenomena();
  while (e.hasMoreElements() )
  {
   Phenomenon each = (Phenomenon) e.nextElement()
   if (each.name() = = name)
    return each;
  };

  return null;
 }
```

Después de crear las observaciones, necesito ser capaz de encontrar el fenómeno más reciente.

```
class Patient
 public Phenomenon phenomenonOf
  (PhenomenonType phenomenonType)
 {
  return (latestObservation (phenomenonType) = =
   null ? new NullPhenomenon() :
   latestObservation(phenomenonType).phenomenon() );
 }

 private Observation
  latestObservation(PhenomenonType value) {
  return latestObservationIn(observationsOf(value) );
 }

 private Enumeration
  observationsOf(PhenomenonType value) {
   Vector result = new Vector();
   Enumeration e = observations();
   while (e.hasMoreElements() )
   {
    Observation each = (Observation) e.nextElement();
    if (each.phenomenonType() = = value)
     result.addElement (each);
 };
 return result.elements();
}
```

```
private Observation latestObservationIn
 (Enumeration observationEnum) {
 if (!observationEnum.hasMoreElements () )
  return null;
 Observation result =
  (Observation) observationEnum.nextElement ();
 if (!observationEnum.hasMoreElements () )
  return result;

 do
 {
  Observation each =
   (Observation) observationEnum.nextElement ();
  if (each.whenObserved ().
   after (result.whenObserved () ) )
    result = each;
 }

 while (observationEnum.hasMoreElements () ) ;

 return result;
}

class Observation
 public PhenomenonType phenomenonType () {
  return _phenomenon.phenomenonType ();
 }
```

Existen varios métodos que se combinan para lograr esto. Se podría dibujar un diagrama que muestre lo anterior, pero yo no suelo tomarme la molestia. El modo en que descompongo un método tiene que ver más con la reestructuración de factores (véase la página 35) que con el diseño previo.

Podemos ahora encargarnos de añadir la conducta de las mediciones.

En primer lugar, veamos la definición de la clase Medición (Measurement) y su constructor.

```
public class Measurement extends Observation {
 public Measurement (Quantity amount,
  PhenomenonType phenomenonType,
  Patient patient, Date whenObserved)  {
   initialize (patient, whenObserved);
```

```
    _amount = amount;
    _phenomenonType = phenomenonType;
  };

  public PhenomenonType phenomenonType () {
    return _phenomenonType;
  };

  public String toString () {
    return _phenomenonType + ": " + _amount;
  };

  private Quantity _amount;

  private PhenomenonType _phenomenonType;
}

class Observation
  protected void initialize (Patient patient,
    Date whenObserved) {
      patient. observationsAdd (this);
      _whenObserved = whenObserved;
  }
```

Nótese que un diagrama de clases es un buen comienzo para desarrollar esto. De nuevo necesitamos la medición más reciente.

```
Class Patient
  public Quantity latestAmountOf (PhenomenonType value) {
    return ((latestMeasurement (value) = = null)  ?
    new NullQuantity () :latestMeasurement (value)
    .amount ();
  }

  private Measuremet (PhenomenonType value) {
    if (latestObservation (value) = = null)
      return null;
    if (!latestObservation (value) .isMeasurement () )
      return null;
    return (Measurement) latestObservation (value);
```

En ambos casos, el diagrama de clases sugiere una estructura básica y le añadimos comportamiento para poder manejar consultas más interesantes.

En este punto, podemos describir nuestra posición con respecto al diagrama de clases de perspectiva de especificación que se muestra en la figura 11-7.

Obsérvese cómo este diagrama enfatiza la interfaz sobre la implementación. He modelado el papel de Tipo de fenómeno (Phenomenon type) a Fenómeno (Phenomenon) como un papel calificado, pues es la interfaz primaria en Tipo de fenómeno. De manera similar, he mostrado Observación (Observation) con un vínculo hacia Tipo de fenómeno, porque la interfaz existe ahí, aunque la medición es la única con un apuntador directo a Fenómeno.

Al observar este diagrama, podemos ver que la única diferencia entre Medición (Measurement) y Observación (Observation) es que la Medición tiene una cantidad. Podríamos eliminar la clase Medición del modelo de especificación permitiendo que cualquier observación tenga una cantidad (potencialmente nula).

Incluso, podríamos tener una clase Medición separada, que podría tener campos de *cantidad* y *tipo de fenómeno*, pero nadie fuera del paquete tendría conocimiento de la existencia de la clase. Necesitaríamos añadir métodos Factory (Gamma *et al.* 1994), ya sea en Observación o en Paciente (Patient), para permitir la creación de la clase apropiada.

Dejaré esa modificación como ejercicio para el lector y continuaré con la asignación automática de un Fenómeno a una Medición.

La figura 11-7 ilustra el proceso general.

En primer lugar, necesitamos añadir una llamada de método al constructor de Medición.

```
Class Measurement
 public Measurement (Quantity amount,
 PhenomenonType phenomenonType,
 Patient patient, Date whenObserved)
  initialize (patient, whenObserved);
  _amount = amount;
  _phenomenonType = phenomenonType;
  _phenomenon = calculatePhenomenonFor (_amount);
```

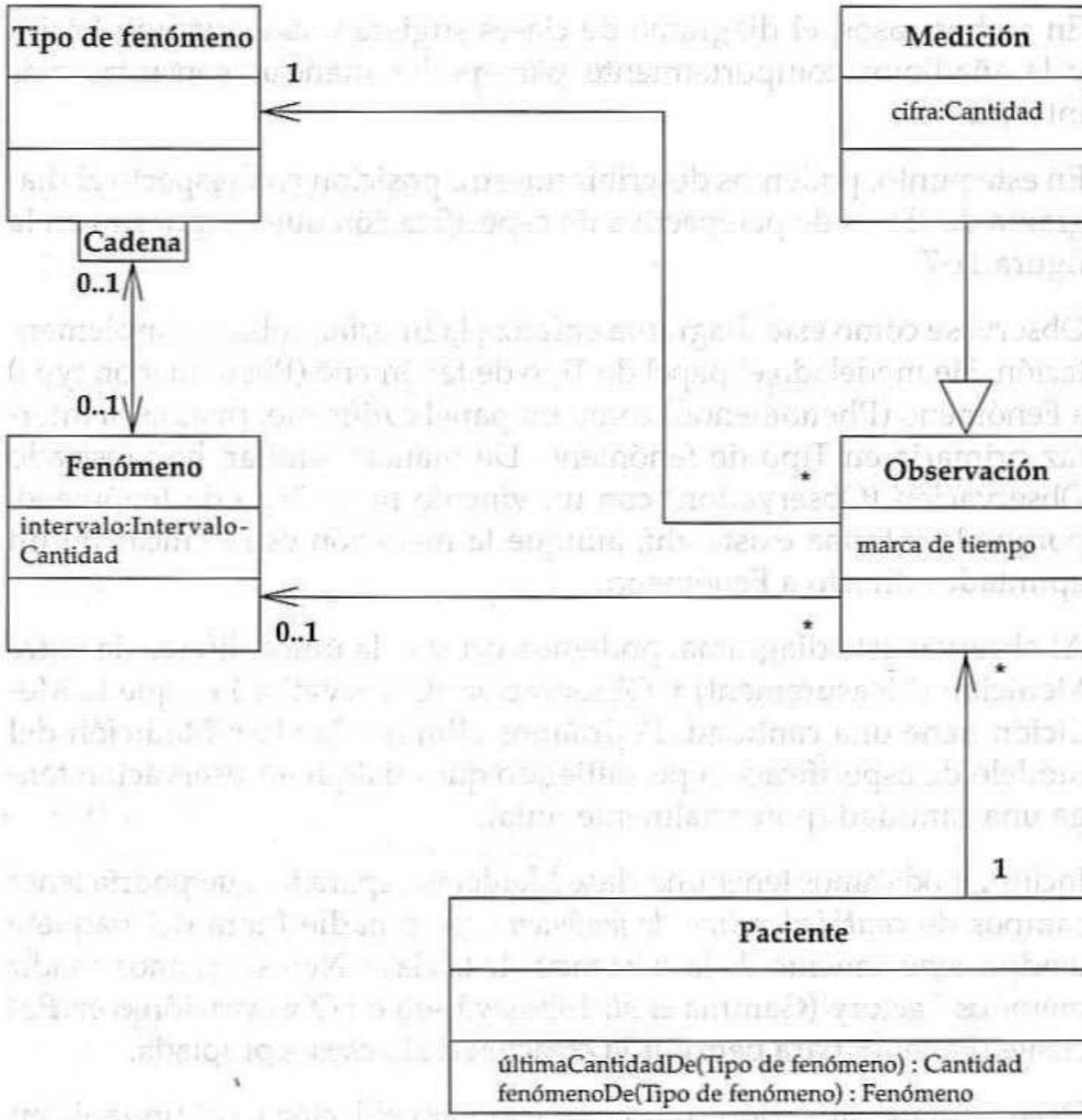


Figura 11-7: *Otro modelo de especificación de observación del paciente*

Esto solicita cada fenómeno a su vez.

```
Class Measurement
 public Phenomenon calculatePhenomenonFor (Quantity arg)
 {
  return _phenomenonType.phenomenonIncluding (arg);
 }
```

Esto solicita cada fenómeno a su vez.

```
Class PhenomenonType
 public Phenomenon phenomenonIncluding (Quantity arg) {
  Enumeration e = phenomena();
  while (e.hasMoreElements())
  {
   Phenomenon each = (Phenomenon) e.nextElement();
   if (each.includes(arg))
    return each;
  };

  return null;
 }
Class Phenomenon
  public boolean includes (Quantity arg) {
  return (_range == null ? false:_range.includes(arg));
 }
```

El código surge con naturalidad del diagrama de secuencia. En la práctica, generalmente me sirvo de un diagrama de secuencia para bosquejar la interacción y después hago algunos cambios a medida que lo codifico. Si la interacción es importante, entonces actualizo la gráfica de secuencia como parte de mi documentación. Si considero que tal gráfica no añadirá mucha claridad al código, archivo el borrador de la gráfica de secuencia en el archivero circular.

Éste es un ejemplo breve de cómo utilizar el UML con un lenguaje de programación; sin embargo, le debe dar a usted una buena idea del proceso. No tiene que estar trabajando en un proyecto de altos vuelos para determinar que le viene a la mano utilizar partes del UML. No es necesario utilizarlo en su totalidad; basta con las partes que le sean convenientes.

El esbozar un diseño con un diagrama de clases y con un diagrama de interacción le puede ayudar a poner en orden sus pensamientos y a facilitarle la codificación. Yo considero estos borradores como prototipos rápidos. No debe necesariamente conservar los diagramas después, pero podría ser más fácil que usted y otros comprendan el código, si lo hace.

No es necesario que disponga de una herramienta CASE cara y rebuscada. Un pizarrón y una simple herramienta de dibujo en su computadora serán suficientes. Existen, por supuesto, herramientas CASE muy útiles y, si está involucrado en un proyecto mayor, podría considerar disponer de una de ellas.

Si es así, entonces compare tal herramienta con la base de una herramienta de dibujo sencilla y un procesador de textos (es extraordinario lo mucho que puede hacerse con, por ejemplo, Visio y Word). Si la herramienta tiene generación de código, observe con cuidado cómo genera el código. La generación de código obliga a la herramienta CASE a interpretar de manera muy particular los diagramas, lo cual afectará el modo en que usted dibujará los diagramas y el significado de los mismos.

He realizado proyectos de dimensiones medias en los que comenzamos con una herramienta CASE y terminamos dejándola de lado. No tema hacerlo si considera que la herramienta no le ayuda.

Puede encontrar más información sobre este ejemplo en mi dirección Web (<**our-world.compuserve.com/homepages/Martin_Fowler**>). La versión del ejemplo en dicho sitio ahonda más en algunas de las cuestiones relacionadas con el proceso de capas que interviene en llevar este modelo a una interfaz de usuario.

Apéndice A

Técnicas y sus usos

Técnica	*Propósito*
Diagrama de actividades	Muestra el comportamiento dentro de una estructura de control. Puede mostrar muchos objetos a través de muchos usos, muchos objetos en un solo caso de uso, o la implementación de un método. Alienta el comportamiento paralelo.
Diagrama de clase	Muestra la estructura estática de conceptos, tipos y clases. Los conceptos indican cómo piensan los usuarios acerca del mundo; los tipos muestran las interfaces de los componentes de software; las clases muestran la implementación de los componentes de software.
Tarjetas CRC	Ayudan a encontrar la esencia del propósito de una clase. Son buenas para explorar la forma de implementar un caso de uso. Úselas cuando esté empantanado en los detalles o si está aprendiendo el enfoque del diseño orientado a objetos.

Técnica	*Propósito*
Diagrama de emplazamiento	Muestra la disposición física de los componentes en los nodos de hardware.
Diseño por contrato	Provee una definición rigurosa del propósito de una operación y del estado de licitud de una clase. Codifique éstos dentro de las clases para mejorar la depuración.
Diagrama de interacción	Muestra cómo colaboran varios objetos en un solo caso de uso.
Diagrama de paquetes	Muestra grupos de clases y las dependencias entre ellas.
Patrones	Ofrecen buenos fragmentos de técnicas de análisis, diseño y codificación. Son buenos ejemplos de los que se puede aprender; constituyen un punto de partida para los diseños.
Reestructuración de factores	Ayuda a realizar cambios que mejoren la estructura de un programa funcional. Úsese cuando el código se interponga en el camino hacia un buen diseño.
Diagrama de estados	Muestra cómo un objeto particular se comporta a lo largo de muchos casos de uso.
Caso de uso	Ayuda a los usuarios a plantear sus requerimientos en bloques significativos. La planificación de la construcción se realiza en torno a la entrega de algunos casos de uso en cada iteración. Es la base para las pruebas del sistema.

Apéndice B

Cambios del UML 1.0 al 1.1

En septiembre de 1997 presenciamos el lanzamiento del UML 1.1, el cual incorpora muchos cambios que resultaron del proceso de normalización del OMG. Las impresiones de *UML destilado* a partir de la sexta impresión en adelante se ajustarán para reflejar estos cambios. Sin embargo, para aquellos que tienen impresiones anteriores de este libro, he recopilado este documento para resumir los cambios. No voy a analizar todos los cambios del UML 1.0 al 1.1, sino únicamente aquellos que

- cambien algo de lo que dije en *UML destilado*, o
- que representen características importantes que no hubiera explicado en *UML destilado*

Tenga presente que hay un complicado problema de numeración de versiones. Cuando se lanzó la nueva versión de UML, se le llamó 1.1, pues se derivaba de la 1.0; bastante lógico. Esta lógica fue aniquilada cuando el OMG adoptó la norma y la llamó versión 1.0. Por tanto, la versión 1.1 de UML es la versión 1.0 de UML del OMG, que es distinta de la versión 1.0 original. Enredado, ¿verdad?

Continúo apegándome al espíritu de *UML destilado*: estudiar los elementos clave de UML, según afecten la aplicación del UML en los pro-

yectos reales. Como siempre, las selecciones y recomendaciones son mías. Si hay algún conflicto entre lo que digo y los documentos oficiales del UML, habrá que hacerle caso a los documentos del UML (pero tenga la bondad de informarme, para que yo proceda a corregirlo).

También he aprovechado la ocasión para indicar errores y omisiones importantes. Agradezco a los lectores que me los han hecho saber.

Tipos y clases de implementación

En la página 63 de *UML destilado* hablé sobre las perspectivas y cómo alteraban la manera en que la gente dibuja e interpreta los modelos, en particular, los diagramas de clase. El UML ahora toma en cuenta esto, diciendo que todas las clases en un diagrama de clase se pueden especializar como tipos o como clases de implementación.

Una **clase de implementación** corresponde a una clase en el ambiente del software en el que usted se esté desarrollando. El **tipo** es algo un poco más nebuloso; representa una abstracción menos ligada a la implementación. Esto podría ser un tipo CORBA, una perspectiva de especificación de una clase o una perspectiva conceptual. De ser necesario, se pueden añadir estereotipos para hacer aún mayor la diferenciación.

Se puede indicar que, para un diagrama en particular, todas las clases se apegan a un estereotipo particular. Esto es lo que se haría al dibujar un diagrama desde una perspectiva en particular. La perspectiva de implementación utilizaría clases de implementación, mientras que la perspectiva de especificación y la conceptual emplearían tipos.

Se usa la relación de realización para indicar que una clase de implementación implementa uno o más tipos.

Hay una distinción entre tipo e interfaz. Se pretende que una interfaz corresponda directamente a una interfaz estilo Java o COM. Las interfaces, por tanto, sólo tienen operaciones, no atributos.

Se puede emplear una sola clasificación estática con las clases de implementación, pero se pueden utilizar clasificaciones múltiples y dinámicas con los tipos (supongo que esto es porque los principales lenguajes OO se apegan a una clasificación simple y estática. Si uno de

estos días usted trabaja con un lenguaje que maneje clasificación múltiple o dinámica, en realidad esta restricción no procedería).

Operaciones y atributos con alcance de clase

El UML se refiere a una operación o atributo que tiene aplicabilidad con la clase, en lugar de con una instancia, es decir, que tiene **alcance de clase**. Esto es equivalente a los miembros static de C++ y Java y a las variables y métodos de clase de Smalltalk. Las características de alcance de clase aparecen subrayadas en los diagramas de clase. Esta notación estaba disponible en el UML 1.0, pero pasé por alto hablar sobre ello.

Restricciones discriminatorias completas e incompletas

En la página 88 de las impresiones previas de *UML destilado* dije que la restricción {completo} sobre una generalización indicaba que todas las instancias del supertipo deben ser también una instancia de un subtipo dentro de dicha partición. UML 1.1 define, en cambio, que {completo} indica que todos los subtipos dentro de dicha partición han sido especificados, lo cual no es precisamente lo mismo. En distintas conversaciones he encontrado algunas inconsistencias en la interpretación de esta restricción, así que tenga cuidado con ella. Si lo que quiere hacer es indicar que todas las instancias del supertipo deben ser una instancia de uno de los subtipos, entonces le sugiero que emplee otra restricción, a fin de evitar confusiones. Actualmente, yo utilizo {obligatorio}.

Composición

En el UML 1.0, la composición implicaba que el vínculo era inmutable (o estaba congelado; véase más adelante), cuando menos en el caso de componentes de un solo valor. Esta restricción ya no es parte de la definición. Con toda justeza, aún hay un poco de confusión sobre la interpretación de esto.

Inmutabilidad

El UML define que la restricción {congelado} significa la inmutabilidad de las relaciones de asociación. Según la definición actual, no parece aplicarla a atributos ni a clases. En mi trabajo práctico, ahora empleo el término congelado en lugar de inmutabilidad, y me encuentro satisfecho aplicando la restricción a las relaciones de asociación, clases y atributos.

Regresos en los diagramas de secuencia

En el UML 1.0, el regreso en un diagrama de secuencia se identificaba poniendo una punta de flecha con guión, en lugar de una punta de flecha rellena (véase la página 116). Esto resultaba un tanto incómodo, pues la diferencia era demasiado sutil y fácil de pasar por alto. El UML 1.1 utiliza una flecha punteada para indicar un regreso, lo que me agrada, y hace mucho más obvios los regresos (dado que yo utilicé regresos punteados en *Análisis de patrones*, esto también me hace parecer influyente). Lo que se regresa se puede nombrar para su uso posterior con un nombre de la forma "hayExistencia:=revisa()".

Empleo del término "papel"

En UML 1.0, el término **papel** indicaba principalmente una dirección en una asociación (véase la página 66). UML 1.1 se refiere a este uso como **papel de asociación**. También hay un **papel de colaboración**, que es el que tiene la instancia de una clase en una colaboración. El UML 1.1 pone mucho más énfasis en las colaboraciones, así que parece que este empleo de "papel" se convertirá en el principal.

Marcadores de iteraciones

Tanto en UML 1.0 como en UML 1.1 se puede emplear un * para marcar los mensajes que suceden muchas veces en un diagrama de

secuencia, diagrama de colaboración o diagrama de actividades. Con frecuencia, conviene indicar cuál es la base de la iteración. Esto se puede hacer utilizando una expresión dentro de corchetes []: por ejemplo, *[para cada artículo de línea], o *[i=1..n] (el UML no indica lo que debe ser el contenido de la expresión; escriba en español o en su lenguaje de programación).

Bibliografía

Kent Beck: *Smalltalk Best Practice Patterns.* Prentice Hall, 1996.

Kent Beck: "Make It Run, Make It Right: Design Through Refactoring." *The Smalltalk Report*, Vol. 6, No. 4, pp. 19-24, SIGS Publications, January 1997.

Kent Beck and Ward Cunningham: "A Laboratory For Teaching Object-Oriented ThinKing." *Proceedings of OOPSLA 89.* SIGPLAN Notices, Vol. 24, No. 10, pp. 1-6. Véase <**http://c2.com/doc/oopsla89/paper.html**>

Grady Booch: *Object-Oriented Analysis and Design with Applications, Second Edition.* Addison-Wesley, 1994.

Grady Booch: *Object Solutions: Managing the Object-Oriented Project.* Addison-Wesley, 1996.

Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal: *Pattern-Oriented Software Architecture: A System of Patterns.* John Wiley & Sons, 1996.

Peter Coad and Jill Nicola: *Object-Oriented Programming.* Yourdon, 1993.

Peter Coad, David North, and Mark Mayfield: *Object Models: Strategies, Patterns and Applications.* Prentice Hall, 1995.

Peter Coad and Edward Yourdon: *Object-Oriented Analysis.* Yourdon, 1991.

Peter Coad and Edward Yourdon: *Object-Oriented Design.* Yourdon, 1991.

Steve Cook and John Daniels: *Designing Object Systems: Object-Oriented Modeling with Syntropy.* Prentice Hall, 1994.

James Coplien: "A Generative Development Process Pattern Language." In Coplien and Schmidt, 1995, pp. 183-237.

James O. Coplien and Douglas C. Schmidt, eds: *Pattern Languages of Program Design* [PLoPD1]. Addison-Wesley, 1995.

Ward Cunningham: "EPISODES: A Pattern Language of Competitive Development." In Vlissides *et al.* 1996, pp. 371-388.

Martin Fowler: *Analysis Patterns: Reusable Object Models*. Addison-Wesley, 1997.

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides [Gang of Four]: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.

Adele Goldberg and Kenneth S. Rubin: *Succeeding With Objects: Decision Frameworks for Project Management*. Addison-Wesley, 1995.

Graham, Ian: *Métodos orientados a objetos*, Segunda edición, Addison-Wesley/Díaz de Santos, 1996.

David Harel: "Statecharts: A Visual Formalism for Complex Systems." In *Science of Computer Programming*, Vol. 8, 1987.

Ivar Jacobson, Magnus Christerson, Patrik Jonsson, and Gunnar Övergaard: *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, 1992.

Ivar Jacobson, Maria Ericsson, and Agneta Jacobson: *The Object Advantage: Business Process Reengineering with Object Technology*. Addison-Wesley, 1995.

Andrew Koenig and Barbara Moo: *Ruminations on C++: A Decade of Programming Insight and Experience*. Addison-Wesley, 1997.

Martin, James; Odell, James J.: *Métodos orientados a objetos: Conceptos fundamentales*, Prentice-Hall Hispanoamericana, México, 1997.

Martin, James; Odell, James J.: *Métodos orientados a objetos: Consideraciones prácticas*, Prentice-Hall Hispanoamericana, México, 1997.

Robert Cecil Martin: *Designing Object-Oriented C++ Applications: Using the Booch Method*. Prentice Hall, 1995.

Steve McConnell: *Rapid Development: Taming Wild Software Schedules*. Microsoft Press, 1996.

Bertrand Meyer: *Object-Oriented Software Construction*. Prentice Hall, 1997.

William F. Opdyke: *Refactoring Object-Oriented Frameworks*. Ph.D. Thesis, University of Illinois at Urbana-Champaign, 1992. Véase <**ftp://st.cs.uiuc.edu/pub/papers/refactoring/opdyke-thesis.ps.Z**>

James Rumbaugh: *OMT Insights*. SIGS Books, 1996.

James Rumbaugh, Michael Blaha, William Premerlani, Frederick Eddy, and William Lorenzen: *Object-Oriented Modeling and Design*. Prentice Hall, 1991.

Sally Shlaer and Stephen J. Mellor: *Object Lifecycles: Modeling the World in States*. Yourdon, 1991.

Sally Shlaer and Stephen J. Mellor: *Object-Oriented Systems Analysis: Modeling the World in Data*. Yourdon, 1989.

Sally Shlaer and Stephen J. Mellor: "Recursive Design of an Application Independent Architecture". *IEEE Software*, Vol. 14, No. 1, 1997.

John M. Vlissides, James O. Coplien, and Norman L. Kerth, eds.: *Pattern Languages of Program Design 2* [PLoPD2]. Addison-Wesley, 1996.

Kim Walden and Jean-Marc Nerson: *Seamless Object-Oriented Software Architecture: Analysis and Design of Reliable Systems*. Printice Hall, 1995.

Rebecca Wirfs-Brock, Brian Wilkerson, and Lauren Wiener: *Designing Object-Oriented Software*. Prentice Hall, 1990.

Índice

A

clase abstracta
  comparada con interfaz 98
  notación 95
restricción abstracto 96
acción 138
activaciones 119
actividad
  descomposición 158
  en diagrama de actividades 147
  en diagrama de estados 138
diagramas de actividades 10, 22, 38, 40, 124, 125, 144, 185
  definición 147
  ejemplos 148, 152, 154, 155, 157, 158
  cuándo utilizar 159
actor
  definición 52
  relación con casos de uso 53
agregación
  definición 90
  ejemplo 92
patrones de análisis
  definición 44
  ejemplo 44
  mapa histórico 107
  correlación indizada 104
  observación 166
  fenómeno con intervalo 166
  cantidad 166
  intervalo 110, 166, 169
  modelos de papeles 90
  escenario 44
riesgo arquitectónico 30
afirmación
  definición 80
  papel en la subclasificación 81
asociación 61
  como responsabilidad 67
  bidireccional 71
  definición 66
  nombrado 71
  navegabilidad 69
  papeles 66
  unidireccional 71
clase de asociación
  definición 104
  ejemplo 104
  promoción a clase completa 105
  sutilezas 106
mensaje asíncrono 120
atributo
  definición 72
  notación 72

B

restricción bolsa 100
base arquitectónica, componentes 29
Beck, Kent 3, 33, 37, 46, 48, 74, 76, 193
bidireccional, asociación 71
vínculo, estereotipo 109
Booch, Grady xiii, xv, 1, 3, 4, 12, 48, 65, 68, 84, 135, 137, 145, 193
  Véase también Tres amigos
elemento enlazado 108
Brandt, John 37
Brooks, Fred 33
Buschmann, Frank 46, 117, 125, 193

C

ceremonia 17
diagramas de clase 9, 10, 11, 22, 26, 38, 39, 40, 133, 154, 180, 181, 185
  notaciones de cardinalidad 68
  definición 61
  ejemplos 43, 44, 62, 70, 88, 90, 93, 94, 96, 97, 103, 104, 105, 106, 107, 108, 109, 167, 171, 172, 182
  perspectivas 63
  terminología 65
  cuándo emplear 83
clasificación 102
  definición 87
  tipos 87, 89
tarjetas de Clase-Responsabilidad -Colaboración (CRC)
  Véase tarjetas CRC
Coad, Peter 3, 65, 68, 90, 193
Cockburn, Alistair 59
ejemplos de código (Java) 67, 69, 103, 107, 108, 175, 176, 177, 178, 179, 180, 181, 182, 183
diagrama de colaboración
  comparación con el diagrama de secuencia 123
  definición 121
  ejemplos 122, 123
calendario de compromisos 28
componente 161
patrón Compuesto 93
composición
  definición 91
  notación 92
perspectiva conceptual
  actividades 147
  asociaciones 66
  atributos 72
  definición 63
  asociaciones derivadas 95
  atributos derivados 95
  ejemplo 167
  generalización 77
  operaciones 73
  asociaciones calificadas 103, 104
  cuándo emplear 83
diagrama de estados concurrentes
  ejemplo 143
condición 117
conexión 161
restricciones 79
  abstracta 96
  bolsa 100
  gad 101
  global 118
  jerarquía 101
  congelado 101
  ordenado 100
  sólo lectura 101
fase de construcción
  y UML 38
  definición 17
  descripción 33
Cook, Steve 63, 83, 84, 145, 193
Coplien, James 48, 194
tarjetas CRC 3, 8, 34, 74, 155, 167
  definición 74
  ejemplo 74
  empleo con casos de uso 75
  cuándo usar 75
Cunningham, Ward 3, 30, 39, 46, 48, 74, 76, 193, 194

D

restricción Gad 101
Daniels, John 63, 83, 84, 145, 193
borrado 120
dependencia
  definición 128
  sobre diagramas de clase 97
  sobre diagramas de emplazamiento 161
diagramas de emplazamiento 27, 40, 186
  definición 161
  ejemplo 162
  cuándo utilizar 163
asociación derivada
  definición 93

ejemplo 93
atributo derivado
definición 93
Diseño por contrato 80, 186
definición 80
cuándo utilizar 82
modelo de diseño 22
patrones de diseño
Compuesto 93
definición 43
ejemplo 43
Fachada 131
Suplente 43
discriminador 88
modelo de dominio
diagramas de actividades 156
construcción 22, 24
definición 21
equipo 24, 25
empleo con casos de uso 23
clasificación dinámica
definición 89
ejemplo 90

E

fase de elaboración
construcción del modelo de dominio 21
definición 17
descripción 18
descubrimiento de casos de uso 21, 50, 58
terminación 30
categorías de riesgo 19
evento entrada 141
patrón Episodios 30
diagramas de eventos 159, 160
excepción
definición 81
evento salida 141
relación extends
definición 55
ejemplo 52
cuándo usar 57

F

patrón Fachada 131
métodos Factory 181
campo 72
Fowler, Martin 41, 45, 46, 78, 89, 104, 107, 110, 166, 194
amigo 113
descomposición funcional 127

G

pandilla de los cuatro (gang of four) 41, 43, 46, 64, 78, 93, 131, 181, 194
generalización 102
definición 77
utilización con paquetes 133
método de obtención 76
restricción Global 133
Goldberg, Adèle 48, 194
Graham, Ian 48, 59, 194
guardia
en diagrama de actividades 149
en diagrama de estados 139

H

Hadfield, Tom xvi, 8
Harel, David 137, 194
restricción Jerarquía 101
patrón Mapa Histórico 107
estereotipo Historia 107

I

congelado 101
restricción inmutable 101
perspectiva de implementación
actividades 147
asociaciones 69
atributos 72
definición 64
asociaciones derivadas 93, 95

atributos derivados 93
generalización 77
operaciones 73
asociaciones calificadas 103, 104
refinamiento 97
subclasificación 97
cuándo usar 84
fase de concepción
definición 17
descripción 18
Ingeniería de información 127
diagramas de interacción 8, 11, 23, 26, 39, 41, 75, 144, 156, 160, 186
definición 115
ejemplos 116, 122, 123, 174
tipos 116
cuándo usar 124
interfaz
comparada con clase abstracta 98
definición 95
estereotipo Interfaz 97
invariante 80
definición 81
similitud con afirmación 80
ISE 83
iteración
asignación de casos de uso 32
determinación de longitud 31
determinación de cantidad 32
elementos 17
naturaleza incremental 34
naturaleza iterativa 34
marcador de iteración 117
desarrollo iterativo 9
definición 17
cuándo usar 47

J

Jacobson, Ivar xiv, xv, 1, 3, 4, 12, 48, 49, 51, 59, 65, 68, 86, 87, 117, 194
Véase también Tres amigos
Johnson, Ralph 37, 194
Véase también Pandilla de los cuatro

K

patrón Correlación Indizada 104
Koenig, Andrew 110

L

línea de vida 117
notación de paletas 98
Loomis, Mary 4

M

Martin, James 3, 10, 68, 78, 84, 160, 194
Martin, Robert 135, 194
Mellor, Steve 2, 65, 68, 195
tutoría 28
metamodelo
definición 6
extracto de UML 1.0 6
método 76, 77
Meyer, Bertrand 80, 83, 195
lenguaje de modelado 1
modificador 76
clasificación múltiple
definición 87
ejemplo 88
disparador múltiple 151
multiplicidad
definición 66
en los diagramas de actividades 151
relaciones de valor múltiple 100

N

navegabilidad
definición 69
ejemplos 70
tipos 71
Nerson, Jean-Marc 83, 195
nodo 161
notación 5

O

diagrama de objetos, ejemplos 168, 169
Object Managemente Group (OMG) xvi, 1, 4, 5
Técnica de modelado de objetos (OMT)
  Véase OMT
Objectory 2, 4, 16, 21, 41, 48, 49
estado observable 76
patrón Observación 166
Odell, Jim xvi, 3, 4, 10, 65, 68, 78, 84, 87, 95, 147, 159, 160, 194
OMT 1, 3, 95, 137
estereotipo Opaco 131
Opdyke, William 37, 195
operación
  definición 73, 76
optimización 47
restricción Ordenado 100

P

paquete 128
diagramas de paquetes 11, 27, 39, 161, 186
  definición 128
  ejemplos 130, 132
  cuándo utilizar 135
visibilidad de paquete 112, 131
clase con parámetro
  definición 108
  ejemplo 108
patrones 9, 11, 13, 41, 186
  definición 42
  Véase también patrones de análisis
  Véase también patrones de diseño
  cuándo utilizar 45
perspectivas, tipos 63
redes de Petri 147
Fenómeno con patrón Intervalo 166
  adición durante el desarrollo iterativo 38
  construcción 30
riesgo político
  manejo de 29
  definición 20
poscondición 80
precondición 80
visibilidad privada
  en C++ 111
  en Smalltalk 112
proceso
  definición 1
  panorámica 16
  etapas 17
visibilidad protegida
  en C++ 111
  en Java 112
prototipos 25, 159
patrón Suplente 43
visibilidad pública
  en C++ 111
  en Java 112
  en Smalltalk 112

Q

asociación calificada
  definición 103
  ejemplo 103
patrón Cantidad 166
consulta 76

R

patrón Intervalo 110, 166, 169
Rational Objectory Process 15
Rational Software 3, 4, 48
restricción Solo Lectura 101
realización 58
Diseño Recursivo 2
reestucturación de factores 35, 179, 186
  definición 35
  principios 36
  cuándo reestructurar 37
Refactory 37
objetos de referencia 98
refinamiento 88

definición 97
riesgos de requerimientos
manejo de 20
definición 19
responsabilidad 74
Diseño guiado por responsabilidad 3
regreso 117
riesgos
categorías 19
manejo de 20, 25, 27, 29
Roberts, Don 37
papeles 66
patrón Modelos de papeles 90
Rubin, Kenneth S. 48, 194
Rumbaugh, Jim xiii, xv, 1, 3, 4, 12, 65, 68, 90, 145, 195
Véase también Tres amigos

S

escenario 57
patrón Escenario 44
riesgo de calendarización 31
SDL 147
autodelegación
consecuencias 120
definición 117
autotransición 142
diagrama de secuencia 183
comparación con diagrama de colaboración 123
definición 116
ejemplos 116, 119, 121, 174
método de establecimiento 77
Shlaer, Sally 2, 65, 68, 195
Siemens 124
clasificación simple 87
riesgo de habilidad
manejo de 27
definición 20
origen 66
perspectiva de especificación
actividades 147
asociaciones 65
atributos 72
definición 63
asociaciones derivadas 93
atributos derivados 93
ejemplos 171, 172, 182
generalización 77
operaciones 73
asociaciones calificadas 103
refinamiento 97
subclasificación 97
subtipificación 133
cuándo emplear 83
diagramas de estados 23, 40, 125, 160, 186
definición 137
ejemplos 138, 140, 141, 143
cuándo utilizar 144
tabla de estados 137
clasificación estática 89
estereotipo
como mecanismo de extensión 9
definición 86
notación 87
estereotipos
vínculo 109
historia 107
interfaz 97
opaco 131
transparente 131
tipo 97
STL 110
subclasificación 78, 81, 95, 98
sustituibilidad
y afirmaciones 82
definición 78
subtipo 61
subtipificación 78, 110
superestado 140, 141
carril
definición 156
ejemplos 157
barra de sincronización 149
interacciones del sistema 50

T

destino 66
riesgo tecnológico
  manejo de 25
  definición 19
plantilla
  Véase clase con parámetro
pruebas 34, 135
tres amigos xiii, xv, 2, 4, 9, 12, 15, 46, 48, 84, 97, 147
etapa de transición
  definición 17
  descripción 47
relación transitiva 129
estereotipo Transparente 131
tabla de verdad 159
estereotipo Tipo 97

U

asociación unidireccional 71
Método Unificado 4
UML xi, xii, xiii, xiv, xv, 1, 2, 12, 26, 33, 39, 51, 67, 91, 183, 190,191
diagramas de casos de uso
  definición 51
  ejemplo 52
casos de uso 186
  y riesgo por requerimientos 20
  y riesgo tecnológico 26
  asignación a iteraciones 32
  categorización 30
  definición 10, 49
  sobre diagramas de actividades 150, 160
  sobre diagramas de interacción 115
  niveles de prioridad 30
  propiedades 49
  división 56
  usando tarjetas CRC 75
  cuándo emplear 58
objetivos del usuario 50
relaciones de uso
  definición 56
  ejemplo 52
  cuándo usar 57

V

visibilidad
  definición 110
  dentro de C++ 111
  dentro de Java 112
  dentro de Smalltalk 112
Vlissides, John 194, 195
  Véase también Pandilla de los cuatro

W

Walden, Kim 83, 195
Wirfs-Brock, Rebeca 3, 76, 86, 135, 195
flujo de trabajo 11, 22, 147, 160

Y

Yourdon, Ed 3, 193

IMPRESORA ROMA, S. A. DE C. V.
TOMÁS VÁZQUEZ No. 152
COL. SAN PEDRO IXTACALCO
C. P. 08220 MÉXICO, D. F.

Diagrama de paquetes

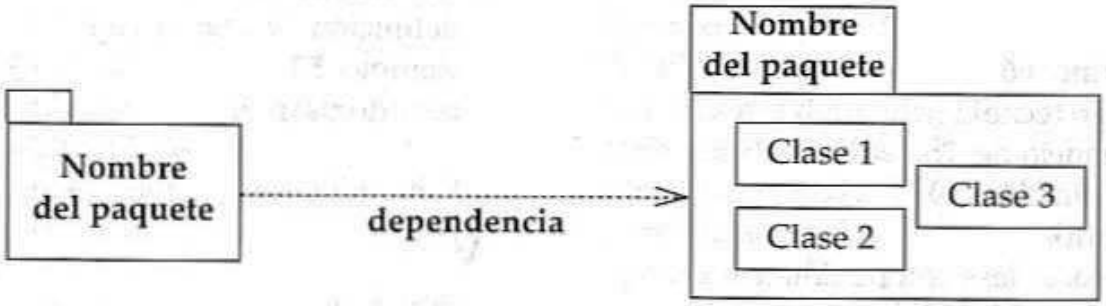


Diagrama de caso de uso

Diagrama de secuencia

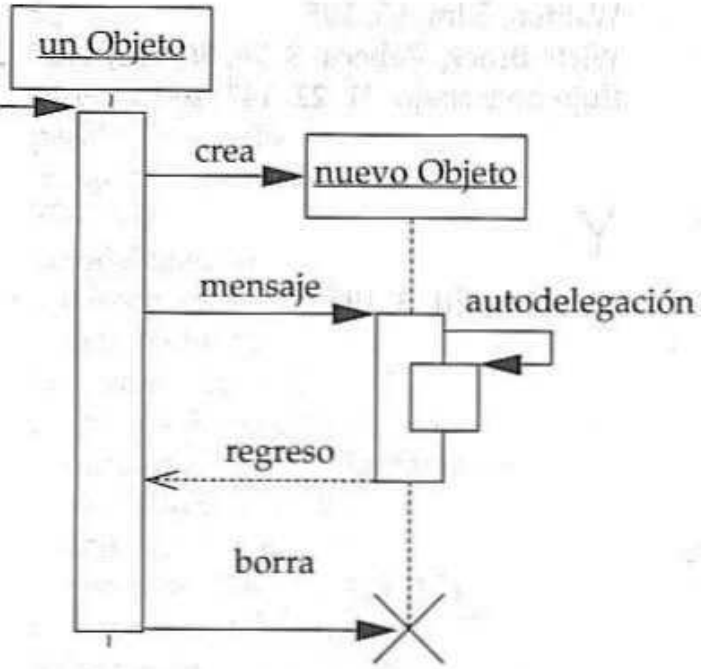


Diagrama de emplazamiento

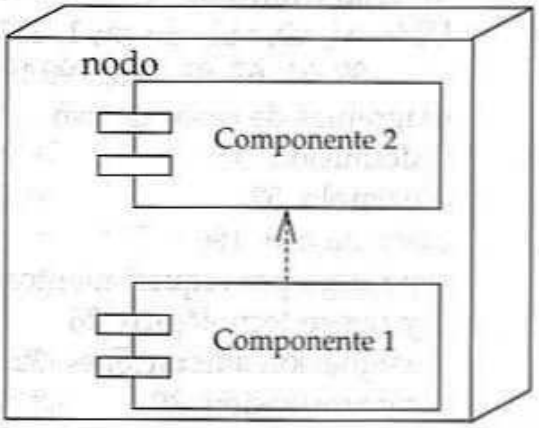


Diagrama de colaboración

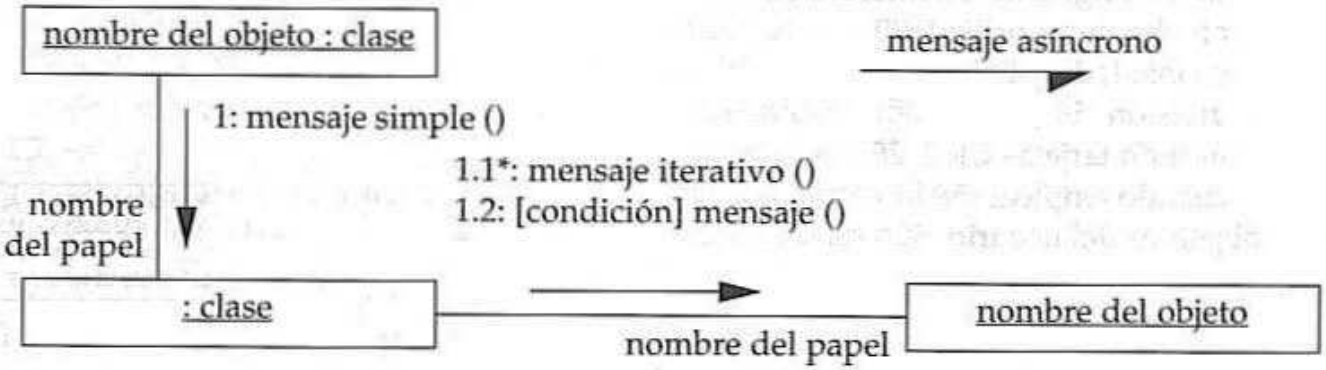
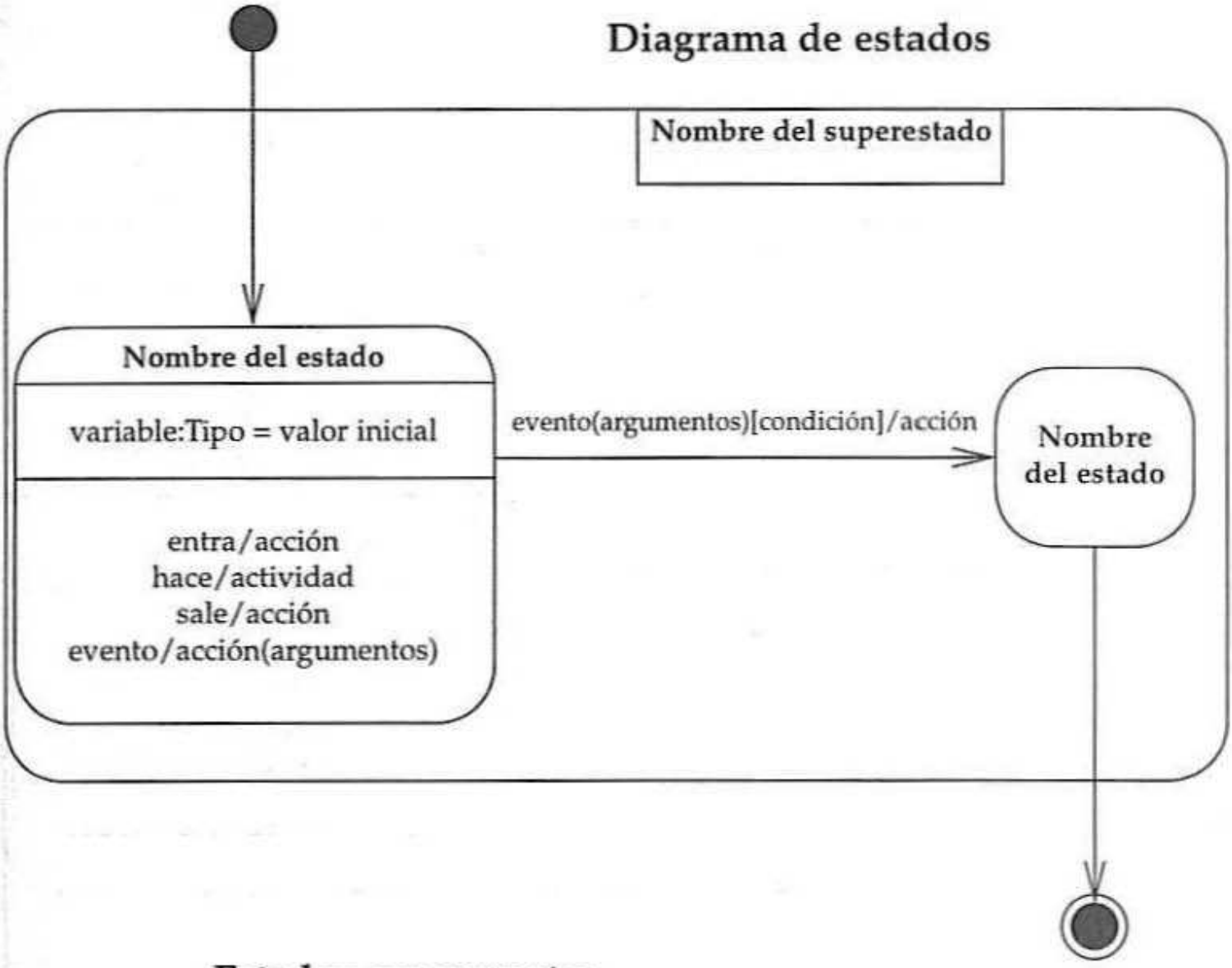


Diagrama de estados



Estados concurrentes